

How this repository actually works

Every important flow traced from the click to the committed row, with each path a link into the checkout. Written from the code as it stands, not from the documents that describe it.

COMMIT

7bee05e

TRACED BY

22 parallel readers

BUILT

1 October 2026

PATHS

clickable, and checked against disk

What is in here

Before you start	2
How to read this	2
What this is a map of	2
The repository, counted	2
What a reader should distrust	4
Part I — The live sitting	5
The sitting, from the bell to the claim	5
Marking the paper	11
The score card, the key and the per-question report	14
Rank, percentile, the board and the rollups	19
Part II — Making a paper	24
Building a test, and freezing it	24
The question bank	28
The authoring workshop	33
Importing in bulk	38
Part III — Who may do what	43
Signing in	43
What a student may reach	47
The roster, the admins and the catalog	51
Events and their candidates	58
Watching a live test, and intervening	62
Part IV — The clients	68
The student portal	68
The admin portal	71
The mobile client	75
What the three clients share	75
Part V — The platform underneath	83
The machinery every module leans on	83
Telling a student something happened	87
A student's own account	93
Part VI — The data model	99
Every table, and what the database refuses	99
Part VII — What it runs on	104
The infrastructure, the cost, and where the plan has drifted	104
Half 1 — what the plan now says	105
Half 2 — where the document and the repo disagree	107
Blocking	107
Stale claims	107
Present-tense claims with no repo artifact	107
Verified correct (so the handbook can assert them)	107
Internal arithmetic worth a second look	108
Part VIII — What is wrong with it	108
Blocking — fixed in this pass	109
Correctness — a user gets a wrong answer	109
Privacy and exposure	109
Prose that is not true of the code	110
Dead weight	110
Everything else	110

Before you start

How to read this

This document is a map of the code, written by reading the code. It is not a summary of the other documents in `docs/` — those were treated as claims to be checked, not as sources, and where a document and the repository disagree the disagreement is recorded rather than smoothed over.

Every path in here is a link. A monospaced path like `apps/api/src/attempts/submit.service.ts:150` opens that file in your checkout, and a directory like `apps/api/src/attempts` opens the folder. The boxes inside the diagrams are links too. Nothing was typed from memory: the build refuses to turn a path into a link unless the file is on disk at build time, so a dead link in this document is impossible — the path would simply have rendered as plain text.

The flow traces are the point. Each one starts at a real entry — an HTTP method and route, a queue job, a cron tick, or the click in a component — and does not stop until the value is durable: a column written, a Redis key set with its TTL, a job enqueued, a response shape returned. Guards, interceptors, transaction boundaries and every branch that changes the outcome are steps in their own right. Read a trace top to bottom and you have the whole path; click any step to land in the file at the line that does it.

Note Where something could not be verified it says so. "Read, not measured" means the figure was derived from the source rather than observed on a running system — it is kept because it is still a fact about the code, and marked because it is not a fact about production.

What this is a map of



One NestJS codebase runs as **three containers from one image**, differing only in the `API_ROLE` environment variable: `exam` registers the handful of controllers a live sitting touches, `core` registers everything else, and `worker` registers no HTTP surface worth speaking of but owns every queue processor and scheduler. Caddy decides which container a request reaches by matching the exam role's seven routes exactly. That split is why a results rush cannot slow a hall that is still writing.

The repository, counted

WHAT	HOW MUCH
HTTP routes	206 across 32 controller files — 141 admin, 56 student, 9 auth
Prisma models / enums	41 / 36
Migrations	106 files, 5,672 lines of hand-written SQL
Things Prisma cannot express	20 triggers, 24 functions, 2 views
TypeScript	~158,000 lines across 1,054 tracked files
Test files	104 unit, 74 against a real database, plus the two SPA suites
Commits since the last handbook	492

The shape of those numbers matters more than the numbers. Admin routes outnumber student routes two and a half to one, because the student surface is small and hot while the admin surface is broad and cold. The SQL is large for a Prisma project because everything that has to be true no matter which code path runs — a sat paper that cannot move, an option list that cannot drift from its question — is a trigger, not a service method.

What a reader should distrust

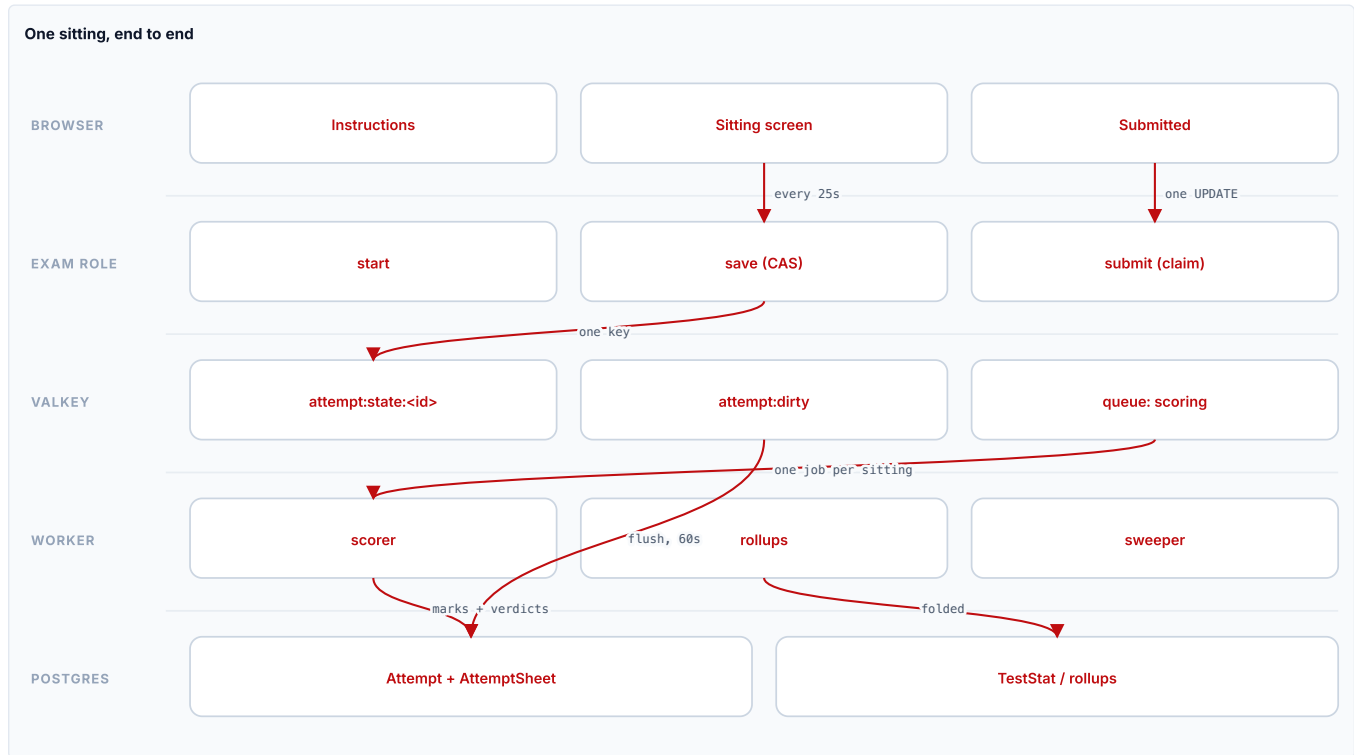
Three things in this repository are easy to misread, and each one has caught a reader before:

- `apps/api/src/events` is not an event bus. It is an admin entity — an exam event with a roster
- `AttemptSheet.answers` has no question ids in it. It is one slot per paper row, by position.
- A comment naming a gate is not a gate. Several comments describe checks that were deleted;

Part I — The live sitting

Everything in this part happens between a student pressing Begin and that student reading a mark. It is the only part of the system with a hard scaling constraint — three thousand people sitting the same paper in the same minute, designed to hold at six to eight — and every decision in it was made against that number.

The shape to hold in your head is that **the hot path does not touch Postgres**. A live sitting is one Redis key, rewritten whole under a compare-and-swap. The timer runs in the browser against a deadline the server granted. Marks are computed by a worker after the fact. Postgres is written three times in a whole sitting — at the start, on a one-minute flush, and at the submit — and the rest is cache, queue and arithmetic.



The sitting, from the bell to the claim

Where it lives

`apps/api/src/attempts/` (API), `packages/app-kit/src/exam/` + `packages/app-kit/src/autosave-policy.ts` (client engine), `packages/contracts/src/attempts.ts` (schemas + routes).

- `apps/api/src/attempts/attempts.controller.ts` — all six `/me` sitting routes; the only controller on `API_ROLES.EXAM` besides the leaderboard (`attempts.module.ts:47`).
- `apps/api/src/attempts/attempts.service.ts` — start and resume; the only writer of a new `Attempt` row.
- `apps/api/src/attempts/attempt-state.service.ts` — the Redis sitting: `open`, `save`, `current`, `take`, the compare-and-swap `patch`.
- `apps/api/src/attempts/attempt-state.ts` — pure merge rules (`applyBatch`, `isStale`, `isInTime`, `packHeld`, credit arithmetic).
- `apps/api/src/attempts/submit.service.ts` — ends a sitting exactly once, student or sweeper.
- `apps/api/src/attempts/attempt-sheet.service.ts` — the one writer of `AttemptSheet.answers`.
- `apps/api/src/attempts/answer-sheet.ts` — the positional slot codec.
- `apps/api/src/attempts/scoring-queue.ts` — the keyed BullMQ add.

What it is for

A candidate's 100-question sitting must not touch Postgres per keystroke, because 6–8K of them sit the same paper in the same minute. This module keeps the whole live sitting in one Redis key, writes Postgres three times per sitting (start, flush-on-a-timer, submit), and hands scoring to a worker. The server owns `startedAt` / `endsAt`; the client only counts down to the `endsAt` it is told. Every write of the live key is a compare-and-swap, so no concurrent save lands on a state it did not read.

FLOW — START A SITTING

- 1 `POST /me/tests/:testId/attempt`, from the exam screen's mount
`apps/exams/src/features/exam/exam.tsx:35` `useStartedSitting()`
`body {languages?, tab, resume?, holdsPaper}`; one query, `staleTime: Infinity`, retried only on a hang or fault (`packages/app-kit/src/exam/start-sitting.ts:65`).
- 2 Request budget, exam role only
`apps/api/src/common/request-budget.ts:61` `RequestBudgetInterceptor.intercept()`
`8s, then SERVICE_UNAVAILABLE (503).`

- 3 **JwtAuthGuard**
`apps/api/src/auth/guards/jwt-auth.guard.ts:20`
verifies the access token and that the Redis session still exists; sets `request.user`.
- 4 **ActorGuard** against `@Actors(ActorTypes.STUDENT)` on the class
`apps/api/src/auth/guards/actor.guard.ts:11`
an admin token is `FORBIDDEN` here.
- 5 **ThrottlerGuard**, named bucket `sitting` from `@SittingRateLimit()`
`apps/api/src/common/throttling/rate-limits.ts:25`
60 hits/min per `student:<id>` (`apps/api/src/config/env.schema.ts:135`), counted in `rateLimit:sitting:student:<id>`.
- 6 **Body validated**
`apps/api/src/common/zod-validation.pipe.ts:10` `ZodBody(startAttemptSchema)`
- 7 **Handler**, decorated `@SittingRateLimit()` `@Post('tests/:testId/attempt')` `@HttpCode(HttpStatus.OK)` with `@Param('testId')`, `@Body(new ZodBody(startAttemptSchema))`, `@CurrentUser()`
`apps/api/src/attempts/attempts.controller.ts:66` `AttemptsController.start()`
- 8 **Two independent reads in one wave**
`apps/api/src/attempts/attempts.service.ts:74` `AttemptsService.start()`
every `Attempt` of this student on this test (`orderBy attemptNo desc`) plus the `Test` with its `baseConfig` and sections.
- 9 **Branch: a row with `status = IN_PROGRESS` exists and the body named it or named nothing**
`apps/api/src/attempts/attempts.service.ts:84`
resume (flow 4), not a start.
- 10 **Branch: `resume` was sent but no live sitting matches**
`apps/api/src/attempts/attempts.service.ts:95` `AppException(SITTING_ENDED) (409)`; the SPA redirects to the submitted screen (`apps/exams/src/features/exam/exam.tsx:43`)
- 11 **Access gate, deliberately after the resume check**
`apps/api/src/access/access-resolver.service.ts:157` `assertCanStart()`
`FORBIDDEN` unless the catalog says this student may start now.
- 12 **Sittability**
`apps/api/src/attempts/attempt-rules.ts:13` `testStartBlocker()`
`CONFLICT` unless `status = ACTIVE` and `finalizedAt !== null`.
- 13 **Slot arithmetic**
`apps/api/src/attempts/attempt-rules.ts:36` `slotsAfter()`
`attemptNo = ended.length + 1` (voids counted), `ranksAgain = no ended sitting still holds isGraded`.
- 14 **Paper length read **outside** the transaction**
`apps/api/src/attempts/attempt-sheet.service.ts:21` `sizeOf()`
from the per-process paper cache.
- 15 **Transaction opens, `TX_LIMITS.SHORT` (maxWait 5s, timeout 10s)**
`apps/api/src/attempts/attempts.service.ts:131` `AttemptsService.create()`
- 16 **Attempt INSERT**
`apps/api/src/attempts/attempts.service.ts:132`
writes `testId`, `studentId`, `attemptNo`, `isGraded`, `startedAt = new Date()`, `endsAt = startedAt + scopedDurationSec×1000` (`attempt-rules.ts:46`), `shuffleSeed = randomInt(2×31)`, `languages`; defaults supply `id` (`uuid v7`), `status = IN_PROGRESS`, `scoredRevision = 0`.
- 17 **AttemptSheet INSERT**
`apps/api/src/attempts/attempt-sheet.service.ts:26`
`answers` = an array of `null`, one per paper row.
- 18 **Lock on first attempt**
`apps/api/src/attempts/attempts.service.ts:155` `lockTheBlueprint()`
`UPDATE BaseConfig SET locked = true WHERE id = ? AND locked = false`, inside the same transaction. Commit.
- 19 **Redis seeded**
`apps/api/src/attempts/attempt-state.service.ts:77` `open()`
`SET attempt:state:<id>` via CAS to `{attemptId, studentId, testId, startedAt, endsAt, revision: 0, lastSeenAt: now, answers: {}, sections: {}, tab, forwardOnly?}`, EX 172800 (48h).
- 20 **One sitting per student**
`apps/api/src/attempts/attempt-state.service.ts:129` `takeClaim()`
reads `sitting:claim:<studentId>`; a different attempt id there gets its `tab` patched to `null` (so it refuses with `SITTING_SET_ASIDE`), then `SET sitting:claim:<studentId> <attemptId>` EX 172800.
- 21 **Branch: two starts raced and the `@unique([testId, studentId, attemptNo])` rejected one**
`apps/api/src/attempts/attempts.service.ts:111`
the loser re-reads the winner and opens its key; `CONFLICT` only if it has already ended.
- 22 **Paper attached unless the screen says it holds one**
`apps/api/src/attempts/attempts.controller.ts:73`
`AttemptPaperService.paper()` (`attempt-paper.service.ts:142`); a build failure is logged and answered as `paper: null`.
- 23 **ResponseInterceptor** wraps
`apps/api/src/common/response.interceptor.ts:24`
{success: true, data: StartedAttempt, meta: {requestId}}, HTTP 200 (not 201, by `@HttpCode`).

A second start of the same test is **not** a second row: step 9 resumes it, `startedByThisCall: false`, same `startedAt`.

FLOW — AUTOSAVE AN ANSWER

- 1 The engine records a change locally and queues it
`packages/app-kit/src/exam/use-attempt-state.ts:347` `record()`
local state is what the screen draws; it never waits on the server.
- 2 Timer fires
`packages/app-kit/src/exam/use-attempt-state.ts:326` `autosaveDelayMs() = 25 000 ms ± 5 000 ms jitter (packages/app-kit/src/autosave-policy.ts:8); or early at 20 unsent answers (autosave-policy.ts:14)`
- 3 `PATCH /me/attempts/:id/state` with `{revision, answers[], sections?, tab}`, aborted at 15 000 ms
`packages/app-kit/src/exam/use-attempt-state.ts:241` `save()`
- 4 Same guard stack as the start, same `sitting` bucket
`apps/api/src/attempts/attempts.controller.ts:121`
decorators `@SittingRateLimit()` `@Patch('attempts/:id/state')` (no `@HttpCode`, so 200).
- 5 CAS loop, up to 5 tries
`apps/api/src/attempts/attempt-state.service.ts:241` `patch()`
GET the raw bytes, then a Lua `replaceJson` that writes only if the key still holds exactly those bytes, re-applying `EX 172800`. Five lost races → `CONFLICT`, "This sitting is being written to right now."
- 6 Key missing → rebuild from Postgres
`apps/api/src/attempts/attempt-state.service.ts:304` `durableState()`
one `Attempt` read (with `baseConfig.navigation/shuffleQuestions` and `sheet.answers/updatedAt`); another student's id is `NOT_FOUND`, a non-`IN_PROGRESS` row is `CONFLICT`. This is the only Postgres read on the save path, and only when Redis lost the key.
- 7 Staleness judged inside the swap
`apps/api/src/attempts/attempt-state.ts:172` `isState()`
`batch.revision <= held.revision` means a newer save already landed; the batch is dropped and the ack says `applied: false`.
- 8 Forward-only narrowing
`apps/api/src/attempts/attempt-state.ts:177` `withinReach()`
on `NAVIGATION_POLICY.FORWARD_ONLY`, changes to seats behind the furthest reached in that section are filtered out of the batch.
- 9 Ownership, clock and tab
`apps/api/src/attempts/attempt-state.service.ts:366` `answered()`
`NOT_FOUND` for another student; `CONFLICT` past `endsAt + 30s` grace (`attempt-state.ts:20`); `SITTING_TAKEN_OVER` (409) if `held.tab` is another tab, `SITTING_SET_ASIDE` (409) if it is `null`.
- 10 Merge
`apps/api/src/attempts/attempt-state.ts:197` `applyBatch()`
per question: state re-derived server-side from the answer and the mark, `timeSpentSec = max(held, sent)`, `answeredAt` kept from the first time it was given, `firstActionAt` the earliest of either side; `revision = batch.revision`, `lastSeenAt = now`; section `openedAt` stamped by the server.
- 11 Packed and written
`apps/api/src/attempts/attempt-state.ts:79` `packHeld()`
answers become positional arrays `[stateIndex, optionId/null, timeSpentSec, firstActionSec, answeredSec, typedAnswer?]` with the option kept as an id here (`KEPT_AS_AN_ID`) and times as whole seconds after `startedAt`. Measured at 10.3 KB a sitting against 24.6 KB for named fields.
- 12 Marked for the flusher
`apps/api/src/attempts/attempt-state.service.ts:286` `markDirty()`
`SADD attempt:dirty <attemptId>`, after the write.
- 13 Ack
`apps/api/src/attempts/attempt-state.service.ts:390` `acked()`
`{revision, applied, endsAt, serverNow}`. **Nothing is written to Postgres.** The client re-seeds its counter from `saved.revision`, re-anchors its clock from `endsAt / serverNow`, and only de-queues the answers if `applied` (`use-attempt-state.ts:254`).

Postgres catches up on a timer: the `attempt-flush` scheduler runs every 60 s (`apps/api/src/queue/queues.ts:155`), drains `attempt:dirty` in lanes of 8 and rewrites `AttemptSheet.answers` whole, gated on the sitting still being `IN_PROGRESS` (`apps/api/src/attempts/attempt-flush.processor.ts:28`). A crash costs at most a minute of answers.

Multi-device / version checking. The only concurrency controls are the raw-bytes CAS (step 5), the monotonic `revision` (step 7) and the tab claim (step 9). There is **no per-question version**. `ef74144` added a per-question compare on the client and `29269a8` reverted it entirely — its message states the reason: a stored queue can reach the server before the seed judges it, and a tab's own applied-but-unacknowledged save was being dropped. So a tab stood down keeps its unsent queue, and "Continue here" resends it as it stands, overwriting whatever the other device wrote in the meantime. Known limit, documented as needing a server-side per-question version.

FLOW — SUBMIT

- 1 The student confirms; the open question's seconds are banked
`packages/app-kit/src/exam/use-exam-view.ts:137`
`submit.mutationFn` → `state.finish()`.
- 2 Waits at most 5 000 ms for a save in the air, stops all saving, drains any backlog past 200 answers as ordinary saves
`packages/app-kit/src/exam/use-attempt-state.ts:286` `finish()`
then hands the remaining unsent batch to the submit call.
- 3 `POST /me/attempts/:id/submit` with `{tab, last?}`, aborted at 10 000 ms
`packages/app-kit/src/exam/use-exam-view.ts:144`
retried 3× at 1 s/2 s/4 s, and only for a hang, 503 or throttle (`packages/app-kit/src/autosave-policy.ts:39`). The server's own budget is 8 s, deliberately under it (`apps/api/src/common/request-budget.ts:20`).
- 4 Same guards and the `sitting` bucket
`apps/api/src/attempts/attempts.controller.ts:112`
decorators `@SittingRateLimit()` `@Post('attempts/:id/submit')` `@HttpCode(HttpStatus.OK)`.

- 5 Row read, ownership checked
`apps/api/src/attempts/submit.service.ts:55` `require()` + `:56`
another student's id is `NOT_FOUND` and counts a `refused` metric.
- 6 Live key read (`GET`, never taken) and the tab judged
`apps/api/src/attempts/submit.service.ts:62`
a tab that no longer holds the sitting is refused, so a stood-down tab cannot end a sitting being answered elsewhere.
- 7 The riding batch saved as the newest
`apps/api/src/attempts/submit.service.ts:88` `lastAnswers()`
calls `state.save(..., {last: true})`, which forces `revision = max(sent, held + 1)` (`attempt-state.service.ts:152`) so a reloaded screen's low counter cannot make its final batch stale. Dropped silently if the row is not `IN_PROGRESS` or the grace has passed.
- 8 Answers written **before** the claim
`apps/api/src/attempts/submit.service.ts:115` `AttemptSheetService.write(held, true)`
`raw UPDATE "AttemptSheet" SET answers = $1::jsonb, updatedAt = now() WHERE attemptId = $2 AND EXISTS (... status = 'IN_PROGRESS') (attempt-sheet.service.ts:35)`. A throw here leaves the sitting open.
- 9 The one gate
`apps/api/src/attempts/submit.service.ts:150` `claim()`
`UPDATE "Attempt" SET status = 'SUBMITTED', submittedAt = now() WHERE id = ? AND status = 'IN_PROGRESS'. count = 0` means somebody else won.
- 10 Loser path
`apps/api/src/attempts/submit.service.ts:118` `alreadySubmitted()`
re-reads the row and answers `{status, submittedAt, submittedByThisCall: false, answeredCount}`. **This is the idempotency**: a resend after a 10 s give-up, a second tab, or the sweeper all converge on one claim and one answer. HTTP 200, not an error.
- 11 Redis key taken, behind the claim
`apps/api/src/attempts/submit.service.ts:121` `AttemptStateService.take()`
one `GETDEL` on `attempt:state:<id>` plus `SREM attempt:dirty <id>`; anything arriving after it finds no key and is refused by `durableState` because the row is no longer `IN_PROGRESS`.
- 12 Last write, ungated
`apps/api/src/attempts/submit.service.ts:123` `sheets.write(last, false)`
always, even unchanged, because a stale flush may have landed between steps 8 and 9.
- 13 Scoring enqueued, after the last write
`apps/api/src/attempts/submit.service.ts:161` `queue() → apps/api/src/attempts/scoring-queue.ts:27`
`addBulk` to queue `scoring` with `name: 'scoring', data: {attemptId, testId}, opts: {jobId: 'scoring-<attemptId>', removeOnFail: true}`. The keyed `jobId` makes a duplicate add a no-op, so a retried submit cannot score twice. A queue failure is caught and logged, never fails the committed submit — the sweeper re-queues (`apps/api/src/attempts/attempt-sweeper.processor.ts:46`).
- 14 Count read off the sheet just written
`apps/api/src/attempts/submit.service.ts:140` `answeredOn()`
no read-back when a sheet was written.
- 15 Response
(attemptId, status: 'SUBMITTED', submittedAt, submittedByThisCall: true, answeredCount) wrapped in the success envelope; the client clears its local queue, marks the catalog stale without refetching, exits fullscreen and navigates to the submitted screen (use-exam-view.ts:163)

Continues in: `ScoringProcessor` (`apps/api/src/attempts/scoring.processor.ts`, queue `scoring`, concurrency 8, 5 attempts, 5 s exponential backoff — `apps/api/src/queue/queues.ts:22`), which writes `score`, `correctCount`, `wrongCount`, `unattemptedCount`, `sectionScores`, `timeTakenSec`, `scoredRevision`, `evaluatedAt` and `AttemptSheet.verdicts`, then `RollupProcessor`. Not traced here.

FLOW — RESUME A SITTING (RELOAD MID-EXAM)

- 1 The reloaded screen mounts and starts the sitting again
`apps/exams/src/features/exam/exam.tsx:35`
`POST /me/tests/:testId/attempt` with the same `tab` and, if the URL carried `?resume=`, that attempt id.
- 2 The live row is found
`apps/api/src/attempts/attempts.service.ts:83`
the gate at `:98` is **skipped**: resume asks whether a sitting may continue, not begin.
- 3 Away time judged
`apps/api/src/attempts/attempt-state.service.ts:110` `resume()`
`require()` reads the key, or rebuilds it from `AttemptSheet` if it is gone.
- 4 Branch: away longer than 48 h, or the 48 h credit cap spent
`apps/api/src/attempts/attempt-state.ts:280` `isAbandoned()`
no credit; the deadline stands and the sweeper will end it.
- 5 Clock credit
`apps/api/src/attempts/attempt-state.ts:248` `creditMs()`
`now - lastSeenAt`, less a 60 s present-grace, capped at `48h - creditedMs`. Open sections' `openedAt` are pushed by the same amount (`attempt-state.ts:260`).
- 6 Key rewritten with the new `endsAt`, `lastSeenAt = now`, this `tab`, and `creditedMs` accumulated
`apps/api/src/attempts/attempt-state.service.ts:117`
two `patch` calls, so the cap cannot be farmed by reloading.
- 7 The claim is re-taken, standing down whatever tab held the student's previous sitting
`apps/api/src/attempts/attempt-state.service.ts:106`
- 8 The only Postgres write
`apps/api/src/attempts/attempts.service.ts:90`
`UPDATE Attempt SET endsAt = ?` and only when the credited deadline differs, so the sweeper does not judge a resumed sitting by the deadline it walked away from.

- 9 Response `startedByThisCall: false` with the real `startedAt / endsAt` .
- 10 **Changed by e20bb7e** : the screen now asks `GET /me/attempts/:id/state` only when `startedByThisCall` is false
`packages/app-kit/src/exam/use-attempt-state.ts:166`
a sitting this screen just started skips the request entirely (it would return `revision: 0` and an empty object), while still setting `sectionsSettled` . At 7K simultaneous opens that is 7K fewer requests on the exam role.
- 11 On a true reload the GET answers `{attemptId, revision, answers, sections, endsAt, serverNow}` from the key
`apps/api/src/attempts/attempt-state.service.ts:167`
and the client merges it **under** anything already answered locally, seeding its revision to `max(own, held)` .

The data it touches

COLUMN	TYPE	HOLDS	WRITTEN BY
<code>Attempt.id</code>	uuid v7	sitting id	default, at start
<code>Attempt.testId / .studentId</code>	uuid	the paper, the candidate	start
<code>Attempt.attemptNo</code>	int	<code>ended.length + 1</code> , voids included	start
<code>Attempt.isGraded</code>	bool	this is the one ranked sitting	start (<code>slotsAfter</code>)
<code>Attempt.status</code>	enum	<code>IN_PROGRESS</code> → <code>SUBMITTED</code>	start (default); <code>claim()</code>
<code>Attempt.startedAt</code>	timestampz(3)	server's start instant	start
<code>Attempt.endsAt</code>	timestampz(3)	server's deadline	start; resume credit; support extend
<code>Attempt.submittedAt</code>	timestampz(3)	the claim's own instant	<code>claim()</code>
<code>Attempt.shuffleSeed</code>	int	<code>randomInt(2**31)</code> , orders this student's paper	start
<code>Attempt.languages</code>	enum[]	what this sitting is sat in	start
<code>Attempt.sectionState</code>	jsonb?	nothing — dead column	nobody
<code>Attempt.score</code> , <code>correctCount</code> , <code>wrongCount</code> , <code>unattemptedCount</code> , <code>sectionScores</code> , <code>timeTakenSec</code> , <code>scoredRevision</code> , <code>evaluatedAt</code>	—	marks	the scoring worker, not this module
<code>AttemptSheet.attemptId</code>	uuid PK	—	start
<code>AttemptSheet.answers</code>	jsonb	positional array, one slot per <code>PaperQuestion.order</code> : <code>[stateIdx, optionPosition]</code>	<code>rawId\</code> null, <code>timeSpentSec</code> , <code>firstActionSec\</code> null, <code>answeredSec\</code> null, <code>typedAnswer?</code> , null' for untouched start (blank); flusher; submit
<code>AttemptSheet.updatedAt</code>	timestampz(3)	last durable write — doubles as the rebuild's <code>lastSeenAt</code>	same
<code>BaseConfig.locked</code>	bool	blueprint frozen on first sitting	start's transaction

Redis:

KEY	VALUE	TTL
<code>attempt:state:<attemptId></code>	<code>StoredState</code> : <code>attemptId</code> , <code>studentId</code> , <code>testId</code> , <code>startedAt</code> , <code>endsAt</code> , <code>lastSeenAt?</code> , <code>creditedMs?</code> , <code>revision</code> , <code>answers</code> (<code>questionId</code> → slot array, option kept as an id), <code>sections</code> (<code>sectionId</code> → { <code>remainingSec</code> , <code>closed</code> , <code>openedAt?</code> }), <code>forwardOnly?</code> { <code>shuffleSeed</code> , <code>shuffleQuestions</code> } , <code>tab?</code> (string holds / null stood down / absent cold)	172 800 s (48 h), re-applied on every write
<code>attempt:dirty</code>	SET of attempt ids with writes Postgres has not seen	none
<code>sitting:claim:<studentId></code>	the one attempt id this student is answering	172 800 s
<code>ratelimit:sitting:student:<id></code>	hit counter	60 s

What must already be true

a `Student` row the token resolves to; a live Redis session for that token; a `Test` with `status = ACTIVE` and `finalizedAt != null` ; `assertCanStart` satisfied by the catalog (enrolment/branch reach and the schedule window); `PaperQuestion` rows for the test (the sheet is sized from them); Redis reachable (a save with Redis down has no path — the rate limiter fails open, the state CAS does not); `DATABASE_URL` , `REDIS_URL` , `RATE_LIMIT_SITTING_PER_MIN` ; `API_ROLE` including `exam` for the controller to be registered at all and `worker` for the flush/sweep schedulers.

Why it is shaped this way

- **One key per sitting, rewritten whole, over a row per answer.** A per- answer table at 6K×100 is 600K rows on the hot path; this is one `GET` + one Lua `SET` . Cost: a save is O(whole sitting) in bytes, which is why `packHeld` encodes positionally.

- **Monotonic revision over per-question versions.** The alternative (a server-side version per question) is what 29269a8 says is actually needed and is not built; a client-side compare was tried and reverted. Cost: the documented multi-device overwrite.
- **Claim-by-UPDATE over a transaction or a lock.** UPDATE ... WHERE status = 'IN_PROGRESS' is one statement and needs no advisory lock; the loser reads the winner's outcome. Cost: the answers write happens twice (steps 8 and 12) rather than once inside a transaction.
- **Keyed BullMQ job over an idempotency table.** jobId = scoring-<attemptId> makes duplicate asks free; the alternative is a dedupe row per submit. Cost: keyedJob sets removeOnFail: true, so a retained failure cannot silently swallow a re-add.
- **endsAt in Postgres as well as Redis.** The sweeper reads the row, so a resume's credit must reach both. Cost: the one Postgres write on the resume path.

Which screen calls it

ROUTE	UI FILE	SCREEN
POST /me/tests/:id/attempt	packages/app-kit/src/exam/start-sitting.ts:63	apps/exams /exam/:testId (apps/exams/src/features/exam/exam.tsx); mobile apps/mobile/app/exam/[testId].tsx
GET /me/tests/:id/brief	packages/contracts/src/client/me.ts:143	instructions screen, both clients
GET /me/tests/:id/paper	me.ts:155	instructions screen — the paper is held before the bell
GET /me/attempt/:id/paper	me.ts:151	fallback inside useStartedSitting
PATCH /me/attempt/:id/state	packages/app-kit/src/exam/use-attempt-state.ts:249 (and :428 on keepAlive when the page closes)	the sitting, both clients
GET /me/attempt/:id/state	use-attempt-state.ts:177	the sitting, reload only
POST /me/attempt/:id/submit	packages/app-kit/src/exam/use-exam-view.ts:147	the sitting, both clients

Is it optimal

- Start: 2 parallel reads + 1 PaperQuestion read (cached per process after the first) + a 3-statement transaction + 1–3 Redis commands. The attempts read uses Attempt_testId_studentId_attemptNo_key. The paper build is skipped when holdsPaper is true (c653471) — the expensive part of the bell. Read, not measured.
- Resume: the same 2 reads, plus 1 UPDATE only if the deadline moved. Optimal for what it has to decide.
- Autosave: 0 Postgres queries, 2 Redis round trips (GET + Lua CAS) plus one SADD — the stated design goal, met. Rebuild-from-Postgres is 1 query and only fires when the key is gone.
- GET /state : 1 Redis GET, and now 0 requests at all for a sitting the screen just started (e20bb7e).
- Submit: 2 Postgres reads + 2 raw UPDATE s + 1 updateMany + 2 Redis commands + 1 queue add. No N+1. The two sheet writes are deliberate (step 12's comment), not a bug.
- Payload: a save carries only changed answers, capped at 200 (SAVE_BATCH_MAX); the held state is ~10.3 KB per sitting, measured in the packHeId comment.

What it leans on

ResponseInterceptor + AllExceptionsFilter (no controller builds an envelope); JwtAuthGuard / ActorGuard / ThrottlerGuard with the Redis-backed throttler storage (four containers, one limit); RequestBudgetInterceptor + statement_timeout=6s on the exam role's connection (a9ae56d) so a client retry does not land on a container still doing the first copy; packages/contracts for the schemas both sides validate against and the ME_ATTEMPT_ROUTES both sides build URLs from; redisKeys so no feature collides on a prefix; RedisService.replaceJson / takeJson / removeIfUnchanged, three Lua scripts that are what makes the CAS and the flush-settle atomic; PaperSheetService 's per-process paper cache, which is why a forward-only save reads no Postgres once warm; AccessResolverService for the start gate; BullMQ scoring for everything after the bell.

What changed recently

(last commits over these paths)

- f472a4d — the live-ops board's status is now a SQL literal so its partial index is provable; the paper caches hold the promise, not the rows, so a cold read is paid once; the start transaction no longer counts paper questions inside itself.
- c653471 — holdsPaper on the start body; the server skips the ExamPaper build for a screen that already has it.
- 29269a8 — reverted the per-question client-side drop; a stood-down tab keeps and resends its queue. SITTING_SET_ASIDE kept so the screen can name which move stopped it.
- 27b50f0 — a sitting set aside for another test keeps what it never saved (the tab: null stand-down rather than a drop).
- e20bb7e — startedByThisCall now rides ExamSitting and suppresses the state GET on a freshly started sitting, while still setting sectionsSettled.

What is wrong with it

- prisma/schema.prisma:703 — Attempt.sectionState (jsonb) is written and read by nothing in the repo (grepped repo-wide; only the schema, the 20260820000000 migration and docs/schema.dbml mention it). Dead column.
- Section clocks are never durable. HeldState.sections lives only in attempt:state:<id>, and AttemptSheetService.write persists answers only (attempt-sheet.service.ts:35). take() at submit discards them. Bites when a candidate submits a sectional paper and the per-section time has to be reconstructed — the submitted screen gets it from the client's own sectionEffort(...) (packages/app-kit/src/exam/use-exam-view.ts:169), so a reload of that screen cannot show it.
- apps/api/src/attempt/attempt.service.ts:103 — slotsAfter is handed every non-live sitting, so attemptNo counts voided ones. Correct per its comment, but combined with @@unique([testId, studentId, attemptNo]) it means a hard-deleted attempt row (none exist today) would make the next start collide and take the isUniqueViolation path, which then looks for a live sitting and raises CONFLICT.
- apps/api/src/attempt/submit.service.ts:96 — when the Redis key is gone, lateness is judged against attempt.endsAt from the row. A support extension that moved only the key and not the row cannot happen (LiveOpsService writes both), but a resume whose UPDATE at attempt.service.ts:90 failed after the key was credited leaves the row behind the key — the final batch would then be dropped as late while the student still sees time on the clock.

- The brief's `a50fb58` is a docs commit; the submit-timeout change is `a450f57` `fix(exam): a submit that hangs is given up after 10 seconds and sent again`.

Marking the paper

Where it lives

`apps/api/src/queue/`, `apps/api/src/attempts/`, and the hand-written SQL in `prisma/migrations/20260917100000_*` / `20260918120000_*` / `20260918110000_*`.

- `apps/api/src/queue/queues.ts` — the one table of queue names, concurrency, attempts, backoff, retention, and every job id.
- `apps/api/src/queue/queue.module.ts` — `BullModule.forRootAsync` on a second ioredis connection; registers every queue with `jobOptionsFor(name)`.
- `apps/api/src/queue/reporting-worker-host.ts` — base class carrying the two `@OnWorkerEvent` listeners, found on subclasses via Nest's prototype-chain scan.
- `apps/api/src/attempts/scoring.processor.ts` — the worker: read, score, one transaction, notify, re-count.
- `apps/api/src/attempts/score-paper.ts` — pure evaluation plus the packed `sectionScores` codec.
- `apps/api/src/attempts/answer-sheet.ts` — the positional sheet encode/decode, `SLOT_STATES`, `verdictsOf`.
- `apps/api/src/attempts/paper-sheet.service.ts` — per-process LRU of a frozen paper's terms, keyed `testId:paperRevision`.
- `apps/api/src/attempts/attempt-sweeper.processor.ts` — the repair: never-scored and behind-the-paper sittings.

What it is for

Marking must be off the request path: a hall of 6–8K submits inside minutes, and evaluating inline would put the answer key, per-question writes and a rollup fold on the HTTP handler. A submit claims the row and enqueues an id; a pool of 8 workers evaluates and writes the durable columns. The same processor is the re-score mechanism — a dropped or bonus question only bumps `Test.paperRevision`, and the sweeper re-asks for every sitting stamped behind it. It is the only writer of `Attempt.score/correctCount/wrongCount/unattemptedCount/sectionScores/timeTakenSec/scoredRevision/evaluatedAt` and of `AttemptSheet.verdicts`.

FLOW — THE SCORING JOB

- Submit enqueues after the final sheet write
`apps/api/src/attempts/submit.service.ts:128`
`queue()` → `scoring-queue.ts:39` `addBulk`, job name `'scoring'`, job id `scoring-<attemptId>` (`queues.ts:72`), `removeOnFail: true` (`:61`). A throw is caught and logged (`submit.service.ts:162`); the submit still succeeds.
- Worker picks it up
`apps/api/src/attempts/scoring.processor.ts:52` `@Processor('scoring', { concurrency: 8 })`; concurrency, attempts: 5 and backoff { `exponential, 5000` } all from `queues.ts:22` + `:49`. Registered only when `API_ROLE` is worker or all (`attempts.module.ts:81–84`, `config/api-role.ts:42`)
- `process()` → `score(attemptId)`
`apps/api/src/attempts/scoring.processor.ts:67`
nothing branches on the job name.
- One `findUnique` with `SCORING_SELECT`
`:73`
`status, current score, isGraded, startedAt, submittedAt, test.paperRevision, test.scope, sheet.answers`. Missing row → warn, return `null`, job succeeds (`:78`).
- Status gate
`:81`
`SCORABLE = {SUBMITTED, EVALUATED}` (`:50`); `IN_PROGRESS` or `VOIDED` returns `null` and the job succeeds.
- `papers.termsOf(testId, paperRevision)`
`apps/api/src/attempts/paper-sheet.service.ts:157`
one `findMany` over `PaperQuestion` `ORDER BY order`, held as a promise in a 64-entry LRU; a hit is zero queries, and a drop unkeys the entry by bumping the revision.
- `scorableOf` joins sheet to terms by `SLOT`
`apps/api/src/attempts/scoring.processor.ts:207` `terms.map((term, slot) => decodeAnswer(sheet[slot], term.optionIds, startedAt))`
- `scorePaper(served)`
`apps/api/src/attempts/score-paper.ts:81` `pure.verdictOn` (`:113`) is `correctOptionIds.includes(selectedOptionId)`, null when nothing was selected or the row has no key; `TEXT_FIELD` goes to `typedVerdict` (`:120`)
case/whitespace-folded compare, or for `ANSWER_MODE.NUMERIC` `/given - wanted/ <= tolerance + 1e-9`. An empty or unusable key reads as untouched, never as wrong.
- `awardFor`
`:142`
`BONUS` pays `marks` to everyone; `DROPPED` pays `marks` to anyone who attempted and `0` otherwise (the negative is given back); else `marks / -negativeMarks / 0`. `isCorrect` stays the key's verdict either way.
- Totals sum in whole hundredths (`:75`) and divide once; `tallySection` (`:153`) keys a `Map` on `baseConfigSectionId` with the same counters plus `timeSpentSec`, divided by `divideSection` (`:175`).
- `persist()` opens `prisma.$transaction(..., TX_LIMITS.SHORT)`
`apps/api/src/attempts/scoring.processor.ts:112`
`maxWait 5s, timeout 10s` (`prisma.service.ts:8`).
- `mark()` runs ONE raw statement
`:137`
CTE `held` does `SELECT ... FOR UPDATE` and captures `evaluatedAt IS NULL AS first`; `marked` updates `Attempt` (`status EVALUATED, evaluatedAt = COALESCE(old, now)`, `score, three counts, packed sectionScores, timeTakenSec, scoredRevision = paperRevision, updatedAt`) only `WHERE status = ANY(SCORABLE) AND scoredRevision <= paperRevision`; `sheet` updates `AttemptSheet.verdicts` gated on `EXISTS (SELECT 1 FROM marked)`.
- Zero rows → `null` → `{applied:false}` → `score()` returns `null` (`:88`): voided under the scorer, or overtaken by newer marks. Nothing written, nothing counted.

- 14 `row.sheets !== 1` → throw
:169
marks without verdicts rolls the transaction back.
- 15 First evaluation → `rollups.foldStudentSitting(tx, ...)`
`apps/api/src/attempts/rollup.service.ts:314`
two raw upserts incrementing `StudentStat` then `StudentSubjectStat`, in the SAME transaction as the marks.
- 16 First evaluation → `announce()`
`scoring.processor.ts:176` → `notifications.tell(tx, ...)`
`notifications.service.ts:51` `createMany({skipDuplicates:true})`, `dedupeKey = result:<attemptId>` against the partial unique index `Notification_student_dedupe_key` (migration `20260907120000 :75`).
- 17 Re-score instead → `announceCorrection()`
:187
silent when `Number(attempt.score ?? 0) === score`; otherwise a `RESULT_UPDATED` row keyed on the NEW total.
- 18 Commit. On a re-score only, `recount()`
:95
`rollup.rebuild(testId)` (1 min debounce) and `rollup.rebuildStudent(studentId)`; a failure here is caught and logged, never failing a committed score.

FLOW — THE STALLED-ATTEMPT REPAIR

- 1 `attempts.module.ts:103`
`sweepQueue.upsertJobScheduler('attempt-sweep', { every: ATTEMPT_SWEEP_EVERY_MS })` = every 2 minutes (`queues.ts:158`), only on a worker container (:98)
- 2 `attempt-sweeper.processor.ts:41`
`process()` runs four arms in order, each later one .catch-logged so one failure does not skip the rest
- 3 `endStranded()`
:55
`loops.expired()` (`IN_PROGRESS`, `endsAt` past `SAVE_GRACE_SEC`, `LIMIT 200`, `Attempt_expiring_idx`) → `abandoned()` checks the live Redis key → `submit.expire()` in lanes of 8; stops on a short batch or a batch nothing ended.
- 4 `askAgainForUnscored()`
:96
gauge first: `unscoredCount(now - 5 min)` (`SCORING_RETRY_AFTER_MS`, `queues.ts:81`) → `metrics.setScoringBacklog` (`metrics.service.ts:175`).
- 5 `neverScored()`
:150
`SELECT id, testId FROM "Attempt" WHERE status='SUBMITTED' AND score IS NULL AND submittedAt < now - 30s ORDER BY submittedAt LIMIT 1000` (`SUBMIT_QUEUE_GRACE_SEC`, `NEVER_SCORED_BATCH_CEILING`). The status is a raw literal, not a bind, so the planner can prove `Attempt_unscored_idx` ("submittedAt") WHERE status='SUBMITTED' AND score IS NULL (migration `20260926130000 :30`; the reason is at `attempt-sweeper.processor.ts:170`).
- 6 → `scoring.queue(page)` under the SAME `scoring-<attemptId>` id, so a job BullMQ still holds swallows the re-ask and only a genuinely lost job is re-created; a non-empty page logs a count (`:103`).
- 7 `askForRescores()`
:107
`pages.behindThePaper()` (`:124`): `EVALUATED` sittings with `scoredRevision < Test.paperRevision` where `paperRevision > 0`, keyset-ordered on (`testId, scoredRevision, id`) to match `Attempt_rescore_idx` (migration `20260930160000 :20`), `RESCORE_PAGE = 1000`, looping until a short page.
- 8 → `scoring.rescore(page)`
`jobId scoring-rescore-<attemptId>-<revision>` (`queues.ts:77`): one drop queues once, a second drop is a new ask

The commit the brief cites, `See4915`, is NOT this repair — it only touched `apps/api/src/notifications/notification-delivery.processor.ts`, adding a full-batch warning to the stalled `delivery` sweep (`REPAIR_BATCH` = 25, once a minute). The scoring repair carries no such ceiling marker.

The data it touches

MODEL.COLUMN	TYPE	HOLDS	WRITER
<code>Attempt.status</code>	enum	<code>EVALUATED</code>	<code>scoring.processor.ts:142</code>
<code>Attempt.evaluatedAt</code>	timestamptz(3)	first evaluation only (<code>COALESCE</code>)	:143
<code>Attempt.score</code>	Decimal(8,2)	paper total	:144
<code>Attempt.correctCount</code> / <code>wrongCount</code> / <code>unattemptedCount</code>	Int	verdict tallies	:145–147
<code>Attempt.sectionScores</code>	jsonb	packed tuples, below	:148
<code>Attempt.timeTakenSec</code>	Int	start→submit capped at 86400; 86400 if never submitted (<code>leaderboard-score.ts:7</code>)	:149
<code>Attempt.scoredRevision</code>	Int	the <code>Test.paperRevision</code> marked against	:150
<code>AttemptSheet.verdicts</code>	jsonb	[<code>isCorrect</code> , <code>marksAwarded</code>] per slot	:159
<code>Notification</code> row	—	<code>RESULT_READY</code> / <code>RESULT_UPDATED</code>	:176 , :194
<code>StudentStat</code> , <code>StudentSubjectStat</code>	—	incremental add, first evaluation only	<code>rollup.service.ts:321–322</code>

`Attempt.sectionScores` — array of fixed 6-tuples, one per section (`score-paper.ts:44-57`): `[0]` `baseConfigSectionId` (string), `[1]` `score`, `[2]` `correctCount`, `[3]` `wrongCount`, `[4]` `unattemptedCount`, `[5]` `timeSpentSec`. Read back only through `sectionScoresIn` (`:61`), which drops any element that is not a 6-length array. Raw SQL readers index it positionally: `rollup.service.ts:209` `sums (part-->5)`, `:234-235` `reads (part-->0)` and `(part-->1)`.

`AttemptSheet.answers` — one slot per paper row in `PaperQuestion.order`; `null` is the unanswered/untouched sentinel (`blankSheet`, `answer-sheet.ts:78`). A present slot is `[stateIndex, option, timeSpentSec, firstActionSec, answeredSec]`, with a 6th element `typedAnswer` only when one was typed (`:32-41`). `stateIndex` indexes `SLOT_STATES`, append-only (`:15`). `option` is a `POSITION` into `PaperQuestion.optionIds`, or the raw id when that array does not hold it (`optionOf`, `:87`) — which then scores wrong. Times are whole seconds after `Attempt.startedAt`. `CHECK (jsonb_typeof("answers") = 'array')` and `fillfactor` (50, raised to 70 in `20260918100000` :54) exist in SQL only.

Redis: the scorer reads and writes no application key — `apps/api/src/redis/redis.keys.ts` mints none for it. BullMQ's own keyspace is the only Redis traffic, on a dedicated connection with `maxRetriesPerRequest: null` (`queue.module.ts:14`); no `prefix` is configured, so the default applies — exact key names *not verified*.

The `AttemptSheetAnswer` view — SQL only, invisible to Prisma; dropped and recreated by the uuid migration, current text at `prisma/migrations/20260918120000_every_id_is_a_uuid/migration.sql:418-443`:

```
CREATE VIEW "AttemptSheetAnswer" AS
SELECT sheet."attemptId", paper.id AS "paperQuestionId", paper."questionId",
  slot."position"::integer AS slot,
  COALESCE((ARRAY['NOT_VISITED','NOT_ANSWERED','ANSWERED','MARKED_REVIEW','ANSWERED_MARKED'])
    [((slot.value --> 0)::integer) + 1], 'NOT_VISITED') AS state,
  CASE jsonb_typeof(slot.value --> 1)
    WHEN 'number' THEN paper."optionIds"[((slot.value --> 1)::integer) + 1]
    WHEN 'string' THEN slot.value --> 1 ELSE NULL END AS "selectedOptionId",
  slot.value --> 5 AS "typedAnswer",
  COALESCE((slot.value --> 2)::integer, 0) AS "timeSpentSec",
  sitting."startedAt" + make_interval(secs => ((slot.value --> 3)::integer)::double precision) AS "firstActionAt",
  sitting."startedAt" + make_interval(secs => ((slot.value --> 4)::integer)::double precision) AS "answeredAt",
  ((sheet.verdicts --> (slot."position"::integer - 1)) --> 0)::boolean AS "isCorrect",
  ((sheet.verdicts --> (slot."position"::integer - 1)) --> 1)::numeric AS "marksAwarded"
FROM "AttemptSheet" sheet
JOIN "Attempt" sitting ON sitting.id = sheet."attemptId"
CROSS JOIN LATERAL jsonb_array_elements(sheet.answers) WITH ORDINALITY slot(value, "position")
JOIN LATERAL ( SELECT row_on_paper.id, row_on_paper."questionId", row_on_paper."optionIds",
  row_number() OVER (ORDER BY row_on_paper."order") AS "position"
  FROM "PaperQuestion" row_on_paper
  WHERE row_on_paper."testId" = sitting."testId") paper ON paper."position" = slot."position";
```

`WITH ORDINALITY` is 1-based, so `answers` matches on `position` and `verdicts` on `position - 1`. Nothing in TypeScript queries it (no reference anywhere under `apps/` or `packages/`); the originating migration header calls it support-only.

What must already be true

an `AttemptSheet` row (created inside the start transaction, `attempt-sheet.service.ts:27`); the sitting `SUBMITTED` or `EVALUATED` with `submittedAt` set; `PaperQuestion` rows frozen by `paper_question_sat_guard` and their pinned `QuestionVersion` by `question_version_sat_guard` (migration `20260917100000` :58-168) — the terms LRU has no invalidation, so the database has to promise the key cannot move; `PaperQuestion.optionIds` filled by the `paper_question_option_ids` trigger (`:44`); `REDIS_URL`; `API_ROLE` $\in$ { `worker`, `all` } for the processor and the sweep scheduler to exist at all.

Why it is shaped this way

- **Positional, not keyed by question id.** A row per answer was measured at "~220 MB and 442 MB of WAL per 5K x 100 exam before anyone scored it" (commit `b46c8e1`); a keyed jsonb object would still repeat a 36-char uuid per slot. The cost: positions mean something only while the paper cannot move, hence two triggers and an `optionIds` array copied onto each row so decoding never loads a question version — and a reader without the paper in paper order cannot decode a sheet at all.
- **Packed `sectionScores`.** Field names repeated per section per sitting were most of the value: migration `20260918110000` :1-22 measured 383 → 151 bytes and `Attempt` 61.05 → 38.16 MB over 105,000 sittings. The section id deliberately STAYS in each tuple; dropping it saves ~60 bytes but forces cross-test readers to load each blueprint's section order.
- **A stamp, not an outbox.** `scoredRevision` written in the same UPDATE as the marks replaced an `OutboxEvent` per ended sitting, a relay, a repair join that grew with lifetime volume, and a nightly prune (`cdfab62`). The cost: finding re-score work is a partial-index scan instead of a queue of rows.
- **Ids in the job, never data.** `ScoringJobData` is { `attemptId`, `testId` } (`queues.ts:65`), so a job held across a deploy or retry cannot carry a stale sheet; the price is one read per attempt per attempt-made.

Which screen calls it

the scorer has NO endpoint. Re-scoring is reached from admin: `PATCH /admin/tests/:id/paper/:rowId/status` (`tests.controller.ts:132-142`, decorated `@Audit(AUDIT_FEATURE.TEST, AUDIT_ACTION.UPDATE)`, `@RequiresFeature(FEATURE_KEYS.TEST_MANAGEMENT, PERMISSION_LEVELS.WRITE)`, `@Patch`; class-level `@Controller('admin/tests')` + `@Actors(ACTOR_TYPES.ADMIN)`), called from `apps/admin/src/features/tests/paper-disposition.tsx:106` on the Paper screen, route `/tests/:id/paper` (`apps/admin/src/lib/constants.ts:123-124`). Results are read in `apps/exams`: `GET /me/attempts/:id/scorecard` and `/solutions` (`attempt-report.controller.ts:25,31`), from `/attempts/:attemptId/score-card` and `/attempts/:attemptId/report`; both refuse with `CONFLICT` until the status is `EVALUATED` (`attempt-report.service.ts:134`). The post-submit screen does NOT poll — an Alert says marking is queued, beside a link (`apps/exams/src/features/exam/submitted.tsx:25-32`).

Is it optimal

per job: 1 `findUnique`, 0-1 paper read (an LRU hit is free), then one transaction of 1 raw CTE statement covering both table updates, 2 rollout upserts and 1 `createMany`. No N+1 — the sheet-to-terms join is an array index, not a query. Measured in `cf0ff89` on real Postgres/Valkey/BullMQ, 1000 sittings of a 100-question paper with 85 answers: ~1,435/s at 32 in flight (p50 19 ms, p95 24 ms), ~1,635/s at 64 (p50 33 ms); shipped concurrency is 8. Sweeper reads use `Attempt_unscored_idx`, `Attempt_rescore_idx` and `Attempt_expiring_idx`, each with its status as a raw literal so the partial predicate is provable. `optimal` on the hot path. The one cost I read rather than measured: the view re-derives `row_number()` over the whole paper for each slot — quadratic in paper length per attempt, acceptable only because nothing but ad-hoc SQL reads it.

Shared machinery — `ReportingWorkerHost` + `QueueFailures` (two `@onWorkerEvent` listeners no processor restates); `QUEUE_POLICY` / `jobOptionsFor` (one table, not per-queue options); `keyedJob` (BullMQ's one-job-per-id IS the dedupe mechanism); `PaperSheetService.hold()` (a cold read is shared, a failed one is not cached);

`TX_LIMITS` ; `NotificationsService.tell` plus the SQL partial unique index (replay safety for free); `RollupQueue` (debounced keyed rebuilds); `@iace/contracts` enums. No envelope, guards or app-kit — no HTTP surface here.

What changed recently

- `bdf8c1` — added `AND a."scoredRevision" <= paperRevision` to the marking UPDATE. Before it a scorer that read the paper before a drop could commit after the re-score, putting old marks back with the old stamp; the next ask was then swallowed by BullMQ's hour-long completed-id retention.
- `6949f2b` — the re-score arm now pages keyset-style. Before, it read an unordered 1,000 per sweep, so a drop on a 6,000-sitting test took ~12 minutes just to queue.
- `cdfab62` — replaced outbox/relay/repair/prune with `Attempt.scoredRevision` plus the sweeper scan; a drop now commits in a SHORT transaction.
- `cf0ff89` — submit stopped writing an `OutboxEvent`: the claim is one UPDATE; `SUBMITTED` with a null score IS the request.
- `f472a4d` — live-test read reductions, including binding statuses as literals so these sweeps' partial indexes are provable.

Idempotency, failure and re-score

- A job that throws is retried up to `attempts: 5` with exponential backoff from 5000 ms (`queues.ts:22`, `:53`). Each failure hits `ReportingWorkerHost.onFailed` → `QueueFailures.record`, which increments a metric and logs at `error` once spent (`queue-failures.ts:22`). Because every scoring add goes through `keyedJob` (`removeOnFail: true`, `queues.ts:61`), a spent scoring job is removed at once — `FAILED_JOB_RETENTION`'s seven days never applies to it, and the log line is the only record. The sitting stays `SUBMITTED` with a null score, so the sweeper re-asks within 2 minutes and the gauge rises after 5.
- Idempotent on a retry, from the code: nothing is written outside the one transaction, so a throw leaves no partial state; `evaluatedAt = COALESCE(a."evaluatedAt", now)` (`:143`) keeps the first stamp; every other column is an absolute assignment of a pure function of (sheet, terms); the ONLY incremental write, `foldStudentSitting`, is gated on `first` (`:117`), which is `evaluatedAt IS NULL` read under `FOR UPDATE`, so a second delivery cannot double-add `StudentStat`. The notification is deduped on `result:<attemptId>` against a unique index. `apps/api/test-db/scoring-processor.db.test.ts:156` asserts a full row + per-question snapshot (minus `updatedAt`) is identical on a second `score()`; `:143` that a stamped `evaluatedAt` does not move.
- Re-score is a first-class path: `SCORABLE` includes `EVALUATED` (`:50`). It is triggered by `PaperService.setQuestionStatus` incrementing `Test.paperRevision` (`paper.service.ts:566`), found by the sweeper, queued under `scoring-rescore-<attemptId>-<revision>`. On a re-score the processor skips the incremental fold and queues a test-wide and student-wide rebuild instead (`:90`); rebuilds write their answer outright, so running them twice is safe. There is no "re-score this attempt" admin endpoint — the paper-row status is the only lever.

What is wrong with it

- `queues.ts:39-40` — the `FAILED_JOB_RETENTION` comment ("kept a week") holds only for the scheduler queues. Every `scoring`, `rollup` and `notification-delivery` add goes through `keyedJob`, whose `removeOnFail: true` overrides it. Bite: a sitting whose scoring fails deterministically five times — the one coded throw is `scoring.processor.ts:170`, marks landing with no sheet row — leaves nothing inspectable in BullMQ, only a log line and a rising `scoringBacklog`.
- `prisma/migrations/20260918120000_every_id_is_a_uuid/migration.sql:423` duplicates `SLOT_STATES` as a SQL literal array. Bite: appending a sixth state to `answer-sheet.ts:15` silently makes the view report `NOT_VISITED` for every slot in that state; nothing compares the two lists.
- `scoring.processor.ts:244-250` — `foldableOf` indexes `scored.questions[slot]` and `served[slot]` by the `TERMS` index, correct only because `scorePaper` pushes exactly one verdict per row handed to it (`score-paper.ts:92`). The comment at `:243` says so; no type enforces it, so an early `continue` added to that loop would shift every subject's correct-count by one.
- `answer-sheet.ts:171-172` — two consecutive doc comments on `answeredRows`, the first a leftover describing the wrong thing. Dead text, no behaviour.
- `scorableOf` (`:208`) uses `sheetIn`, which returns `[]` for a non-array column, so a malformed sheet would score as all-unanswered rather than failing. Latent only: the SQL `CHECK` makes it unreachable today.

The score card, the key and the per-question report

Where it lives

`apps/api/src/attempts/` (API), `apps/exams/src/features/performance/` and

`apps/exams/src/features/review/` (web), `apps/mobile/src/components/report/` (mobile), `packages/contracts/src/{attempts,stats}.ts` (shapes), `packages/app-kit/src/student-queries.ts` (cache keys).

- `apps/api/src/attempts/attempt-report.controller.ts` — all three student read routes: `scorecard`, `solutions`, `performance`.
- `apps/api/src/attempts/performance.service.ts` — `scoreCard()` and the admin `report()`, both through one `figuresOf`.
- `apps/api/src/attempts/attempt-report.service.ts` — `solutions()` (the only route carrying an answer key) and the trend.
- `apps/api/src/attempts/paper-sheet.service.ts` — the four per-paper in-process LRU caches and the `hold` / `recall` / `remember` primitives.
- `apps/api/src/attempts/answer-sheet.ts` — the positional sheet codec; `answeredRows` / `servedSheet`.
- `apps/api/src/attempts/question-report.service.ts` — the per-question table, its own paper read, its own key read.
- `apps/exams/src/App.tsx:174-185` — the five report tabs as routes under `/attempts/:attemptId/report`.
- `apps/api/test-db/attempt-report.db.test.ts:38` — `QUERY_BUDGET = { SCORE_CARD: 14, REFUSAL: 2 }`, the gate that keeps the card at one request's worth of queries.

What it is for

Once the scoring job has marked a sitting, the student has to be able to read

what it was worth, where it put them in the cohort, and why each answer was right or wrong. This module is every read that happens *after* the paper is handed in, and it is deliberately on the `CORE` API role (`attempts.module.ts:49-60`) so a results rush never lands on the containers serving the live hall. It must do that for a 6K cohort all opening their result within minutes of each other, which is why the score card is one request and the answer key is held per paper rather than per reader. It is also the module that owns the only path on which a `QuestionVersion` answer key can reach a student.

FLOW — THE SCORE CARD

- Student lands on `/attempts/:attemptId/report` (index tab)
`apps/exams/src/App.tsx:177` `ScoreCardPanel`
the panel mounts; the shell above it reads only the trend (`report.tsx:131`), not the card.

- 2 `useQuery(scoreCardQuery(attemptId))`
`apps/exams/src/features/performance/score-card.tsx:24`
`key ['me', 'attempts', id, 'score-card'], meta.silent = isMarkingPending`.
- 3 `GET /me/attempts/:id/scorecard`
`packages/contracts/src/attempts.ts:444 + client/me.ts:181`
`response parsed against scoreCardSchema`.
- 4 `JwtAuthGuard` → `ActorGuard` → `FeaturePermissionGuard` → `SuperAdminGuard` → `ThrottlerGuard`
`apps/api/src/app.module.ts:77-82` `@Actors(ActorTypes.STUDENT)` on the controller class restricts it to a student token
- 5 `@Get('attempts/:id/scorecard')` with `@Param('id')`, `@CurrentUser()`
`attempt-report.controller.ts:26-28`
`delegates straight to PerformanceAnalyticsService`.
- 6 `attempt.findFirst({ where: { id, studentId, status: EVALUATED }, select: SCORE_CARD_SELECT })`
`performance.service.ts:190-193`
`ownership and markedness are both in the WHERE, so a refusal pays for no nested read (~4 queries: Attempt, Test, TestBaseConfig, BaseConfigSection).`
- 7 No row → `refuseCard` reads `{ id: true }` alone
`performance.service.ts:238-245`
`NOT_FOUND` if not theirs, `CONFLICT` ("This paper has not been marked yet...") if theirs and unmarked. 2 queries total.
- 8 `Promise.all([testStats([testId]), leaderboard.standing(testId, id)])`
`performance.service.ts:196-199`
`TestStat` row (`computedAt` doubles as the curve's cache watermark) + one `standingsSql` raw query.
- 9 `standingsSql({ attemptId })`
`ranking-sql.ts:46-82`
`WITH chosen ... CROSS JOIN LATERAL` count; cohort filter is literal SQL so the planner matches `Attempt_ranking_idx (testId, score DESC, timeTakenSec, id)`.
- 10 `figuresOf({ scope: ATTEMPT, attemptId }, attempt, standing, testStats)`
`performance.service.ts:201,248`
`the one method the admin report also calls, so card and admin view cannot drift.`
- 11 Inside, five reads in parallel
`performance.service.ts:255-272`
`paperQuestion.findMany(PERFORMANCE_ROW_SELECT)`; `attemptSheet.findUnique({answers, verdicts})`; `testSectionStat.findMany`; `topperOf`; `curveOf`.
- 12 `topperOf`
`topper.ts:44-75`
`rankOneSql` (one index probe) → `topper` `Attempt` + its `AttemptSheet` → that paper's `SHEET_ROW_SELECT` rows. 4 queries. Only the topper's **time and marks** leave the file.
- 13 `curveOf`
`performance.service.ts:289-316`
`hold(this.curves, \ ${testId}:${computedAt}\, ...)` → `cohortCurveOf` raw `GROUP BY score`; cold only, keyed on the rollup watermark.
- 14 `servedSheet(paper, {...anchor, sheet}, shuffleQuestions)`
`apps/api/src/attempts/answer-sheet.ts:199`
`the sheet is joined to the paper positionally: slot N ↔ the Nth PaperQuestion by order ( answer-sheet.ts:179 ), then re-ordered into the student's display order.`
- 15 `compositionOf`, `sectionalStandingOf`, `timeUseOf`, `paceOf`
`performance-analytics.ts:34,141, attempt-analytics.ts:20, performance.service.ts:442`
`pure; every rate carries the n it was measured over, so an uncounted bucket is null, never 0`.
- 16 Card assembled
`performance.service.ts:207-234`
`maxMarks` comes from `figures.composition`, `totalQuestions / durationSec` from `scopedQuestionCount / scopedDurationSec` over `test.scope` + `scopeRef`.
- 17 `ResponseInterceptor` wraps it as `{ success: true, data, meta:{requestId} }`
`apps/api/src/app.module.ts:75`
`14 queries total, pinned by attempt-report.db.test.ts:171-180`.

It is one request, but not for all five tabs. `scoreCardQuery` is read by Score card (`score-card.tsx:24`), Subject report (`subject-report.tsx:30` , reads only `card.sections`) and Compare (`compare.tsx:23` , reads `testId` , `cohort` , `maxMarks`) — and by the mobile `score-card-panel.tsx:17` , `subjects-panel.tsx:27` , `compare-panel.tsx:25` . Solution report and Question report have their own endpoints by design: they are the two payloads that carry the key. The shell itself fetches no card (`864c567`): it takes `isGraded` off the trend.

FLOW — THE PER-QUESTION REPORT

- 1 `/attempts/:id/report/questions`
`apps/exams/src/App.tsx:181-183`
`QuestionReportPanel`; `PageFrame` fills so the table owns the scroll (`report.tsx:41`).
- 2 `useQuery(questionReportQuery(attemptId))`
`apps/exams/src/features/performance/question-report.tsx:48`
`GET /me/attempts/:id/question-report ( attempts.ts:446 )`.
- 3 `@Get('/:id/question-report')`, `@Actors(STUDENT)`
`question-report.controller.ts:6-17`
`QuestionReportService.forAttempt`.
- 4 `attempt.findFirst({ id, studentId }, REPORT_SELECT)`
`question-report.service.ts:89-92`
`includes sheet:{answers, verdicts}`; `no questionVersion`.

- 5 Gate: `!attempt → NOT_FOUND`; `status !== EVALUATED → CONFLICT`
question-report.service.ts:93-96
| unlike the score card this is checked *after* the wide read, so a refusal still pays for it.
- 6 Five reads in parallel
question-report.service.ts:99-109
| `testQuestionStat.findMany`; `testStat.findUnique`; `topperOf`; `paperQuestion.findMany(REPORT_ROW_SELECT)`; `leaderboard.cohortSize`.
- 7 REPORT_ROW_SELECT
question-report.service.ts:62-74
| this is where the correct answer comes from: `questionVersion.options` (each carrying `isCorrect`) and `questionVersion.answerKey`. Read **direct from Postgres, not through PaperSheetService**.
- 8 `servedSheet(rows, attempt, shuffleQuestions)`
apps/api/src/attempts/question-report.service.ts:110
| the student's answer is decoded off `AttemptSheet.answers` slot-by-slot, the verdict off `AttemptSheet.verdicts[slot]`; the join is positional, by `PaperQuestion.order`.
- 9 `keyOf(rows)`
question-report.service.ts:198-213
| a `Map<questionId, {options, correctAnswer}>`; `correctAnswer` is the first string in `answerKey.answers` and only for `TEXT_FIELD`.
- 10 assemble
question-report.service.ts:121-153
| `compared = evaluatedCount >= COHORT_COMPARISON_FLOOR`; below it every cohort column is forced `null` rather than shown over a tiny n.
- 11 `questionReportRow(sat, cohort, topper, keyed)`
apps/api/src/attempts/question-report.ts
| one row per served question: the student's half joined by slot, the cohort's half keyed by `paperQuestionId`, the key's half keyed by `questionId`.
- 12 Response `{attemptId, testId, testTitle, cohortSize, paceIndex, sections, questions}`
~15 queries, unpinned by any budget test

FLOW — SOLUTION REVIEW (THE ANSWER KEY)

- 1 `/attempts/:id/report/solutions`
App.tsx:179 → SolutionPanel
| `apps/exams/src/features/review/review.tsx:13`; mobile `solutions-panel.tsx:20`.
- 2 `useState('')` for the section, then `useQuery(solutionsQuery(attemptId, sectionId))`
review.tsx:15-17
| `key ['me', 'attempts', id, 'solutions', sectionId]`, `placeholderData: (held) => held` so the open section stays on screen while the next loads (`student-queries.ts:112-119`).
- 3 `GET /me/attempts/:id/solutions?sectionId=...`
attempts.ts:445, client/me.ts:185-189
- 4 `@Get('attempts/:id/solutions')`, `@Query(new ZodQuery(solutionsQuerySchema))`, `@Actors(STUDENT)`
attempt-report.controller.ts:32-39
| `{ sectionId?, questionId? }`, both `.min(1).optional()`.
- 5 `attempt.findFirst({ id, studentId }, SOLUTION_SELECT)`
attempt-report.service.ts:129-132 5 queries (`Attempt`, `AttemptSheet`, `Test`, `TestBaseConfig`, `BaseConfigSection`). `SOLUTION_SELECT` carries ****no**** `questionVersion` the key rides the paper, two reads later.
- 6 The gate: `!attempt → NOT_FOUND` "No such sitting"; `status !== ATTEMPT_STATUS.EVALUATED → CONFLICT`
attempt-report.service.ts:133-136
| that is the whole gate. Pinned by `attempt-report.db.test.ts:339-369`, which asserts exactly one `findFirst` and that no `questionVersion` appears in it.
- 7 `papers.solutionsOf(testId, test.paperRevision)`
attempt-report.service.ts:139 → paper-sheet.service.ts:146-154
| `hold(this.solutions, \ ${testId}:${paperRevision}\ , ...)`; cold adds 3 queries (`PaperQuestion`, `Question`, `QuestionVersion`), warm adds 0.
- 8 The cache: a plain `Map<string, Promise<SolutionPaperRow[]>>` field on the singleton `PaperSheetService` (`paper-sheet.service.ts:119`). `hold` holds the **promise**, so 6K concurrent cold readers share one read; a rejected read deletes itself so the next reader retries (`paper-sheet.service.ts:108-111`). Eviction is LRU at `HELD_PAPERS = 64`
recall re-inserts on a hit, remember drops `held.keys().next()` (insertion-order first = longest unread). No TTL, no size bound in bytes, process lifetime only
- 9 `answeredRows(paper, attempt)`
answer-sheet.ts:173-196
| each paper row gets this sitting's answer and verdict, positionally.
- 10 `.map(toSolutionQuestion)`
attempt-report.service.ts:204-217
| narrows `questionVersion.content` (stem and worked solution) and each option's text to `attempt.languages`, and attaches `answerKey`. Options keep `isCorrect`.
- 11 `servedQuestions(rows, shuffleSeed, shuffleQuestions, shuffleOptions)`
exam-order.ts:56-73 via attempt-report.service.ts:141-146
| the same sequencer the exam paper used; one option generator spent across the **whole** paper in display order.
- 12 `wantedOf(served, query, sections[0]?.id ?? null)`
attempt-report.service.ts:171-185
| narrowing happens **after** the sequencer: `questionId` wins and answers one row (a saved question carries no section); otherwise `sectionId`, absent meaning the paper's first section.
- 13 `imageUrlsIn(storage, asked.questions.flatMap(htmlOfQuestion))`
attempt-report.service.ts:156, exam-images.ts:5-11
| image keys in the narrowed section only are resolved to URLs.

- 14

Response

attempt-report.service.ts:158-166

```
{attemptId, testId, testTitle, languages, sections[], sectionId, questions[]}: every section listed so the screen can offer the ones it has not asked for, one section's questions carried.
```
- 15

Section switch

review.tsx:31 onSectionChange: setSectionId

```
a new query key, a new request, the previous section kept as placeholder.
```
- 16

A saved question opened alone

saved-question-dialog.tsx:23-24 → savedSolutionQuery → ?questionId=

```
key [..,'solutions',' ', 'question', questionId] .
```

Is there a `solutionsAreOpen` gate? No. It existed and was deleted. `solution-gate.ts`, `solutionsOpening()`, `solutionsOpen`, `closedReason` and `openedAt` all went in 7b71af8 ("feat(tests): a test opens and never shuts"), whose body states the trade in the author's own words: a student who has sat a paper can hand the key to one who has not, and nothing shuts that window. The only surviving mentions are a historical migration comment (`prisma/migrations/20260907090000_a_practice_test_opens_and_never_shuts/migration.sql:8`) and two **stale comments in contracts** that still describe a gate the code does not have: `packages/contracts/src/stats.ts:686` ("waits for `solutionsAreOpen`, exactly as the Solution Report does") and `:717` ("Gated. Empty and null until `solutionsAreOpen`"). The enforced gate is two lines and nothing else: `attempt-report.service.ts:134-136` and `question-report.service.ts:94-96` — your own sitting, `status = EVALUATED`.

Can an early submitter read the key before the batch finishes? Yes. There is no cohort-wide instant anywhere in the path: the moment the BullMQ scoring job sets `status = EVALUATED` (`scoring.processor.ts:142`), that student's `/solutions` and `/question-report` both answer with `questionVersion.options().isCorrect` and the worked solution, while every other student is still sitting. Nothing on `Test` or `Attempt` carries a solutions-release instant to check.

The data it touches

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT	
Attempt.status	enum	EVALUATED is the entire gate for all three reads	scoring.processor.ts:142	
Attempt.score / correctCount / wrongCount / unattemptedCount	Decimal(8,2) , Int?	the card's headline figures	scoring processor	
Attempt.sectionScores	jsonb	positional array per section: [baseConfigSectionId, score, correctCount, wrongCount, unattemptedCount, timeSpentSec] (score=paper.ts:61-71)	scoring processor	
Attempt.shuffleSeed	Int	re- derives the student's display order on every read	attempt start	
Attempt.isGraded / evaluatedAt / submittedAt / startedAt	bool / timestamptz	retake flag; the clock the sheet's second -offsets are relative to	start + scoring	
AttemptSheet.answers	jsonb	array, one slot per PaperQuestion.order ; slot = [stateIndex, optionPosition0Id, timeSpentSec, firstActionSec, answeredSec] , plus a 6th typedAnswer only when given (answer=sheet.ts:23-43); null slot = untouched	exam autosave/flush	
AttemptSheet.verdicts	jsonb?	same positions: [isCorrect \	null, marksAwarded] (answer-sheet.ts:46')	scoring processor only
PaperQuestion.order / optionIds	Int , String[]	the positional join key, and what an answer slot's integer indexes into	paper finalize (trigger-copied)	
PaperQuestion.marks / negativeMarks / status	Decimal(6,2) , enum	the arithmetic shown, and why a dropped question reads oddly	paper finalize	
QuestionVersion.options	jsonb	the key: each option's id , text , isCorrect	authoring; frozen by question_version_sat_guard	
QuestionVersion.answerKey	jsonb	TEXT_FIELD accepted answers	authoring	
QuestionVersion.content	jsonb	per-language {stem, solution} rich nodes	authoring	
Test.paperRevision	Int	the solutions cache's revision key; +1 on a paper edit (tests/paper.service.ts:566)	paper service	
TestStat.evaluatedCount / sumTimeSec / computedAt	Int/BigInt/ts	pace index denominator; computedAt is the curve cache's watermark	rollup	
TestSectionStat.attempted / sumScore / sumTimeSec	Int/Decimal/BigInt	the per- section cohort mean	rollup	
TestQuestionStat.correctCount / wrongCount / skippedCount / sumTimeSec / optionCounts	Int/BigInt/jsonb	the question report's cohort half	rollup	

No Redis at all on this path. `AttemptReportService`, `PerformanceAnalyticsService` and `QuestionReportService` take only `PrismaService`, `LeaderboardService`, `StorageService` and `PaperSheetService`. Every cache here is an in-process `Map`: `PaperSheetService`.`{rows, terms, served, solutions}` (`paper-sheet.service.ts:116-119`, LRU 64 each) and `PerformanceAnalyticsService`.`curves` (`performance.service.ts:139`, LRU 64).

`AttemptSheetAnswer` — the SQL view that decodes a sheet into one row per slot (`prisma/migrations/20260918120000_every_id_is_a_uuid/migration.sql:418`, originally `20260917100000...:171`) — is **not read by any TypeScript in this module or anywhere else**. It is not in `schema.prisma` and has no Prisma model. It exists for support and ad-hoc SQL only; its own migration comment says so. Every runtime decode goes through `answer-sheet.ts`.

What must already be true

an `Attempt` row owned by the caller with `status = EVALUATED` ; an

`AttemptSheet` with `verdicts` written (the scoring transaction writes totals and verdicts together or neither, `scoring.processor.ts:105`); `PaperQuestion` rows with `optionIds` populated, in a stable `order` ; a student actor token (`@Actors(ActorTypes.STUDENT)`); the API process serving `API_ROLES.CORE` — on the `EXAM` role these controllers are not registered at all (`attempts.module.ts:49–60`). The cohort columns additionally need a `TestStat` / `TestSectionStat` / `TestQuestionStat` row; without one they read as `null` or a dash, never as a live scan. `Attempt_ranking_idx` must exist for the standing and topper probes (`prisma/migrations/20260929090000...:32`).

Why it is shaped this way

- The sheet is **one positional jsonb array**, not a row per answer. A 6K × 200-question paper would
- The answer key is **held per paper in process memory**, not per reader from Postgres and not in
- The curve and the standing are **counted live**, not stored. `TestStat.scoreHistogram` was
- The score card deliberately **carries no question**. `2975989` removed the `questions` array that

Which screen calls it

ROUTE	WEB (<code>APPS/EXAMS</code>)	MOBILE
GET /me/attempts/:id/scorecard	<code>features/performance/score-card.tsx:24</code> at <code>/attempts/:id/report</code> ; <code>subject-report.tsx:30</code> at <code>../report/subjects</code> ; <code>compare.tsx:23</code> at <code>../report/compare</code>	<code>src/components/report/score-card-panel.tsx:17</code> , <code>subjects-panel.tsx:27</code> , <code>compare-panel.tsx:25</code> on <code>app/attempts/[attemptId]/report.tsx</code>
GET /me/attempts/:id/solutions	<code>features/review/review.tsx:17</code> at <code>../report/solutions</code> ; <code>features/review/saved-question-dialog.tsx:23</code> (by <code>?questionId=</code>)	<code>src/components/report/solutions-panel.tsx:20</code>
GET /me/attempts/:id/question-report	<code>features/performance/question-report.tsx:48</code> at <code>../report/questions</code>	<code>src/components/report/questions-panel.tsx:39</code>
GET /me/performance	<code>features/performance/report.tsx:131</code> (shell), <code>overview.tsx:61</code> , <code>compare.tsx:24</code>	<code>compare-panel.tsx:26</code>

No admin SPA screen reads any of these; the admin side reads `PERFORMANCE_SCOPES.ATTEMPT` through `AdminPerformanceController` instead. `ROUTES.SCORE_CARD_PATTERN` , `REVIEW_PATTERN` and `QUESTION_REPORT_PATTERN` (`apps/exams/src/lib/constants.ts:29–31`) are kept as redirects into the tabbed shell (`App.tsx:186–197`).

Is it optimal

- `/scorecard` : **14 queries**, counted from the code and pinned at `attempt-report.db.test.ts:38,179`
- `/solutions` : **8 queries cold, 5 warm** (derived from the selects, not measured) — the 5 are the
- `/question-report` : **~15 queries** (derived) and, unlike `/solutions` , ****it re-reads the whole**
- **No byte measurement of any of these payloads exists in the repo** — no test, script or harness

What it leans on

`ResponseInterceptor` / `AllExceptionsFilter` for the envelope and

for mapping `AppException(ErrorCodes.CONFLICT)` to a 409 the client reads by code (`isMarkingPending` , `packages/app-kit/src/markings.ts:5`), which is what lets "No marks yet" be an `EmptyState` rather than an error; `JwtAuthGuard` + `ActorGuard` + `@Actors` for ownership of the actor type, with row ownership done in every WHERE rather than in a guard; `ZodQuery` + `solutionsQuerySchema` for the two query params; `packages/contracts` for the shapes and for `servedQuestions` / `displayOrder` , the sequencer shared with the live exam paper — which is the guarantee that the option a student remembers as "C" is "C" in review (`attempt-paper.db.test.ts:347–376`); `packages/app-kit/src/student-queries.ts` for one cache key per read across web and mobile; `packages/ui` `EmptyState` / `DataTable` / `Combobox` / `Tabs` ; `StorageService` for image URLs. No queue and no Redis on any read in this module.

What changed recently

(`git log -15` over these paths)

- `7912c96` — the student SPA moved from `apps/test` to `apps/exams` ; the graft index still has the old paths.
- `b610946` — the leaderboard's board is read once per rollup and sliced per reader; the Compare tab's board read got cheaper, the score card's standing did not change.
- `3e10825` — added `PaperSheetService.solutions` (the fourth LRU, keyed `testId:paperRevision`), moved the key off a per-reader Postgres read, added `?sectionId=` / `?questionId=` with the narrowing placed *after* the sequencer, and deleted the gate's own second read so the status check rides the sitting's one select. Also deleted `PRICED_ROW_SELECT` .
- `864c567` — the report shell stopped reading the score card for the retake notice; landing on Solutions or Questions no longer pays for a 14-query card. `isGraded` now rides the trend's sittings.
- `0d69861` — `status = EVALUATED` moved into the card's WHERE, so a refusal costs 2 queries instead of ~4; the narrow second read decides `NOT_FOUND` vs `CONFLICT` .
- `2975989` — merged the student `performanceReport` into the score card: one request, 14 queries instead of 24 over two; deleted `GET /me/performance/report` , `api.me.performanceReport` , `attemptReportQuery` ; the card dropped its `questions` and `sections` arrays and gained the report's composition, sectional standing, time use, pace and curve.

What is wrong with it

1. `packages/contracts/src/stats.ts:686` and `:717` — both comments assert a `solutionsAreOpen` gate
2. `packages/contracts/src/stats.ts:514–515` — `scoreHistogramSchema` and its comment "The shape

3. `apps/api/src/attempts/question-report.service.ts:103-107` — reads the whole paper's
4. `apps/api/src/attempts/attempt-report.service.ts:171-185` — `wantedOf` with a `questionId` that is
5. `apps/exams/src/features/review/review-paper.tsx:54` — the `notice` prop ("What the gate is still
6. `apps/api/src/attempts/attempt-report.service.ts:187-202` — `toScoreCardQuestion` is named for a
7. `apps/api/src/attempts/answer-sheet.ts:171-172` — two consecutive doc comments on

Rank, percentile, the board and the rollups

Where it lives

`apps/api/src/attempts/` (ranking + rollups), `apps/api/src/dashboard/`,
`prisma/migrations/**` for the SQL function and indexes.

- `apps/api/src/attempts/ranking-sql.ts` — every ranking query as `Prisma.Sql`; the only place rank SQL is written.
- `apps/api/src/attempts/leaderboard.service.ts` — `standing`, `cohortSize`, `standingsOf`, `standingsOfStudent`.
- `apps/api/src/attempts/leaderboard-view.service.ts` — the student board: own seat live, neighbours held.
- `apps/api/src/attempts/rollup.service.ts` — the five aggregates: sweep, recount, rebuild, delta fold.
- `apps/api/src/attempts/rollup-fold.ts` — what one sitting adds, with no database.
- `apps/api/src/attempts/rollup.processor.ts` + `rollup-queue.ts` — the `rollup` BullMQ worker and its four jobs.
- `apps/api/src/dashboard/dashboard.service.ts` — the admin dashboard counts, gated by `dashboard-bands.ts`.
- `apps/api/src/attempts/overview.service.ts` — `/me/overview`: stored tallies plus five live standings.

What it is for

A standing must be true at the instant it is read, while the live hall must not

pay for it. Rank, percentile, cohort size, the score curve and the topper are counted from `Attempt` on every read and stored nowhere: `prisma/schema.prisma:1166` says so, and no `rank / percentile / topper` column exists (grep of the schema returns only comments). What is stored is the additive part — career tallies, per-test and per-section totals, item analysis — as a delta inside the scorer's transaction or by a periodic recount.

FLOW — RANK AND PERCENTILE FOR ONE SITTING

- 1 GET `/me/attempts/:id/scorecard`
`apps/api/src/attempts/attempt-report.controller.ts:25` `@Get('attempts/:id/scorecard')` on `@Controller('me') + @Actors(ACTOR_TYPES.STUDENT) (:16--:18)`
`|` `admitted`
- 2 Guards `JwtAuthGuard` → `ActorGuard` → `FeaturePermissionGuard` → `SuperAdminGuard` → `ThrottlerGuard`
`apps/api/src/app.module.ts:77--82`
`|` `user.id`
- 3 — `apps/api/src/attempts/performance.service.ts:186`
`scoreCard()`
`|` `entered`
- 4 Ownership + `status: EVALUATED` in the WHERE
`:188--192`
`|` `row, or refuseCard`
- 5 Parallel reads
`:195--198`
`|` `testStats` and the standing
- 6 — `apps/api/src/attempts/leaderboard.service.ts:25`
`standing()` `$queryRaw(standingsSql({ attemptId })))`
- 7 chosen CTE picks the sitting under `IN_COHORT`
`ranking-sql.ts:54--59`
`|` 0 or 1 row
- 8 `CROSS JOIN LATERAL` counts the test's cohort
`:66--80`
`|` `cohort`, `outscored`, `tied`, `rank`
- 9 `sitting_percentile(outscored, tied, cohort)::float8`
`:63`
`|` percentile
- 10 `row?.test_id === testId else null`
`apps/api/src/attempts/leaderboard.service.ts:26`
`|` `{ rank, percentile, cohortSize }`, null for a retake, void or unscored sitting
- 11 `figuresOf`
`apps/api/src/attempts/performance.service.ts:200`
`|` `ScoreCard`
- 12 `ResponseInterceptor`
`apps/api/src/app.module.ts:75`
`|` `{ success: true, data, meta }`

Verbatim (ranking-sql.ts:11, :14, :66 - :80):

```
-- IN_COHORT      "isGraded" AND "status" = 'EVALUATED' AND "score" IS NOT NULL
-- RANK_ORDER      "score" DESC, "timeTakenSec" ASC NULLS LAST, "id" ASC
SELECT COUNT(*) AS cohort,
       COUNT(*) FILTER (WHERE b."score" < a."score") AS outscored,
       COUNT(*) FILTER (WHERE b."score" = a."score") AS tied,
       COUNT(*) FILTER (
         WHERE b."score" > a."score"
             OR (b."score" = a."score"
                 AND COALESCE(b."timeTakenSec", 2147483647) < COALESCE(a."timeTakenSec", 2147483647))
             OR (b."score" = a."score"
                 AND COALESCE(b."timeTakenSec", 2147483647) = COALESCE(a."timeTakenSec", 2147483647)
                 AND b."id" < a."id")
         ) + 1 AS rank
FROM "Attempt" b
WHERE b."testId" = a."testId" AND <IN_COHORT, unqualified so it binds to b>
```

sitting_percentile is still there — prisma/migrations/20260911190000_rank_is_counted_in_postgres/migration.sql:20, replaced body-unchanged but PARALLEL SAFE at prisma/migrations/20260912010000_percentile_is_parallel_safe/migration.sql:5:

```
CREATE OR REPLACE FUNCTION sitting_percentile(outscored bigint, tied bigint, cohort bigint)
RETURNS numeric LANGUAGE sql IMMUTABLE PARALLEL SAFE AS $$
SELECT CASE WHEN cohort <= 1 THEN 100::numeric ELSE
  ROUND((LEAST((LEAST(GREATEST(outscored, 0), cohort)
    + LEAST(GREATEST(tied, 1), GREATEST(cohort - LEAST(GREATEST(outscored, 0), cohort), 1)) / 2.0
  ) / cohort, 1) * 100, 2) END $$;
```

Indexes — Attempt_ranking_idx exists and is the only index serving ranking: created 20260911190000/migration.sql:31, dropped and recreated with INCLUDE at prisma/migrations/20260929090000_the_bank_boards_and_ledger_read_through_indexes/migration.sql:31 - :34 as ("testId", "score" DESC, "timeTakenSec", "id") INCLUDE ("studentId", "submittedAt") WHERE "isGraded" AND "status" = 'EVALUATED' AND "score" IS NOT NULL. Attempt_testId_score_idx was dropped — created prisma/migrations/20260820000000_schema_realignment/migration.sql:793, dropped as redundant at prisma/migrations/20260918100000_the_live_sittings_are_indexed_apart_from_the_finished_ones/migration.sql:52. Also hand-written: Attempt_graded_per_test_key (unique (testId, studentId) WHERE isGraded, 20260820000000:1155) and Attempt_student_drift_idx (("studentId", "updatedAt") WHERE status IN ('EVALUATED', 'VOIDED') AND "updatedAt" > "evaluatedAt", prisma/migrations/20260928203000_the_drift_index_holds_only_sittings_that_moved/migration.sql:22).

FLOW — THE LEADERBOARD

- 1 GET /me/leaderboard?testId=...
apps/api/src/attempts/leaderboard.controller.ts:23 @Get() on @Controller('me/leaderboard') + @Actors(STUDENT), @Query(new ZodQuery(leaderboardQuerySchema)), @CurrentUser()
mounted on the EXAM role only (attempts.module.ts:46)
- 2 — leaderboard-view.service.ts:39
board()
| entered
- 3 Gate read: the reader's own graded EVALUATED sitting + test.title + test.stat.computedAt
:42-:49
| no row → AppException(NOT_FOUND)
- 4 Own standing counted live, the score card's own call
:54
| one LATERAL cohort count
- 5 standing === null → emptyBoard (:108)
:56
| cohortSize: 0, nobody seated
- 6 heldSeats()
:59, :75
| null when computedAt is undefined, i.e. no TestStat yet
- 7 hold(rankings, \ \${testId}:\${countedAt.getTime()}\ , ..., HELD_BOARDS)
:83-:88, helper paper-sheet.service.ts:96
| in-process LRU of promises, ceiling 8 (:27)
- 8 On a miss, rankedCohortSql() reads the whole ranking
apps/api/src/attempts/ranking-sql.ts:126
| SEATED windows (ROW_NUMBER, COUNT(*) OVER (), sitting_percentile(RANK() OVER (ORDER BY score ASC) - 1, COUNT(*) OVER (PARTITION BY score), COUNT(*) OVER ()), :104 - :114) + NAMED 's Student join and Branch left join (:117)
- 9 Agreement test: held seat's rank and cohort equal the live standing's
:90
| disagree → null
- 10 Agree → seatsNear()
:96-:102
| slice(0,3) plus slice(rank-4, rank+3), podium ranks filtered out so no seat twice
- 11 Disagree or no watermark → live testBoardSql
:60, ranking-sql.ts:135
| same halves, WHERE r.rank <= 3 OR r.rank BETWEEN mine-3 AND mine+3
- 12 Rows shaped in Node
:62-:69
| isYou from the gate's attemptId; percentile only on the reader's row

- 13 `splitBoard / boardName`
leaderboard-board.ts:16, :10
podium (rank ≤ 3), neighbourhood (> 3); a blank name shows as 'Student'
- 14 Response
packages/contracts/src/leaderboard.ts:34
{ testId, label, cohortSize, podium, neighbourhood, you, generatedAt }, ≤ 10 rows

Cached: nothing in Redis. Only the per-process `Map<string, Promise<CohortSeatRow[]>>` at `leaderboard-view.service.ts:31`, keyed `testId:TestStat.computedAt` epoch ms, 8-entry LRU, **no TTL** — invalidated by the watermark moving, and a rejected read is deleted so the next reader retries (`paper-sheet.service.ts:108`).
`LEADERBOARD_PODIUM = 3`, `LEADERBOARD_NEIGHBOURS = 3` (`packages/contracts/src/leaderboard.ts:17`, :20).

FLOW — A ROLLUP REBUILD

- 1 Scheduler `attempt-sweep`, every: 2 min
apps/api/src/queue/queues.ts:158, registered attempts.module.ts:103, worker role only (:98)
- 2 — `attempt-sweeper.processor.ts:43`
rollup.sweep()
enqueues `sweep-cohorts` under the one id `rollup-sweep-cohorts`, 5 s delay (`rollup-queue.ts:28`, `queues.ts:101`, :104): a burst is one job
- 3 `rollup` worker, concurrency 2
rollup.processor.ts:16, queues.ts:23
dispatch on `job.name`; legacy `fold-attempt / fold-pending` remap to the sweep (:29)
- 4 `sweepCohorts()`
apps/api/src/attempts/rollup.service.ts:116
three selections, each arm in `tried()` (:135)
- 5 `changedTests()`
:146-:157
tests whose `Attempt.updatedAt` beats `TestStat.computedAt - 2 min` under `MOVES_THE_COHORT` (VOIDED, or graded+EVALUATED), `LIMIT 50`
- 6 `recountTest()` in a `MEDIUM` transaction (10 s wait / 15 s, `prisma.service.ts:10`)
:193
- 7 `writeTestTotals`
:206-:221
one `INSERT ... ON CONFLICT` → `TestStat.evaluatedCount`, `sumTimeSec` (from `sectionScores[*][5]`), `computedAt`
- 8 `writeSectionTotals`
:230-:240
deleteMany, then one `INSERT ... SELECT` over `jsonb_array_elements` → `TestSectionStat.attempted`, `sumScore`, `sumTimeSec`, `computedAt`
- 9 `staleItems()`
:161-:174
evaluatedCount > 0 and no `TestQuestionStat` newer than `TestStat.computedAt` or now - 15 min (`COHORT_COUNT_EVERY_MIN`, `packages/contracts/src/stats.ts:783`, `LIMIT 50`)
- 10 `recountTestItems` in `BULK` (120 s)
:244
firstSittings: one sitting per student, oldest first (:419)
- 11 `replay`
:444-:452
batched, `REBUILD_PAGE = 200` ids a round trip (:86); `foldablesOf` reads the page's attempts plus each test's paper once (:70, :51)
- 12 `addToQuestion` per served question
apps/api/src/attempts/rollup-fold.ts:142
in-memory `Map<paperQuestionId, QuestionTotals>`
- 13 `writeItemTotals`
:256-:269
deleteMany + createMany → `TestQuestionStat.correctCount`, `wrongCount`, `skippedCount`, `sumTimeSec`, `optionCounts`, `computedAt`
- 14 `driftedStudents()`
:178-:189
StudentStat rows with a sitting whose `updatedAt > evaluatedAt` beats `computedThrough - 2 min`, `LIMIT 50`, through `Attempt_student_drift_idx`
- 15 `rebuildStudent` in `BULK`
:278-:292
upserts `StudentStat` first (lock order), replays every EVALUATED sitting through the same paged `replay`
- 16 `writeStudent`
:382-:416
all `StudentStat` tallies outright plus `computedThrough / computedAt`, then `deleteMany` + `createMany` on `StudentSubjectStat`

Other triggers: a **re-score** queues `rebuild-test` + `rebuild-student`, debounced 1 min (`scoring.processor.ts:96`, `queues.ts:129`); a **void** the same (`attempt-resolution.service.ts:146`); `POST /admin/tests/:id/analytics/resync` queues `rebuild-test` at zero delay under a separate `-now` id (`test-analytics.controller.ts:43`, `test-analytics.service.ts:98`, `rollup-queue.ts:42`). A **first** evaluation queues nothing — the student's two tables fold as a delta inside the scorer's transaction (`scoring.processor.ts:118`, `rollup.service.ts:314`, incrementing `INSERT ... ON CONFLICT DO UPDATE` at :332, :365). `rebuild-all` has one producer, the manual script `apps/api/src/enqueue-rollup-rebuild-all.ts:29`.

No "regenerate" step — a case-insensitive grep for `regenerate` across `apps/api/src`, `apps/admin/src`, `apps/exams/src`, `packages/contracts/src` and `prisma/schema.prisma` returns nothing. The curve (`cohort-curve.ts:11`) and the topper (`topper.ts:39`, via `rankOneSql`, `ranking-sql.ts:17`) are read-time counts too; `TestStat.topperAttemptId` was dropped at `prisma/migrations/20260930120000_the_topper_is_the_boards_rank_one/migration.sql:10` - :14.

The data it touches

MODEL.COLUMN	TYPE	HOLDS	WHO WRITES IT
Attempt.score , correctCount , wrongCount , unattemptedCount , sectionScores , timeTakenSec , scoredRevision	Decimal(8,2)?, Int? ×3, Json?, Int?, Int	the marks ranking reads; timeTakenSec is the tiebreak (schema.prisma:704 — :712)	ScoringProcessor.mark , one statement (scoring.processor.ts:138 — :164)
Attempt.isGraded	Boolean	true only for the one ranked sitting (:694)	start; cleared by a regranting void
StudentStat.studentId	Uuid @id	the student (:1134)	—
StudentStat.testsEvaluated , sumScore , totalCorrect , totalWrong , totalUnattempted , sumTimeSec , retakeCount , lastAttemptAt	Int, Decimal(16,2), Int ×3, BigInt, Int, Timestamptz?	career tallies (:1135 — :1142)	addToStudent delta; writeStudent outright
StudentStat.computedThrough , computedAt	Timestamptz?, Timestamptz	drift watermark / last touch (:1144)	writeStudent both; the delta only computedAt
StudentSubjectStat (studentId , subjectId , scope) attempted , correct , sumTimeSec , computedAt	Int ×2, BigInt, Timestamptz	per-subject, per-scope accuracy (:1152 — :1163)	addToSubjects delta; writeStudent delete+create
TestStat.testId @id, evaluatedCount , sumTimeSec , computedAt	Int, BigInt, Timestamptz (:1168 — :1172)	ranked cohort size and total clock; computedAt is the held-ranking cache key	writeTestTotals only
TestSectionStat (testId , baseConfigSectionId) attempted , sumScore , sumTimeSec , computedAt	Int, Decimal(18,2), BigInt, Timestamptz (:1178 — :1189)	per-section cohort totals	writeSectionTotals
TestQuestionStat (testId , paperQuestionId) questionId , correctCount , wrongCount , skippedCount , sumTimeSec , optionCounts , computedAt	Uuid, Int ×3, BigInt, Json?, Timestamptz (:1194 — :1211)	item analysis; optionCounts is { [optionId]: count } (rollup-fold.ts:160 , read via optionCountsIn :109)	writeItemTotals

sectionScores is a packed positional array per section: [sectionId, score, correct, wrong, unattempted, timeSpentSec] (rollup.service.ts:224) — index 5 feeds both sumTimeSec columns, index 1 feeds TestSectionStat.sumScore . **Redis keys: none** — no file here imports Redis.

What must already be true

both sitting_percentile migrations applied, or every standing and board query

errors. A sitting must be isGraded AND status = 'EVALUATED' AND score IS NOT NULL to be in a cohort; a board needs the reader's own such sitting on that paper; held neighbours need a TestStat row, so one rollup pass. Both schedulers register only on API_ROLES.WORKER (attempts.module.ts:98); MeLeaderboardController runs only on EXAM, MeOverviewController only on CORE (:46 — :59). REDIS_URL for the rebuild-all script.

Why it is shaped this way

- Counted, not stored: a stored rank is wrong the moment the next student submits. It replaced a Redis sorted set (20260911190000/migration.sql:1), which was a second source of truth needing its own rebuild.
- The cohort filter stays literal SQL in IN_COHORT / RANK_ORDER , so the planner can prove the partial index's predicate; bound parameters — the alternative — make it skip Attempt_ranking_idx (reasoning at attempt-sweeper.processor.ts:170 — :177).
- Student tallies delta, cohort tallies recount. Folding the cohort per submit would serialise 5K writers on one TestStat row; recounting students would replay their whole history every pass.
- Reversal is a bounded rebuild, never an undone delta (rollup.service.ts:1 — :5): an undo run twice corrupts the total, a rebuild run twice writes the same answer.
- A ranking is held per TestStat.computedAt , not per reader, costing neighbour names up to one sweep of staleness — but the reader's own rank stays live, so board and score card cannot disagree.

Which screen calls it

- GET /me/leaderboard — apps/exams/src/features/leaderboard/leaderboard.tsx:52 — :54 , route /leaderboard (apps/exams/src/lib/constants.ts:33) , rows in board.tsx:20 / :73 ; mobile at apps/mobile/src/components/performance/leaderboard-panel.tsx:28 . Key [ME, 'leaderboard', testId] (packages/app-kit/src/student-queries.ts:62) , client packages/contracts/src/client/me.ts:206 .
- GET /me/overview — apps/exams/src/features/dashboard/dashboard.tsx:64 , SPA home; "(last 5)" at :209 , :216 , also apps/exams/src/features/performance/overview.tsx:242 and apps/mobile/src/components/performance/overview-panel.tsx:119 .
- GET /me/attempts/:id/scorecard — apps/exams performance/report screens.
- GET /admin/dashboard — apps/admin/src/features/dashboard/dashboard.tsx:55 , route / (apps/admin/src/App.tsx:168) . POST .../analytics/resync — apps/admin/src/features/tests/test-analytics.tsx .

Is it optimal

- Score-card standing: 1 query, one LATERAL pass, index-only through Attempt_ranking_idx (the INCLUDE keeps it off the heap). O(cohort); no N+1.
- /me/leaderboard : on a hit 2 queries plus a ranking read amortised over the rollup window; on a miss or disagreement, 2 + a full window pass. Payload ≤ 10 rows. b610946 's figure for the pre-hold version: 6K readers × 6K rows = 36M rows of work for 78K rows of answer.
- /me/overview : 3 queries, exactly 5 laterals. /admin/dashboard : ≤ 13 queries, all count / groupBy /indexed findMany with take — optimal for what it shows.
- Rollup sweep: 3 selections, then per test either 2 statements or a 200-id-paged replay of every sheet. All numbers quoted are read from migration headers (20260918100000 , 20260929090000) and commit bodies, not measured here.

What it leans on

`ResponseInterceptor` + `AppException / ErrorCodes` (the board

throws, never builds an envelope); the four global guards + `@Actors` (the board's privacy argument is that it needs a STUDENT token); `ZodQuery` and `packages/contracts/src/leaderboard.ts` (one schema both sides; podium and neighbour widths shared by SQL and splitter); `RECENT_STANDINGS` and `COHORT_COUNT_EVERY_MIN` in contracts because screens say both numbers out loud; `hold / recall / remember` from `paper-sheet.service.ts`; `keyedJob` ids (a burst of re-scores collapses to one rebuild); `TX_LIMITS`; `readInBatches`; app-kit query keys; `@iace/ui` frames.

What changed recently

- `312278f` — the board's `you`, `rank`, `percentile`, `cohortSize` now come from `LeaderboardService.standing`, the score card's own call; the held ranking supplies only the seats around the reader, and only when its rank **and** cohort size match. Costs one index-only cohort count a read.
- `b610946` — `testBoardSql` stopped running per reader: a ranking is read once per `TestStat.computedAt` and sliced in memory, `is_you` left the SQL, `SEATED / NAMED` became shared so held and live cannot drift. `HELD_BOARDS` is 8 against `HELD_PAPERS` 64.
- `35cae83` — `standingsOfStudent` gained an in-query bound: one cohort count per sitting ever sat became five.
- `0f94ae9` — `pointsBoardSql` and the board-scope idea (ALL_TIME/SERIES) deleted, with `GET /me/performance/series`, the places-moved arrow, the sittings count and the measure choice. ALL_TIME had no `testId` filter and ran on exam-role containers.
- `ad4139e / 44a0925 / b96a89d / 7c9edee` — the stored topper, a test's mean/highest/lowest, and every stored sum-of-sums or average left the rollup tables; `TestStat` is down to three columns.

What is wrong with it

- `rollup.service.ts:236 - :240` — `writeSectionTotals` filters `status = EVALUATED AND isGraded` but omits `score IS NOT NULL`, where `writeTestTotals` (:213) uses the full `IN_COHORT`: two halves of one transaction describing different populations. Unreachable today — `mark` always writes `score` in the statement that sets `EVALUATED` (`scoring.processor.ts:145`), and such a row carries no section entries. A divergence to close, not a live bug.
- `rollup.service.ts:101 - :104` — `staleItems` is bounded at 50 tests, each a `BULK` (120 s) replay of every sheet of its cohort, on a queue of concurrency 2 (`queues.ts:23`). A pass can outlast the 2-minute interval, and while it runs the next sweep's `add` is silently dropped by the shared job id (`queues.ts:60`). Self-correcting, but item figures can fall behind the 15-minute clock the copy promises.
- `ranking-sql.ts:73 - :76` — the tiebreak coalesces a missing `timeTakenSec` to the bare literal `2147483647`, three times, where `RANK_ORDER` (:14) states the same rule as `NULLS LAST`. Change one and the other does not follow; two disagreeing orders is the "two numbers for one sitting" class `312278f` fixed.
- `ranking-sql.ts:108 - :112` with `leaderboard-view.service.ts:67` — `sitting_percentile` is computed for every seat in both board queries, then discarded for all but the reader's row, since by contract only `isYou` carries one (`packages/contracts/src/leaderboard.ts:28`). A window computed for nothing on 5,999 of 6,000 rows.

Part II — Making a paper

A paper does not appear; it is assembled, reviewed and then frozen. This part follows that assembly — the blueprint a test is cut from, the bank the questions come from, the two-person workflow that writes them, and the spreadsheet importer that fills the bank in bulk.

The hinge is the **freeze**. Up to it, almost everything is editable; past it the database itself stops you, through triggers rather than service code, because a sat paper whose rows moved would silently mis-decode every answer sheet stored against it. That is the single most important constraint in this part, and it is enforced in SQL precisely so that no new code path can forget it.

Building a test, and freezing it

Where it lives

`apps/api/src/tests/` is the whole module (10 files, 2521 lines), plus

`apps/api/src/common/paper-edit.ts` and the trigger SQL under `prisma/migrations/`.

- `tests/tests.controller.ts` — `TestsController` (`admin/tests`), `SeriesTestsController`
- `tests/tests.service.ts` — owns `Test`: list, detail, create, update, delete.
- `tests/paper.service.ts` — the paper: read, add, fill, replace, remove, typist Done, hand-over, and
- `tests/finalize.service.ts` — the freeze and every gate in front of it.
- `tests/offering.service.ts` — the Offer step: opening, program openings, series move.
- `tests/draw-engine.ts` — the one pure draw (`drawSection` , seeded, pins-first).
- `tests/test-rules.ts` — pure rules: scope, title, completeness, what moves a paper.
- `common/paper-edit.ts` — `beginPaperEdit` / `beginDraftPaperEdit`: the `SELECT ... FOR UPDATE` on

What it is for

A `BaseConfig` describes a paper's shape; a `Test` is one instance of it with

its own scope, skin and series. This module fills `PaperQuestion` rows from the bank or from what a typist wrote, proves the paper whole and read, then freezes it by stamping `Test.finalizedAt` and flipping `status` to `ACTIVE`. From the freeze, immutability is enforced in Postgres by triggers, not by services. **There is no Paper model and no per-test section row**: a test has exactly one paper — its `PaperQuestion` rows (`prisma/schema.prisma:576`) — and its sections are the `BaseConfigSection` rows its scope covers (`packages/contracts/src/tests.ts:130` `scopedSections`).

Flow trace: create a test from a config

1. `POST /admin/tests`, decorated `@Audit(AUDIT_FEATURE.TEST, AUDIT_ACTION.CREATE)`,
2. Config read, proved active — `apps/api/src/configs/base-configs.service.ts:326` —
3. Series read, proved to be for the config's stage — `apps/api/src/tests/tests.service.ts:203` —
4. Title free inside that series — `apps/api/src/tests/tests.service.ts:330` — `assertTitleFree()` —
5. Scope names a real part of this config — `apps/api/src/tests/tests.service.ts:316` —
6. One insert, no transaction — `apps/api/src/tests/tests.service.ts:184` — `prisma.test.create()` —
7. Read back whole — `apps/api/src/tests/tests.service.ts:163` — `detail()` — `TestDetail`: the row,

No `PaperQuestion` row exists yet and `paperSource` is null, which blocks every paper action (`assertSourceChosen` , `common/paper-edit.ts:34`).

Flow trace: populate a paper — PICKED (drawn from the bank)

`PaperSource` is `FRAMED` | `PICKED` (`prisma/schema.prisma:134-137`), chosen once by `PATCH /admin/tests/:id { paperSource }` from a dialog (`apps/admin/src/features/tests/test-builder-assign-step.tsx:357-364`).

1. `POST /admin/tests/:id/paper/sections/:sectionId/fill`, `@Audit(TEST, UPDATE)`,
2. Test read with `_count.attempts` — `paper.service.ts:793` — `requireTest()`.
3. Advisory claim in Redis — `tests/edit-lock.ts:10` → `common/edit-lock.ts:29` —
4. Refusals: offered or sat (`assertAssemblable` , `paper.service.ts:719`); source unchosen; section
5. Every row of the paper read once — `paper.service.ts:231`.
6. Candidate pool — `paper.service.ts:666` — `poolFor()` — `PICKED` narrows in SQL by
7. Existing section rows become pins **at the version the row pinned**, not the question's current
8. Pins judged against the split — `paper.service.ts:839` — `assertWithinSplit()` — `CONFLICT`.
9. The draw — `draw-engine.ts:64` — `drawSection()` — sorts the pool by id, shuffles with
10. Short pool → `DRAW_SHORTFALL` (HTTP 422), field error per section — `paper.service.ts:288-297`.
11. Transaction `TX_LIMITS.SHORT` (maxWait 5 s / timeout 10 s) — `paper.service.ts:240` —
12. `paperQuestion.createMany`, numbered from the paper's highest `order` + 1 —
13. A now-full section holding an unchecked question goes back to its released reader —
14. Paper re-read and returned — `paper.service.ts:728` — `paperOf()` — `TestPaper` with

`POST :id/paper/questions` (`addQuestions` , `paper.service.ts:127`) is the hand-pick variant: same gates plus `assertNoRepeats`, an already-on-paper set check, a section-full check and `quotaWithPicks` against the split; it never draws. `PATCH :id/paper/:rowId` (`replaceQuestion` , `:319`) swaps one row's `questionId` / `questionVersionId` in place.

Flow trace: populate a paper — FRAMED (typed for this paper)

1. POST /admin/sections/:testId/:sectionId/done , @Audit(TEST, UPDATE) ,
2. Assignment proved a live, unfinished typist row — paper.service.ts:350-363 — NOT_FOUND or
3. Test not offered, not sat (assertAssemblable), and the Done open for this source (doneOpen ,
4. Everything any typist wrote for the section — paper.service.ts:481 — typedFor() .
5. The selection validated whole before any write — paper.service.ts:440 —
6. Transaction TX_LIMITS.SHORT — paper.service.ts:383 — beginDraftPaperEdit , then
7. The section's rows become exactly the choice — paper.service.ts:406 — placeTyped() — deletes
8. The reader's handedAt stamped in the same transaction — paper.service.ts:392 .
9. Leftovers settled in that same transaction — questions/questions.service.ts:475 —

A PICKED section reaches its reader by POST :id/paper/sections/:sectionId/hand-over (paper.service.ts:489), gated by handOverGap (:818): not offered, source PICKED, section full, a reader exists, not already handed.

Flow trace: finalize (the freeze)

One road only: the Offer step's save.

1. PUT /admin/tests/:id/offering , @Audit(TEST, UPDATE) , `@RequiresFeature(TEST_MANAGEMENT,
2. One transaction, TX_LIMITS.MEDIUM (10 s/15 s) — offering.service.ts:245 — beginPaperEdit
3. Optimistic lock — offering.service.ts:248 — version !== body.expectedVersion → CONFLICT
4. writeOpening — offering.service.ts:274 — unchanged to the minute is a no-op (sameMinute:91);
5. writeProgramOpenings — offering.service.ts:294 — duplicate codes → VALIDATION_ERROR
6. offered === true && status !== ACTIVE — finalize.service.ts:41 — offerWithin() .
7. Row re-read inside the lock — finalize.service.ts:46 — NOT_FOUND if gone.
8. Gate — every seat's work is finished — finalize.service.ts:100 assertAssignmentsRead() —
9. Gate — every question on a read section carries its reader's tick — finalize.service.ts:130
10. Already finalized → status is lifted back to ACTIVE and it returns; the completeness gate is
11. Gate — the paper is whole — finalize.service.ts:75 assertPaperIsWhole() →
12. The freeze — finalize.service.ts:61 — one UPDATE : finalizedAt = now , status = ACTIVE ,
13. offered === false && status === ACTIVE → status = INACTIVE only — offering.service.ts:258 .
14. version + 1 again, closing the transaction — offering.service.ts:261 .
15. Catalog invalidated, audit row addressed — offering.service.ts:442 — announce() —

What finalizedAt gates afterwards. A sitting cannot start until it is set (attempts/attempt-rules.ts:19 , PAPER_NOT_READY_MESSAGE). Every paper edit is refused (common/paper-edit.ts:27-30 ; paper.service.ts:719 ; tests.service.ts:228 for any field that movesThePaper , test-rules.ts:126). Hand-over and release close (paper.service.ts:824 ; assignments/assignment-gates.ts:22,30). A reworded question only un-ticks drafts (assignments/unread-questions.ts:46,51). It enables the one post-freeze write: PATCH :id/paper/:rowId/status refuses while finalizedAt === null (paper.service.ts:551 , NOT_FROZEN_MESSAGE) and on success bumps Test.paperRevision in the same transaction (:566). Immutable from the freeze: every column of PaperQuestion but status , and the QuestionVersion rows the paper pins. Still movable: title , examTemplate (PAPER_NEUTRAL_FIELDS , test-rules.ts:99); once anybody has sat it, title alone (TEST_UNFROZEN_FIELDS , :96).

The triggers. All from prisma/migrations/20260917100000_a_sitting_keeps_its_answers_on_one_sheet/migration.sql , re-created verbatim in 20260918120000_every_id_is_a_uuid/migration.sql:415-417 . They enforce the stronger rule — a paper somebody has sat — not the freeze.

paper_question_sat_guard , BEFORE INSERT OR UPDATE OR DELETE ON "PaperQuestion" FOR EACH ROW (:141-143), body :123-139 :

```
IF TG_OP = 'UPDATE' AND (to_jsonb(NEW) - 'status') IS NOT DISTINCT FROM (to_jsonb(OLD) - 'status') THEN
  RETURN NEW;
END IF;
PERFORM 1 FROM "Test" WHERE "id" IN (NEW."testId", OLD."testId") FOR UPDATE;
IF NOT EXISTS (SELECT 1 FROM "Attempt" WHERE "testId" IN (NEW."testId", OLD."testId")) THEN
  IF TG_OP = 'DELETE' THEN RETURN OLD; END IF;
  RETURN NEW;
END IF;
RAISE EXCEPTION 'A paper somebody has sat cannot change: only a question's status may move'
USING ERRCODE = 'integrity_constraint_violation';
```

The FOR UPDATE is the point: an Attempt insert takes only KEY SHARE on Test through its FK, which conflicts with nothing an ordinary UPDATE holds, so without it a paper edit and the first sitting each pass their own EXISTS and both commit (20260918150000/migration.sql:1-7).

question_version_sat_guard , BEFORE UPDATE ON "QuestionVersion" FOR EACH ROW WHEN (options, answerKey or content IS DISTINCT) (:160-168). Current body — 20260918160000_the_version_guard_locks_its_tests_in_one_order/migration.sql:20-36 :

```
PERFORM 1 FROM "Test"
WHERE "id" IN (SELECT "testId" FROM "PaperQuestion" WHERE "questionVersionId" = OLD."id")
ORDER BY "id"
FOR UPDATE;
IF EXISTS (SELECT 1 FROM "PaperQuestion" paper
  JOIN "Attempt" sitting ON sitting."testId" = paper."testId"
  WHERE paper."questionVersionId" = OLD."id") THEN
  RAISE EXCEPTION 'A question version on a paper somebody has sat cannot change its options, key or content'
  USING ERRCODE = 'integrity_constraint_violation';
END IF;
```

ORDER BY "id" was added because an unordered FOR UPDATE over two or more tests let two backends take them in different orders and deadlock with 40P01.

paper_question_option_ids , BEFORE INSERT OR UPDATE OF "questionVersionId", "optionIds" (:58-60), body :44-56 :

```
NEW."optionIds" := ARRAY(
  SELECT listed.option->>'id'
  FROM "QuestionVersion" version,
    jsonb_array_elements(
      CASE WHEN jsonb_typeof(version."options") = 'array' THEN version."options" ELSE '{}'::jsonb END
    ) WITH ORDINALITY AS listed(option, position)
  WHERE version."id" = NEW."questionVersionId"
  ORDER BY listed.position
);
```

An `AttemptSheet` stores a **position** into that array, so it is load-bearing for scoring. `question_version_option_ids_fanout`, AFTER UPDATE OF "options" ON "QuestionVersion" (`20260918140000/migration.sql:25-37`), re-assigns `"optionIds" = "optionIds"` on every pinning row so the BEFORE trigger rebuilds it. It leans on alphabetical BEFORE-trigger order (`o` before `s`) — renaming either `PaperQuestion` trigger silently drops `sat_guard`'s own check (`20260918140000/migration.sql:16-23`).

Also on `Test`: `CREATE UNIQUE INDEX "Test_testSeriesId_title_key" ON "Test" ("testSeriesId", lower("title"))` (`20260912170000/migration.sql:67`) — the real guarantee behind `assertTitleFree`. No CHECK constraint survives on `Test`: `Test_ranked_requires_fixed_check` was dropped in `20260911150000/migration.sql:19`, `Test_practice_has_no_window_check` in `20260909180000/migration.sql:36`.

Flow trace: offer / schedule to students

1. The same `PUT :id/offering` save sets `Test.opensAt` and the `TestProgramUnlock` rows (steps 4-5)
2. Visibility: the shared catalog selects `tests: { where: { status: TEST_STATUS.ACTIVE } }` —
3. A student's effective opening is the **earliest** of their own programs' unlocks, falling back to
4. Openness is derived on every read, never stored — `packages/contracts/src/access.ts:374` —
5. Startability — `attempts/attempt-rules.ts:13` — `testStartBlocker()` — `status ACTIVE` and
6. Students are told once — `notifications/test-opening.service.ts:38` — `sweep()` — ACTIVE tests

The "in-use counter on the offer" does not exist. `docs/03-conventions.md:169` claims `tests` moves a `Question` in-use counter on the offer, once per question served, watermarked by `finalizedAt`. `prisma/schema.prisma:456-497` has no such column and the offer writes nothing to `Question` (`finalize.service.ts:41-65`). In-use is derived live: `inUse: isReferenced(row) = paperQuestions.length + questionStats.length > 0` (`questions/questions.service.ts:785,800`). The counters the offer does move are `Test.version` (+1 at the freeze, +1 closing the transaction); the separate `Test.paperRevision` moves only on a disposition (`paper.service.ts:566`).

Data touched

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
<code>Test.title</code>	text?	unique per series, case-insensitively	create / update
<code>Test.baseConfigId</code> , <code>.examStageId</code>	uuid	stage denormalised so the composite FK holds	create only
<code>Test.scope</code>	FULL/MODULE/SECTIONAL	which slice of the config	create / update
<code>Test.scopeRef</code>	jsonb?	<code>{ moduleId?: string, sectionId?: string }</code> (<code>contracts/src/tests.ts:75</code>)	create / update
<code>Test.questionPoolFilter</code>	jsonb?	<code>{ sections: Record<sectionId, { topicIds?: string[], tags?: string[], mix?: { LOW: int, MEDIUM: int, HIGH: int } }> }</code> (<code>contracts/src/tests.ts:88-119</code>)	update (<code>test-paper.tsx:161</code>)
<code>Test.paperSource</code>	<code>PaperSource?</code>	FRAMED or PICKED, null until declared	update, once
<code>Test.examTemplate</code>	enum	the skin, copied off the config at creation	create / update
<code>Test.status</code>	DRAFT/ACTIVE/INACTIVE	ACTIVE is the only state the catalog carries	<code>finalize.service.ts:53,63</code> ; <code>offering.service.ts:259</code>
<code>Test.version</code>	int	the Offer step's optimistic lock	paper-moving updates, the freeze, each offering save, a series move
<code>Test.paperRevision</code>	int	monotonic cache key and re-score watermark	<code>paper.service.ts:566</code> only
<code>Test.finalizedAt</code>	timestamptz?	the freeze; set once, never cleared	<code>finalize.service.ts:62</code> only
<code>Test.opensAt</code>	timestamptz?	the whole of its timing — it opens and never shuts	<code>offering.service.ts:290</code>
<code>Test.announcedAt</code>	timestamptz?	the sweep's watermark, never read as a date	<code>notifications/test-opening.service.ts:84</code>
<code>Test.seriesOrder</code>	int?	position in the series; cleared by a move	<code>offering.service.ts:226</code>
<code>PaperQuestion.</code> <code>{testId,baseConfigId,baseConfigSectionId}</code>	uuid	the row's place; composite FKs	paper writes
<code>PaperQuestion.</code> <code>{questionId,questionVersionId}</code>	uuid	the pinned version	paper writes
<code>PaperQuestion.order</code>	int	unique per test; the sheet's slot order	paper writes
<code>PaperQuestion.</code> <code>{marks,negativeMarks}</code>	decimal(6,2)	copied from the section	paper writes
<code>PaperQuestion.status</code>	ACTIVE/DROPPED/BONUS	the only post-freeze change	<code>paper.service.ts:560</code>
<code>PaperQuestion.optionIds</code>	text[]	the version's option ids in stored order	trigger only

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
TestProgramUnlock. {testId,programCode,opensAt}	pk (testId, programCode)	which programs open it early	offering.service.ts:327–336
QuestionAssignment. {finalizedAt,handedAt,replacedAt}	timestamptz?	the whole state of a seat's work	paper.service.ts:385,392,511 ; tests.service.ts:254 ; unread- questions.ts:35

Redis: `test:edit:<testId>` — the advisory edit claim, value the admin id, TTL `EDIT_LOCK_TTL_SEC = 900` s (`redis/redis.keys.ts:68,79`). Taken **before** the Postgres transaction deliberately, with no release and no heartbeat.

Prerequisites

An active `BaseConfig` with sections (`assertUsable` refuses `isActive: false`); a `TestSeries` whose `examStageId` is null or matches the config's stage; an admin with `TEST_MANAGEMENT` WRITE (the typist Done needs `QUESTION_AUTHORING` WRITE); `Test.paperSource` set before any paper action; for FRAMED, live TYPIST `QuestionAssignment` rows with questions written under them; to finalize without a super admin, every PROOFREADER (and FRAMED TYPIST) row done and every paper question on a read section ticked; migration `20260917100000` applied (the triggers and `optionIds`).

Why this shape

- **No `Paper` table and no per-test section rows.** Sections are read back off
- **`finalizedAt` is the freeze, not the existence of rows.** The paper is built incrementally in
- **Marks and negatives are copied onto each row.** It costs duplication and a stale copy if the
- **Immutability is in Postgres, not the service.** A script or a bug bypasses an application guard;

Frontend

All `apps/admin` — the student SPA never calls these. Wrappers: `packages/contracts/src/client/admin.ts:532–596` ; route strings `ADMIN_TEST_ROUTES` / `ADMIN_TEST_PAPER_ROUTES` (`packages/contracts/src/tests.ts:484,616`).

ENDPOINT	UI FILE	SPA ROUTE
GET /admin/tests	features/tests/tests.tsx:130 ; lib/use-suggested-name.ts:37	/tests
GET /admin/tests/:id	features/tests/test-builder.tsx:90 , test-paper.tsx:83	/tests/:id , /tests/:id/paper
POST /admin/tests	test-builder.tsx:170	/tests/new
PATCH /admin/tests/:id	test-builder.tsx:169 (Setup) , test-builder-assign-step.tsx:364 (source) , test-paper.tsx:161 (questionPoolFilter)	/tests/:id , /tests/:id/paper
DELETE /admin/tests/:id	tests.tsx:181	/tests
GET :id/paper	test-paper.tsx:89 , test-builder-assign-step.tsx:113	/tests/:id/paper , /tests/:id
POST :id/paper/questions	test-paper.tsx:447	/tests/:id/paper
DELETE :id/paper/questions	paper-questions.tsx:147	/tests/:id/paper
POST .../fill	test-paper.tsx:459	/tests/:id/paper
POST .../hand-over	test-paper.tsx:603 , test-builder-assign-step.tsx:453	/tests/:id/paper , /tests/:id
PATCH :id/paper/:rowId/status	paper-disposition.tsx:106	/tests/:id/paper
PUT :id/offering	test-builder-offering.tsx:349	/tests/:id (Offer step)
POST :id/series	features/test-series/test-series-tests.tsx:202	/test-series/:id
GET /admin/test-series/:id/tests	features/test-series/test-series-tests.tsx	/test-series/:id
PATCH :id/paper/:rowId	no UI, no client wrapper, no route constant	—

Optimality

Query counts are read off the code; indexes are read off `prisma/schema.prisma` and the migrations. Nothing here was measured with EXPLAIN.

- `GET /admin/tests` — 2 queries in one `$transaction` (page + count) plus 2 `groupBy` scoped to the
- `GET /admin/tests/:id` — ~5 Postgres + 1 Redis `GET` ; `configs.detail` is a second config read on
- `POST :id/paper/questions` — ~13 queries, none per question: drawability is three batched reads in
- `POST .../fill` — ~11. `poolFor` is one unbounded `findMany` over the drawable bank narrowed to the
- `PUT :id/offering` — 15–20, with **two N+1s over program openings**: `requireProgram` per changed
- `offerWithin` runs `paperOf` (`finalize.service.ts:49`) **before** the already-finalized early
- `PaperQuestion` reads by `testId` lead on `PaperQuestion_testId_questionId_key` ; `anySitting`
- `TestPaper` payload: every row carries a ≤140-char `stemPreview` (`questions/question-core.ts:153`),

Shared machinery it leans on

The response envelope and `ErrorCodes` (interceptor wraps, filter maps; `DRAW_SHORTFALL` → 422, `packages/contracts/src/envelope.ts:28,58`); `@Actors / @RequiresFeature` from `common/security`; `ZodBody / ZodQuery`; `@Audit + AuditContext` (`setEntityId`, `setChanged`, `fieldDiff` over `AUDITED_TEST_FIELDS`, `tests.service.ts:87`); `formRefusal + FORM_LEVEL_FIELD`, so a refusal lands on a control rather than a toast; `common/pagination`, `common/relation-counts`, `common/search-terms`, `common/prisma-json`, `common/series-order`; `common/edit-lock + redisKeys`; `DomainEventBus` for `ACCESS_CATALOG_CHANGED`; `BaseConfigsService` (the shape is never copied); `QuestionsService` (`drawableFor`, `settleTyped`, `stemPreviewOf`); `assignments` (`doneOpen`, `uncheckedOn`, `reopenReadingIfUnchecked`); `TX_LIMITS`. **No queue and no BullMQ job** — the re-score is found by the sweep reading `paperRevision`. UI side: the typed `client/admin.ts` wrappers, `@iace/app-kit` `applyFieldErrors`, and `packages/ui` dialogs / `EmptyState` / `Alert` / `RadioGroup`.

Changed recently

```
git log --oneline -15 -- apps/api/src/tests/ :
```

- `f472a4d` — `_count` relation selects over `Attempt` and `PaperQuestion` replaced by one `groupBy`
- `cdfab62` — the whole `OutboxEvent` re-score machinery (write, relay, repair, nightly prune) was
- `f9745a4` — `handOverGap` extracted as the one rule and `canHandOver` now rides on every paper
- `091f0d6` — the typist Done moved to `POST /admin/sections/:testId/:sectionId/done`; the client no
- `bd07de9` — `settleTyped` moved inside the Done transaction; before it, a failure part-way left

Inconsistencies

1. `prisma/schema.prisma:535` says `paperSource` is "set once, and only a super admin ever moves it
2. `tests.service.ts:219-221` runs that assert on `input.paperSource !== undefined`, **before**
3. `docs/03-conventions.md:169` describes a `Question` in-use counter moved on the offer. No such
4. `finalize.service.ts:49` reads the whole paper before the `finalizedAt !== null` early return at
5. `finalize.service.ts:51-56` — a re-offer of an INACTIVE-but-finalized test lifts `status` back to
6. The FRAMED branch of `poolFor` (`paper.service.ts:675-677`) and `SECTION_UNDER_TYPED_MESSAGE`
7. `PATCH /admin/tests/:id/paper/:rowId` (`replacePaperQuestion`, `tests.controller.ts:119`) is fully
8. `removeQuestions` (`paper.service.ts:531-532`) resolves each row with its own `await requireRow`
9. **A finalized test cannot be thawed.** `finalizedAt` is written at `finalize.service.ts:62` and
10. `tests.service.ts:245-258` — a *scope* narrowing deletes out-of-scope `PaperQuestion` rows and

The question bank

Where it lives

`apps/api/src/questions/` (24 files), `prisma/schema.prisma:434-517`, `packages/contracts/src/questions.ts`, admin UI in `apps/admin/src/features/questions/`.

- `questions.service.ts` — owns `Question` and `QuestionVersion`; the only module that writes them (`:117`).
- `questions.controller.ts` — `admin/questions`: list, export, images, availability, detail, versions, create, update, archive/unarchive, delete.
- `question-core.ts` — `buildContent`, `computeStemHash`, `storedStemHash`, `drawableFor`.
- `question-query.ts` — `reachableTest`, `questionWhere`, `questionOrderBy`.
- `question-images.ts` + `image-bytes.ts` — key minting, magic-byte sniffing, `<img>` src stripping and rewriting.
- `authoring.service.ts` — the typist's author-fenced view over the same service.
- `prisma/migrations/20260918140000_an_edited_version_resyncs_every_paper_that_pins_it/migration.sql` — the `optionIds` fan-out trigger.
- `prisma/migrations/20260925120000_the_bank_is_searched_through_an_index/migration.sql` — `pg_trgm`, `tagsText()`, the three search indexes.

What it is for

Holds every question the platform can put on a paper, in up to three languages, under a `Subject` → `Topic` taxonomy plus free-text tags. A paper does not copy a question: `PaperQuestion` pins one `QuestionVersion`, so fixing a typo must never change what a student already sat. The module enforces the two rules the schema cannot state — when an edit may rewrite the existing version row and when it must never append a new one, and that one stem is never typed into the bank twice. It is also the write path for authored figures and formulas, which arrive as sanitized HTML carrying S3 keys.

FLOW — AUTHOR A QUESTION

1. `POST /admin/questions` from the bank workspace
`apps/admin/src/features/questions/bank-question.ts:87` `api.admin.questions.create()`
`packages/contracts/src/client/admin.ts:661`.
2. Global JWT guard, then `@Actors(ACTOR_TYPES.ADMIN)`, `@RequiresFeature(QUESTION_MANAGEMENT, WRITE)`, `@Audit(QUESTION, CREATE)`
`questions.controller.ts:67, :142-144`
`request` carries `AuthenticatedUser`.
3. `ZodBody(questionDraftSchema)` parses; tags lowercase and collapse, code upper-cases
`packages/contracts/src/questions.ts:363-376`
`a QuestionDraft`.
4. `create(body, user.id)`
the author comes from the token, never the body
`questions.controller.ts:150`.

5	<code>validated(draft) → taxonomyForIds</code> reads the one subject and one topic by id, 2 queries <code>apps/api/src/questions/taxonomy-context.ts:13</code>
6	<code>validateQuestion(draft, taxonomy)</code> with KaTeX bound in <code>apps/api/src/questions/question-core.ts:112</code> the first issue throws <code>VALIDATION_ERROR</code> with <code>fieldErrors</code> (<code>questions.service.ts:564-571</code>).
7	<code>buildContent(draft)</code> <code>apps/api/src/questions/question-core.ts:55</code> per language: <code>DOMPurify</code> (<code>question-sanitize.ts:37</code>), <code>stripImageSrc</code> , then <code>asContentHtml</code> wrapping in exactly one <code><div></code> . A language reaches the row only if it has a stem.
8	<code>computeStemHash = sha256 of canonicalStemKey</code> <code>apps/api/src/questions/question-core.ts:122</code> plus <code>stemHashVersion = STEM_HASH_VERSION</code> (<code>2, :120</code>).
9	<code>assertNotDuplicate(stemHash, null)</code> 1 query on <code>Question_stemHash_idx</code> questions.service.ts:589; a hit throws <code>CONFLICT</code> with <code>fieldErrors['stem.en']</code> and <code>details.duplicateOf</code> .
10	<code>prisma.\$transaction(..., TX_LIMITS.SHORT)</code> maxWait 5s / timeout 10s (<code>apps/api/src/prisma/prisma.service.ts:8</code>) questions.service.ts:267.
11	<code>tx.question.create</code> with <code>columnsOf</code> (<code>:544</code>) one <code>Question</code> row; <code>currentVersionId</code> still <code>NULL</code>
12	<code>tx.questionVersion.create(versionDataOf(id, 1, built, optionsWithIds(built, []), createdById))</code> :275,:672 version = 1; every option gets a fresh <code>randomUUID()</code> (<code>:662-669</code>).
13	<code>tx.question.update { currentVersionId }</code> with <code>QUESTION_INCLUDE</code> :284 the composite FK <code>[id, currentVersionId] → QuestionVersion[questionId, id]</code> is now satisfied. No <code>optionIds</code> trigger fires: the fan-out is <code>AFTER UPDATE OF "options"</code> only.
14	<code>served(toDetail(row))</code> imageKeysIn over every HTML string, each key mapped to <code>storage.publicUrl(key) = \${MEDIA_BASE_URL}/\${key}</code> , src rewritten :127, <code>apps/api/src/storage/storage.service.ts:68</code> .
15	<code>ResponseInterceptor</code> wraps <code>apps/api/src/common/response.interceptor.ts:13</code> { success: true, data: <code>QuestionDetail</code> , meta: { <code>requestId</code> } }.

The option-id triggers, quoted. `paper_question_option_ids` (`prisma/migrations/20260917100000_.../migration.sql:44-60`) is `BEFORE INSERT OR UPDATE OF "questionVersionId", "optionIds" ON "PaperQuestion"` and sets `NEW."optionIds" := ARRAY(SELECT listed.option->'id' FROM "QuestionVersion" version, jsonb_array_elements(...) WITH ORDINALITY AS listed(option, position) WHERE version."id" = NEW."questionVersionId" ORDER BY listed.position)`. `question_version_option_ids_fanout` (`...20260918140000...:25-37`) re-fires it from the version side: `UPDATE "PaperQuestion" SET "optionIds" = "optionIds" WHERE "questionVersionId" = NEW."id"`, as `AFTER UPDATE OF "options" ON "QuestionVersion" ... WHEN (OLD."options" IS DISTINCT FROM NEW."options")`. Naming the column in the SET clause is what re-fires the `BEFORE` trigger — no second copy of the array SQL. That file's header also records a dependence on Postgres running same-event `BEFORE` triggers alphabetically: `paper_question_option_ids` ('o') rebuilds the array before `paper_question_sat_guard` ('s') compares `NEW` to `OLD`, so renaming either trigger reverses the order.

FLOW — EDIT A PUBLISHED QUESTION

1	<code>PATCH /admin/questions/:id</code> <code>apps/admin/src/features/questions/bank-question.tsx:72</code>
2	<code>@Audit(QUESTION, UPDATE)</code> , <code>@RequiresFeature(QUESTION_MANAGEMENT, WRITE)</code> <code>questions.controller.ts:153-163</code>
3	<code>validated(draft)</code> , then <code>assertNotDuplicate(built.stemHash, id)</code> 3 queries questions.service.ts:296-297.
4	<code>\$transaction(TX_LIMITS.SHORT) → writeEdit</code> ; everything below is read inside it :299,:310
5	<code>tx.question.findUnique(QUESTION_INCLUDE)</code> ; <code>assertScreenIsCurrent</code> compares <code>draft.expectedUpdatedAt</code> with the row's <code>updatedAt</code> :317-319,:649 a stale form throws <code>CONFLICT</code> on field <code>status</code> . No Redis edit lock on this path, unlike section work.
6	<code>assertIdentitySettled</code> changing type, subjectId or topicId on a question any <code>PaperQuestion/TestQuestionStat</code> points at is refused :508-525.
7	<code>tx.\$queryRaw SELECT 1 FROM "Test" WHERE "id" IN (...PaperQuestion... UNION ...QuestionReview where t."finalizedAt" IS NULL) ORDER BY "id" FOR UPDATE</code> :323-329 one lock order, Test before Question.
8	<code>tx.question.updateMany({ where: { id, updatedAt: question.updatedAt }, data: columnsOf(...) })</code> ; <code>count !== 1</code> throws <code>editedElsewhere</code> :332-336
9	<code>optionsWithIds(built, optionsIn(question.currentVersion?.options))</code> an option id is inherited **by position**, minted only for a position that had none :339, :662.

- 10 `versionFor` computes `fingerprint(content, options, answerKey)`
sha256 over a key-sorted canonical form, 16 hex chars and when it matches the stored version returns `{ id: currentVersionId, rewored: false }`: no version row is written at all
`:360-363`, `:744-766`.
- 11 The rule, `apps/api/src/questions/questions.service.ts:380-383`: `const reached = await tx.paperQuestion.count({ where: { questionId: question.id, questionVersionId, test: reachableTest(new Date()) } }); return reached > 0 ? null : questionVersionId;`
`reachableTest` at `question-query.ts:15-26` is `finalizedAt: { not: null } AND (opensAt: null OR opensAt <= now + START_GRACE_MS OR some programUnlocks.opensAt <= now + START_GRACE_MS), START_GRACE_MS = 5 * 60 * 1000 (packages/contracts/src/access.ts:371)`
- 12 Zero reached → `revise()` mutates the existing row's `content`, `options`, `answerKey` and reassigns `createdById` / `createdAt` to this admin
`:387-406`. Otherwise `insertVersion()` writes `version = (currentVersion.version ?? 0) + 1`
`:409-426`.
- 13 `revise()`'s UPDATE fires `question_version_sat_guard` first (BEFORE UPDATE ... WHEN `options/answerKey/content` changed), which takes `FOR UPDATE ... ORDER BY "id"` on every pinning Test and raises `integrity_constraint_violation` if any has an `Attempt`
...20260918160000...20-36. Then the fan-out rebuilds `optionIds` on every unreached pinning paper
- 14 `if (version.rewored) await uncheckRewored(tx, id)`
clears `QuestionReview.checkedAt` on every unfinalized test and reopens the reader's section
`:342`, `apps/api/src/assignments/unread-questions.ts:40-57`.
- 15 `tx.question.update { currentVersionId }` (a no-op when revised in place), then `auditContext.setChanged(questionDiff(before, after))`
one leaf per stem.EN, solution.HI, option.3.TE, plus `correctOptionPositions`
`:344`, `:690-738`.

What breaks if the rule is wrong. `AttemptSheet.answers` stores a position into `PaperQuestion.optionIds`, not an option id (`apps/api/src/attempts/answer-sheet.ts:87-91`, `:108-123`). Rewrite a version a reachable test pins and three things follow: the stem and options change under people mid-sitting; the fan-out rebuilds `optionIds`, so a stored position decodes to a different option and scores against the new key; and if the rewrite dropped an option, `optionIds[option]` yields `undefined` → `null` and the answer silently vanishes. The sat-guards are the backstop, but they surface as a raw Postgres error rather than a refusal an admin can read. Commit `5bc625f` fixed exactly this: before it `reachableTest` compared `opensAt` to the bare clock, so between T-5min and T an edit still rewrote a startable paper in place.

FLOW — SEARCH THE BANK

- 1 `GET /admin/questions?q=...`
`apps/admin/src/features/questions/questions.tsx:147` via `useListScreen`
- 2 `@RequiresAnyFeature([QUESTION_MANAGEMENT, TEST_MANAGEMENT], READ)`
the test owner's picker reads the same endpoint
`questions.controller.ts:64`, `:75-81`.
- 3 `ZodQuery(questionListQuerySchema)`
`packages/contracts/src/questions.ts:524-547`; `q` is trimmed and capped at 64 chars with `**no minimum length**` (`common.ts:242-250`), `pageSize ≤ 100`
- 4 `query.q ? await this.searchIds(query.q) : null`
`apps/api/src/questions/questions.service.ts:149`
- 5 `searchIds` issues one raw three-branch UNION
`:615-631`: `v."content"::text ILIKE '%...%'` over the joined current version, `q."questionCode" ILIKE '%...%'`, `"tagsText"(q."tags") ILIKE '%...%'`. The term is `escapeForContent`-ed for the first two (`content` is stored HTML, so `&` is `&`) and raw for tags. A UNION, not an OR, because Postgres builds a `BitmapOr` within one relation, never across a join
- 6 The indexes serving it, all GIN `gin_trgm_ops` from `...20260925120000...:37-44`: `QuestionVersion_content_trgm_idx` on `("content"::text)`, `Question_questionCode_trgm_idx` on `questionCode`, `Question_tags_trgm_idx` on `tagsText("tags")`
`tagsText` is `array_to_string($1, ',')` pinned to `text[]` so it is IMMUTABLE, joining on a comma because a tag may hold a space but never one
- 7 `questionWhere(query, matchedIds)`
`question-query.ts:37`. Always narrowing: `status != ARCHIVED` unless a status is named, `drawableFor(forTestId)` when `drawable`, the `writtenFor` split, and `{ id: { in: matchedIds } }`. Reader-chosen filters (`subject`, `topic`, `type`, `difficulty`, `status`, `tag`, `author`, `date range`, `language`) pass through `matchFilters`, ANDed or ORed by match
- 8 `$transaction([findMany(QUESTION_INCLUDE, orderBy, pageArgs), count])`
2 queries
`:153-161`. `QUESTION_INCLUDE` resolves `subject`, `topic`, `author`, `assignment+test+section`, `currentVersion`, and `paperQuestions: { take: 1 } / questionStats: { take: 1 }` for `inUse`.
- 9 `orderBy` is `[[createdAt:'desc'],{id:'desc'}]` (or `asc/asc`)
`:99-102`
served by `Question_createdAt_id_idx` (`...20260929090000...:21`), scanned backwards.
- 10 `paged(...)` → `{ items, page, pageSize, total }`, which the interceptor turns into `meta.page/pageSize/total`.

FLOW — A QUESTION'S MEDIA

- 1 `POST /admin/questions/images`, multipart field `file`, from `apps/admin/src/lib/upload-question-image.ts:6`
the browser shrinks it first via `shrunkForUpload`
- 2 `@Audit(QUESTION, CREATE)`, `@RequiresAnyFeature([QUESTION_MANAGEMENT, QUESTION_AUTHORIZING, QUESTION_PROOFREAD], WRITE)`, `FileInterceptor` capped at `QUESTION_IMAGE_MAX_BYTES = 2 MB`
`questions.controller.ts:103-119`
- 3 `checkQuestionImage` refuses an absent or empty file
`apps/api/src/questions/question-images.ts:55`
- 4 `sniffImage` reads PNG/JPEG/GIF/WebP dimensions from the leading bytes; the browser's content type is a hint about the filename, never a fact (`image-bytes.ts:1-12`, `:94`). SVG is excluded on purpose as a script container (`packages/contracts/src/questions.ts:792`).

- 5

Over `QUESTION_IMAGE_MAX_PIXELS = 12 Mpx` is refused by dimensions, not bytes
`apps/api/src/questions/question-images.ts:87`
- 6

`questionImageKey(contentType)` → `questions/images/<uuid>.<ext>`
unscoped, because the image is uploaded before the question has an id (:19–22). The importer instead keys by content digest, so one picture imported twice is one object (:28)
- 7

`storage.upload(key, buffer, contentType, IMMUTABLE_CACHE_CONTROL)`
one `PutObjectCommand` to `S3/MinIO` with `Cache-Control: public, max-age=31536000, immutable`
`questions.service.ts:139`, `question-images.ts:25`.
- 8

Response `{ key, url }`; the editor puts the **key** into the content as `<img data-key="...">`. On save `stripImageSrc` removes any `src` (:112) and `withoutForeignImages` drops any `<img>` with no key (:107); on read `applyImageUrls` builds a fresh unsigned ``${MEDIA_BASE_URL}/${key}`` (:119). There is no `Attachment` table and no row linking an image to a question. Formulas are not media: they are `<span data-type="inline-math" data-latex="...">` in the same HTML (`question-content.ts:108`).

Multi-language, and the absent render mode — Language is a property of the content JSON, not a column: `SUPPORTED_LANGUAGES = { EN: 'en', HI: 'hi', TE: 'te' }` (`packages/contracts/src/questions.ts:17`) are the **keys** inside `QuestionVersion.content` and inside each option's `text`. `DEFAULT_LANGUAGE` is `en` and mandatory — it is what the bank is searched and deduped by (:35). `languagesIn(draft.stem)` decides which languages reach the row, so a translated option with no translated stem is dropped (`question-core.ts:55–74`); `QuestionSummary.languages` is recomputed from the stored content on read, never stored (`questions.service.ts:838–843`). **Neither `Question` nor `QuestionVersion` carries a render mode.** `TestUi` (`CBT/OMR/GENERIC/TYPING`), `ExamTemplate`, `LanguageMode` (`SINGLE/DUAL`) and the `languages` array live on `BaseConfig` (`prisma/schema.prisma:353–358`), and the chosen languages are copied onto `Attempt.languages` (:702).

Question vs QuestionVersion — `Question` is identity and filing; `QuestionVersion` is everything the question says. The split exists because `PaperQuestion` holds a composite FK to `[questionId, id]` with `onDelete: Restrict`, so "what did this student read" is a database fact rather than a convention (schema comment, :499–500).

The data it touches

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
<code>Subject.id/name/code</code>	<code>uuid v7 / text @unique / text? @unique</code>	top taxonomy level	<code>TaxonomyService.createSubject</code> (<code>taxonomy.service.ts:107</code>)
<code>Topic.id/subjectId/name</code>	<code>uuid v7 / uuid / text, @unique({subjectId,name})</code>	a pool under a subject, not a scope	<code>TaxonomyService</code>
<code>Question.id</code>	<code>uuid v7</code>	identity	<code>questions.service.ts:268</code>
<code>Question.questionCode</code>	<code>text? @unique</code>	the institute's own reference, upper-cased	<code>columns0f</code> (:544)
<code>Question.type</code>	<code>QuestionType SINGLE_MCQ\</code>	TEXT_FIELD	which shape <code>columns0f</code>
<code>Question.subjectId / topicId</code>	<code>uuid / uuid? (Restrict)</code>	taxonomy; frozen once anything uses the row	<code>columns0f</code> , guarded by <code>assertIdentitySettled</code>
<code>Question.difficulty</code>	<code>DifficultyLevel LOW/MEDIUM/HIGH</code>	what a draw spec asks for	<code>columns0f</code>
<code>Question.status</code>	<code>QuestionStatus ACTIVE\</code>	ARCHIVED, default ACTIVE	in or out of circulation <code>setStatus</code> (:428)
<code>Question.currentVersionId</code>	<code>uuid?</code>	the version the bank shows; composite FK, <code>Restrict</code>	<code>create</code> , <code>writeEdit</code> , <code>remove</code> , <code>settleTyped</code>
<code>Question.tags</code>	<code>text[] default []</code>	free text, lowercased, ≤10 tags, ≤32 chars each	<code>columns0f</code>
<code>Question.stemHash</code>	<code>text?</code>	sha256 of <code>canonicalStemKey</code> ; the dedup key	<code>columns0f</code> , <code>StemRehashService</code>
<code>Question.stemHashVersion</code>	<code>int default 1</code>	the fold that hash used; the code's is 2	<code>columns0f</code> , <code>StemRehashService</code>
<code>Question.assignmentId</code>	<code>uuid? (SetNull)</code>	the typing assignment it was written under	<code>create</code> , <code>settleTyped</code>
<code>Question.createdById</code>	<code>uuid? (Restrict)</code>	author, from the token	<code>create</code> only
<code>Question.createdAt / updatedAt</code>	<code>timestamptz(3)</code>	<code>updatedAt</code> is the optimistic-lock token	Prisma
<code>QuestionVersion.id</code>	<code>uuid v7</code>	identity; <code>@unique({questionId,id})</code> anchors the composite FKs	<code>create</code> , <code>insertVersion</code>
<code>QuestionVersion.questionId / version</code>	<code>uuid / int, @unique({questionId,version})</code>	the 1-based chain	same
<code>QuestionVersion.content</code>	<code>jsonb NOT NULL</code>	<code>{ en?: { stem: Node[], solution?: Node[] }, hi?.., te?... }, Node = { type: 'TEXT', text: html }, one <div> root per field</code>	<code>versionData0f</code> , <code>revise</code>
<code>QuestionVersion.options</code>	<code>jsonb?</code>	<code>{ { id: uuid, position: int, isCorrect: bool, text: { en?: Node[], ... } } }</code> ; stored order is <code>optionIds</code> order	<code>versionData0f</code> , <code>revise</code>
<code>QuestionVersion.answerKey</code>	<code>jsonb?</code>	TEXT_FIELD only: <code>{ mode: 'EXACT'\</code>	'NUMERIC', answers: <code>{ en: string, ... }, tolerance?: number }</code> <code>versionData0f</code> , <code>revise</code>

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
QuestionVersion.createdById / createdAt	uuid? (no FK) / timestampz(3)	the hand and the moment; both reassigned by an in-place revision	versionDataOf, revise:401-402
PaperQuestion.optionIds	text[]	copy of the pinned version's ids in stored order	trigger only

Question indexes: (subjectId, difficulty, status), (topicId), (stemHash), (createdById), (assignmentId), GIN (tags), GIN trgm (questionCode), (status), (createdAt, id) — prisma/schema.prisma:485-495 . No Redis key and no BullMQ job is written on any question path. StemRehashService runs once at worker boot (stem-rehash.service.ts:19-29): 500 rows a batch, keyset-paged on id, raw UPDATE ... WHERE q."updatedAt" = v.seen so it never clobbers a concurrent edit and never moves updatedAt .

What must already be true

a Subject row, and the Topic (if named) must be under it; an Admin with QUESTION_MANAGEMENT at the route's level, or QUESTION_AUTHORIZING for admin/authoring/*, or TEST_MANAGEMENT for list and availability; MEDIA_BASE_URL plus the S3/MinIO vars for StorageService; CREATE EXTENSION pg_trgm and the tagsText function from migration 20260925120000 for search; the process must serve API_ROLES.CORE for the controllers to mount (questions.module.ts:31) and WORKER for the rehash.

Why it is shaped this way

- Two tables, not one self-versioning row. The composite FK with Restrict makes what a student read a database fact. A single mutable row plus an audit trail cannot answer it; copying the question into the paper would leave no canonical row for the bank to show.
- Options in a jsonb column, not an Option table. prisma/schema.prisma:500 gives the reason: one read is the whole question on the exam path. The cost — no SQL constraint can say "exactly one isCorrect" (validateQuestion carries it), and optionsWithIds must hand-merge ids by position.
- optionIds denormalized onto PaperQuestion by trigger. The sheet then stores a small integer per slot, keeping AttemptSheet one row per sitting at fillfactor = 50 . Option ids in the sheet would be larger and still would not survive a reorder. The cost is the two-trigger dance and its dependence on alphabetical trigger order.
- stemHash as a plain index, deduped in application code. A @unique would be race-free but would hard-fail an import batch on one repeat; assertNotDuplicate instead points the admin at the existing question, naming it when archived (:596-605).

Which screen calls it

(admin SPA only — apps/exams and mobile never call these). GET /admin/questions → features/questions/questions.tsx:147 and features/tests/question-picker.tsx:249, at /questions and inside /tests/:id . GET .../export → questions.tsx:188 . POST .../images → lib/upload-question-image.ts:6, from every authoring surface. GET .../availability → features/tests/draw-spec.tsx:45 . GET .../:id → features/questions/bank-question.tsx:28, at /questions/:id (read-only) and /questions/:id/edit . GET .../:id/versions → features/questions/question-history.tsx:99, a sheet on /questions . POST /admin/questions → bank-question.tsx:87 at /questions/new . PATCH .../:id → bank-question.tsx:72 . Archive / unarchive / delete → questions.tsx:226-228, the row menu. /admin/subjects and /admin/topics → features/questions/taxonomy.tsx at /questions/taxonomy . admin/authoring/* → features/authoring/* . Route constants apps/admin/src/lib/constants.ts:79, :95-104; wiring apps/admin/src/App.tsx:180-189 .

Is it optimal

- GET /admin/questions without q : 2 queries in one transaction; Question_createdAt_id_idx for the sort, Question_subjectId_difficulty_status_idx or Question_status_idx for the filters. No N+1 — take: 1 on both child relations replaced the _count that grouped whole child tables per page (f472a4d). 20 summaries, each a 140-char stemPreview, no content. Optimal.
- GET /admin/questions?q=... : 3 queries, and the hot spot. searchIds has no LIMIT, and the service funnels the whole result into { id: { in: matchedIds } }, used twice — by findMany and by count . q has no minimum length, so q=a matches nearly every row: on a 50K bank that is 50K uuids out of Postgres and 50K back in for a 20-row page. Short terms are also where trigram GIN degrades — under 3 characters there are no trigrams to probe. Read from the SQL, not measured; the migration header records the pre-index state as a sequential scan with a per-row jsonb → text cast.
- Language filter currentVersion: { content: { path: [lang], not: DbNull } } (question-query.ts:93) is a relation subquery with no index on the JSON path; it degrades with bank size.
- GET .../export : 1 count + 2 parallel queries + 1 author lookup, capped at EXPORT_MAX_ROWS = 50_000; the workbook is built in memory.
- PATCH .../:id : 3 queries before the transaction, 5-6 inside, plus triggers; the fan-out is one BEFORE-trigger invocation per pinning paper, each re-reading the version jsonb.
- GET .../:id/versions : 2 queries, and the chain is short by construction. Optimal.

What it leans on

ResponseInterceptor + AllExceptionsFilter for the envelope and AppException → status mapping; the global JWT guard with @Actors / @RequiresFeature / @RequiresAnyFeature / @RequiresExport (apps/api/src/common/security/decorators.ts); ZodBody / ZodQuery over packages/contracts, so the form, the importer and the API validate identically; @Audit + AuditContext.setChanged; pageArgs / paged; matchFilters; editedElsewhere from common/edit-lock; optionsIn / answerKeyIn from common/prisma-json (the only Json casts in the API); StorageService for the one upload path; common/exporting; common/time/institute-day for the civil-date filters. The admin UI goes entirely through the typed api.admin.questions client (packages/contracts/src/client/admin.ts:639-685), which re-parses every response against the same schemas. Coverage: apps/api/test-db/questions-service.db.test.ts carries ~40 cases, nine of them pinning the version rule across draft / unreached / inside-grace / open papers, and apps/api/test-db/question-version-option-ids.db.test.ts pins the fan-out.

What changed recently

(git log -15 over the module, the schema and the contracts file)

- 5bc625f — behaviour: reachableTest now compares openings to now + START_GRACE_MS . Before it, an edit in the five minutes before a test opened rewrote a startable paper's version in place.
- f472a4d — deleted QuestionsService.page() as dead; collapsed servedAll into served; replaced relation _count with a scoped groupBy; pulled the five hand-written option/answer-key casts into common/prisma-json .
- da1e4fb — an import run can only be committed through the importer that previewed it (question-import.service.ts plus two schema columns).
- 3cc1fba — findLast for the last typist row; no behaviour change.

- `a5112aa`, `dd680b3` — schema and contract tidying only (outbox removal; one composed schema un-exported).

What is wrong with it

1. `questions.service.ts:615-631` + `question-query.ts:63` — `searchIds` returns every match with no `LIMIT`, funnelled into `id: { in: [...] }`. Input: `GET /admin/questions?q=a&pageSize=20`. The entire matching id set crosses the wire twice for one page.
2. `prisma/schema.prisma:467` + `:489` — `stemHash` is `@@index`, not `@unique`, and `assertNotDuplicate` (`questions.service.ts:589`) reads **outside** the write transaction. Input: two admins, or two import commits, post the same stem concurrently — both reads miss, both inserts succeed, and the bank holds the duplicate it exists to prevent.
3. No `storage.remove` call exists anywhere in `apps/api/src` (verified by grep). `POST /admin/questions/images` writes `questions/images/<uuid>.<ext>` before the question has an id, so an admin who uploads a figure and abandons the form leaves the object orphaned permanently. No sweeper, no reference count.
4. `questions.controller.ts:103` — `@Audit(QUESTION, CREATE)` on `uploadImage` never records anything: the handler returns `{ key, url }`, there is no `:id` param and nothing calls `setEntityId`, so `audit.interceptor.ts:55-56` resolves no `entityId` and returns early. A dead decorator.
5. `QuestionVersion_content_trgm_idx` and `Question_tags_trgm_idx` exist only in migration SQL — the `QuestionVersion` model (`prisma/schema.prisma:501-517`) declares no indexes at all — whereas `Question_questionCode_trgm_idx` is declared at `:493`. Both are expression indexes Prisma's datamodel cannot express. I did not run `pnpm db:check`, so whether `prisma migrate diff` reports this as drift is **not verified**.
6. `question-query.ts:54` vs `:77` — the `status != ARCHIVED` floor is skipped whenever a status is named, but the named-status filter is itself subject to the `match` toggle. Input: `?status=ACTIVE&difficulty=HIGH&match=any` returns archived HIGH questions, because the OR widens past the status the reader asked for and nothing narrows it back.
7. `questions.controller.ts:130` vs `:64` — `GET :id` requires `QUESTION_MANAGEMENT` while `GET /` and `/availability` also accept `TEST_MANAGEMENT`. A test owner can page and draw from the bank but gets 403 opening any single question; the picker works only because the list already carries `stemPreview`.

The authoring workshop

Where it lives

`apps/api/src/assignments/`, the section-work half of `apps/api/src/questions/`,
`apps/admin/src/features/authoring/`, `apps/admin/src/components/section-thread.tsx`.

- `assignments/assignments.service.ts` — assign, the per-admin queue, the progress grid, a reader's release and its gap rule.
- `assignments/assignment-gates.ts` — `doneOpen` and `readOpen`, the two predicates API and UI share.
- `assignments/unread-questions.ts` — "unchecked on this test", and the two reopens a paper edit calls.
- `assignments/section-thread.service.ts` — the (test, section) discussion and its revision trail.
- `questions/section-work.service.ts` — the screen's whole read, seat resolution, every per-question verdict.
- `tests/paper.service.ts:349` — `typistDone`, here because its effect is the paper.
- `apps/admin/src/features/authoring/section-authoring.tsx` — the single screen all three seats work from.
- `prisma/schema.prisma:608` / `:638` / `:665` — `QuestionAssignment`, `QuestionReview`, `SectionComment`.

What it is for

A paper is typed and proof-read before it is offered. Two people hold one

section of one test: a typist writes its questions, a proof-reader reads them. The module records who holds which half, moves the section between them, and carries the per-question verdict — checked, sent back with a reason, fixed — until a reader can release a whole section and the owner can offer the test. `Test.finalizedAt` ends all of it.

FLOW — ASSIGN A TYPIST AND A PROOFREADER

- 1 Admin submits the Assign dialog in test-builder step 2
`apps/admin/src/features/tests/test-builder-assign-step.tsx:633` `assign.mutationFn()`
`body {baseConfigSectionId, assigneeId, role, dueAt}`
- 2 Typed client writes it
`packages/contracts/src/client/admin.ts:841` `assignments.assign()`
`POST /admin/assignments/tests/:testId`
- 3 Route: `@Actors(ADMIN)` + `@RequiresFeature(TEST_MANAGEMENT, WRITE)` + `@Post('tests/:testId')` + `@HttpCode(CREATED)` + `ZodBody(createAssignmentSchema)`
`apps/api/src/assignments/assignments.controller.ts:76` `assign()`
`validated body, user.id as actor`
- 4 Test must exist
`apps/api/src/assignments/assignments.service.ts:254` `requireTest()`
`id, baseConfigId, paperSource, finalizedAt`
- 5 An offered test takes nobody new
`apps/api/src/assignments/assignments.service.ts:255`
`CONFLICT`
- 6 Source must be declared
`apps/api/src/common/paper-edit.ts:34` `assertSourceChosen()`
`paperSource` is `FRAMED` or `PICKED`
- 7 Section must belong to the test's config
`apps/api/src/assignments/assignments.service.ts:710` `requireSection()`

- 8 Assignee must hold the role's key at WRITE
apps/api/src/assignments/assignments.service.ts:727 assertHoldsFeature()
QUESTION_AUTHORIZING for TYPIST, QUESTION_PROOFREAD for PROOFREADER (:93); else a fieldErrors.assigneeId refusal
- 9 Nobody reads their own typing
apps/api/src/assignments/assignments.service.ts:742 assertNotTheOtherRole()
- 10 Transaction opens on the app's one lock order
common/paper-edit.ts:27 beginDraftPaperEdit()
SELECT ... FROM "Test" ... FOR UPDATE
- 11 Current holder re-read under the lock
apps/api/src/assignments/assignments.service.ts:270
the same admin ⇒ VALIDATION_ERROR (:273)
- 12 Progress carries to the new holder
apps/api/src/assignments/assignments.service.ts:931 handedOnArrival()
a reader arriving on a FRAMED section whose typist is done starts handed
- 13 The old row stands down, never deleted
apps/api/src/assignments/assignments.service.ts:283
replacedAt = now()
- 14 New row inserted
apps/api/src/assignments/assignments.service.ts:286
role, dueAt, inherited finalizedAt / handedAt, createdById
- 15 A race loses on the partial unique
apps/api/src/assignments/assignments.service.ts:303
CONFLICT (QuestionAssignment_active_role_key, WHERE "replacedAt" IS NULL)
- 16 Counts attached, interceptor wraps
assignments.service.ts:657, common/response.interceptor.ts
{success:true, data: Assignment, meta}

Enums (prisma/schema.prisma:270): AssignmentRole = TYPIST | PROOFREADER, AdminRole (:283) TYPIST | PROOFREADER | ADMIN | SUPER_ADMIN only labels and presets grants — no guard reads it.

FLOW — THE TYPIST HANDS THE SECTION OVER (THERE IS NO PER-QUESTION TYPIST TICK)

- 1 "Mark done"
apps/admin/src/features/authoring/section-authoring.tsx:227
opens TypistDoneDialog
- 2 Dialog re-reads the section
apps/admin/src/features/authoring/typist-done-dialog.tsx:50
GET /admin/sections/:testId/:sectionId
- 3 Submit
apps/admin/src/features/authoring/typist-done-dialog.tsx:63
POST /admin/sections/:testId/:sectionId/done, body {selected, discard}
- 4 Route: @Audit(TEST, UPDATE) + @RequiresFeature(QUESTION_AUTHORIZING, WRITE) + @Post('done') + @HttpCode(OK) + ZodBody(typistDoneSchema)
apps/api/src/tests/tests.controller.ts:229 TypistDoneController.done()
- 5 Seat resolved to a typing row
apps/api/src/questions/section-work.service.ts:184 actingTypist()
a super admin acts through the holder (:564)
- 6 Row re-checked
apps/api/src/tests/paper.service.ts:360
role, replacedAt, finalizedAt
- 7 Gate
apps/api/src/assignments/assignment-gates.ts:17 doneOpen()
TYPIST, replacedAt null, finalizedAt null, test.paperSource === FRAMED, test.finalizedAt null
- 8 The choice must be exactly the section's count, inside its mix
apps/api/src/tests/paper.service.ts:380 assertTypedSelection()
- 9 Transaction + FOR UPDATE on the test
apps/api/src/tests/paper.service.ts:384
- 10 QuestionAssignment.finalizedAt = now(), guarded on finalizedAt: null
apps/api/src/tests/paper.service.ts:385
a second Done gets CONFLICT
- 11 The choice becomes the paper
apps/api/src/tests/paper.service.ts:406 placeTyped()
unticked PaperQuestion rows deleted, new ones appended after the highest order
- 12 The unlock: QuestionAssignment.handedAt = now() on the active reader
apps/api/src/tests/paper.service.ts:392

13 Unchosen drafts settled
apps/api/src/questions/questions.service.ts:477 settleTyped()
discarded deleted, the rest get assignmentId = null

14 Fresh SectionWork returned
apps/api/src/questions/section-work.service.ts:140

handedAt is the whole unlock: readOpen (assignment-gates.ts:25) requires it, scoped() returns [] to a reader without it (section-work.service.ts:449), and requireReading refuses with "has not reached you yet" (:544). On a PICKED paper the owner hands over instead (paper.service.ts:512).

FLOW — A PROOFREADER SENDS ONE QUESTION BACK

- 1 "Send back" on the card
apps/admin/src/features/authoring/section-authoring.tsx:487
SendBackDialog (reason radio + optional note ≤500)
- 2 Submit
apps/admin/src/features/authoring/section-authoring.tsx:566
POST /admin/sections/:testId/:sectionId/questions/:questionId/send-back
- 3 Route: @RequiresAnyFeature([QUESTION_AUTHORIZING, QUESTION_PROOFREAD, TEST_MANAGEMENT], WRITE) + @HttpCode(OK) + ZodBody(sendBackSchema)
apps/api/src/questions/section-work.controller.ts:195
- 4 Context read once
apps/api/src/questions/section-work.service.ts:395 load()
- 5 Must be the section's active reader, test not offered, handed, not released
apps/api/src/questions/section-work.service.ts:538 requireReading()
- 6 A section with no active typist refuses
apps/api/src/questions/section-work.service.ts:355
CONFLICT
- 7 Question must be on this section's paper
apps/api/src/questions/section-work.service.ts:548 requireOnPaper()
- 8 Durable write
apps/api/src/questions/section-work.service.ts:368 questionReview.upsert on @@unique([testId, questionId])
sets sentBackAt, reason, note, sentBackById; clears fixedAt, fixedById, checkedAt, checkedById
- 9 Response re-derived
apps/api/src/questions/section-work.service.ts:145 workOf()
review.state is now SENT_BACK (:116)
- 10 No notification fires. Nothing in apps/api/src/assignments/** or section-work.* touches NotificationsService; the only notification.createMany is apps/api/src/notifications/notifications.service.ts:51, used by access, students, me and scoring. The typist learns of it on the screen.
- 11 The typist's card gains a warning Alert with the reason and note
apps/admin/src/features/authoring/section-authoring.tsx:425
- 12 It becomes editable again even after Done
apps/api/src/questions/section-work.service.ts:515
editable() returns true on SENT_BACK regardless of own.finalizedAt
- 13 "Mark fixed" appears
apps/admin/src/features/authoring/section-authoring.tsx:501 POST ../fixed
section-work.service.ts:388 — fixedAt / fixedById, state FIXED
- 14 The reader meanwhile sees With the typist
apps/admin/src/features/authoring/section-authoring.tsx:472
and check refuses (section-work.service.ts:325)
- 15 A reword drops every tick
questions.service.ts:342 → apps/api/src/assignments/unread-questions.ts:40 uncheckReworded()
and a released reader is pulled back (:10, reopenReadingIfUnchecked)

Flow trace: the comment thread — scoped to (testId, baseConfigSectionId): the section, never a question.

1. "Thread" button, on the section page or builder step 2 — apps/admin/src/components/section-thread.tsx:95 — GET
/admin/sections/:testId/:sectionId/comments
2. Route: @Actors(AADMIN) + @RequiresAnyFeature([QUESTION_AUTHORIZING, QUESTION_PROOFREAD, TEST_MANAGEMENT], READ) —
apps/api/src/assignments/assignments.controller.ts:149 — passes {id, managesTests: can(user, TEST_MANAGEMENT)}
3. Read rule — apps/api/src/assignments/section-thread.service.ts:115 — assertMayRead() — managesTests short-circuits; else a count of
QuestionAssignment on the pair with assigneeId = reader.id, replaced rows included, so a stood-down holder still reads. Zero ⇒ NOT_FOUND "No such section"
4. Whole thread, unpaginated, createdAt asc — section-thread.service.ts:45
5. Image keys signed once per distinct key — section-thread.service.ts:108 — servedAll() → storage.publicUrl
6. Post — same path, WRITE, 201 — assertMayWrite() (:128) takes only rows with replacedAt: null; a super admin otherwise, with requireSection() (:146)
proving the pair exists
7. Reword — PATCH ../comments/:commentId — :70 — author only, not a super admin; the previous {body, at} is pushed onto revisions, editedAt set

The data it touches

COLUMN	TYPE	HOLDS	WRITTEN BY
QuestionAssignment.testId/baseConfigId/baseConfigSectionId	uuid	composite FKs to Test(id,baseConfigId) and BaseConfigSection(baseConfigId,id)	assign
.assigneeId	uuid	the admin; FK RESTRICT	assign
.role	AssignmentRole	TYPIST \	PROOFREADER
.dueAt	timestampz(3)	informal reminder; gates nothing	assign
.finalizedAt	timestampz(3)	the whole state: null outstanding, set done	paper.service.ts:385 (typist), assignments.service.ts:(reader); cleared by unrea questions.ts:35
.handedAt	timestampz(3)	when the section reached the reader	paper.service.ts:392 , :512 , assignments.service.ts:
.replacedAt	timestampz(3)	role passed on, or section left scope	assignments.service.ts:
QuestionReview.checkedAt/checkedById	timestampz(3)/uuid, no FK	the reader's tick	check (:328); cleared by uncheck , sendBack , uncheckReworded
.sentBackAt/reason/note/sentBackById	timestampz(3)/ SendBackReason /text/uuid	the rejection; `SPELLING \	DATA_CORRECTION \
.fixedAt/fixedById	timestampz(3)/uuid	the typist's answer	fixed (:388)
SectionComment.body	text	the message	comment , editComment
.images	text[]	S3 storage keys, signed on read	comment
.revisions	jsonb[]	[{ body: string, at: ISO string }] , oldest first; the current body is never in it	editComment (:95)
.editedAt	timestampz(3)	last reword	editComment
Question.assignmentId	uuid, SetNull	the typing job a draft was written under	questions.service.ts:27 nulled by settleTyped (:490)

Redis: one key, section:edit:{testId}:{baseConfigSectionId} , TTL 900s (apps/api/src/redis/redis.keys.ts:74 , EDIT_LOCK_TTL_SEC :79). Advisory, taken before the transaction, no release and no heartbeat (apps/api/src/common/edit-lock.ts:1-5); read back into SectionWork.editingBy . No queue and no BullMQ job anywhere in this module.

What must already be true

a Test with paperSource set and finalizedAt null; a BaseConfigSection on that test's baseConfigId and inside its scope / scopeRef ; an active Admin holding QUESTION_AUTHORIZING or QUESTION_PROOFREAD at WRITE (feature keys are code-owned); the assigning admin holding TEST_MANAGEMENT WRITE; migrations 20260928160000 (QuestionReview, handedAt , replacedAt , the partial unique) and 20260928120000 (section-at-a-time Done); a reachable Redis.

Why it is shaped this way

- Stamps, not a status enum (schema.prisma:617): each of finalizedAt / handedAt / replacedAt says *when*, so no state machine can drift from the dates. Cost: "what state is this" is a TS predicate (assignment-gates.ts) SQL cannot enforce.
- Passing a role inserts a row and marks the old one replacedAt rather than updating assigneeId , so the record survives. Cost: a plain @@unique cannot say it — one-active-holder is a hand-written partial index, invisible to Prisma.
- SectionComment keys on the pair, not on an assignment row, because the two seats share one thread (migration 20260919140000 header). Cost: no composite FK to BaseConfigSection ; the section id is an unvalidated column, proved only by the service's lookup.
- revisions is a Json[] on the row, not a second table. Cost: no index on, and no query over, an earlier wording.

Which screen calls it

all apps/admin . The one screen is SectionAuthoringPage (features/authoring/section-authoring.tsx), mounted at three patterns rendering the same component (apps/admin/src/App.tsx:190): /authoring/assignments/tests/:testId/sections/:sectionId , /proofreading/assignments/tests/:testId/sections/:sectionId , /tests/:testId/sections/:sectionId (apps/admin/src/lib/constants.ts:86 , :91 , :93).

ENDPOINT	CALLER (APPS/ADMIN/SRC/_)	SCREEN ROUTE
GET /admin/assignments/tests/:testId	features/tests/test-builder-assign-step.tsx:108 , features/tests/test-paper.tsx:95	/tests/:id , /tests/:id/paper
POST /admin/assignments/tests/:testId	test-builder-assign-step.tsx:633	/tests/:id
DELETE /admin/assignments/:id	test-builder-assign-step.tsx:699	/tests/:id

ENDPOINT	CALLER (APPS/ADMIN/SRC/_)	SCREEN ROUTE
GET /admin/assignments/assignable	test-builder-assign-step.tsx:626	/tests/:id
GET /admin/assignments/mine	features/authoring/assignment-queue.tsx:176, assignment-picker.tsx:16	/authoring/assignments, /proofreading/assignments
GET /admin/assignments/progress	features/authoring/section-progress.tsx:187	/tests/section-progress
GET /admin/assignments/tests	features/authoring/assignment-scope-picker.tsx:30	both queues + progress
GET /admin/assignments/tests/:testId/sections	assignment-scope-picker.tsx:56	both queues + progress
GET /admin/sections/:t/:s	section-authoring.tsx:99, typist-done-dialog.tsx:50	the three section routes
POST .../questions; GET / PATCH / DELETE .../questions/:q	section-authoring.tsx:212, :185, :195, :534	same
GET .../questions/:q/other-tests	section-authoring.tsx:431	same
POST / DELETE .../questions/:q/check; POST .../send-back; POST .../fixed	section-authoring.tsx:464, :465, :566, :466	same
POST .../done	typist-done-dialog.tsx:63	same + /authoring/assignments
POST .../release	finalize-assignment-dialog.tsx:28	same + /proofreading/assignments
GET / POST .../comments, PATCH .../comments/:id	components/section-thread.tsx:95, :107, :111	section routes + /tests/:id
POST .../import/preview, .../import/commit	features/imports/import-questions.tsx:51, :55	.../sections/:sectionId/import

Authorisation rule — `SectionWorkService.load`, decision at `apps/api/src/questions/section-work.service.ts:413–420`. It reads exactly three things: `QuestionAssignment.assignedId` (=== `viewer.id`), `replacedAt` (prefer the active row, else `own.at(-1)`, the last row they held), and, as the only fallback, the grant `can(viewer, FEATURE_KEYS.TEST_MANAGEMENT)`. It ignores `role`, `finalizedAt`, `handedAt`, `dueAt`, `createdById` and `Test.createdById` for reachability — `role` is read afterwards only to label the seat (:423). The refusal is `NOT_FOUND` "No such section", never `FORBIDDEN`: not yours reads as not there. Per-action gates layer on top: `heldNow` (:584), `requireReading` (:538), `typingRow` (:573), `actingAs` (:564 — a super admin acts *through* the holder), `editable` (:509), `deletable` (:526), `doneOpen` / `readOpen` (`assignment-gates.ts:17` / :25). The thread has its own pair (`section-thread.service.ts:115`, :128): read counts replaced rows, write does not. `AssignmentsController` has **no** per-row ownership — `forTest`, `assign` and `remove` turn on `TEST_MANAGEMENT` alone; `mine` is hard-scoped to `user.id` (`controller:93`); `tests / sections` scope by `ownScope` (`assignments.service.ts:765`), which pins every non-super-admin to their own rows.

Is it optimal

(all read, not measured — no database was queried)

- `GET mine` : 2 statements in a `$transaction`, 2 for written counts, ≤3 for release counts ⇒ ≤7 per page, batched across the page on purpose (`assignments.service.ts:453`, :601). Index `QuestionAssignment.assignedId_finalizedAt_idx`. No N+1.
- `GET progress` : one `findMany` of every unfrozen test with its sections and active assignments, then the cross expanded, filtered and paged **in memory** (:483–495). Correct, but the whole set is materialised per request and no index serves `finalizedAt IS NULL AND paperSource IS NOT NULL`.
- `GET /admin/sections/:t/:s` : ~12 statements — test, section, `sectionAssignments` + 2 counts, `scoped` (paper, typed, questions, reviews), one Redis `GET`, `canRelease` ≤3. Reviews come back in one `IN (...)` (:477), so no N+1 over questions.
- `POST .../check | send-back | fixed` : `load` (≈4) + 1–2 reads + 1 upsert + a full `workOf` re-read (≈7) ⇒ ~13 statements for one tick. The re-read is the payload, not a correctness need (:144).
- `GET assignable` : `holdersOf` reads **every** active admin then filters in memory (`apps/api/src/admins/admins.service.ts:100–105`) — a deliberate scan, fine at dozens of admins.

What it leans on

the response envelope + exception filter (controllers return

data or throw `AppException`); `ZodBody` / `ZodQuery` over contracts schemas (`createAssignmentSchema`, `typistDoneSchema`, `sendBackSchema`, `sectionWorkSchema`); the four security decorators (`common/security/decorators.ts`) for the coarse gate, leaving the per-section rule to the services; `common/paper-edit.ts` for the one lock order; `common/edit-lock.ts` + Redis for the advisory claim; `common/pagination.ts`, `common/time/institute-day.ts` (due dates as institute days), `common/relation-counts.ts`, `common/search-terms.ts`; `@Audit` on the typing writes; app-kit's `useListScreen` for both queues; `packages/ui` `ListView`, `FormDialog`, `FormCombobox`, `DatePicker`, `ConfirmDialog`, `Alert`, `Badge`, `EmptyState`, `TruncatedText`, `Tabs`.

What changed recently

(`git log -15` over these paths)

- `10f929d` — pure move, `src/routes/*` → `src/features/<area>/*`; no behaviour change.
- `f472a4d` — per-row reads became per-page batches: `releaseCounts` and `sectionWrittenCounts` (:601, :657) answer a whole queue page in a fixed number of statements instead of three per row, and `releaseGapOf` became the one release rule the queue flag and `finalize` both use.
- `da1e4fb` — an import run commits only through the importer it was previewed in.
- `3cc1fba` / `d2e1ef9` — a stood-down typist keeps only their own last-held drafts (`typingOrHeld`, :588); a super admin acts through the current holder, not beside them (`actingAs`, :564).
- `24e45f8` / `9c8a43d` — Mark done / Mark read are offered only where the API would take them: `canRelease` comes from the server's `releaseGap` (:464).

The two commits the brief names, `7a95db8` and `cc88ce0`, changed the **bank question page** (`features/questions/bank-question.tsx`) and added one prop to `authoring-workspace.tsx`: a question is read in its own tab, and the editor asks before leaving for the reading view. Neither touched `apps/api/src/assignments/**` or the section screen's rules.

What is wrong with it

1. `apps/api/src/questions/section-work.service.ts:339` — `unchecked` never calls `requireOnPaper`, unlike `check` (`:323`) and `sendBack` (`:356`), and its `updateMany` keys on `{testId, questionId}` only. The reader of section A sending `DELETE /admin/sections/<testId>/<sectionA>/questions/<questionOnSectionB>/check` clears another section's tick — and `reopenReadingIfUnchecked` is not called, so section B's released reader is never pulled back.
2. Same file `:384` — `fixed` likewise skips `requireOnPaper`, so the typist of section A can mark fixed a question sent back on section B of the same test.
3. `apps/api/src/assignments/assignments.service.ts:937-946` — `handedOnArrival` selects `finalizedAt` and never reads it; the return is `new Date()`. Dead select.
4. `apps/api/src/assignments/section-thread.service.ts:70` — `editComment` checks authorship and nothing else, so an admin whose row carries `replacedAt` can still reword an old message, against the rule stated at `:134`. Migration `20260919140000`'s header still claims the table is "append-only by construction"; `20260920120000` made it editable and the comment was never revised.
5. `apps/api/src/assignments/assignments.controller.ts:127` — `remove(id)` takes no `testId`, and `assignments.service.ts:312` reads only the row's own stamps. Any admin with `TEST_MANAGEMENT` WRITE can delete a work-free assignment on any test in the institute.
6. `apps/api/src/assignments/unread-questions.ts:6` — `uncheckedOn` builds a correlated `reviews: { none: ... }`; there is no `PaperQuestion(testId, baseConfigSectionId)` index (only `@index([baseConfigId, baseConfigSectionId])` and `@unique([testId, order])`), so the `releaseCounts` `groupBy` at `:628` must lead on one of those two.

Importing in bulk

Where it lives

`apps/api/src/imports/` (the three roster importers), `apps/api/src/common/importing/` (the sheet-reading half both share), `apps/api/src/questions/question-import*.ts` (the question importer, which also owns `ImportLog.target`), `apps/admin/src/features/imports/` (four screens).

- `apps/api/src/imports/imports.controller.ts` — 12 routes: `template/preview/errors/commit` for students, event candidates, program students.
- `apps/api/src/imports/imports.service.ts` — planning context loads, per-row writes, and the `ImportLog` lifecycle (`withRun / openRun / closeRun`).
- `apps/api/src/imports/student-import.ts` — the roster row validators, and `planRoster / readContact / missingColumnErrors` reused by the other two.
- `apps/api/src/common/importing/sheet-reader.ts` — `xlsx/CSV` sniffing, the zip-bomb ceiling, floating pictures, one `CsvTable` for every path.
- `apps/api/src/common/importing/date-cell.ts` — `toDate`, normalising any date shape a spreadsheet produced.
- `apps/api/src/questions/question-import.service.ts` — the two-step question run: preview stores the file and opens the run, commit re-reads it.
- `apps/api/src/questions/question-import.ts` — the question row validators and the in-file/in-bank dedup.
- `apps/api/src/audit/audit.service.ts` — the only read-back of a run (`listImports`, `importFile`).

What it is for

IACE's rosters and question bank arrive as spreadsheets from branch offices, so every bulk entry point is upload-then-review rather than a form. Four things import: students (full profile + access), event candidates, program enrolments, and questions. Every importer is the same two-request shape — a preview that writes nothing and reports errors per line, then a commit that re-reads the same input and applies only the rows that pass. The admin never sees a half-applied file without being told which lines were refused, and can download the refused lines with an `Errors` column appended.

FLOW — PREVIEW (STUDENTS)

- 1 POST `/imports/students/preview`, multipart field `file`
`apps/api/src/imports/imports.controller.ts:55` `ImportsController.preview()`
decorators: `@RequiresFeature(STUDENT_MANAGEMENT, READ)`, `@Post`, `@HttpCode(OK)`, `@UseInterceptors(FileInterceptor(IMPORT_FILE_FIELD))`; class-level `@Actors(ADMIN)`.
- 2 Global guards run in order `JwtAuthGuard` → `ActorGuard` → `FeaturePermissionGuard` → `SuperAdminGuard` → `ThrottlerGuard`
`apps/api/src/app.module.ts:77-82`
an admin with the READ grant.
- 3 Multer buffers the body with `fileSize: BODY_LIMIT_IMPORT`, `files: 1`
`apps/api/src/common/importing/upload.ts:7-13`
the sheet is in memory, never S3, at preview.
- 4 A missing file becomes a field error on `file`
`apps/api/src/common/importing/upload.ts:24-30` `requireFile()`
a `Buffer`.
- 5 Zero-length refused, then the first 4 bytes are matched against `PK\x03\x04`; non-zip bytes containing a NUL in the first 512 are refused as "not a spreadsheet", otherwise parsed as CSV
`apps/api/src/common/importing/sheet-reader.ts:79-92` `readUploadedTable()`
- 6 For a workbook, the central directory's declared uncompressed total is summed without inflating anything and refused over 200MB
`apps/api/src/common/importing/sheet-reader.ts:98-105`, ceiling at `upload.ts:21` `declaredInflationBytes()`
- 7 `ExcelJS.xlsx.load`, then the named sheet or the first; row 1 becomes `headers` (lower-cased, spaces/underscores/hyphens stripped) plus `labels` as written
`sheet-reader.ts:107-135`
`normaliseHeader()` at `csv.ts:109`.
- 8 Each later row is read cell by cell: rich text joined, a formula's result not its text, a `Date` through `toDate`, numbers without exponent form; all-empty rows dropped;
`line` is the sheet's own row number
`sheet-reader.ts:139-172`
a `CsvTable`.
- 9 `contextFor` loads four queries in parallel
active branches, active exam codes, active program codes, and only the students whose mobiles the file mentions `apps/api/src/imports/imports.service.ts:334-383`
an `ImportContext`.

- 10 File-level refusals first: empty file, missing required column (named as the sample names it), more than `IMPORT_MAX_ROWS` (1000)
apps/api/src/imports/student-import.ts:72-90, 154-173
`planRoster()` ; on any of these, zero rows are planned.
- 11 Per row, in order: `readName` (`personNameSchema` : ≥ 2 chars, ≤ 120 , letters/marks/space/ `.` / `'` / `-` only), `readContact` (`mobileSchema` : normalised then `^[6-9]\d{9}$` , plus "already on line N" for a repeat inside the file), `readStudentType` (ONLINE/OFFLINE/NON_IACE, any casing or hyphenation), `readBranch` (active branch by canonical **name**, plus `studentBranchBlocker` for a type/branch pairing the screens refuse; blank is legal only for NON_IACE), `readCourses` (against `EXAM_COURSES`), exam and program codes against the loaded catalogs, `readProfile` , and `reachesNothing` (a row granting no course, exam or program is refused)
student-import.ts:321-378
- 12 The date cell: `toDate` tries ISO-like, named month (either order), numeric parts, then an Excel serial ≥ 61 ; nothing matched means the value is returned unchanged
apps/api/src/common/importing/date-cell.ts:76-88 `readProfile` then validates the `**date**` with `dobSchema` (real calendar day, not future, year $\geq$ `EARLIEST_BIRTH_YEAR`) and only says "is not a date we can read" when `toDate` changed nothing
`student-import.ts:277-286` .
- 13 `actionFor` $\rightarrow$ `skip` on any error, else `update` if the mobile is known, else `create` ; `willReceiveDefaultPin` is false for a student who already chose a PIN
student-import.ts:176-179, 374
- 14 Response envelope wrapped by `ResponseInterceptor`
{ success: true, data: { rows[], summary: { total, willCreate, willUpdate, invalid }, fileErrors[] }, meta }
`student-import.ts:142-151` . Nothing was written: no `ImportLog` , no S3 object (`imports.service.ts:418`).
- 15 `POST /imports/students/errors` posts the same file back and gets a one-sheet workbook of only the refused lines, original columns plus an `Errors` column (a re-uploaded errors file's old column is replaced)
apps/api/src/common/importing/rows-with-errors.ts:11-38

Question preview differs in two ways: it reads the sheet named `Questions` when the workbook has several (`question-import.service.ts:367-369`), and it **does** open a run — see the next trace.

FLOW — COMMIT

- 1 `POST /imports/students/commit`
apps/api/src/imports/imports.controller.ts:75 `commit()`
`@RequiresFeature(STUDENT_MANAGEMENT, WRITE)` (preview is READ), `@HttpCode(OK)` , `FileInterceptor` ; the caller arrives via `@CurrentUser()` .
- 2 The file is re-read and **re-planned from scratch** against the live context
imports.service.ts:72-75
the client's plan is never trusted and permission is re-judged.
- 3 `openRun` creates the `ImportLog` row (`feature`: `STUDENT` , `source`: `SHEET` , `actorId` , `total` , `status`: `PREVIEWED` , `errors.fileErrors` if any), uploads the bytes to `imports/student/<logId>.xlsx` , then writes `fileS3Key` back
imports.service.ts:419-441
one durable run row + one S3 object.
- 4 Every PIN the run will issue is hashed up front in parallel batches (argon2 ≈ 13 ms each)
imports.service.ts:86-92, apps/api/src/auth/pin/starting-pin.service.ts:37-52
- 5 Row loop, **not in a transaction**: a `skip` row is passed over, an existing student is `update` d (name only if present, access merged, branch disconnected when the sheet has none, profile upserted), a new one `create` d
imports.service.ts:94-109, 246-284
`Student` rows and `StudentProfile` rows written one at a time.
- 6 Counts accumulate on the in-memory outcome; `failed` was seeded with `plan.summary.invalid` , so **invalid rows are never written and are counted as failed**
imports.service.ts:83, 399-402
- 7 A throw anywhere in the loop closes the run as `FAILED` with the counts so far and the error message in `errors.message` , then rethrows
imports.service.ts:404-412
- 8 On success, `recordImportRows` writes one `RowActionLog` per touched student (`changed`: `null` , `importLogId` set); a failure here is logged and swallowed
imports.service.ts:452-455, apps/api/src/audit/audit.service.ts:146-165
- 9 `closeRun` writes the counts, `status` , `finishedAt`
imports.service.ts:457-473
- 10 Only now are the PINs texted, one SMS per minted row, each failure logged not thrown
imports.service.ts:113, starting-pin.service.ts:55-70
- 11 Response: { `...plan.summary` , `created` , `updated` , `skipped` } .
- 1 `POST /imports/questions/commit` with { `importLogId` }
apps/api/src/questions/question-import.controller.ts:121-128 `@RequiresFeature(QUESTION_MANAGEMENT, WRITE)`; or `POST /admin/sections/:testId/:sectionId/import/commit`
apps/api/src/questions/section-work.controller.ts:100-109 — `@RequiresFeature(QUESTION_AUTHORIZING, WRITE)` + `@Audit(QUESTION, CREATE)` , which first asserts the caller holds the section's open, unreplaced, unfinalized typing row (`section-work.service.ts:210` , `typingRow()` at :573-581).
- 2 **Guard 1 (c64d6f8)**
apps/api/src/questions/question-import.service.ts:258-260: if (log.actorId !== actorId) throw NOT_FOUND. ImportTarget.actorId became required and the bank route now passes @CurrentUser(); before, the bank's commit passed no actor and the check was `actorId !== undefined && ...`, so ****any admin with question-management write a super admin included**
could commit a run someone else previewed*, landing a typist's section run loose in the bank (and making the typist's own commit fail as already imported).
- 3 **Guard 2 (da1e4fb)**
apps/api/src/questions/question-import.service.ts:125-127: if (log.target !== null && log.target !== targetOf(into.section)) throw formRefusal(CONFLICT, ...). Preview writes target as 'bank' or `$(testId)/$(baseConfigSectionId)` (:86, targetOf()) at :388). This stops the ****same**** admin cross-committing: a section run into the bank loses its typing job, a bank run through a section lands under a typist's row. A null target (previewed before the migration) still commits anywhere, by design

- 4

openRun

 also refuses a run whose feature is not `QUESTION` , whose `fileS3Key` is null, or whose status is already `COMMITTED` (`CONFLICT` , "already imported")
:253-265
and returns the stored bytes.
- 5

Re-read, re-plan with the `ImportRowEdit` overlay laid over the file, keep only `action === 'create'` rows
:128-132
- 6

Pictures are uploaded before any row
:134, 194-199
- 7

One `$transaction` with `TX_LIMITS.BULK` (maxWait 10s, timeout 120s): per row three statements
Question, its `QuestionVersion` v1, then the `currentVersionId` pointer `with assignmentId: into.section?.assignmentId`
:137-157 .
- 8

`recordImportRows` (swallowed on failure), then the `ImportLog` closes with `status: COMMITTED` , `finishedAt` , counts
:160-188

Program enrolment collapses step 5 to **one statement** — `UPDATE "Student" SET programs = array_append(...) WHERE id = ANY($1) AND NOT (programs @> ARRAY[$2]) RETURNING id` — so a student enrolled since the preview cannot gain the code twice (`imports.service.ts:231-234`). Event candidates `create` only unknown mobiles, as `NON_IACE` with `pinIsDefault: true` , then hand every id to `events.addCandidates` (`imports.service.ts:170-187`). Question commit is the one that takes an `importLogId` rather than a file:

The data it touches

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
<code>ImportLog.id</code>	uuid v7	run id, also the S3 key and the review - window handle	both services
<code>ImportLog.feature</code>	<code>AuditFeature</code>	<code>STUDENT</code> for all three rosters, <code>QUESTION</code> for questions	<code>imports.service.ts:427</code> , <code>question-import.service.ts:83</code>
<code>ImportLog.source</code>	<code>ImportSource</code>	always <code>SHEET</code> here (<code>INDIVIDUAL</code> / <code>SCRIPT</code> / <code>SELF_SIGNUP</code> exist unused by this module)	both
<code>ImportLog.actorId</code>	uuid?	the previewer; the whole of the question importer's authorisation	both
<code>ImportLog.fileS3Key</code>	text?	<code>imports/<feature>/<logId>.xlsx</code>	<code>import-log.ts:2</code>
<code>ImportLog.target</code>	text?	'bank' or <code>testId/baseConfigSectionId</code> ; null for every roster run and for pre-migration question runs	<code>question-import.service.ts:86</code> only
<code>ImportLog.total/created/updated/skipped/failed</code>	int	row counts	<code>closeRun</code> , question commit's update
<code>ImportLog.status</code>	text (not an enum column)	<code>PREVIEWED</code> → <code>COMMITTED</code> \	<code>FAILED</code> both
<code>ImportLog.errors</code>	jsonb?	<code>{ fileErrors?: string[], message?: string }</code> — the file-level refusals, and the thrown error's message on a <code>FAILED</code> roster run	<code>imports.service.ts:432, 465-470</code>
<code>ImportLog.startedAt/finishedAt</code>	timestamptz(3)	run window	default now / <code>closeRun</code>
<code>ImportRowEdit.importLogId, line</code>	composite PK	one corrected or set-aside previewed question row	<code>question-import.service.ts:243-247</code>
<code>ImportRowEdit.draft</code>	jsonb?	a full <code>QuestionDraft</code> with image <code>src</code> stripped to keys; null when the row was only left out	<code>saveRow</code>
<code>ImportRowEdit.leftOut</code>	bool	row excluded from the import	<code>leaveOutRow</code>
<code>RowActionLog.*</code>	—	one thin row per written entity, <code>changed: null</code> , <code>importLogId</code> set	<code>recordImportRows</code>
<code>Student.*</code> , <code>StudentProfile.*</code> , <code>EventCandidate</code> , <code>Student.programs</code>	—	what a roster run writes	<code>writeRow</code> , raw <code>UPDATE</code> , <code>events.addCandidates</code>
<code>Question</code> , <code>QuestionVersion</code>	—	what a question run writes, v1 only	question commit transaction

No Redis key in this module. S3: `imports/student/<logId>.xlsx` , `imports/question/<logId>.xlsx` , plus content-addressed picture keys from `importedImageKey` .

What must already be true

an authenticated active admin actor; `STUDENT_MANAGEMENT` READ to preview and WRITE to commit a roster, `QUESTION_MANAGEMENT` for the bank importer, `QUESTION_AUTHORING` plus the section's own open typing row for a section import. At least one active `Branch` , and the exam/program codes a row names must exist and be active. A program import needs the program to exist and be offered (`programs.assertUsable`); an event import needs the event (`events.detail`). `BODY_LIMIT_IMPORT` must be set (schema-validated; `.env.example:141` = `10mb`), and S3/MinIO reachable — a commit cannot open a run without it. Migration `20260930150000_an_import_run_records_where_it_lands` for `target` .

Why it is shaped this way

- **The file is the record, not a client-side plan.** Both services re-read and re-plan at commit (`imports.service.ts:70` , `question-import.service.ts:58`) rather than accepting the preview back. The alternative — post the plan — was rejected because a client that can send a plan can send any plan, and the bank may have gained the same question in between. Cost: the roster importer parses and re-queries twice per run.

- **A roster preview stores nothing; a question preview stores everything.** Rosters keep no run for an abandoned preview (imports.service.ts:418), so the roster commit must carry the file again. The question importer needs a review window (corrections, left-out rows, drawable pictures) so it must persist at preview — at the cost of an ImportLog row plus an S3 object per abandoned preview.
- **target is a string, not two columns.** A nullable TEXT added nothing to rewrite and needed no backfill, so a pre-deploy run keeps null and still commits anywhere. The alternative, a testId + sectionId pair with FKs, would have forced a decision about pre-existing runs and a table rewrite.
- **Invalid rows are dropped, never queued.** There is no "fix later" state for a roster row: it is reported, excluded, and downloadable with its reasons. Only the question importer has ImportRowEdit, and only because its rows are expensive to re-type.

Which screen calls it

(all apps/admin; nothing in apps/exams or mobile imports)

ENDPOINT	UI FILE	SPA ROUTE
GET/POST /imports/students/{template,preview,errors,commit}	apps/admin/src/features/imports/import-students.tsx:30-43	/students/import (constants.ts:77)
/imports/events/:eventId/candidates/*	apps/admin/src/features/imports/import-event-candidates.tsx	/cohorts/events/:id/import (constants.ts:135)
/imports/programs/:code/students/*	apps/admin/src/features/imports/import-program-students.tsx	/cohorts/programs/:code/import (constants.ts:138)
/imports/questions/* and /admin/sections/:testId/:sectionId/import/*	apps/admin/src/features/imports/import-questions.tsx:51-56, 99-102 — one page, the section variant chosen from the URL params	/questions/import (constants.ts:95) and /authoring/assignments/tests/:testId/sections/:sectionId/import (constants.ts:111)
GET /admin/audit/imports, GET /admin/audit/imports/:id/file	apps/admin/src/features/admins/audit.tsx:282 (AuditImportsPage), :161-175 (ImportFileAction)	/audit/imports (constants.ts:147)

All four preview/commit pairs share useErrorRows (apps/admin/src/features/imports/use-error-rows.ts) and ImportViewProps from packages/ui.

Is it optimal

- Roster preview: 4 queries, flat in row count — three small catalog reads plus one Student findMany on mobile IN (...) using Student_mobile_idx. optimal as read; not measured.
- Roster commit (students): 4 + one write per non-skip row, each a separate round trip; a nested profile upsert makes it 2-3 statements. A 1000-row roster is ~1000 round trips **with no enclosing transaction** — a partial roster survives a mid-run failure. This is the module's one real N+1, and it is why IMPORT_MAX_ROWS is 1000 (the comment at contracts/src/imports.ts:207 blames argon2, not the writes).
- Program commit: optimal — one UPDATE ... WHERE id = ANY(\$1), deliberately replacing one statement per student (5412bfe).
- Question preview/commit: planTable loads the taxonomy once and plans twice, the first pass only harvesting keys; dedup is one findMany with OR of two IN lists, hitting Question_stemHash_idx and the questionCode unique index. Commit is 3 statements per created row inside one 120s transaction — 1000 rows is ~3000 statements, the largest single transaction in the module.
- GET /admin/audit/imports : 2 queries in a transaction (page + count) plus one name lookup, ordered startedAt desc, id desc. **No index serves it** — ImportLog has only @index([feature, startedAt]) (prisma/schema.prisma:998), and the list filters on actorId for a non-super admin with no feature filter, so it is a sort over the whole table. Harmless at today's row count; read, not measured.

What it leans on

the response envelope and ApplicationException / ErrorCodes (controllers never build a body); JwtAuthGuard / ActorGuard / FeaturePermissionGuard via @Actors / @RequiresFeature / @RequiresAnyFeature; formRefusal for a refusal the whole form owns (apps/api/src/common/form-refusal.ts); packages/contracts for the column definitions that generate the sample workbook and drive the parser (STUDENT_IMPORT_COLUMNS, QUESTION_IMPORT_COLUMNS), so a template can never document a format the parser rejects; the field schemas (mobileSchema, personNameSchema, dobSchema, emailSchema, tagSchema, questionCodeSchema) shared with the single-entity forms; common/exporting for the errors workbook; StorageService (the one S3 path); AuditService for row actions; StartingPinService for hashing and SMS. No queue, no Redis, no WebSocket.

What changed recently

(git log -15 over apps/api/src/imports, apps/api/src/common/importing)

- e66ab91 — preTestReady / profileCompleted stopped being stored flags the importer maintained; readiness is now derived from the profile, so writeRow no longer sets them.
- 14f8e4c — the roster importer's own mobile/name reading collapsed into the one readContact the candidate and program importers already used.
- 3e7ba94 / 8d32f14 / bc3cd5f — a NON-IACE row may leave the branch column blank; a re-import adds access and never removes it, and the preview, the guard and the write were made to agree on that merged set (student-import.ts:343-345).
- c93a386 — added the three .../errors routes: post the previewed file back, get the refused lines with an Errors column.
- cefc97a — the zip-bomb check moved before ExcelJS.xlsx.load instead of after it.

What is wrong with it

- apps/api/src/questions/question-import.service.ts:174 vs :182 — the same run reports two different skipped values: the client gets total - created (which folds in invalid rows), the ImportLog gets duplicates + leftOut. Import a 3-row sheet with 1 create and 2 invalid rows: the screen says 2 skipped, the audit list says 0 skipped / 2 failed.
- apps/api/src/imports/imports.service.ts:116 vs :83 — commitStudents returns skipped: invalid while the run row records those same rows as failed with skipped: 0. commitProgramStudents is worse: the run gets skipped: alreadyEnrolled (:222) and the response skipped: invalid (:242), so for a file of 5 already-enrolled students and 1 bad row the screen says 1 skipped and the audit row says 5.
- apps/api/src/common/importing/import-log.ts:2-3 — the stored key always ends .xlsx, even when the admin uploaded a CSV, and audit.service.ts:254 derives the download filename's extension from that key while audit.controller.ts:80 sets the xlsx Content-Type. Upload a .csv roster, commit, then download

the run's file from `/audit/imports`: a CSV arrives named `.xlsx`.

- `ImportLog.target` is written and never read back — neither `toImportSummary` (`audit.service.ts:276–294`) nor `importLogSchema` (`packages/contracts/src/audit.ts:160–177`) carries it, so the audit screen cannot distinguish a bank run from a section run even though the column exists to make that distinction enforceable.
- `apps/api/src/imports/imports.service.ts:425–440` — the run row is created, *then* the file uploaded, *then* `fileS3Key` written. If S3 is down the commit throws out of `openRun` before `withRun`'s try block, leaving a `PREVIEWED` row with a null key, no `finishedAt`, and nothing written; the audit list shows it as an eternally-in-progress run with no file.
- `apps/api/src/questions/question-import.service.ts:79–96` — a question preview opens a run and uploads the file before any file-level check, so a sheet with a missing required column still leaves an `ImportLog` row (`total: 0`) and an S3 object. Nothing ever cleans those up; the roster importers write nothing on preview for exactly this reason (`imports.service.ts:418`).
- `apps/api/src/common/importing/date-cell.ts:71–73` — when both numeric parts could be a month, day-first wins. `03/04/2001` imports as 3 April. Correct for this institute, silently wrong for a sheet exported by a US-locale tool, and nothing warns.

Part III — Who may do what

Three separate questions get confused with one another, and this part keeps them apart. **Who are you** is authentication — a token, a session in Redis, an OTP or a four-digit PIN. **What kind of actor are you** is a guard — student or admin, and for an admin which feature keys they hold. **What may you reach** is resolution — the catalog a particular student can see, which depends on their branch, their enrolment, their programs, their grants and their event candidacies, and which is recomputed rather than stored.

The last chapter is the exception to all of it: the live-ops screens, where an admin reaches into a sitting that is currently happening.

Signing in

Where it lives

`apps/api/src/auth/` (+ `guards/`, `otp/`, `pin/`); route vocabulary in `common/security/`, limits in `common/throttling/`, keyspace in `redis/redis.keys.ts`, wire types in `packages/contracts/src/auth.ts`.

- `auth/auth.controller.ts` — nine `/auth/*` routes; all but `logout` and `me` are `@Public()`.
- `auth/auth.service.ts` — every flow's orchestration; owns no table, reads `Student`.
- `auth/otp/otp.service.ts` — code minting, cooldown, three daily caps, delivery, verify.
- `auth/pin/pin.service.ts` — argon2id hashing, the logout ladder, the setup ticket.
- `auth/session.service.ts` — Redis sessions, rotation with replay detection, revocation.
- `auth/token.service.ts` — access and refresh JWTs under two separate secrets.
- `auth/guards/jwt-auth.guard.ts` — the global guard; builds `request.user`.

What it is for

Two unrelated identity tables log in two unrelated ways: a student proves a

mobile once by OTP, sets a 4-digit PIN, and uses that PIN daily; an admin signs in by email OTP every time. There is no password column anywhere in `prisma/schema.prisma`. The module issues a short access token plus a rotating refresh token and keeps the session in Redis, so a revoke bites before the access token expires. A 4-digit secret is defended by an escalating Redis logout ladder, not by entropy, and paid sends are capped three ways.

FLOW — STUDENT OTP LOGIN (SIGNUP / PIN RESET)

- 1 `POST /auth/student/otp/request`, `@Public()` `@OtpRequestRateLimit()`
`apps/api/src/auth/auth.controller.ts:37-46` `requestStudentOtp()`
`body` via `ZodBody(requestStudentOtpSchema)`, `request.ip` passed through.
- 2 The global JWT guard sees `IS_PUBLIC_KEY` and returns `true` without setting `request.user`
`apps/api/src/auth/guards/jwt-auth.guard.ts:21-25`
anonymous allowed; only the `otpRequest` throttler applies, 15/min on tracker `ip:<ip>` (`apps/api/src/common/throttling/throttling.module.ts:26-38`).
- 3 Cooldown read first
`apps/api/src/auth/otp/otp.service.ts:40-48` `request()`
`otp:cooldown:student:<mobile>` `TTL > 0` → `RATE_LIMITED` with `details.retryAfterSec`.
- 4 Three daily counters INCR'd, each given 86400s TTL if it has none
`otp.service.ts:50-54,94-138` `countTowardsDay()/assertIpDailyBudget()/assertGlobalDailyBudget()`
over `OTP_MAX_PER_DAY` (5), 500/IP, or `floor(300000/18)=16666` globally → `RATE_LIMITED`.
- 5 CSPRNG code minted, HMAC-SHA256'd under `JWT_ACCESS_SECRET` and stored
`otp.service.ts:60-64,206-215`
`otp:student:<mobile>` = `{codeHash, createdAt}`, TTL `OTP_TTL_SEC` 300s. The code itself is never stored.
- 6 Stale attempts key deleted, cooldown set
`otp.service.ts:66-69`
`otp:cooldown:student:<mobile>` = `"1"`, TTL `OTP_RESEND_COOLDOWN_SEC` 45s.
- 7 WhatsApp-then-SMS, or SMS
`otp.service.ts:141-166` `deliver()`
a throw becomes `SERVICE_UNAVAILABLE`, and counters and cooldown are deliberately not refunded (`otp.service.ts:71`).
- 8 `{sent, expiresInSec, resendAfterSec, codeLength, devCode?}` wrapped as `{success:true,data,meta:{requestId}}`
`apps/api/src/common/response.interceptor.ts:24-32`
`devCode` only when `OTP_SENDER=CONSOLE` in development (`otp.service.ts:87`).
- 9 `POST /auth/student/otp/verify`, `@Public()` `@AuthRateLimit()`
`auth.controller.ts:49-57`
the `auth` throttler, 120/min per IP.
- 10 Hash compared constant-time
`otp.service.ts:169-196` `verify()`
a miss INCRs `otp:attempts:*` (TTL cloned from the OTP key); at `OTP_MAX_VERIFY_ATTEMPTS` 5 both keys are deleted and the code is `RATE_LIMITED`, below it `OTP_INVALID` with `details.attemptsRemaining`. A hit deletes the OTP key, its cooldown and its attempts key (:199-203) — single use, resend immediate.
- 11 Standing checked
`auth.service.ts:72-78`
`findFirst({mobile, deletedAt:null})`; `isActive === false` → `FORBIDDEN`.

- 12 32 random bytes peppered with `PIN_PEPPER` and stored
`apps/api/src/auth/pin/pin.service.ts:130-139` `issueSetupToken()`
`pin:setup:<mobile>`, TTL `PIN_SETUP_TTL_SEC` 600s. Returns `{setupToken, expiresInSec, pinAlreadySet}` — no session yet.
- 13 `POST /auth/student/pin/set`, `@Public()` `@AuthRateLimit()`
`auth.controller.ts:60-74` `deviceFrom(request, body.device)` reads `x-client/x-device-name` (`auth/device.ts:48-58`)
- 14 Ticket compared and deleted
`pin.service.ts:142-153`
mismatch or expiry → `OTP_EXPIRED`. PIN then hashed `argon2id`, 19456 KIB / t=2 / p=1, over the peppered value (`pin.service.ts:63-65`).
- 15 Branch
`auth.service.ts:103-114`
an existing live row → `update({where:{id, isActive:true}})` (`auth.service.ts:297-309`; P2025 → `FORBIDDEN`), else create with `pinIsDefault:false` and `studentType: NON_IACE`. `Student.pinHash` written.
- 16 Prior sessions revoked and the ladder cleared
`auth.service.ts:117-118`; `STUDENT_SIGNED_UP` (new rows only) and `STUDENT_PIN_RESET` emitted (:119-120)
- 17 Tokens signed
`auth.service.ts:315-327` + `auth/token.service.ts:37-54`
`randomUUID()` `sid`; access `{sub,actor,sid}` under `JWT_ACCESS_SECRET` exp 15m; refresh `{sub,actor,sid,jti}` under `JWT_REFRESH_SECRET` exp 30d.
- 18 Session written
`apps/api/src/auth/session.service.ts:54-70` `create()`
`session:student:<id>:<sid>` = `{refreshTokenHash, deviceName, ip, userAgent, createdAt, lastSeenAt, client}`, TTL 30d; `sessions:student:<id>` SADD + EXPIRE 30d. First, `replaceSameKind()` tombstones and revokes same-kind sessions (:199-215). Returns `{tokens, identity}` (`auth.service.ts:122-123`).

Flow trace: admin login — the brief's premise is wrong: **no admin password, no argon2 on the admin path**. `grep password prisma/schema.prisma` returns nothing; `argon2` appears only in `auth/pin/`.

- `POST /auth/admin/otp/request`, `@Public()` `@AuthRateLimit()` — `auth.controller.ts:90-98`.
- Admin by email (`@unique`, `prisma/schema.prisma:828`) — `auth.service.ts:210-218` — unknown → `ADMIN_NOT_REGISTERED` + `fieldErrors.email`; deliberately discloses whether an email is an admin.
- `otp.request(ADMIN, email)` — `otp.service.ts:39-91` — same cooldown and `otp:admin:<email>` TTL 300s, but **no daily counters**: `otp.service.ts:50` meters students only. Channel EMAIL (`otp.service.ts:219-225`).
- `POST /auth/admin/otp/verify`, `@Public()` `@AuthRateLimit()` — `auth.controller.ts:100-109`.
- `otp.verify(ADMIN, ...)` — same attempt cap, same single-use delete.
- Admin re-read for id, then `identityOf()` — `auth.service.ts:230-232` — deliberately NOT gated on `isActive` (`auth.service.ts:229`): a deactivated admin signs in and can do nothing.
- `isSuperAdmin = admin.isActive && admin.isSuperAdmin` — `apps/api/src/admins/admins.service.ts:85`. The stored column (`prisma/schema.prisma:831`) is itself written from `role === SUPER_ADMIN` on create and update (`admins.service.ts:158,185`), so `AdminRole` is the source and the boolean its cache. `permissions` returns `{}` for a super admin because `can()` bypasses (`admins.service.ts:86`, `packages/contracts/src/admins.ts:134-141`).
- `issue()` → the same pair and `session:admin:<id>:<sid>` TTL 30d. `replaceSameKind` is skipped for admins (`session.service.ts:51`), so an admin may hold many sessions.

FLOW — MOBILE 4-DIGIT PIN LOGIN

- `POST /auth/student/login`, `@Public()` `@AuthRateLimit()`
`auth.controller.ts:77-86`
`body {mobile, pin, device?}`.
- Lockout checked before anything else
`pin.service.ts:88-97`
`pin:lock:<mobile>` present → `PIN_LOCKED` + `details.retryAfterSec`.
- Exactly one student resolved
`auth.service.ts:185-188` `findFirst({mobile, deletedAt:null, include:{profile}})`. Uniqueness is the hand-written partial index `Student_mobile_live_key ON "Student" ("mobile") WHERE "deletedAt" IS NULL` (`prisma/migrations/20260820000000_schema_realignment/migration.sql:1152`), which Prisma cannot express
hence `findFirst` (`auth.service.ts:95`).
- `pinHash` carries no unique constraint
`prisma/schema.prisma:761` is a bare `String?`, and it appears in the migrations only as "pinHash" TEXT, (...20260820000000.../migration.sql:345). Confirmed: students share digits freely, and the PIN is checked only against the row the mobile resolved to
- `argon2` verify of the peppered PIN, or an equal-cost decoy verify when the number is unknown or has no PIN
`auth.service.ts:189-191` + `pin.service.ts:78-81`
`burnVerifyTime()` keeps timing flat.
- Failure → `pin:attempts:<mobile>` INCR (900s TTL on the first)
`pin.service.ts:100-114`
at `PIN_MAX_ATTEMPTS` 5, `pin:lockouts:<mobile>` INCR picks the rung from `PIN_LOCKOUT_STEPS_SEC` `[900,3600,86400]`, sets `pin:lock:<mobile>` for it, re-expires `pin:lockouts` to rung + `PIN_LOCKOUT_DECAY_SEC` 86400 so waiting one out climbs. Error `PIN_INVALID` 'Incorrect mobile number or PIN' for both wrong PIN and unknown number.
- `isActive === false` → `FORBIDDEN`
`apps/api/src/auth/auth.service.ts:197`
- `clearFailures()` deletes all three PIN keys
`pin.service.ts:117-123`
- `issue()` → tokens + session. `device.client` from `x-client`, so a MOBILE sign-in replaces the previous MOBILE session and leaves WEB alone
`session.service.ts:199-215`

Flow trace: a request on a protected route — guards run in registration order (`apps/api/src/app.module.ts:77-82`), after `RequestIdMiddleware` / `AuditContextMiddleware` and inside `RequestBudgetInterceptor` → `ResponseInterceptor` → `AuditInterceptor` .

1. `JwtAuthGuard` — `guards/jwt-auth.guard.ts:20` — rejects a missing or non-`Bearer` header (`UNAUTHENTICATED` 'Missing access token', :29), a bad or expired signature (`token.service.ts:73-79`), a payload failing `accessTokenClaimsSchema` ('Malformed token', `token.service.ts:59`), a session gone from Redis (:33-35 → `SESSION_REPLACED` if a tombstone exists, else `UNAUTHENTICATED`), and an admin whose row is gone (:42-43). On success writes `request.user = {id, actor, sessionId, isSuperAdmin, isActive, permissions}` — grants read from Postgres per request, never from the token (:37).
2. `ActorGuard` — `guards/actor.guard.ts:11` — no-ops without `@Actors(...)` ; else `FORBIDDEN` when `user.actor` is not listed.
3. `FeaturePermissionGuard` — `guards/feature-permission.guard.ts:22` — no-ops without `@RequiresFeature` / `@RequiresAnyFeature` / `@RequiresExport` ; rejects a student, then a deactivated admin **before** the super-admin bypass (:33-39), then `!keys.some(can(...))` , then `required.export && !can(user, DATA_EXPORT)` .
4. `SuperAdminGuard` — `guards/super-admin.guard.ts:11` — no-ops without `@RequiresSuperAdmin()` ; deactivated admin first, then anyone not an active super admin.
5. `ThrottlerGuard` last, so an authenticated caller counts as `<actor>:<id>` and only anonymous traffic shares `ip:<ip>` — `common/throttling/rate-limit.ts:12` — 1-minute windows in `ratelimit:<name>:<tracker>` ; a Redis failure returns `ALLOWED` rather than blocking a hall mid-exam (`rate-limit.ts:17-22`).
6. **FEATURE_KEYS** is owned at `packages/contracts/src/admins.ts:20` — ten keys, code-only; `AdminFeaturePermission.featureKey` is a bare `String` with no FK (`prisma/schema.prisma:853`). The decorators citing it are in `common/security/decorators.ts:34-51` , deliberately in the shared kernel, not in `auth/` .

FLOW — REFRESH AND REVOKE

1

POST /auth/refresh , @Public() @AuthRateLimit()

auth.controller.ts:113-119

body is {refreshToken} only.

2

Verified under JWT_REFRESH_SECRET , narrowed to {sub,actor,sid}

token.service.ts:63-71

3

One column read, not an identity

auth.service.ts:355-363 mayRefresh()

student select:{isActive} ; admin identityOf() !== null .

4

Next refresh token signed with a fresh jti

token.service.ts:46-54

5

Compare-and-swap rotation, 3 tries

session.service.ts:79-120 rotate()

missing session → throwEnded() ; a token the session never held and that is not the previous one inside the 60s REFRESH_RETRY_GRACE_SEC (:23) → session revoked, UNAUTHENTICATED ; a retry inside the grace keeps the consumed hash in siblingRefreshTokenHashes (capped 8); three lost CAS rounds → CONFLICT , not a sign-out.

6

Session JSON rewritten at TTL 30d, index re-SADD'd and re-EXPIRE'd

session.service.ts:107-112

7

Access token re-signed; {accessToken, refreshToken, expiresInSec} returned

auth.service.ts:262-268

8

Client side: a 401 fires one refreshTokens() with timeout and backoff; a refusal ends the session

packages/contracts/src/client/core.ts:159-186

9

POST /auth/logout (not @Public)

auth.controller.ts:122-126 sessions.revoke(actor, id, sessionId)

session.service.ts:130-133 — DEL the one key + SREM from the index. This device only.

Every revocation call site (`graft callers revokeAll --depth 2` , `graft grep "sessions.revoke"`):

- `revoke` : `auth.service.ts:272` (logout); `session.service.ts:98` (replay detected in `rotate`); `session.service.ts:184` (`signOutOther` , i.e. `DELETE /me/sessions/:id`); `session.service.ts:213` (`replaceSameKind`).
- `revokeAll` : `auth.service.ts:117` (`setStudentPin`), `auth.service.ts:170` (`changeStudentPin` , `POST /me/pin`), `auth/student-sessions.listener.ts:17` (`STUDENT_DEACTIVATED`), `auth/admin-sessions.listener.ts:17` (`ADMIN_DEACTIVATED`). One Lua `EVAL` over the index set (`redis/redis.service.ts:190`), so a racing sign-in lands wholly before or after.
- **Does not revoke**: `POST /auth/refresh` on a deactivated account (refuses, leaves the Redis session), student soft-delete and erasure, an admin role or permission change (covered instead by the guard's per-request read), and a PIN change does not end the session it was made from — it issues a fresh one.

The data it touches

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
Student.mobile	String , partial-unique live	the login identifier	students module; <code>auth.service.ts:107</code> on signup
Student.pinHash	String?	argon2id encoded hash of the peppered PIN; no unique index	<code>auth.service.ts:104,163,301</code> ; <code>pin/starting-pin.service.ts</code> for imports
Student.pinIsDefault	Boolean default true	institute-issued PIN not yet replaced; surfaces as <code>identity.hasDefaultPin</code>	set false at <code>auth.service.ts:108,165,302</code>
Student.isActive	Boolean	login gate, read on login, refresh and <code>/auth/me</code>	students module
Student.deletedAt	Timestamptz(3)?	soft delete; releases the mobile for reuse	students module
Admin.email	String @unique	admin login identifier	admins module
Admin.isSuperAdmin	Boolean	the bypass; derived from <code>role</code> on every write	<code>admins.service.ts:158,185</code>
Admin.isActive	Boolean	zeroes <code>isSuperAdmin</code> and <code>permissions</code> in <code>identityOf</code>	admins module

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
AdminFeaturePermission(adminId, featureKey, level)	composite PK	one grant; featureKey a bare String matched against code-owned FEATURE_KEYS	admins module

OTP, sessions, tokens and lockouts are **Redis only** — `grep '^model' prisma/schema.prisma` has no Session, Otp or Token model, and `auth.types.ts:5` says so.

KEY	VALUE	TTL
otp:{student\	admin}<identifier>	{codeHash, createdAt} OTP_TTL_SEC 300s
otp:cooldown:{actor}<identifier>	"1"	OTP_RESEND_COOLDOWN_SEC 45s
otp:attempts:{actor}<identifier>	counter	the OTP key's remaining TTL
otp:daily:<mobile> / otp:daily:ip:<ip> / otp:daily:global	counters	86400s
pin:setup:<mobile>	peppered ticket	PIN_SETUP_TTL_SEC 600s
pin:attempts:<mobile>	counter	900s (ladder rung 1)
pin:lock:<mobile>	"1"	900 / 3600 / 86400s by rung
pin:lockouts:<mobile>	counter	rung + 86400s
session:{actor}<subjectId>:<sessionId>	StoredSession (auth.types.ts:6-20)	JWT_REFRESH_TTL 30d
sessions:{actor}<subjectId>	SET of session ids	30d, refreshed on create and rotate
session-replaced:{actor}<subjectId>:<sessionId>	{replacedBy: ClientKind\	null} 30d
ratelimit:<name>:<tracker>	counter	60s

StoredSession : refreshTokenHash (sha256 hex of the live token); siblingRefreshTokenHashes? (≤8 hashes minted by grace-window retries); previousRefreshTokenHash? + rotatedAt? (the 60s retry window); deviceName / ip / userAgent / client (device binding); createdAt ; lastSeenAt .

Daily OTP cap — 5 codes per mobile per rolling 24h (OTP_MAX_PER_DAY , env.schema.ts:114), 500 per IP (:116), and a platform kill switch of OTP_GLOBAL_DAILY_BUDGET_PAISE 300000 ÷ NOTIFICATION_COST_SMS_PAISE 18 = 16666 sends (:118,186 ; otp.service.ts:125-138). Admin OTPs count towards none of them.

What must already be true

JWT_ACCESS_SECRET , JWT_REFRESH_SECRET and PIN_PEPPER ≥24 chars and not the

.env.example placeholders (config/env.schema.ts:244,276-285); TRUST_PROXY_HOPS > 0 in production, refine-enforced (:237,272) — without it every student shares the proxy's address and the 500/day per-IP cap becomes a platform cap. Redis is declared non-optional (auth.module.ts:21). Student_mobile_live_key must be deployed, or two live rows share a mobile and findFirst picks arbitrarily. An admin row must pre-exist — no self-signup, first super admin by pure SQL.

AuthController mounts only on the CORE API role (auth.module.ts:33).

Why it is shaped this way

- Sessions in Redis at TTL = refresh lifetime, not a Session table: expiry needs no sweeper, the hot path is one EXISTS . The table alternative puts a Postgres write on every refresh during a 6-8K live test.
- Rotation stores only sha256 of the live token, one previous hash and ≤8 siblings. A blacklist grows without bound; no rotation cannot detect theft. Cost: a token older than the 60s grace signs the client out rather than retrying.
- pinHash carries no unique constraint by rule — a @unique on 4 digits caps the platform at 10,000 students. Cost: brute force is stopped by the Redis ladder, not by entropy.
- isSuperAdmin is stored and re-derived from role on every write, not computed at read time. Deriving it live saves a column but makes a role rename a behaviour migration; cost is two columns that can disagree if anything writes role outside admins.service .

Which screen calls it

/login in both SPAs (apps/exams/src/lib/constants.ts:9 , apps/admin/src/lib/constants.ts:70).

Student OTP request / verify / pin set / login → apps/exams/src/features/auth/login.tsx:196,243,312,130 and apps/mobile/app/login.tsx:185,240,299,122 . Admin OTP request/verify → apps/admin/src/features/auth/login.tsx:87,141 . GET /auth/me → the three providers/auth.ts files plus apps/exams/src/features/exam/system-check.tsx:18 and apps/mobile/src/components/tests/system-check.tsx:16 . POST /auth/logout → apps/admin/src/providers/auth.ts:25 , apps/exams/src/providers/auth.ts:17 , apps/mobile/src/providers/auth.ts:16 . POST /auth/refresh → no UI; packages/contracts/src/client/core.ts:169 , fired on any 401. POST /me/pin , GET /me/sessions , DELETE /me/sessions/:id → apps/api/src/me/me.controller.ts:182,192,200 ; apps/mobile/app/change-pin.tsx calls the first.

Is it optimal

all read from code; nothing measured here.

- student/otp/request : 0 Postgres, 6-8 Redis trips plus one send. Each daily counter costs an extra TTL call because INCR/EXPIRE is not one Lua call (otp.service.ts:94-98) — ~3 avoidable trips, off any hot path.
- student/login : 2 Postgres queries, not 1 — include:{profile} is a second query, relationJoins not being enabled (prisma/schema.prisma:14-16). mobile = \$1 AND deletedAt IS NULL is exactly Student_mobile_live_key ; Student_mobile_idx and the trigram index are redundant for it. ~15-20ms is the argon2 verify (pin.service.ts:18), and a miss pays the same by design.
- Protected route: student 0 Postgres + 1 Redis EXISTS (optimal — jwt-auth.guard.ts:41 short-circuits with a literal); admin 2 Postgres (row + permissions relation), uncached.

- `GET /auth/me`: **4 queries** for an admin — the guard's `identityOf` and `me()` → `loadIdentity` → `identityOf` read the same rows twice (`jwt-auth.guard.ts:40` , `auth.service.ts:375`). Student: 3.
- `refresh`: 1 Postgres query (student) or 2 (admin); 2–5 Redis ops by CAS retries. `revokeAll`: one Lua `EVAL` over the index set, no `KEYS` scan (`redis/redis.service.ts:190`) — optimal.

What it leans on

`common/response.interceptor.ts` + `AllExceptionsFilter`, so no

handler builds an envelope and every error is an `AppException` mapped from `ErrorCodes`; `ZodBody` for bodies and `fieldErrors`; `packages/contracts` for `AUTH_ROUTES`, every schema, `ActorTypes`, `FEATURE_KEYS` and `can()` — one rule the guard and every screen read; `common/security` decorators as the route vocabulary; `common/throttling` with its Redis storage; `RedisService` (`setJson` / `replaceJson` / `deleteIndexedSet`); `common/messaging` `MESSAGE_SENDER`, so `auth` decides to send and owns no channel; `DomainEventBus` for the PIN events and the two deactivation listeners; `common/same-hex` for constant-time comparison; `app-kit` `create-auth.tsx` plus `contracts/src/client/core.ts` for token storage and the 401-refresh loop.

What changed recently

(`git log --oneline -15 -- apps/api/src/auth ...`)

- `62c882d` — deleted `AuthService.verifyStudentPin` and with it `POST /me/erasure`; student self-erasure is gone, erasure is super-admin-only from the admin portal. The PIN-change path uses the private `verifyCurrentPin` and was untouched.
- `3b71758` — `AuthenticatedUser` now extends `AdminAuthority` instead of restating `isActive` / `isSuperAdmin` / `permissions`. Same runtime shape, one definition.
- `f472a4d` — removed a duplicated `throwEnded` from `jwt-auth.guard.ts` (it now calls `sessions.throwEnded`), and made `rotate` re-SADD/re-EXPIRE the session index so a refreshing session keeps its place and `revokeAll` cannot lose it.
- `d258c56` — `mayRefresh` reads `select:{isActive}` instead of building a whole identity, cutting a mid-paper refresh to one column.
- `e66ab91` — `preTestReady` / `profileCompleted` stopped being stored columns; `studentIdentity` derives them via `readinessOf(student.profile)`, which is why every identity read carries `include:{profile}` and therefore a second query.

What is wrong with it

- `auth/otp/otp.service.ts:185` — the attempt-cap branch deletes the OTP and attempts keys but **not** `otp.cooldown:*`, while the error says 'Request a new code'. Five wrong codes within 45s of the send: the client is told to request a new code and the next request is refused `RATE_LIMITED` 'Please wait Ns'. The success path (:199–203) does clear the cooldown — the two exits disagree.
- `config/env.schema.ts:186` — `NOTIFICATION_COST_SMS_PAISE` is `nonnegative()`, so `0` is valid, and `otp.service.ts:129` divides by it, making `maxSends` `Infinity`. `NOTIFICATION_COST_SMS_PAISE=0` silently switches off the platform-wide OTP spend kill switch.
- `guards/jwt-auth.guard.ts:40` + `auth.service.ts:375` — `identityOf` has no per-request memoization, so `GET /auth/me` reads the same admin row and permissions twice on any admin page load.
- `auth.service.ts:242–269` — `refresh` refuses a deactivated account but never revokes its Redis session. Deactivate an admin whose `ADMIN_DEACTIVATED` listener throw was swallowed (`admin-sessions.listener.ts:19–21`) and the sessions survive: the holder keeps working until the 15m access token expires, and the key sits for 30d.
- `guards/actor.guard.ts` and `guards/feature-permission.guard.ts` still run on `@Public()` routes, because `JwtAuthGuard` returns true without setting `request.user`. No live route combines `@Public()` with `@Actors` / `@RequiresFeature`, so this bites only the next one that does — it would answer `FORBIDDEN` to every anonymous caller.
- `auth.service.ts:210–218` — `requestAdmin0tp` sits on the shared `auth` bucket (120/min per IP) and is metered by none of the three daily caps (`otp.service.ts:50`). A script enumerating emails gets a free, unbudgeted "is this an IACE admin" oracle at 120 tries a minute per address.
- `auth.service.ts:295–296` — two stacked one-line doc comments on `replaceStudentPin`; the first ("Announces a PIN change...") describes `announcePinReset`, not this method.

What a student may reach

Where it lives

`apps/api/src/access/`, read from `me/`, `attempts/`, `notifications/`.

- `access/access-resolver.service.ts` — the one reach rule; owns the held catalog and the Redis epoch.
- `access/access-cache.listener.ts` — two domain events become one `INCR`.
- `access/test-series.service.ts` — `TestSeries` CRUD + branch list; emits the bust.
- `access/student-grants.service.ts` — owns `StudentGrant`: grant, revoke, export, admin reach read.
- `access/access.controller.ts` — four admin controllers; no student route lives here.
- `me/me.controller.ts:91` — `GET /me/catalog`, the only student entry.
- `prisma/schema.prisma:877–975` — `Program`, `TestSeries`, `StudentGrant`, `Event`, `EventCandidate`, `TestProgramUnlock`.

What it is for

A test reaches a student only through a series, and a series reaches them by one

of four automatic routes (its `kind`) plus a deliberate per-student grant. This module holds that rule in one function and answers three questions with it: what a student may see (the catalog), what they may read about (the brief), and what they may begin (the start guard). It answers it backwards too — who does one series reach — for opening notifications, absentee reports and the admin's reach count. Nothing a student may do is stored; it is derived on every read.

- 1 A student screen mounts `catalogQuery`
`packages/app-kit/src/student-queries.ts:87 createStudentQueries()`
`GET /me/catalog , key ['me','catalog'] ( :18 ), no staleTime .`
- 2 Route matched
`me/me.controller.ts:90-93 MeController.catalog()`
`@Get('catalog') under @Controller('me') + @Actors(ACTOR_TYPES.STUDENT) ( :66-67 ); CORE role only ( me.module.ts:24 ).`
- 3 Global guards
`app.module.ts:77-82 JwtAuthGuard → ActorGuard → FeaturePermissionGuard → SuperAdminGuard → ThrottlerGuard`
`subject is user.id ; no id in the path.`
- 4 Pass-through
`me/me.service.ts:47-49 MeService.catalog()`
`straight to the resolver.`
- 5 Two reads together
`access/access-resolver.service.ts:150 catalog()`
`Promise.all([standingOf, shared]).`
- 6 Student facts read live
`:273-279 standingOf()`
`student.findUnique with STANDING_FACTS ( :110-118 ). Measured: 4 SQL statements (Student, StudentGrant, EventCandidate, Attempt); Prisma 6.19.3 without relationJoins ( schema.prisma:14-16 ) issues one batched IN (...) per relation, so no N+1.`
- 7 Absence
`:288-289 isPresent()`
`isActive && deletedAt === null , else NOT_FOUND ( :151 ).`
- 8 Facts folded
`:291-300 reachFrom()`
`{ isTestBlocked, currentBranchId, programs, enrolledCourses, grantedSeries:Set, events:Set }.`
- 9 Sitings folded
`:283`
`attempts arrive attemptNo asc , VOIDED excluded, so the highest attemptNo wins per test.`
- 10 Epoch read
`:234-236 catalogEpoch()`
`Redis GET access:catalog:epoch ( redis/redis.keys.ts:53 ); counterOf ( :346-349 ) coerces a non-integer or negative to 0 .`
- 11 Hold checked
`:243 shared()`
`the held copy serves only if held.epoch === epoch and age < CATALOG_MAX_AGE_MS (15 min, :31 ).`
- 12 Miss → rebuild
`:250-254 testSeries.findMany({ where: { isEnabled: true }, select: SHARED_SELECT })`
`measured: 7 SQL statements (TestSeries, ExamStage, Exam, Test, TestProgramUnlock, BaseConfig, BaseConfigSection). The promise is held ( :247-261 ): readers share one build, a rejection unholds itself.`
- 13 Reach filter
`:316,:302,:369-386 seriesSources()`
`FREE always; STANDARD needs branch ∈ branchIds and exam.course ∈ enrolledCourses ; PROGRAM needs programCode ∈ programs ; EVENT needs eventId ∈ candidacies; a grant appends GRANT on any kind.`
- 14 Rows shaped
`:444-488 toResolvedTest()`
`the three scoped* helpers narrow a scoped test; opensFor ( :463-471 ) takes the earliest matching TestProgramUnlock.opensAt , else Test.opensAt ; sorted byOrderThenId .`
- 15 Sequential gate
`:490-502 project()`
`waiting = findIndex(!isSat(status)) ; index <= waiting reachable, -1 opens all.`
- 16 Clock applied
`:508-511 projectTest()`
`canStart = reachable && testIsOpen(opensAt, now) , which allows a 5-minute grace ( contracts/src/access.ts:374-376 ).`
- 17 Envelope
`apps/api/src/app.module.ts:75 ResponseInterceptor`
`data = { testBlocked, series:[{ id, name, examStage/null, kind, sequentialTests, tests:[{ id, title, durationSec, sectionCount, totalQuestions, order, opensAt, attemptStatus, canStart }]} } ( contracts/src/access.ts:388-454 ).`

What is cached — Nothing per student. Only the *shared* series list: one `Promise<SharedSeries[]>` in **per-process memory** on the service instance (:141), tagged with the epoch it was built under, valid until the epoch moves or 15 minutes pass. Redis holds one integer under the literal key `access:catalog:epoch` , **no TTL**. One sibling cache of the same shape holds a single test's paper row keyed by `testId` + epoch, with no age ceiling (`attempts/attempt-paper.service.ts:169-180`).

- 1 Admin write, e.g. `PUT /admin/test-series/:id/branches`
`access.controller.ts:174-180 @Audit(TEST_SERIES, UPDATE) + @RequiresFeature(BRANCH_TEST_MANAGEMENT, WRITE)`

- 2 Row written
apps/api/src/access/test-series.service.ts:244 setBranches()
TestSeries.branchIds updated.
- 3 Fact published
:250 events.emit(DOMAIN_EVENTS.ACCESS_CATALOG_CHANGED, { testSeriesId })
- 4 Bus
common/events/domain-event-bus.ts:12-18 DomainEventBus.emit()
in-process EventEmitter2, fire-and-forget; a throwing listener is logged, the producer unaffected.
- 5 Listener
access-cache.listener.ts:16-19 onCatalogChanged()
the testSeriesId reaches only a log line; the bust is global.
- 6 Bust
access-resolver.service.ts:229-231 invalidateAll()
redis.client.incr('access:catalog:epoch'). INCR, never DEL. It must increment: held copies are compared for equality (:243) and counterOf(null) === 0 (:346-349), so a DEL would be a no-op for any process holding epoch 0 — every fresh process before the first bump.
- 7 Failure swallowed
access-cache.listener.ts:29-32 bump()
a Redis error logs only; the write landed and every held copy ages out inside 15 minutes.
- 8 Next read anywhere
:243 shared()
epoch differs → rebuild. Equality, not >: a Valkey reset sends the counter backwards and later bumps must still bite (:242).
- 9 Same counter busts the paper-row cache
apps/api/src/attempts/attempt-paper.service.ts:170 testOf()
- 10 Other emitters
test-series.service.ts:176/201/219/250/285 (create, update, remove, setBranches, toggleBranch **only when a row moved**); tests/tests.service.ts:281 (title or examTemplate change) and :302 (delete), both behind inTheCatalog(test) i.e. status === ACTIVE (tests/test-rules.ts:45-46); tests/offering.service.ts:232 (series move, same guard) and :444 (announce, from the Offer save at :265)
- 11 EXAM_STAGE_CHANGED busts the same key
access-cache.listener.ts:21-24
from configs/exam-stages.service.ts:128, configs/base-configs.service.ts:229, configs/exams.service.ts:120.
- 12 A grant, revoke, block, branch transfer or enrolment change emits nothing
student-grants.service.ts:109-166 has no emit and the students module is absent from the emitter list. None is needed: every per-student fact is read live at step 6. student.access_changed was deleted in 4ead5d3 because nothing heard it

FLOW — THE START GUARD

- 1 POST /me/tests/:testId/attempt
attempts/attempts.controller.ts:63-66 @SittingRateLimit(), @Post('tests/:testId/attempt'), @HttpCode(HttpStatus.OK), on @Controller('me') + @Actors(ACTOR_TYPES.STUDENT) (:50-51)
- 2 Sitings + test in one wave
attempts/attempts.service.ts:74-81 start()
- 3 Resume short-circuits above the guard
:82-92
an IN_PROGRESS sitting reopens with no access check; a stale resume id → SITTING_ENDED (:94).
- 4 Guard
:98 access.assertCanStart(studentId, testId)
the first refusal, so a nonexistent test reads like an unreachable one. attempt-paper.service.ts:126 calls it too (GET /me/tests/:testId/paper), so a candidate pays it twice.
- 5 Two reads
apps/api/src/access/access-resolver.service.ts:158
4 statements + 1 Redis GET, plus 7 on a cold hold.
- 6 Cond. 1
:162-164
student absent (unknown id, isActive false, deletedAt set) → undefined.
- 7 Cond. 2
:163
isTestBlocked → undefined, short-circuiting before projection so the message cannot become "not opened yet".
- 8 Cond. 3
:327 startableTest()
no held series holds this id (unknown test, isEnabled false, status !== ACTIVE) → undefined.
- 9 Cond. 4
:329 sourcesOf(holder, standing).length === 0
no route into that one series → undefined.
- 10 Cond. 5
:331,:499 project()
in a sequentialTests series, a test past the first unsat one → canStart false.

11	Cond. 6	:510 <code>projectTest()</code> the effective opening is more than 5 minutes away → <code>canStart false</code> .
12	Pass	:165 <code>if (test?.canStart) return;</code>
13	Refusal	:167, :514-518 <code>AppException(ErrorCodes.FORBIDDEN, refusalFor(test, now)).</code> **Every condition throws the same code, FORBIDDEN** ; only the message differs 'This test has not opened yet' for a reached-but-early test, 'This test is not open to you right now' for conditions 1-4. <code>AllExceptionsFilter</code> (<code>app.module.ts:72</code>) maps the code to its status.

What `7b8b5c1` changed (from the diff) — `assertCanStart` previously did `catalogOf(series, standing, now).series.flatMap(r => r.tests).find(r => r.id === testId)` : it resolved and projected *every* series the student reaches and every ACTIVE test in each, then kept one. It now calls `startableTest` (:321–334), which finds the single series holding the id, checks reach on that series alone, and projects only it. That series is still projected whole, and must be — `sequentialTests` decides a test by its index among its siblings and every earlier sitting's status. The block short-circuit was left above the projection deliberately, or a blocked student would read "not opened yet". The verdict is unchanged: `apps/api/test-db/access-resolver.db.test.ts:652–675` pins the guard's answer for every test in a sequential series to the `canStart` the same student's catalog reports. The commit names the removed cost as the 3.7–6.9 ms resolve, paid twice per candidate.

The data it touches

MODEL.COLUMN	TYPE	HOLDS	WRITTEN BY
<code>Student.isActive</code> , <code>.deletedAt</code>	bool, timestamptz(3)	read as absent	students
<code>Student.isTestBlocked</code>	bool	listed, nothing startable	students
<code>Student.currentBranchId</code>	uuid?	STANDARD branch match	students
<code>Student.programs</code>	text[] GIN	<code>Program.code</code> s: PROGRAM arm + unlocks	students / import
<code>Student.enrolledCourses</code>	<code>ExamCourse[]</code> GIN	STANDARD course match	students / import
<code>StudentGrant.studentId</code> , <code>.testSeriesId</code>	uuid, composite PK, <code>@index</code> on the second	the grant itself	<code>StudentGrantsService.grant</code>
<code>StudentGrant.createdAt</code> , <code>.createdById</code>	timestamptz(3), uuid?	"Granted" column, name on export	same
<code>EventCandidate.eventId</code> , <code>.studentId</code>	uuid, composite PK	the EVENT arm	events
<code>TestSeries.kind</code>	enum	which automatic arm applies	<code>TestSeriesService</code>
<code>TestSeries.branchIds</code>	uuid[] GIN	STANDARD reach	<code>setBranches</code> , <code>toggleBranch</code>
<code>TestSeries.programCode</code> , <code>.eventId</code>	text?→ <code>Program.code</code> , uuid?	PROGRAM / EVENT reach	<code>TestSeriesService</code>
<code>TestSeries.isEnabled</code>	bool	false → invisible to every student	<code>TestSeriesService</code> only
<code>TestProgramUnlock.testId</code> , <code>.programCode</code> , <code>.opensAt</code>	composite PK + timestamptz	per-program stagger on one opening	tests / offering
<code>Attempt.status</code> , <code>.testId</code> , <code>.attemptNo</code> , <code>.voidedAt</code>	enum, uuid, int, timestamptz	<code>attemptStatus</code> + the sequential gate	attempts

`Test.scopeRef` is the only jsonb read, passed opaquely through `scopeRefOf` into the three `scoped*` helpers — the resolver never looks inside it. Redis: `access:catalog:epoch` , one integer, no TTL, `INCR` on write, `GET` on read. That is the module's entire Redis surface.

Scoped by branch, by enrolment, or both — Both, and two more. Four independent automatic routes plus the grant, OR'd: FREE reaches every live student unconditionally; STANDARD needs branch **and** course together (:375–377); PROGRAM is by `Student.programs` alone, branch-blind; EVENT by `EventCandidate` alone, branch-blind; a `StudentGrant` adds reach to any kind. `Student.enrolledExams` is **not** consulted in the resolve — only `enrolledCourses` , via the stage's exam.

What must already be true

`TestSeries.isEnabled = true` (:251) and `Test.status = ACTIVE` (:35), or the test is invisible to everyone; `isActive = true` , `deletedAt = null` ; `isTestBlocked = false` to start. STANDARD also needs a branch in `branchIds` , an `ExamStage` → `Exam.course` , and that course enrolled. Redis reachable — the epoch `GET` at :235 is unguarded. Admin writes need `TEST_MANAGEMENT` , `BRANCH_TEST_MANAGEMENT` or `STUDENT_MANAGEMENT` at `WRITE` ; `Program` writes need `@RequiresSuperAdmin()` .

Why it is shaped this way

- Derived, never stored. The per-student catalog in Valkey that existed before `821f8db` was ~184 MB of a ~280 MB dataset at 8,000 candidates, nearly all of it the same series content copied per student, in the store live sittings depend on. Deriving costs 4 statements.
- One global epoch, not per series or per student. Per-student counters never expired and needed a hand-run deletion on deploy. The price is that any series write rebuilds everything in every process; `81aa696` paid it down by making a draft test's rename, delete and move not bump at all.
- `canStart` and `opensAt` come from the clock on every request and are not part of the hold. Caching availability would need a bump when a clock passes an opening, which nothing can emit — which is why a 15-minute-old hold is still correct about time.
- The hold is a promise, not rows. Holding rows meant every request arriving during a cold read paid its own read (`f472a4d`); a rejected promise is unheld, so a failure is not inherited.

Which screen calls it

`GET /me/catalog` is called from one shared query (`app-kit/src/student-queries.ts:87`)

on four `apps/exams` screens — `dashboard.tsx:66` (`/`), `tests/tests.tsx:41` (`/tests`), `tests/series.tsx:65` (`/series/:seriesId`), `tests/test-about.tsx:90` (`/tests/:testId/about`) — and re-exported for `apps/mobile` (`src/lib/queries.ts:8`). In `apps/admin` , `/students/:id` 's Series tab calls `GET /admin/students/:id/series` (`features/students/student-detail-series.tsx:124`), `POST .../grants` (`:135`) and `DELETE .../grants/:seriesId` behind a `ConfirmDialog` (`:147`); the grant picker calls `GET /admin/test-series?notReachedBy=` (`components/access-picker.tsx:97`). `GET /admin/test-series/:id/grants/export` — UI not verified . The exam screen does not refetch the catalog on exit; it invalidates with `refetchType: 'none'` (`app-kit/src/exam/use-exam-view.ts:164`).

Is it optimal

- `GET /me/catalog` , warm hold: **4 SQL + 1 Redis GET — measured**, by attaching Prisma's `query` event to the real `STANDING_FACTS` select against the dev database. No `N+1`. Indexes **read, not measured** (the dev DB has 1 student and 0 enabled series, so every plan there is a seq scan): `Student_pkey` , `StudentGrant_pkey` prefix on `studentId` , `EventCandidate_studentId_idx` , `Attempt_studentId_createdAt_idx` prefix for the filter — the `attemptNo asc` sort is not served by it.
- Cold hold adds **7 statements — measured**, once per process per epoch, not per request. `821f8db` measured the rebuild at 9.4 ms and the warm catalog at 1.15 ms / 4 statements, against 5.4 ms / 9 before.
- `assertCanStart` : the same 4 + 1, one series projected since `7b8b5c1` . Still called twice per candidate (start and paper prefetch) — the duplication that commit names and does not remove.
- `reachableTest` (the brief): 3 statements (`REACH_FACTS` , no `attempts`), but it still flat-maps every reached series to find one test (`:174–178`); the `startableTest` narrowing was not applied here.
- `studentsReaching` / `audienceCount` : 2 queries; the `grants.some` arm uses `StudentGrant_testSeriesId_idx` , PROGRAM the `Student_programs` GIN, STANDARD `Student_currentBranchId_idx` + `Student_enrolledCourses` GIN. `studentsReaching` pulls every matching id into memory uncapped.
- Payload: every reached series with every ACTIVE test, unpaged. Not measured.

What it leans on

`ResponseInterceptor` + `AllExceptionsFilter` for the envelope and

`AppException` mapping; the four auth guards (`app.module.ts:72–82`), so no controller checks an actor itself; `@Audit` + `AuditContext` , so a write with no `:id` in its path still names its row; `ZodBody` / `ZodQuery` with the contracts schemas; `DomainEventBus` , so a write never awaits a bust; `redisKeys` , so the epoch key is a constant; `packages/contracts` for `testIsOpen` , `isSat` , `testBucket` and the three `scoped*` helpers — the same functions the SPAs call, which is why a card's label and the server's verdict cannot disagree (`930b6e4`); `common/pagination` , `search-terms` , `exporting` , `series-order` ; `packages/ui` `DataTable` , `BadgeList` , `ConfirmDialog` , `RowActions` , `TruncatedText` . No queue, no BullMQ job.

What changed recently

- `7b8b5c1` — the start guard stopped building the whole catalog to answer about one test.
- `f472a4d` — `testPaper` 's test-row read became held on the catalog epoch; the paper caches hold the promise, not the rows.
- `87107bd` — a one-branch PATCH finding the branch already on (or off) stopped auditing and stopped bumping; the `UPDATE` now returns `branchIds` before and after from inside the locked row.
- `de7b625` — added the one-branch `PATCH .../branches` (`array_append` / `array_remove` server-side) so two toggles from the same stale list both land; the whole-list `PUT` stayed for "switch on everywhere".
- `930b6e4` — the in-order gate dropped its private `FINISHED` notion for contracts' `isSat` ; `opensAt > now` became `testIsOpen` , which knows the 5-minute grace.
- `2a21e74` — deleted the Prisma-side copy of the reach rule (`reachableBy` + `REACH_SELECT`); the admin read is `seriesReachedBy` off the held copy. `audienceOf` stayed, being the real other direction.

What is wrong with it

- `access-resolver.service.ts:283` + `:492` — the sittings map keeps the **latest** attempt and the sequential gate reads `!isSat` . Input: a sequential series where test 1 is `EVALUATED` and the student starts a retake of it. The map now says `IN_PROGRESS` , `waiting` becomes index 0, and **test 2's `canStart` flips back to false** — a retake re-locks the later papers.
- `:235` — `catalogEpoch()` awaits `redis.client.get` with no catch. `counterOf` guards a *corrupt* value but not a *thrown* one, so a Redis outage 500s `catalog()` , `assertCanStart()` and `reachableTest()` even though the in-memory copy is valid and in date. The write side guards exactly this (`access-cache.listener.ts:29`); the read side does not.
- `:331` — `startableTest` passes `standing.isTestBlocked` into `project` , but `:163` already returned `undefined` for a blocked student, so that argument is unreachably always `false` .
- `:171–182` — `reachableTest` still projects every reached series to answer about one id, the shape `7b8b5c1` replaced in the guard. Paid on every open of a student's About screen.
- `:372` / `:409` vs `prisma/schema.prisma:897` — the schema comment says "FREE also reaches by an enrolled course", but both directions treat FREE as reaching **every live student** regardless of course. The comment is wrong about the code.
- `:407` — `audienceOf` filters `deletedAt / isActive` only, never `isTestBlocked` . Input: a test-blocked student is counted in the admin's `reachedCount` (`test-series.service.ts:134`) and is sent the "this test is open now" notification (`notifications/test-opening.service.ts:66`) for a test their own catalog marks unstartable.
- `:203–215` — `studentsReaching` returns every id uncapped and `notifications/test-opening.service.ts:66` chunks them into one `$transaction` ; at this module's 6–8K scale one opening writes 6–8K notification rows in a single transaction.

The roster, the admins and the catalog

Where it lives

`apps/api/src/admins/` , `apps/api/src/branches/` , `apps/api/src/students/` , `apps/api/src/configs/` , `apps/api/src/config/` .

- `apps/api/src/admins/admins.service.ts` — admins, the code-owned feature list, grants, and `identityOf` (the facade the JWT guard reads on every admin request).
- `apps/api/src/students/students.service.ts` — the roster: create, patch, the two toggles, and the DTO mapping that signs document URLs.

- `apps/api/src/students/anonymize.ts` — the column-by-column erasure payload; erasure is anonymisation, never a delete.
- `apps/api/src/configs/base-configs.service.ts` — a stage's blueprint: create, locked-aware update, clone-to-evolve, delete.
- `apps/api/src/configs/base-config-rules.ts` — `UNFROZEN_FIELDS`, `locksOutEdit`, the shape rules the DB also enforces.
- `apps/api/src/configs/exams.service.ts` + `exam-stages.service.ts` — the catalog's two levels, and the `assertUsable` gates everything else enters through.
- `apps/api/src/branches/branch-rules.ts` — the virtual-branch singleton rules and the student-type/branch agreement.
- `apps/api/src/config/api-role.ts` — `API_ROLE` decides which controllers a container registers (`onRole([API_ROLES.CORE], ...)` in all four modules).
- `prisma/migrations/20260820000000_schema_realignment/migration.sql:1206-1420` — the `base_config_*_guard` functions and triggers.

What it is for

Everything a test needs to exist *before* anyone sits one: who the staff are and what they may touch, who the students are and where they sit, and the exam blueprint a paper inherits its shape from. The roster is the identity table student auth resolves against; the catalog (`Exam` → `ExamStage`) is the spine that base configs, series and tests all hang off. `apps/api/src/config/` is not an HTTP module at all — it is the zod-validated env plus the container-role switch.

FLOW — CREATE AN ADMIN

- 1 `POST /admin/admins`
`apps/api/src/admins/admins.controller.ts:41` `create()`, decorated `@Audit(AUDIT_FEATURE.ADMIN, AUDIT_ACTION.CREATE)` `@Post('admins')` with `@Body(new ZodBody(createAdminSchema))` and `@CurrentUser()`; the class carries `@Controller('admin')` `@Actors(ActorTypes.ADMIN)` `@RequiresSuperAdmin()` (`:27-30`)
request accepted.
- 2 Global guards run in order `JwtAuthGuard`, `ActorGuard`, `FeaturePermissionGuard`, `SuperAdminGuard`, `ThrottlerGuard`
`apps/api/src/app.module.ts:77-82`
four gates before the handler.
- 3 `JwtAuthGuard` verifies the token, checks the Redis session exists, then reads authority *live*
`apps/api/src/auth/guards/jwt-auth.guard.ts:33-43`
`request.user = { id, actor, sessionId, isSuperAdmin, isActive, permissions }.`
- 4 `AdminsService.identityOf`
`apps/api/src/admins/admins.service.ts:65-88`
`isSuperAdmin: admin.isActive && admin.isSuperAdmin`, and `permissions` is `{}` for a super admin or for a deactivated one. The derivation is `isActive AND isSuperAdmin`, nothing else.
- 5 `SuperAdminGuard`
`apps/api/src/auth/guards/super-admin.guard.ts:20-28`
a deactivated admin gets the "account has been deactivated" FORBIDDEN first; a non-super admin gets "Only a super admin can change this".
- 6 `createAdminSchema` normalises
`packages/contracts/src/admins.ts:177-189`
email trimmed and lowercased *before* `z.email()`, `role` defaulting to `ADMIN_ROLES.ADMIN`.
- 7 Pre-check `findUnique({ where: { email } })`
`admins.service.ts:142-147`
a duplicate raises `CONFLICT` with `fieldErrors.email`; the unique index is still the real arbiter.
- 8 `const preset = ROLE_PERMISSION_PRESET[input.role]`
`packages/contracts/src/admins.ts:96-115`
`TYPIST` → `{QUESTION_AUTHORIZING: WRITE}`; `PROOFREADER` → `{QUESTION_PROOFREAD: WRITE, QUESTION_MANAGEMENT: READ}`; `ADMIN` → six keys; `SUPER_ADMIN` → `{}`.
- 9 `$transaction(..., TX_LIMITS.SHORT)`
`apps/api/src/admins/admins.service.ts:151`
transaction boundary opens.
- 10 `tx.admin.create`
`:152-161`
writes `email`, `fullName`, `role`, `isSuperAdmin: input.role === ADMIN_ROLES.SUPER_ADMIN`, `createdById` taken from the token not the body (`admins.controller.ts:46`).
- 11 `tx.adminFeaturePermission.createMany`
`:164-167`
one row per preset entry, skipped entirely when the preset is empty (super admin).
- 12 Commit, then `toAdminDto(row, preset)`
`:172, :311-322`
response `{ id, email, fullName, role, isSuperAdmin, isActive, createdAt, permissions }`, wrapped by `ResponseInterceptor` into `{ success: true, data, meta }`.

Schema comments on `AdminRole`, and whether the code matches. `prisma/schema.prisma:829-830`: *"Seeds the grants at create, labels the person, and decides isSuperAdmin on every write."* — accurate; `admins.service.ts:158` and `:185` both derive `isSuperAdmin` from `role`. The enum itself (`schema.prisma:283-288`) carries no comment. Two *other* statements of the same rule are misleading: `packages/contracts/src/admins.ts:80` says the role "presets, labels and orders, and **no guard ever reads it**", and `prisma/migrations/20260919140000_.../migration.sql:16-18` says *"The role is DISPLAY ONLY ... no guard anywhere reads it. AdminFeaturePermission stays the only thing authorisation consults."* Literally true of the `role` column, but a write to `role` sets `isSuperAdmin`, which both `SuperAdminGuard` and `can()` read — so setting role to `SUPER_ADMIN` is a full authorisation change, not a label.

What a non-super admin may do. Nothing on `/admin/admins` or `/admin/features` (class-level `@RequiresSuperAdmin`). On branches and exams: `GET` only, behind `STUDENT_MANAGEMENT:READ`; create/update/delete are `@RequiresSuperAdmin` (`branches.controller.ts:49,56,69`; `exams.controller.ts:47,55,67`; `exam-stages.controller.ts:49,57,68`). Base configs are the exception — all six routes are `TEST_MANAGEMENT` `READ/WRITE` with no super-admin gate (`base-configs.controller.ts:42-97`). Students: `READ/WRITE` on `STUDENT_MANAGEMENT`, except `:id/active` and `:id/erasure` which are `@RequiresSuperAdmin` (`students.controller.ts:159,170`), and `:id/sittings` which is `STUDENT_PERFORMANCE:READ` (`:132`).

FLOW — CREATE A STUDENT

- 1** `POST /admin/students`
`apps/api/src/students/students.controller.ts:143` `create()`, decorated `@Audit(AUDIT_FEATURE.STUDENT, AUDIT_ACTION.CREATE)` `@RequiresFeature(FEATURE_KEYS.STUDENT_MANAGEMENT, PERMISSION_LEVELS.WRITE)` `@Post()`; `class @Controller('admin/students') @Actors(ActorTypes.ADMIN) (:65-66)`
- 2** `FeaturePermissionGuard` checks `isActive` **before** the super-admin bypass
`apps/api/src/auth/guards/feature-permission.guard.ts:33-39`
`|` a deactivated super admin is refused.
- 3** `createStudentSchema`
`packages/contracts/src/students.ts:282-291`
`|` `mobile`, `fullName`, `studentType`, `enrolledExams?`, `enrolledCourses?`, `programs?`, `currentBranchId?`. No PIN field: a PIN is never accepted from a form.
- 4** `findLiveByMobile`
`students.service.ts:461-468`
`|` `findFirst({ mobile, deletedAt: null })`, because the uniqueness is partial: `CREATE UNIQUE INDEX "Student_mobile_live_key" ON "Student"("mobile") WHERE "deletedAt" IS NULL` (`prisma/migrations/20260820000000_...:1152`). A clash returns `CONFLICT` with `details.studentId`.
- 5** `ExamsService.assertUsable(codes, 'enrolledExams')`
`apps/api/src/configs/exams.service.ts:139-157`
`|` every code must exist in `Exam` and every matched row must be `isActive`.
- 6** `ProgramsService.assertUsable`
`apps/api/src/students/students.service.ts:262`
`|` same shape for `Program.code`.
- 7** `placementOf`
`students.service.ts:310-324`
`|` a named branch goes through `BranchesService.assertFitsStudent` (`branches.service.ts:49-74`), which applies `studentBranchBlocker`: `NON_IACE` must have none, `ONLINE` must be `VIRTUAL`, `OFFLINE` must not be; an `ONLINE` student naming none is placed in the single virtual branch by `onlineBranchId`.
- 8** `StartingPinService.mint([mobile])`
`apps/api/src/auth/pin/starting-pin.service.ts:37-52`
`|` `randomInt(10 ** 4)` zero-padded (`PIN_LENGTH = 4`, `packages/contracts/src/common.ts:165`), argon2id-hashed at 19 MiB / t=2 / p=1 (`pin.service.ts:18-24`), four at a time.
- 9** `prisma.student.create`
`students.service.ts:273-285`
`|` writes `mobile`, `fullName`, `studentType`, `enrolledExams`, `enrolledCourses`, `programs`, `currentBranchId`, `pinHash`: `issued.hash`, `pinIsDefault`: `true`. No `StudentProfile` row is created here.
- 10** `startingPins.announce([issued])`
`:288, starting-pin.service.ts:54-70`
`|` SMS with the cleartext PIN, after the row is durable, and `.catch` -logged so a failed send cannot roll the row back.
- 11** `this.detail(student.id)`
`:289, :163-181`
`|` re-reads with `profile` and `eventCandidacies`, plus a `HOLDS_OWN_ACCESS` count; returns `StudentDetail`.

The PIN column. There is no plaintext PIN column. `Student.pinHash` `String?` (`prisma/schema.prisma:761`) — **no** `@unique`, no unique index anywhere in the migrations; `pinIsDefault` `Boolean` `@default(true)` sits beside it. The hash is per-student argon2 with its own salt, so two students sharing PIN `1234` have different hashes regardless.

Aadhaar/PAN images are not stored. The schema says so in a comment directly above the model — *"Aadhaar/PAN images are never stored here — only the verification status"* (`prisma/schema.prisma:801-802`) — and the columns bear it out. `StudentProfile` (`:803-822`) holds exactly two document keys — `photoUrl` and `tenthMarksheetUrl` — and two booleans, `aadhaarVerified` and `panVerified`. There is no `aadhaarUrl` / `panUrl` column, and the upload target type is closed: `export type ProfileDocumentColumn = 'photoUrl' | 'tenthMarksheetUrl'` (`apps/api/src/students/student-flags.ts:5`), which is the only thing `saveDocumentKey` accepts (`students.service.ts:412`). Both keys are returned as 300-second signed URLs (`students.service.ts:42`, `:235-238`).

FLOW — A CONFIG CHANGE AGAINST TESTS ALREADY BUILT FROM IT

- 1** `PATCH /admin/base-configs/:id`
`apps/api/src/configs/base-configs.controller.ts:69` `update()`, `@Audit(AUDIT_FEATURE.BASE_CONFIG, AUDIT_ACTION.UPDATE)` `@RequiresFeature(TEST_MANAGEMENT, WRITE)`
- 2** `requireDetail`
`base-configs.service.ts:377-384`
`|` one `findUnique` with `examStage`, `_count.tests`, ordered `modules` and `sections`.
- 3** If `config.locked && locksOutEdit(input)` `→ CONFLICT` `LOCKED_CONFIG_MESSAGE`
`:166-168; locksOutEdit` (`base-config-rules.ts:32-35`) allows only `UNFROZEN_FIELDS = ['name','isDefault','isActive']` plus expected `updatedAt`
- 4** `assertScreenIsCurrent`
`:169, :447-450`
`|` `expectedUpdatedAt` mismatch throws `editedElsewhere`.
- 5** The posted paper is mapped through `paperRowsOf` and compared by `paperKeyOf` to `storedPaperRows`
`:176-178, :472-527`
`|` one mapper feeds both the writer and the comparison.
- 6** If the paper differs and `_count.tests > 0` `→ CONFLICT` `BUILT_ON_CONFIG_MESSAGE`
`:180-182`. Sections are rewritten wholesale (new ids), and a test's paper rows, assignments, scope and draw spec all name sections by id

- 7 `assertShape / assertRenderMode` judge the config as it *will* be
:184-193
so a timer switch alone cannot leave sections in a shape the DB would reject raw.
- 8 `claimEdit`
:195,:344-347
Redis `base-config:edit:<id>`, TTL 900 s (`apps/api/src/redis/redis.keys.ts:71,79`), bypassable by a super admin.
- 9 `$transaction(`, `TX_LIMITS.SHORT`) then `updateMany({ where: { id, updatedAt: config.updatedAt } })`
:199-209
optimistic claim; `count !== 1` throws before any delete.
- 10 `isDefault` is its own write after the claim, preceded by `clearDefault`
:212-215,:388-397
the partial unique `BaseConfig_default_per_stage_key` ON "BaseConfig"("examStageId") WHERE "isDefault" (`migration:1157`) forces both writes into one transaction.
- 11 Rewrite path: `deleteMany` sections, `deleteMany` modules, `writeChildren`
:217-222,:424-444
modules first, sections matched to them by position.
- 12 Audit diff over `AUDITED_CONFIG_FIELDS`
:226,:75-83
- 13 If `_count.tests > 0`, emit `EXAM_STAGE_CHANGED`
:228-230 → `AccessCacheListener.onExamStageChanged` → `AccessResolverService.invalidateAll()` → INCR `access:catalog:epoch` (`apps/api/src/access/access-cache.listener.ts:21-24`, `access-resolver.service.ts:229-231`). Every API process rebuilds its shared catalog on its next read, so a duration or navigation change reaches an already - offered test's card

`locked` is set by the first sitting, not by finalize — `apps/api/src/attempts/attempts.service.ts:154-161`, inside the attempt-start transaction: `updateMany({ where: { id, locked: false }, data: { locked: true } })`.

The `base_config.*guard` triggers (`prisma/migrations/20260820000000_schema_realignment/migration.sql` ; dropped and recreated verbatim with `uuid` parameters at `20260918120000_every_id_is_a_uuid/migration.sql:285-341,407-414`):

- `base_config_assert_section_shape` (:1206-1230) forbids four shapes: "*section %: moduleId is required when the config is SESSION_MODULE_LOCKED*", "*moduleId is only allowed when the config is SESSION_MODULE_LOCKED*", "*durationSec is required when the config is SECTIONAL_LOCKED*", "*perQuestionSec is required when the config is PER_ITEM_TIMED*".
- `base_config_section_shape_guard` (:1232-1252) and `base_config_shape_guard` (:1254-1275) are `CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY DEFERRED`, both re-reading from the table rather than trusting `NEW` — the header explains why: "*Switching a draft config to SESSION_MODULE_LOCKED needs its sections to gain a moduleId, and a section may only carry a moduleId once its config is SESSION_MODULE_LOCKED — checked per statement, neither order succeeds*" (:1191-1196).
- `base_config_assert_unlocked` (:1310-1321) takes `FOR NO KEY UPDATE` and raises "*base config % is locked; its shape cannot change*". The header is explicit that `FOR SHARE` would deadlock: "*two of those deadlock each other. At this strength they queue instead*" (:1302-1308).
- `base_config_guard` (`BEFORE UPDATE OR DELETE`, :1323-1361) raises "*base config % is locked; it cannot be deleted*" on `DELETE`, and "*base config % is locked; clone it to change its shape*" when any of 18 columns is `IS DISTINCT FROM` its old value. `locked` is in that list, so unlocking is refused while re-asserting true is a no-op. `name`, `isDefault` and `isActive` are deliberately out: "*Freezing every column would deadlock the stage on the config that locked first*" (:1288-1290).
- `base_config_child_guard` (:1363-1397 , on both `BaseConfigSection` and `BaseConfigModule`) checks **both** ends of a re-parent, lowest id first: "*Lowest id first, so two transactions re-parenting in opposite directions take the two config rows in the same order and queue instead of deadlocking*" (:1375-1376).
- `base_config_truncate_guard` (:1399-1420) refuses `TRUNCATE` on all three tables: "*truncating % would bypass the locked-config guards; delete the rows instead*".

FLOW — THE CATALOG, AND WHETHER THE RESOLVE IS CACHED

- 1 `GET /me/catalog`
`apps/api/src/me/me.controller.ts:90-92` → `MeService.catalog` → `AccessResolverService.catalog(studentId, now)` (`access-resolver.service.ts:149-153`)
- 2 `shared()`
:239-262
reads `access:catalog:epoch` (a plain `INCR` counter, no TTL, `redis.keys.ts:52-53`), and reuses the in-process `held` copy when `held.epoch === epoch && Date.now() - held.builtAt < CATALOG_MAX_AGE_MS` (15 minutes, :31). Any epoch *difference* invalidates, not just a higher one, so a Valkey reset still bites.
- 3 The held value is the **promise**, not the rows (:250), and a rejection clears it (:258-260), so concurrent cold readers share one query instead of each paying for it.
- 4 The query is one `testSeries.findMany({ where: { isEnabled: true }, select: SHARED_SELECT })` (:250-254)
one copy per process for every student
- 5 The **per-student projection is not cached**: `standingOf` + `reachedBy` + `catalogOf` run on every request (:150-153 , :273-285 , :316-318), and `canStart` is derived from the clock per read (:133).

The **hierarchy**. `ExamCourse` is an enum, not a table — `SSC | RRB | BANKING | AP_TS_POLICE` (`schema.prisma:93-98`). Then `Exam` (:294-308 , `course`, `code @unique`, `name`, `description`, `isActive`) → `ExamStage` (:311-330 , `stageKey @unique` table-wide, `order`, `mode`, `disposition`) → `BaseConfig` (:338-380) → `BaseConfigModule / BaseConfigSection` (:384-396 , :398-427). **Subject and Topic are not part of this module's hierarchy**: they are a question-bank pool (`schema.prisma:434-443` , :445-454), written only by `apps/api/src/questions/taxonomy.service.ts`, and a `BaseConfigSection.subjectId` is an optional pointer into them.

"The catalog stays verbose JSON" — **verified, with the mechanism**. `app.use(compression())` is global (`apps/api/src/main.ts:38`), with the note beside it: "*A paper is a few hundred kilobytes of JSON; the autosave ack is under the default threshold.*" So every catalog response leaves the process gzipped. A positional/array encoding would shorten the *uncompressed* bytes by removing repeated key names — which is precisely the redundancy DEFLATE's LZ77 window already collapses, since the same key strings recur every few bytes across hundreds of series and tests. The recorded measurement is ~12% post-gzip, which does not pay for an encoder, a decoder and two schemas that can drift. I did not re-run that measurement; I verified the compression middleware and that `SHARED_SELECT` emits plain named-field JSON.

The data it touches

MODEL.COLUMN	TYPE	WHAT IT HOLDS	WHO WRITES IT
Admin.email	text @unique	lowercased login identity	admins.service.ts:154
Admin.role	AdminRole	TYPIST/PROOFREADER/ADMIN/SUPER_ADMIN; presets grants, labels, decides isSuperAdmin	admins.service.ts:156,185
Admin.isSuperAdmin	bool	the bypass; derived from role	admins.service.ts:158,185
Admin.isActive	bool	reaches nothing when false, not even the bypass	admins.service.ts:207,220
Admin.createdById	uuid?	from the token, never the body	admins.service.ts:159
AdminFeaturePermission	@id([adminId, featureKey]) + level	one grant; featureKey is a code-owned string, not a FK	admins.service.ts:164,261,267 ; pruned wholesale on deactivate :222
Student.mobile	text	unique only among live rows (partial index)	students.service.ts:275 ; tombstoned to '0000000000' by anonymize.ts:15
Student.pinHash	text?	argon2id of a 4-digit PIN; no @unique	students.service.ts:282 ; nulled by anonymize.ts:19
Student.pinIsDefault	bool	institute-issued vs chosen; surfaces as hasDefaultPin	students.service.ts:283
Student.enrolledExams	String[] GIN	Exam.code values, app-validated	students.service.ts:278,359
Student.enrolledCourses	ExamCourse[] GIN	drives STANDARD-series reach with the branch	students.service.ts:279,360
Student.programs	String[] GIN	Program.code values	students.service.ts:280,361
Student.currentBranchId	uuid? onDelete: Restrict	where they sit; must agree with studentType	students.service.ts:281,362
Student.isActive / isTestBlocked	bool	sign-in vs sitting; separate facts	students.service.ts:432 , :449
Student.deletedAt / anonymizedAt	timestamptz?	account closed vs PII gone — <i>"two separate facts a regulator asks about separately"</i> (schema.prisma:767-768)	anonymize.ts:22-23
StudentProfile.photoUrl , tenthMarksheetUrl	text?	S3 keys, signed for 300 s on read	students.service.ts:416
StudentProfile.aadhaarVerified , panVerified	bool @default(false)	a verification claim; no image stored	nothing sets true (see Inconsistencies); anonymize.ts:41-42 sets false
StudentProfile.educationDetails , pastExamHistory	Json?	arrays of educationEntrySchema / pastExamEntrySchema , re-parsed on read and treated as "nothing recorded" when malformed (students.service.ts:229-231)	students.service.ts:365-371
Branch.type	'PHYSICAL \	VIRTUAL'	VIRTUAL is a singleton: unrenameable, undeactivatable, undeletable branches.service.ts:128
BaseConfig.locked	bool	trips on the first sitting	attempts.service.ts:159 only
BaseConfig.isDefault	bool	one per stage, by partial unique index	base-configs.service.ts:214 , :393
BaseConfigSection.patternNote	text?	workbook wording, reference only	nothing in TS (see Inconsistencies)
BaseConfig.featureFlags	Json?	—	nothing in TS

Redis keys: `base-config:edit:<baseConfigId>` — TTL 900 s, value the editing admin's id (`redis.keys.ts:71` , `EDIT_LOCK_TTL_SEC` : 79). `access:catalog:epoch` — a bare `INCR` counter, **no expiry**, read on every catalog resolve.

What must already be true

- The first super admin exists only by hand-written SQL: `prisma/seed.sql:34-36` inserts `developer@iace.co.in` with `isSuperAdmin = true`. Its own comment: *"Admins cannot self-register and nothing in the application creates one"*.
- The VIRTUAL branch must exist before any ONLINE student can be created — `branches.service.ts:77-84` refuses with `NO_ONLINE_BRANCH_MESSAGE` under the `currentBranchId` field.
- An `Exam` row (active) per code before an enrolment; an active, `CONDUCTED` -disposition `ExamStage` before a base config (`exam-stages.service.ts:144-165`).
- `API_ROLE` must be unset, `all`, or `core` — otherwise none of these four modules' controllers are registered (`config/api-role.ts:42-43`).
- A `MESSAGE_SENDER` provider and a working SMS channel, or the starting PIN is only logged (`starting-pin.service.ts:66-68`).

Why it is shaped this way

- **Grants are rows; roles are not.** `FEATURE_KEYS` is code-owned (`packages/contracts/src/admins.ts:20-31`), so a key nothing checks cannot be granted. The alternative — a `Feature` table the UI registers into — was not taken, which is why `GET /admin/features` is read-only and `ListFeatures` builds itself from

`FEATURE_KEY_VALUES` . `asFeatureKey` (`admins.service.ts:344-346`) drops a stored key code no longer defines rather than trusting it.

- **Enrolment is an array of codes, not a join table.** `Student.enrolledExams` / `programs` / `enrolledCourses` are `String[]` /enum arrays with GIN indexes, so a roster filter is `hasSome` against an index and no `join` (`student-query.ts:25-27`). The cost: `Exam.code` can never be re-used (`schema.prisma:297`), code changes are refused once an enrolment stores one (`exams.service.ts:97-105`), and a code whose row is gone renders as its own code (`students.service.ts:197`).
- **A test inherits its config rather than copying it.** That is what makes the lock necessary at all; the alternative, snapshotting the shape onto every test, was not taken. The price is clone-to-evolve plus a lineage pointer (`clonedFromId` , `onDelete: Restrict`) and the 18-column trigger.
- **Erasure anonymises in place.** `anonymize.ts:1-6` : the sittings and the aggregates they were folded into *"are facts about a PAPER, and a cohort's mean must not move because one person exercised a right"*. A cascade delete would have been simpler and would have moved every percentile.

Which screen calls it

(all `apps/admin` unless stated; `apps/api/src/config/` exposes no endpoint)

ROUTE	UI FILE	SCREEN
GET /admin/admins	<code>features/admins/admins.tsx:109</code> , <code>features/admins/permissions.tsx</code> , <code>components/admin-multi-picker.tsx</code>	/admins , /permissions
POST /admin/admins	<code>features/admins/admins.tsx:254</code>	/admins
PATCH /admin/admins/:id	<code>features/admins/permissions.tsx</code> (<code>role</code> only)	/permissions
PATCH /admin/admins/:id/active	<code>features/admins/admins.tsx:186</code>	/admins
GET /admin/features	<code>features/admins/permissions.tsx</code>	/permissions
PATCH /admin/admins/:id/permissions	<code>features/admins/permissions.tsx</code>	/permissions
GET /admin/branches	<code>features/catalog/branches.tsx</code> , <code>lib/use-branches.ts</code>	/branches
POST /admin/branches	<code>features/catalog/branches.tsx:220</code>	/branches
PATCH /admin/branches/:id	<code>features/catalog/retire-delete-actions.tsx:69</code> via <code>branches.tsx:105</code> (<code>isActive</code> only)	/branches
DELETE /admin/branches/:id	<code>retire-delete-actions.tsx:74</code>	/branches
GET /admin/exams	<code>features/catalog/exams.tsx:151</code> , <code>components/exam-picker.tsx</code> , <code>features/students/use-exams.ts</code>	/exams
POST / PATCH /admin/exams[/:id]	<code>features/catalog/exams.tsx:228</code>	/exams
DELETE /admin/exams/:id	<code>exams.tsx:93</code> (retire/delete menu)	/exams
GET /admin/exam-stages	<code>exams.tsx:387</code> , <code>components/exam-picker.tsx</code>	/exams
POST / PATCH /admin/exam-stages[/:id]	<code>exams.tsx:471</code> , <code>:535</code>	/exams
DELETE /admin/exam-stages/:id	<code>exams.tsx:333</code>	/exams
GET /admin/base-configs	<code>features/catalog/base-configs.tsx</code> , <code>base-config-form.tsx</code> , <code>features/tests/config-picker.tsx</code>	/tests/configs
GET /admin/base-configs/:id	<code>base-config-form.tsx</code> , <code>features/tests/test-builder.tsx</code>	/tests/configs/:id
POST / PATCH /admin/base-configs[/:id]	<code>base-config-form.tsx</code>	/tests/configs/new , /tests/configs/:id
POST /admin/base-configs/:id/clone	<code>base-configs.tsx</code> , <code>base-config-form.tsx</code>	/tests/configs
DELETE /admin/base-configs/:id	<code>base-configs.tsx:208</code>	/tests/configs
GET /admin/students	<code>features/students/students.tsx</code> , <code>components/access-picker.tsx</code>	/students
GET /admin/students/export	<code>students.tsx</code> (both views)	/students
GET /admin/students/:id	<code>features/students/student-detail.tsx</code>	/students/:id
GET /admin/students/:id/sittings	<code>features/students/student-performance.tsx</code>	/students/:id/performance
POST /admin/students	<code>students.tsx</code>	/students
PATCH /admin/students/:id	<code>student-detail.tsx</code>	/students/:id
PATCH /admin/students/:id/active	<code>student-detail-actions.tsx</code>	/students/:id
PATCH /admin/students/:id/test-blocked	<code>student-detail-actions.tsx</code>	/students/:id
POST /admin/students/:id/erasure	<code>student-detail-actions.tsx</code>	/students/:id

No UI: `PATCH /admin/admins/:id` with `fullName` (`updateAdminSchema` accepts it, `packages/contracts/src/admins.ts:198` ; the only caller sends `{ role }`). `PATCH /admin/branches/:id` with `name` (`updateBranchSchema` accepts it; the only caller is `RetireDeleteActions` , which sends `{ isActive }`). `StudentProfile.aadhaarVerified` / `panVerified` have no writer on any route. None of these five modules is called by `apps/exams` or `apps/mobile` ; the student side reads `/me/*` instead.

Is it optimal

- Admin list: 2 queries for rows+count, then **one** grants query for the whole page (`admins.service.ts:132` — the comment names the 51-query alternative). `orderBy createdAt desc` has no index on `Admin` (`schema.prisma:825-844` declares none) — a seq scan + sort on a tens-of-rows table. `orderBy` in practice.
- `identityOf` runs on **every authenticated admin request** (`jwt-auth.guard.ts:40`) and is 2 queries: no `relationJoins` preview feature is enabled (`schema.prisma:14-16`), so the nested `permissions` select is a second round trip.
- `holdersOf` (`admins.service.ts:96-106`) reads every active admin unpaginated, then all their grants, and filters in memory. 2 queries, unbounded rows — fine at staff scale, called only from `assignments.service.ts:247`.
- `listFeatures` reads the entire `AdminFeaturePermission` table (`:240-243`) and filters per key in memory: 1 query, $O(\text{keys} \times \text{rows})$ in JS.
- Student list: `$transaction([findMany, count])` plus **one extra findMany** for `HOLDS_OWN_ACCESS` (`students.service.ts:111-114`) = 3 queries, no $N+1$. Search uses `Student_fullName_trgm_idx` / `Student_mobile_trgm_idx`; the sorts match `Student_fullName_id_idx` and `Student_createdAt_id_idx` (`schema.prisma:794-799`). `detail` is 2 queries plus one S3 presign per non-null document key.
- Branch list: `_count` avoided deliberately — one `groupBy` scoped to the page's ids (`branches.service.ts:197-207` , and the comment saying a relation `_count` "would group every student row for each read").
- `exams.update`: the enrolment count is paid **only** when the code changes (`exams.service.ts:96-105`); a rename or retire does not.
- Base-config `update` is the heaviest: `requireDetail` → `claimEdit` (1 Redis) → `transaction` (1 `updateMany`, optionally `clearDefault` + `update`, optionally 2 `deleteMany` + one `create` **per module and per section**, serially in `writeChildren`) → a second `requireDetail` → `editingBy`. A 20-section rewrite is ~25 statements inside the transaction. `clone` is the same shape, also one `create` per row.
- Catalog resolve: 1 Redis `GET` + (on a miss) 1 `testSeries.findMany` shared process-wide, then 1 student read per request. Measured figures were not re-run here; the structure is as described.

What it leans on

the response envelope (`ResponseInterceptor` + `AllExceptionsFilter`, `app.module.ts:72-75`): no controller here builds `{ success, data, meta }`. The four global guards. `ZodBody` / `ZodQuery` + `packages/contracts` schemas, which are also what the SPA validates responses against (`packages/contracts/src/client/admin.ts:315-422,512+`). `common/pagination` (`pageArgs`, `paged`) for the `meta.page/pageSize/total` every list adds. `common/search-terms` everyTermMatches — one search vocabulary across all five lists. `common/relation-counts` countsBy. `AuditContext` + `@Audit` + `fieldDiff`, so a diff is declared as a field list rather than hand-built. `DomainEventBus` → `AccessCacheListener`, which saves the catalog from being re-queried per student. `common/edit-lock` (Redis) so two admins cannot silently overwrite one blueprint. `common/time/institute-day` `toDateColumn` / `fromDateColumn` for `StudentProfile.dob`. `packages/ui` `ListView`, `FormDialog`, `RowActions`, `ConfirmDialog`, `TruncatedText`, `Badge`, `Alert`, `Accordion` — every screen here is one of those two compositions.

What changed recently

(`git log --oneline -15` over the five paths)

- `f1d5ef4` / `85bd8d9` — env plumbing only: `.env` now loads through Node's own loader instead of `@nestjs/config`, and one variable gates stdout logging. No behaviour change in these modules beyond `AppConfigService` reading an env that is loaded earlier.
- `f472a4d` — the perf pass that actually touched these files: `_count` on seven list selects was replaced with a `groupBy` scoped to the page's ids (which is why `branches.service.ts:197` and `exams.service.ts:26` look the way they do), and base-configs' three hand-copies of the section column list collapsed into the single `paperRowsOf` / `paperKeyOf` pair — the commit body states the bug that caused: "A column added to the writer alone compared as unchanged, so an edit touching only it was silently not written and the save still reported success."
- `e66ab91` — `preTestReady` / `profileCompleted` stopped being stored columns; they are now derived from the profile on read (`readinessOf` in `toSummary`, `students.service.ts:514`) and filtered via `readinessWhere` (`student-flags.ts:8-14`).
- `7bf4f26` — placing an ONLINE student in the virtual branch moved from the form to the server (`placementOf`, `students.service.ts:310-324`), so a client that omits `currentBranchId` now gets the right branch instead of none.
- `3f4153f` — `assertPatchUsable` narrowed to only what a save *adds* (`students.service.ts:297-306`), so a since-retired exam the student still holds no longer blocks removing a different one.

What is wrong with it

- `prisma/migrations/20260820000000_schema_realignment/migration.sql:1332-1351` — `base_config_guard`'s frozen-column list **omits** `examTemplate`, which was added eight days later (`20260828090000_a_paper_carries_its_skin/migration.sql:13`) and never added to the trigger. The invariant test enumerates the same incomplete list (`apps/api/test/migration-invariants.unit.test.ts:166-184`), so nothing catches it. Bites with `UPDATE "BaseConfig" SET "examTemplate" = 'COMPACT' WHERE "locked"` — accepted by the database, re-skinning a paper already sat. The API path is safe: `locksOutEdit` (`base-config-rules.ts:32-35`) refuses any key outside `name/isDefault/isActive`.
- `apps/api/src/admins/admins.service.ts:175-194` — `update` changes `role` and `isSuperAdmin` but **never touches** `adminFeaturePermission`. Create an admin as `ADMIN` (six preset grants), promote to `SUPER_ADMIN`, then set the role to `TYPIST`: the six ADMIN grants are still there and become live again. The Permissions screen cannot show them in between, because its super-admin branch renders an Alert instead of the checkboxes (`apps/admin/src/features/admins/permissions.tsx:225-232`).
- `apps/admin/src/features/admins/permissions.tsx:227-231` — that Alert says "Remove super admin on the Admins screen", but `/admins` has no role editor: `admins.tsx` calls only `list`, `create` and `setActive`. The role control is in `permissions.tsx` itself, inside the `else` branch that a super admin never reaches. A super admin's role therefore cannot be changed from the UI at all.
- `packages/contracts/src/admins.ts:80` and `prisma/migrations/20260919140000_.../migration.sql:16-18` both state the role is display-only and that "no guard anywhere reads it". True of the column; false of the effect, since `admins.service.ts:158,185` derive `isSuperAdmin` from it. `prisma/schema.prisma:829-830` states it correctly.
- `apps/api/src/students/student-query.ts:9-47` — `studentWhere` never filters `deletedAt`. An anonymised student (mobile `0000000000`, `fullName` null, `isActive` false) stays in the roster list and in any export whose filters match. Searching the roster for `0000` after an erasure returns the tombstone.
- `StudentProfile.aadhaarVerified` / `panVerified` — no code path sets either to `true`. `updateStudentProfileSchema` (`packages/contracts/src/students.ts:296-305`) omits them, and the only other writes are `anonymize.ts:41-42` setting them false. They are read by the API (`students.service.ts:225-226`), exported (`student-privacy.service.ts:84-85`) and shown to the student (`apps/exams/src/features/account/profile.tsx:306`), so the screen permanently reads "not verified" unless someone runs SQL.
- `BaseConfigSection.patternNote` (`prisma/schema.prisma:414`) and `BaseConfig.featureFlags` (`:362`) are written by nothing in TypeScript. `patternNote` is set once by the `20260827090000` backfill and by one `prisma/seed.sql` statement; `paperRowsOf` (`base-configs.service.ts:492-505`) omits it, so a **save through PATCH** `/admin/base-configs/:id` that rewrites the paper silently discards every section's `patternNote`, and `clone` (`:284-301`) never copies it. `featureFlags` is likewise

- absent from `clone`'s data (`:244–266`), so a clone loses it too; the only reference anywhere is the trigger test at `apps/api/test/migration-invariants.unit.test.ts:182`.
8. Decorator asymmetry: `branches.controller.ts:66` and `exams.controller.ts:64` stack `@RequiresFeature(STUDENT_MANAGEMENT, WRITE)` on `remove` while their own `create / update` and `exam-stages.controller.ts`'s `remove` carry only `@RequiresSuperAdmin`. Harmless today — `can()` returns true for a super admin — but it reads as a rule where there is none.
9. `prisma/schema.prisma:824` — the comment *"DPDP §6: what the student agreed to, when, and to which version of the notice"* sits directly above `model Admin`. There is no consent model in the schema (no other occurrence of "consent" in the file); the comment is orphaned from a model that was removed, and now reads as a description of `Admin`.

Events and their candidates

The name is a trap. `apps/api/src/events/**` is not an event bus. An `Event` is a **product entity**: a named gathering of people who will sit a paper — an entrance exam, a scholarship round — some of whom are not IACE students yet. `prisma/schema.prisma:933–944` declares `Event` as a plain row (`name`, `description`, `isActive`); `:946–957` declares `EventCandidate` as the membership join (`eventId`, `studentId`). The creating migration says why (`prisma/migrations/20260903080000_an_event_gathers_the_people_sitting_it/migration.sql:1–15`): a candidacy is a **membership**, not a grant, so it earns its own table rather than overloading `StudentGrant`. The repo's real bus is `DomainEventBus`, elsewhere — and the one event this module used to emit (`student.access_changed`) was deleted in `4ead5d3` because nothing listened.

Where it lives

- `apps/api/src/events/events.controller.ts` — the 7 routes; `@Controller('admin/events')`, `@Actors(ActorTypes.ADMIN)`.
- `apps/api/src/events/events.service.ts` — every read and write of both tables, 174 lines.
- `apps/api/src/events/events.module.ts:10` — controller mounted only `onRole( [API_ROLES.CORE], ...)`.
- `apps/api/src/imports/imports.controller.ts:142–177` — the candidate preview/errors/commit routes; template `:87–93`.
- `apps/api/src/imports/imports.service.ts:119–199` — their bodies; `planCandidates` at `:286–306`.
- `apps/api/src/imports/candidate-import.ts` — the pure planner: `create / add / skip`, writes nothing.
- `apps/api/src/access/access-resolver.service.ts` — the only consumer of the roster.
- `packages/contracts/src/access.ts:85–124`, `463–471` — schemas and `EVENT_ROUTES`.

What it is for

A test reaches a student only through a `TestSeries`, whose `kind` picks the

automatic route. Three kinds read facts the student already carries — branch + enrolled course, or a program code. `EVENT` has none to read: an entrance round reaches people with no enrolment, no branch, often no `Student` row. `Event` names the gathering, `EventCandidate` is the roster, and an `EVENT` series addresses its cohort by explicit list; the import lets that roster arrive as a sheet of mobile numbers, minting a bare `NON_IACE` account per unknown number.

FLOW — CREATE AN EVENT AND ATTACH CANDIDATES

- "New event" clicked
`apps/admin/src/features/students/cohorts.tsx:34–37` `CohortsPage`
`EventDialog` mounts.
- Submit
`apps/admin/src/features/catalog/events.tsx:272–273` `EventDialog.save`
`POST /admin/events {name, description}`.
- `JwtAuthGuard` → `ActorGuard` → `FeaturePermissionGuard` → `SuperAdminGuard` → `ThrottlerGuard`
`apps/api/src/app.module.ts:77–82`
`ADMIN` token, `STUDENT_MANAGEMENT / WRITE`.
- `@Audit(AUDIT_FEATURE.EVENT, AUDIT_ACTION.CREATE)`, `@RequiresFeature(STUDENT_MANAGEMENT, WRITE)`, `@Post()`, `new ZodBody(createEventSchema)`
`events.controller.ts:55–60`
`validated CreateEventBody`.
- One INSERT
`events.service.ts:89–91` `create`
`Event` row, `isActive` true, `createdById` null.
- Mapped with a hardcoded zero
`events.service.ts:28–29, 92` `NOTHING_YET`
no count query for a row one statement old.
- Audit tapped off the handler's value, `entityId` from `payload.id`
`apps/api/src/audit/audit.interceptor.ts:45–67`
fire-and-forget `AuditLog`, cannot fail the request.
- Envelope
`apps/api/src/app.module.ts:75`
`{success:true, data:Event, meta:{requestId}}`.
- Row menu → "Add candidates"
`events.tsx:137–140`
`AddCandidatesDialog`, warning `Alert` `:326–332`.
- Add pressed
`apps/admin/src/features/catalog/events.tsx:314`
`POST /admin/events/:id/candidates {studentIds}`.

11 @Audit(EVENT, UPDATE), @HttpCode(HttpStatus.OK), addEventCandidatesSchema min 1

events.controller.ts:81-90

12 requireEvent

events.service.ts:153-157

findUnique else AppException(NOT_FOUND, 'No such event').

13 Roster size for the diff

apps/api/src/events/events.service.ts:131 eventCandidate.count
before .

14 One createMany({skipDuplicates:true})

events.service.ts:135-138

EventCandidate rows; re-posting adds nobody twice.

15 setPatchDiff(rosterDiff(before, before+count))

events.service.ts:140, 161-162

changed is {candidates:{from,to}}, a count not a list.

16 Nothing moved → fieldDiff null → store.unchanged

apps/api/src/audit/audit.context.ts:31-36

interceptor returns early (audit.interceptor.ts:50), no audit row.

17 void → {success:true, data:null, meta}

packages/contracts/src/client/admin.ts:456

FLOW — BULK-IMPORT CANDIDATES

1 "Import candidates"

apps/admin/src/features/catalog/events.tsx:142

route /cohorts/events/:id/import (apps/admin/src/lib/constants.ts:134-135 , mounted App.tsx:224-227).

2 Page reads the event for its title

apps/admin/src/features/imports/import-event-candidates.tsx:37

GET /admin/events/:id .

3 File dropped

apps/admin/src/features/imports/import-event-candidates.tsx:41 useImportScreen.preview

POST /imports/events/:eventId/candidates/preview , multipart.

4 Multer refuses an oversized body as it arrives

apps/api/src/common/importing/upload.ts:7-13

fileSize = BODY_LIMIT_IMPORT , files: 1 .

5 @RequiresFeature(STUDENT_MANAGEMENT, READ) + FileInterceptor(IMPORT_FILE_FIELD) ; no file → VALIDATION_ERROR + fieldErrors.file

imports.controller.ts:143-152, upload.ts:24-30

6 Event must exist

apps/api/src/imports/imports.service.ts:121 events.detail(eventId)

NOT_FOUND on a stale page.

7 Sheet → CsvTable , headers kept as written

apps/api/src/imports/imports.service.ts:122

readUploadedTable .

8 Every readable mobile collected once into a Set

apps/api/src/imports/student-import.ts:62-69

mobilesIn .

9 One findMany on {mobile:{in:...}} , selecting id, mobile, deletedAt

imports.service.ts:291-294

split into existingByMobile (live) and deletedMobiles (:297-304).

10 Headers and ceiling first

student-import.ts:72-90 planRoster

empty / missing column / over IMPORT_MAX_ROWS (1000, packages/contracts/src/imports.ts:208) returns fileErrors and no rows.

11 Per row: mobile parsed, in-file duplicate caught, name trimmed

student-import.ts:113-129

readContact .

12 A number held by a soft-deleted student is an error

candidate-import.ts:11-12, 56-59

DELETED_MESSAGE .

13 Action

candidate-import.ts:44-47 actionFor

errors→ skip , known live number→ add , else create with willReceiveDefaultPin: true (:69).

14 Plan returned, nothing written

candidate-import.ts:31-40

{rows, summary:{total, willCreate, willAdd, invalid}, fileErrors} .

15 Optional errors download posts the same file back

import-event-candidates.tsx:53 → imports.service.ts:125-129

re-planned to an xlsx of failing lines; opens no ImportLog , stores no file.

- 16

Import pressed

apps/admin/src/features/imports/import-event-candidates.tsx:42

POST ../candidates/commit , WRITE .
- 17

Event re-checked, file re-planned from the upload

the client's plan is never trusted

imports.service.ts:137-138 .
- 18

Run opened

imports.service.ts:419-441 openRun

ImportLog{feature: 'STUDENT', source: 'SHEET', status: 'PREVIEWED', total} ; sheet to S3 imports/student/<logId>.xlsx (common/importing/import-log.ts:2-3), fileS3Key written back.
- 19

PINs minted up front, batches of 4

apps/api/src/auth/pin/starting-pin.service.ts:37-52

randomInt 4 digits + argon2, batched because argon2 is memory-hard.
- 20

Serial loop, no transaction: an add row pushes existingStudentId , counts updated , touches nothing on the student (imports.service.ts:159-167); a create row INSERTs a Student with studentType: NON_IACE , pinHash , pinIsDefault: true , no branch/course/exam/program (:170-179).
- 21

One addCandidates for the whole list

imports.service.ts:187 → events.service.ts:129-141

one createMany skipDuplicates .
- 22

Run closed

imports.service.ts:444-473

RowActionLog per entity (failure logged, not thrown), then ImportLog counts, status: 'COMMITTED' , finishedAt .
- 23

PINs texted only now, rows already durable, failures logged

starting-pin.service.ts:55-70
- 24

{total, willCreate, willAdd, invalid, created, added, skipped}

imports.service.ts:193-198

How it differs from the student import Columns: mobile + fullName only, filtered out of the one list (packages/contracts/src/imports.ts:335-337). Actions create/add/skip (imports.ts:101) vs create/update/skip . A known number joins the roster and nothing else is touched (imports.service.ts:159-167) where the student import rewrites name, access, branch and profile (:258-272). A new number is minted NON_IACE with no branch, course, exam or program (:170-179) where the student import takes type and access from the sheet (:486-494). The "default PIN" is not a fixed value: a fresh randomInt 4-digit PIN, argon2-hashed, flagged pinIsDefault: true so it must be changed — handed to every created candidate (candidate-import.ts:69) vs only willReceiveDefaultPin rows there (:98-100). A soft-deleted holder is refused here, invisible there (contextFor filters deletedAt: null , :348). The student commit re-judges the admin's branch scope (:74-75); this plan knows only mobiles. Only this one ends in addCandidates .

FLOW — REMOVE A CANDIDATE

- 1

Student detail → Events tab

apps/admin/src/features/students/student-detail.tsx:382

EventsTab , rows from StudentDetail.events (apps/api/src/students/students.service.ts:168, 176).
- 2

"Remove from event"

student-detail-events.tsx:34-37

ConfirmDialog naming the consequence (:106-114).
- 3

Confirm

apps/admin/src/features/students/student-detail-events.tsx:76

DELETE /admin/events/:id/candidates/:studentId .
- 4

@Audit(EVENT, UPDATE) , WRITE , @HttpCode(HttpStatus.OK)

events.controller.ts:92-98

Express 5 drops a 204 body, so these answer 200.
- 5

requireEvent(id)

apps/api/src/events/events.service.ts:144

NOT_FOUND if the event is gone.
- 6

count , then deleteMany({eventId, studentId})

events.service.ts:145-149

one EventCandidate row deleted, nothing else:: the Student, its PIN, its Attempt rows, scores and rank all stay. Idempotent

0 deleted means no audit row.
- 7

rosterDiff(before, before-count)

apps/api/src/events/events.service.ts:150
- 8

Client invalidates STUDENTS and EVENTS

student-detail-events.tsx:79-80. No catalog bust: reach is read live

Deleting the event is refused while any TestSeries names it (events.service.ts:115-123); else the roster goes with it by ON DELETE CASCADE (20260903080000_../migration.sql:65).

The data it touches

MODEL.COLUMN	TYPE	HOLDS	WRITTEN BY
Event.id	uuid v7 PK	—	default

MODEL.COLUMN	TYPE	HOLDS	WRITTEN BY
Event.name / .description	String / String?	max 120 / max 500 (contracts/src/access.ts:97-101)	create / update
Event.isActive	Boolean, true	catalog visibility only — see Inconsistencies	update
Event.createdById	String? uuid	nothing writes it (verified across apps/api/src)	nobody
Event.createdAt / updatedAt	timestamptz(3)	serialised .toISOString() (events.service.ts:172)	default / @updatedAt
EventCandidate.eventId + studentId	composite PK	the membership; the PK is what makes skipDuplicates idempotent	addCandidates / removeCandidate
EventCandidate.createdAt	timestamptz(3)	when they joined	default
TestSeries.eventId	uuid?, onDelete: Restrict	which roster an EVENT series draws on	TestSeriesService
Student.studentType	enum	NON_IACE for every minted candidate	imports.service.ts:175
Student.pinHash / pinIsDefault	String?/Boolean	random 4-digit PIN, argon2 / true	imports.service.ts:176-177
ImportLog.*	schema.prisma:977- 999	the run; feature always 'STUDENT' (:427); errors jsonb is {fileErrors?: string[], message?: string} (:432, 465- 470)	openRun / closeRun
RowActionLog	—	one row per entity, CREATE or UPDATE	recordImportRows (:452)
AuditLog.changed	jsonb	`(name\`	description\ isActive: {from,to}} or {candidates: {from,to}}`

No Redis key is written here. The one read is access:catalog:epoch (apps/api/src/redis/redis.keys.ts:53) — an INCR counter, no TTL, bumped by series/test writes elsewhere and never by a roster change. S3: imports/student/<importLogId>.xlsx , one path.

What must already be true

API_ROLE unset/ all / core — not mounted on exam or worker
(events.module.ts:10 , config/api-role.ts:9-18). An ADMIN JWT with STUDENT_MANAGEMENT : READ for the GETs and preview/errors, WRITE for the rest; not a catalog key. BODY_LIMIT_IMPORT (config/app-config.service.ts:22-25). S3/MinIO reachable — openRun uploads before the first row. A Student row per posted id (FK enforced). An EVENT series needs its eventId , in SQL by TestSeries_event_kind_names_its_event CHECK ((kind='EVENT') = ("eventId" IS NOT NULL)) (20260904110000_/_migration.sql:58-59) as well as in the app (access/test-series.service.ts:373-377).

Why it is shaped this way

- A membership table, not a column or a grant. Student.events String[] and StudentGrant rows are both rejected at 20260903080000_...:3-10 : no student row exists to hang a column off until the candidate enrolls, and StudentGrant is for exceptions to a rule where a roster is the rule. Cost: a join per catalog read.
- Read live, never cached per student (access-resolver.service.ts:107); only the shared series list is held per process on access:catalog:epoch (:239-262). The alternative, a per-student cached catalog, needed a bust per roster write — 4ead5d3 deleted exactly that. Gain: no invalidation at all.
- Counts via a groupBy scoped to the page's ids (events.service.ts:66-78), not Prisma _count in an include , which groups the whole child table per read (common/relation-counts.ts:1-5).
- The audit log stores a roster size, not a roster (events.service.ts:160-162) — the alternative is thousands of uuids in jsonb per import. Cost: it answers how many, never who.

Does an event change what a student can see or start? Yes, by one path. eventCandidacies:{select:{eventId:true}} is a field of REACH_FACTS (access-resolver.service.ts:107), read inside the single Student.findUnique the resolver already issues (:266-269, 274-277). reachFrom folds it to Reach.events , a Set<eventId> (:291-299 , type :126). sourcesOf pairs it: isCandidate = row.eventId !== null && reach.events.has(row.eventId) (:310). seriesSources has one arm — [TEST_SERIES_KIND.EVENT]: series.isCandidate (:379) — yielding STUDENT_SERIES_SOURCE.EVENT (:389-394). A series with no source is dropped by reachedBy (:316-318), which catalogOf (:336-343), startableTest (:321-334) and reachableTest (:171-182) all run through, so the same resolution gates the catalog, assertCanStart (:157-168) and any single-test read. audienceOf / automaticAudience (:397-432) is the mirror as a StudentWhereInput : {eventCandidacies:{some:{eventId}}} (:422).

An EVENT-reached test looks identical to any other. studentCatalogSeriesSchema (packages/contracts/src/access.ts:406-421) carries kind but no sources — the source list goes only to admin screens via seriesReachedBy (:185-200), badged "Event candidate" (apps/admin/src/lib/constants.ts:225 , student-detail-series.tsx:38-42). apps/exams reads series kind nowhere (verified: its only .kind === hits are login steps and TIMER_KIND). A candidate sees a normal card; canStart turns on opensAt , sequentialTests and their sittings, not the event.

Which screen calls it

(admin SPA only — nothing in apps/exams or apps/mobile) GET /admin/events → features/catalog/events.tsx:213 (/cohorts?tab=events), plus components/access-picker.tsx:43 (EventPicker , activeOnly:'true' , on /tests/series/:id via test-series-access.tsx:54) and :163 (EventMultiPicker , the Students filter, students.tsx:218). POST /admin/events → events.tsx:273 . PATCH :id → events.tsx:273 and features/catalog/retire-delete-actions.tsx:69 (retire/reactivate). DELETE :id → retire-delete-actions.tsx:74 ; the action is left out, not disabled, when seriesCount > 0 (events.tsx:124). POST :id/candidates → events.tsx:314 . DELETE :id/candidates/:studentId → features/students/student-detail-events.tsx:76 (/students/:id , Events tab). GET :id → features/imports/import-event-candidates.tsx:37 . The four import routes

(template/preview/errors/commit) → `import-event-candidates.tsx:45, 41, 53, 42`, all on `/cohorts/events/:id/import`. **No endpoint here is uncalled** — the one dead route, `GET /admin/events/:id/candidates`, went in `2c72ceb`; a roster is read on the Students screen filtered by `eventId` (`apps/api/src/students/student-query.ts:29`), where the `candidateCount` cell links (`events.tsx:95–100`).

Is it optimal

- `GET /admin/events`: **4 queries** — `findMany` + `count` in one `$transaction` (:48–55), then two parallel `groupBy` (:69–72). No N+1. `Event` has **no index beyond its PK** (verified: no `@index` at `schema.prisma:933–944`, no `CREATE INDEX ... ON "Event"` in any migration), so `q` is a seq scan + `ILIKE` and `ORDER BY name` a Sort — read, not measured, and fine at one row per exam round. The `groupBy` s use `EventCandidate_pkey` and `TestSeries_eventId_idx`.
- `GET :id:3`. `POST :id/candidates:3` — `findUnique`, `count`, one `createMany` however many ids; the `count` exists only for the audit diff. `DELETE :id/candidates/:studentId:3`, the delete on the PK. `DELETE :id:3`, the guard on `TestSeries_eventId_idx`, the roster in the cascade.
- Candidate preview: **4 queries for up to 1000 rows** (3 for the event + 1 `findMany` on `Student_mobile_idx`) — optimal; `mobilesIn` deliberately replaces a per-row lookup.
- Candidate commit: `3 + 1 + ImportLog insert + S3 PUT + fileS3Key update + one serially-awaited Student.create per new row (:170) + 3` + the close writes. 1000 new candidates ≈ 1000 round trips and 1000 argon2 hashes at concurrency 4 — the hot spot, the same shape as the student import (:94–109); `createMany` is unavailable because each row's generated `id` is needed for `studentIds`.
- Student hot path: the roster costs **nothing extra** — one more joined select inside a `findUnique` the resolver already issues, on `EventCandidate_studentId_idx`.

Shared machinery `ResponseInterceptor` + `AllExceptionsFilter` (one envelope); `AuditInterceptor` + `AuditContext` (a service sets a diff, `setPatchDiff(null)` suppresses a no-op log); the four global guards plus `@Actors / @RequiresFeature`; `ZodBody / ZodQuery` over contracts schemas; `common/pagination`; `common/search-terms` (each word must land in `name` or `description`); `common/relation-counts`; `common/importing` (`readUploadedTable`, `requireFile`, `rowsWithErrors`, `importFileKey`, the Multer ceiling); `StartingPinService`; `StorageService`; `AccessResolverService`. Client: `useListScreen`, `ListView`, `FormDialog`, `ConfirmDialog`, `TruncatedText`, `RowActions`, `useImportScreen`, `PageCrumbs`, `applyFieldErrors`. **No queue, no WebSocket, no Redis write.**

What changed recently

- `f472a4d` — relation `_count` replaced by a `groupBy` scoped to the page's ids; why `countsOf` exists.
- `2c72ceb` — `GET :id/candidates`, `EventsService.candidates`, its mapper, two schemas and two db tests deleted: no client called it.
- `f7a3268` — `addCandidates` stopped re-reading and returning the roster it had just written; answers `null`.
- `4ead5d3` — `student.access_changed`, its emits and the `DomainEventBus` dependency deleted: the resolver reads live, nothing listened.
- `c93a386` — added the `.../errors` routes; roster planners take a `CsvTable` so the file is read once.

What is wrong with it

- `prisma/schema.prisma:938` — `Event.createdById` is **dead**: declared, migrated (`20260903080000_...:26`), written by nothing (`create` sets `name` and `description` only, `events.service.ts:89–91`), read by nothing. Who made an event is recoverable only from `AuditLog`.
- `Event.isActive` is never consulted in access resolution.** `SHARED_SELECT` (`access-resolver.service.ts:34–78`) takes `TestSeries.eventId` and never joins `Event`; `automaticAudience` (:419–422) filters on `eventCandidates.eventId` alone. Retiring an event revokes nothing — it only hides the event from `EventPicker` (`access-picker.tsx:43`). The UI copy says so correctly (`events.tsx:157, 162–163`), so it is a design choice; the column name invites the opposite reading.
- `packages/contracts/src/access.ts:121` — `studentIds: z.array(z.string().min(1)).min(1), no .uuid(), no .max()`. A well-formed uuid for a nonexistent student raises Prisma `P2003`, which `common/all-exceptions.filter.ts:120–137` does not map (`PRISMA_ERROR_CODES` lists only `P2002/P2025/P2023/P2010`, `common/prisma-errors.ts:4–10`) → **500 INTERNAL**. Input: `POST /admin/events/<real id>/candidates` with `{"studentIds":["01999999-9999-7999-8999-999999999999"]}`. A *malformed* id is handled (`P2023` → `404`). No array ceiling either, so one request can post an unbounded `createMany`.
- `apps/api/src/imports/imports.service.ts:170–187` — the commit loop has **no transaction**, and `addCandidates` runs only after every create succeeds. Input: a 400-row sheet where row 300 trips `Student_mobile_live_key` (a number created concurrently between preview and commit). Rows 1–299 exist as `NON_IACE` students with a texted PIN and **no event** — they reach nothing, and only `RowActionLog` names them. The run is correctly marked `FAILED` (:407), and re-running the fixed sheet repairs it (those rows become `add`), so recoverable, not corrupting.
- `candidate-import.ts:11–12, 56–59` vs `imports.service.ts:348` — **the two importers answer the same cell differently**. A mobile held by a soft-deleted student is a row error here; the student import filters `deletedAt: null` out of its context and creates a *second* account on that number — legal, because `Student_mobile_live_key` is `UNIQUE ... WHERE "deletedAt" IS NULL` (`20260820000000_schema_realignment/migration.sql:1152`).
- `events.service.ts:131–140` — `count` and `createMany` are not in one transaction, so two admins adding overlapping rosters log overlapping `{from,to}` ranges. The rows stay correct (the PK dedupes); only the log's arithmetic is approximate.
- Minor, verified: `starting-pin.service.ts:36` reasons about "a 5,000-row roster" where `IMPORT_MAX_ROWS` is 1000 (`imports.ts:208`); `access-resolver.service.ts:420–421` guards an `EVENT` series with a null `eventId`, which the DB CHECK forbids (`20260904110000_...:58–59`); `imports.service.ts:427` logs a candidate run as `feature: AUDIT_FEATURE.STUDENT` under `imports/student/...` (deliberate, :386–387), so the `eventId` is nowhere on the `ImportLog` row — including its unused `target` column — and the audit screens cannot filter candidate imports by event.

Watching a live test, and intervening

Where it lives

`apps/api/src/attempts/` (API), `apps/admin/src/features/live-ops/` and

`apps/admin/src/features/tests/test-analytics.tsx` (UI), `packages/contracts/src/live-ops.ts` (wire + policy constants).

- `apps/api/src/attempts/live-ops.controller.ts` — the six routes under `admin/live-ops`, all on `TEST_OPERATIONS`.
- `apps/api/src/attempts/live-ops.service.ts` — the picker list and the board; two hand-written SQL reads plus one Redis MGET.
- `apps/api/src/attempts/attempt-resolution.service.ts` — force-submit, extend, reset, void, each audited against the student.
- `apps/api/src/attempts/attempt-resolution.ts` — the pure rules: which action a status admits, and `extendedEndsAt`.
- `apps/api/src/attempts/test-analytics.service.ts` — the cohort report: live curve + the three rollup tables.
- `apps/api/src/attempts/test-report.ts` — the five-sheet workbook the export sends.
- `apps/api/src/attempts/overview.service.ts` / `performance.service.ts:346` — the student's own career figures and the sat-days calendar.
- `apps/admin/src/features/live-ops/live-ops.tsx` — the three-panel board, polling at `LIVE_OPS_POLL_MS`.

What it is for

Two different jobs on either side of a paper. During a live test an ops admin

needs to see, per test, who is still answering, who is past their own deadline and still open, and what has landed in the last half hour — and to reach into one broken sitting without touching the other 2,999. After the paper, a test manager needs the cohort's spread, its section averages and its item analysis, plus a workbook for the branches. The two are split by feature key on purpose (`live-ops.controller.ts:40` vs `test-analytics.controller.ts:35`): an ops admin holds neither `TEST_MANAGEMENT` nor the paper.

FLOW — THE LIVE BOARD

- 1 `GET /admin/live-ops/tests/:testId`
`apps/api/src/attempts/live-ops.controller.ts:50` `board()`
decorators are `@Controller('admin/live-ops')`, `@Actors(ACTOR_TYPES.ADMIN)` (class), `@RequiresFeature(FEATURE_KEYS.TEST_OPERATIONS, PERMISSION_LEVELS.READ)`, `@Get('tests/:testId')`. No `@Audit`.
- 2 Global JWT guard, then the actor and feature guards read `ACTORS_KEY / REQUIRED_FEATURE_KEY`
`apps/api/src/common/security/decorators.ts:24,34`
super admins bypass the feature check (`decorators.ts:33`).
- 3 `test.findUnique({ id }, select: { id, title })`
`apps/api/src/attempts/live-ops.service.ts:81`
`NOT_FOUND 'No such test'` if absent.
- 4 `since = now - 30 min from LIVE_OPS_RECENT_MINUTES`
`live-ops.service.ts:87`, `packages/contracts/src/live-ops.ts:17`
- 5 Seven reads in one `Promise.all`
`live-ops.service.ts:91-108`. No transaction; each is its own statement
- 6 `paperQuestion.count({ testId })`
the denominator of "answered"
served by `PaperQuestion_testId_order_key` (`prisma/schema.prisma:601`).
- 7 `sittings(testId, endsAt >= now) and sittings(testId, endsAt < now)`
`live-ops.service.ts:126-134`
raw SQL, `LIMIT 100` (`LIVE_OPS_ROW_CAP`), joining `Student` and `Branch` by PK; status is interpolated **literally** (`live-ops.service.ts:150`) so the planner can match the partial index.
- 8 `liveCounts`
`live-ops.service.ts:140-144`
one `count(*) FILTER` pass giving both panel sizes.
- 9 `attempt.count(landed) and attempt.count({ status: SUBMITTED, score: null })`
the "landed" and "awaiting scoring" counts
- 10 `attempt.findMany(landed, orderBy submittedAt desc, take 100)` with the student's name and mobile
`live-ops.service.ts:102-107`
- 11 `state.readMany([...active, ...stuck])`
`attempt-state.service.ts:187-198`
one Redis `MGET` of `≤200 attempt:state:<id>` keys. A GET, never a GETDEL: the board must evict nothing.
- 12 `sittingsFrom` lays the held state over each row
`live-ops.ts:38-58`
`endsAt` comes from `Redis` when the key is there and from the row when it has gone (`live-ops.ts:53`); `answeredCount` is null with no key, which is what `hasLiveState: false` means.
- 13 `ResponseInterceptor` wraps `{ success: true, data, meta: { requestId } }`.
- 14 `useQuery` with `refetchInterval: LIVE_OPS_POLL_MS` (15 s) and `meta: { silent: true }`
`apps/admin/src/features/live-ops/live-ops.tsx:64-71`
a failed poll keeps the last good rows and raises one `Alert` (`live-ops.tsx:181-190`) rather than blanking the hall.

FLOW — FORCE-SUBMIT ONE SITTING

- 1 `POST /admin/live-ops/attempts/:attemptId/force-submit`
`apps/api/src/attempts/live-ops.controller.ts:57` `@RequiresFeature(TEST_OPERATIONS, WRITE)`, `@Audit(AUDIT_FEATURE.STUDENT, AUDIT_ACTION.UPDATE)`, `@Post`. Body `{ reason }` through `ZodBody(forceSubmitAttemptSchema)`; the reason is required, trimmed, 1-280 chars (`packages/contracts/src/live-ops.ts:99-105`)
- 2 `require(attemptId, FORCE_SUBMIT)`
`attempt-resolution.service.ts:166-179`
reads eight columns, `resolutionBlocker` refuses anything but `IN_PROGRESS` with `CONFLICT` (`attempt-resolution.ts:26-32`).
- 3 `submit.expire(attemptId)`
`submit.service.ts:106-108`
the **sweeper's own entry point**, re-reading the row and calling the shared `end()`.
- 4 `state.read(attempt.id)`
Redis, not taken: the live state must outlive a write that throws
`submit.service.ts:114`.
- 5 `sheets.write(held, true)`
the answers reach `AttemptSheet.answers **before**` the claim, so a failure here leaves the sitting open
`submit.service.ts:115`.

6	<code>claim()</code> updateMany({ id, status: IN_PROGRESS }, { status: SUBMITTED, submittedAt: now }) <code>submit.service.ts:150-158</code> . The single gate: count 0 means the student's own submit won the race, and the call reports that instead.
7	<code>state.take(attempt.id)</code> GETDEL attempt:state:<id> plus SREM attempt:dirty <code>attempt-state.service.ts:176-180</code> . Behind the claim, so a later save is refused rather than swallowed.
8	Whatever the take returned is written again sheets.write(last, false) <code>submit.service.ts:123</code> .
9	<code>scoring.queue([attempt])</code> BullMQ scoring, keyed scoringJobId(attemptId), failure only logged because the sweeper re-asks <code>submit.service.ts:161-168</code> , <code>scoring-queue.ts:27</code> .
10	<code>require(attemptId, null)</code> reads the row back, then <code>record()</code> files the audit diff against <code>attempt.studentId</code> with <code>supportAction</code> , <code>attemptId</code> , <code>reason</code> and <code>status</code> : {from,to} <code>attempt-resolution.service.ts:64-67</code> , <code>attempt-resolution.ts:41-53</code>
11	<code>AuditInterceptor</code> writes one <code>AuditLog</code> row after the response value, never failing the request <code>apps/api/src/audit/audit.interceptor.ts:48-69</code>
12	Response <code>ResolvedAttempt</code> <code>attempt-resolution.service.ts:182-194</code>

How it differs from the student's own submit (`submit.service.ts:50-85`): the student path adds an ownership check that answers `NOT_FOUND` for someone else's id, a `sittingRefusal` against the tab that holds the sitting, the `last` batch saved as the newest revision inside the deadline's grace (`lastAnswers` , `submit.service.ts:88-103`), and `metrics.countSubmit('accepted'|'refused')` . Force-submit has none of those: no tab check, no last batch, no metric. `end()` and everything after it is identical, which is the point of routing through `expire` .

FLOW — EXTEND, AND HOW IT REACHES A COUNTING BROWSER

1	<code>POST .../extend</code> , body { minutes: 1..180, reason } <code>live-ops.controller.ts:67</code> , <code>packages/contracts/src/live-ops.ts:109-116</code>
2	<code>require(attemptId, EXTEND)</code> <code>IN_PROGRESS</code> only
3	<code>extendedEndsAt(row.endsAt, minutes, now) = max.endsAt, now) + minutes</code> <code>attempt-resolution.ts:35-38</code> : extending a stuck sitting counts from now, or it would buy nothing
4	<code>attempt.update({ endsAt })</code> <code>apps/api/src/attempts/attempt-resolution.service.ts:80</code> <code>Postgres</code> is the deadline of record, and what <code>durableState</code> falls back to.
5	<code>state.pushDeadline</code> <code>attempt-state.service.ts:212-214</code> CAS patch of <code>attempt:state:<id>.endsAt</code> , TTL re-set to <code>STATE_TTL_SEC = PAUSE_LIMIT_SEC = 48 h</code> (<code>attempt-state.service.ts:43</code> , <code>attempt-state.ts:23</code>). With no key it returns null and writes nothing <code>correct</code> , because the row now carries the new deadline.
6	The browser's next autosave fires at 25 s ± 5 s jitter, or at 20 pending answers <code>packages/app-kit/src/autosave-policy.ts:8,11,14</code>
7	<code>save()</code> returns the ack { revision, endsAt: state.endsAt, serverNow } <code>attempt-state.service.ts:390-392</code> : answering <code>**is**</code> the clock check
8	<code>setClock({ endsAt: saved.endsAt, serverNow: saved.serverNow, arrivedAt: Date.now() })</code> <code>packages/app-kit/src/exam/use-attempt-state.ts:256</code>
9	<code>clock = state.clock ?? paperClock</code> <code>packages/app-kit/src/exam/use-exam-view.ts:82</code>
10	<code>useClockCountdown</code> re-keys its <code>secondsLeft</code> callback on <code>endsAt</code> <code>packages/app-kit/src/exam/use-countdown.ts:39-48</code> so the count jumps up on the next tick. No reload, worst case ~30 s plus the request.

Reset — `attempt-resolution.service.ts:91-99` . `state.reestablish(studentId, attemptId)` rebuilds the key from `durableState` (Attempt + `AttemptSheet.answers` + the paper rows, dropping `NOT_VISITED` slots) and merges anything the key still holds over it, held answers winning (`attempt-state.service.ts:201-209` , `380-387`). Writes Redis only; the audit diff carries `answersRestored` . `IN_PROGRESS` only — a marked sitting may never be put back (`attempt-resolution.ts:21`).

Void — `attempt-resolution.service.ts:102-142` . Writes `status=VOIDED` , `voidedAt` , `voidReason` , `voidedById` , and `isGraded=false` only when the sitting was ranked and `regrantRanked` was ticked — clearing the flag `is` the regrant. Then `state.take` (the key goes, and the Postgres fallback now refuses every later save with `ALREADY_ENDED`), then `rollup.rebuild(testId)` and `rollup.rebuildStudent(studentId)` , both delayed 60 s and both only logged on failure (`attempt-resolution.service.ts:145-153`) . **Irreversible**: there is no un-void route, and every other action needs `IN_PROGRESS` , so a void is terminal. What a student mid-sitting sees: their next autosave answers `CONFLICT` , `failed()` sets `hasUnsaved` (`use-attempt-state.ts:215-223`) and the skin shows a "Not saved yet" badge (`apps/exams/src/features/exam/templates/shared/slots.tsx:54`); the countdown keeps running, and their eventual submit lands on `closeOff` and reports `submittedByThisCall: false` .

FLOW — ANALYTICS AND THE RESYNC

1	<code>GET /admin/tests/:id/analytics</code> <code>apps/api/src/attempts/test-analytics.controller.ts:36</code> @Actors(AADMIN), @RequiresFeature(TEST_MANAGEMENT, READ)
---	--

2	<code>requireTest</code> reads <code>{ id, title, testSeriesId }</code> test-analytics.service.ts:103-112
3	Seven reads in <code>Promise.all</code> test-analytics.service.ts:74-83: <code>TestStat</code> by PK; <code>cohortCurveOf</code> ; <code>topperOf</code> ; <code>TestSectionStat</code> ; <code>TestQuestionStat</code> ; <code>attempt.count({ testId })</code> (every sitting, voided included); <code>access.audienceCount(testSeriesId)</code>
4	<code>cohortCurveOf</code> cohort-curve.ts:11-15 raw <code>GROUP BY score</code> over <code>IN_COHORT (isGraded AND status='EVALUATED' AND score IS NOT NULL)</code> , so a voided sitting drops out by itself. <code>cohortShapeOf</code> derives mean, highest, lowest, size and ~N bands in Node (<code>performance-analytics.ts:99-134</code>).
5	<code>topperOf</code> test-analytics.service.ts:148-169 <code>rankOneSql</code> is one index probe of <code>Attempt_ranking_idx</code> (<code>ranking-sql.ts:17-24</code>), then one <code>findUnique</code> for the name, score and sheet time.
6	<code>summaryOf</code> test-analytics.ts:68-92 stored sums become averages here; <code>isSettling = live.size !== stat.evaluatedCount</code> is the only freshness signal.
7	<code>sectionsOf / itemsOf</code> test-analytics.ts:94-118 sorted by paper order, p-value and the three item signals derived on read.
8	<code>POST /admin/tests/:id/analytics/resync</code> <code>apps/api/src/attempts/test-analytics.controller.ts:43</code> <code>TEST_MANAGEMENT WRITE, @HttpCode(202)</code> , empty body.
9	<code>rollups.rebuildNow(testId)</code> rollup-queue.ts:42-44 <code>job REBUILD_TEST</code> under id <code>rollup-rebuild-<testId>-now</code> , delay 0, kept apart from the debounced id so a waiting rebuild cannot swallow it.
10	<code>RollupProcessor</code> (concurrency 2) → <code>rollup.service.ts:272-275</code> <code>rebuildTest</code> = <code>recountTest</code> (upsert <code>TestStat</code> , delete+insert <code>TestSectionStat</code> , <code>TX_LIMITS.MEDIUM</code>) then <code>recountTestItems</code> (replay every first sitting's sheet, delete+ <code>createMany</code> <code>TestQuestionStat</code> , <code>TX_LIMITS.BULK</code>).
11	The screen marks <code>computedAt</code> + <code>wasSettling</code> at the click and polls every 3 s until either moves, giving up at 150 s <code>apps/admin/src/features/tests/test-analytics.tsx:75-116</code>
12	<code>GET /admin/tests/:id/report/export</code> <code>apps/api/src/attempts/test-analytics.controller.ts:51</code> <code>@RequiresExport(TEST_MANAGEMENT)</code> = that key's <code>READ **and** DATA_EXPORT READ</code> (<code>decorators.ts:42-48</code>), <code>@Audit(TEST, EXPORT)</code> , <code>@Res()</code> so no envelope. <code>buildTestReport</code> runs the analytics read again, caps at 50,000 rows twice (<code>EXPORT_MAX_ROWS</code>), then <code>testResultsSql</code> + every sitting + <code>access.studentsReaching</code> minus whoever sat, and writes five sheets Results (rank, percentile, person, marks, per-section triples), Absent, Sections, Questions, Summary — <code>test-report.ts:76-113</code> .

Why a resync exists: nothing folds the cohort on a first evaluation. The sweep picks tests whose `Attempt.updatedAt` passed `TestStat.computedAt - SWEEP_LAG` (`rollup.service.ts:146-157`), and item stats wait on their own clock as well (`rollup.service.ts:161-173`), so a void, a re-score or a late submission can leave the three tables behind for minutes. `isSettling` is how the screen says so.

The data it touches

COLUMN	TYPE	HOLDS	WRITTEN BY
<code>Attempt.status</code>	enum	<code>IN_PROGRESS</code> / <code>SUBMITTED</code> / <code>EVALUATED</code> / <code>EXPIRED</code> / <code>VOIDED</code>	<code>submit.service.ts:153</code> (claim), <code>attempt-resolution.service.ts:112</code> (void)
<code>Attempt.endsAt</code>	timestampz(3)	server-owned deadline	<code>attempt-resolution.service.ts:80</code> (extend), <code>attempts.service.ts:90</code> (resume credit)
<code>Attempt.submittedAt</code>	timestampz(3)	stamped at the claim	<code>submit.service.ts:155</code>
<code>Attempt.isGraded</code>	bool	the one ranked sitting	cleared by a void with <code>regrantRanked</code> — <code>attempt-resolution.service.ts:120</code>
<code>Attempt.voidedAt</code> / <code>voidReason</code> / <code>voidedById</code>	timestampz / text / uuid	the archive stamp	<code>attempt-resolution.service.ts:114-118</code>
<code>AttemptSheet.answers</code>	jsonb	one slot per paper row, paper order	<code>sheets.write</code> on the submit path only
<code>TestStat.evaluatedCount</code> , <code>sumTimeSec</code> , <code>computedAt</code>	int, bigint, timestampz	the ranked cohort's size and total clock	<code>rollup.service.ts:206-229</code>
<code>TestSectionStat.attempted</code> , <code>sumScore</code> , <code>sumTimeSec</code>	int, decimal(18,2), bigint	per-section sums over <code>sectionScores</code>	<code>rollup.service.ts:231-241</code>
<code>TestQuestionStat.correctCount</code> , <code>wrongCount</code> , <code>skippedCount</code> , <code>sumTimeSec</code> , <code>optionCounts</code>	int×3, bigint, jsonb	item analysis and distractor counts	<code>rollup.service.ts:256-269</code>
<code>StudentStat.*</code>	—	one student's career tallies	folded in the scorer's transaction; rebuilt by a void
<code>AuditLog.changed</code>	jsonb	<code>{ supportAction, attemptId, reason, _moved }</code>	<code>audit.interceptor.ts:58-69</code>

Redis: `attempt:state:<attemptId>` (`redis.keys.ts:56`), TTL 48 h (`PAUSE_LIMIT_SEC`), holding `attemptId`, `studentId`, `testId`, `startedAt`, `endsAt`, `lastSeenAt?`, `creditedMs?`, `revision`, `answers`, `sections`, `forwardOnly?`, `tab?` (`attempt-state.ts:49-68`), answers positionally packed (`attempt-state.ts:79`). `attempt:dirty` (a set) — `SREM` 'd by `take` . The board only ever MGETs the first key; `sitting:claim:<studentId>` is never touched here.

What must already be true

an admin JWT; a grant of `TEST_OPERATIONS` at READ to watch and WRITE to act, or

super admin; `TEST_MANAGEMENT` READ for analytics, WRITE for the resync, plus `DATA_EXPORT` READ for the workbook. The picker offers only `status = ACTIVE` tests (`live-ops.ts:7-16`). The board needs `PaperQuestion` rows to show a denominator. The analytics screen needs a `TestStat` row — `summaryOf` returns the empty summary without one (`test-analytics.ts:52-79`) and the body shows "No ranked results yet". The partial index `Attempt_live_by_test_idx` must exist (`prisma/migrations/20260918100000_/_/migration.sql:49`). The resync needs the BullMQ `rollup` worker running, and `sitting_percentile` must exist for every ranking read.

Why it is shaped this way

- The board reads the row for identity and Redis for progress rather than keeping a counter in Postgres: an `answeredCount` column would be a write per autosave, which the scaling rules forbid. The cost is that a lost key shows "Live state lost" instead of a number — which is exactly the signal the reset action exists for.
- Panel rows are capped at 100 and read *apart* from the counts (`live-ops.service.ts:94`): one unbounded query over 3,000 sittings would be the same read, and a hall of stuck sittings would fill the window and empty the running list. The alternative — one query with a window function — would have ranked 3,000 rows to return 200.
- Void archives instead of deleting, and asks for a recount rather than decrementing: a decrement cannot be replayed and drifts the moment one fails. The cost is a ~60 s window where the screen shows figures that still count the voided sitting.
- `TestStat` holds only counts and sums; mean, median, highest, lowest and the topper are counted live off `Attempt_ranking_idx` every read (`b96a89d` , `ad4139e`). A stored mean would be a second place the truth lives, and it was wrong after every void.

Which screen calls it

ROUTE	UI FILE	SCREEN ROUTE
GET /admin/live-ops/tests	apps/admin/src/features/live-ops/live-test-picker.tsx:22	/live-ops (admin)
GET /admin/live-ops/tests/:testId	apps/admin/src/features/live-ops/live-ops.tsx:68	/live-ops (admin), App.tsx:242
POST .../force-submit .../extend .../reset .../void	apps/admin/src/features/live-ops/sitting-actions.tsx:215-229	/live-ops (admin), one ConfirmDialog per panel
GET /admin/tests/:id/analytics	apps/admin/src/features/tests/test-analytics.tsx:89	/tests/:id/analytics (admin), App.tsx:239
POST /admin/tests/:id/analytics/resync	test-analytics.tsx:95 ("Sync now")	same
GET /admin/tests/:id/report/export	test-analytics.tsx:178 (ExportButton)	same
GET /me/overview	apps/exams/src/features/dashboard/dashboard.tsx:64 and features/performance/overview.tsx:60 ; apps/mobile/app/(tabs)/index.tsx:48	/ and /performance (exams); Home tab (mobile)
GET /me/performance/days	apps/exams/src/features/dashboard/dashboard.tsx:69 ; apps/mobile/app/(tabs)/index.tsx:50	same

The nav entry is { label: 'Live operations', featureKey: TEST_OPERATIONS, children: [Sittings] } (apps/admin/src/lib/constants.ts:477-482); the row menu only renders with WRITE (`live-ops.tsx:62,315`). The analytics screen is reached by a link from the test list, the builder and the series screen — no nav entry.

Is it optimal

- **Board: 8 Postgres statements + 1 Redis MGET per poll**, 15 s apart, so ~32 statements/min per watching admin — independent of how many students are sitting. `sittings` ×2 and `liveCounts` ride `Attempt_live_by_test_idx` (partial on `status='IN_PROGRESS'` , so it holds a few thousand entries whatever the history grows to; the migration measured 129 pages/0.9 ms → 3-7 pages/0.07-0.25 ms). `landed` uses `Attempt_testId_updatedAt_idx` , `paperQuestion.count` uses `PaperQuestion_testId_order_key` . The two panel reads each do ≤100 PK lookups into `Student` and `Branch` inside one nested-loop join — not an application N+1.
- The board **does not touch the hot path**: it never reads or writes `AttemptSheet` , never GETDELS a live key, and the `Attempt` rows it reads are not written while a sitting runs. The one shared resource is Redis, and one MGET of ≤200 keys every 15 s against ~120 autosave ops/s at 3,000 sittings is noise.
- The weak query is `awaitingScoring` (`live-ops.service.ts:99-101`): `score: null` is not in `Attempt_testId_status_idx` , so during the submit spike it index-scans every `SUBMITTED` row of the test and heap-checks each. At 3,000 landing at once that is the board's most expensive read, four times a minute. Read, not measured.
- `analytics` : 10 statements (11 with a topper), none per-row. `cohortCurveOf` aggregates in Postgres off `Attempt_ranking_idx` ; `audienceCount` costs two (series + student.count). Payload is one object per paper question, ~100 for a full paper, each carrying `stemPreview` and option shares.
- `resync` is **two statements and an enqueue** — the work is entirely on the worker, which is the whole point.
- `report/export` is the heaviest: `analytics.forTest` again (10), the whole-test window query, every sitting with its student card, `studentsReaching` , and `studentCardsOf` batched. Buffered in memory and `response.send` , capped at 50,000 rows each for results and absentees.
- `GET /me/overview` : 3 statements, one of which (`standingsOfStudent`) runs one LATERAL cohort count per sitting — bounded to `RECENT_STANDINGS = 5` (`packages/contracts/src/stats.ts:41`), which is the bound that keeps it from being an N+1.
- `GET /me/performance/days` : 2 statements; reads `submittedAt` for every EVALUATED sitting since the account opened and buckets them in Node. `optimal` at today's volumes, linear in a student's history.

What it leans on

the response envelope and `AppException` / `ErrorCodes` (so the

controllers never build a body); `Actors` / `RequiresFeature` / `RequiresExport` and `CurrentUser` ; `ZodBody` / `ZodQuery` over the contracts schemas, which is where the 1-280 char reason and the 180-minute cap are enforced; `pageArgs` / `paged` and `everyTermMatches` for the picker; `AuditContext` + the interceptor for a row filed against the student; `AttemptStateService` for every Redis touch; `ScoringQueue` / `RollupQueue` for BullMQ; `common/exporting` for the workbook, the row cap and the headers; and on the UI side `app-kit`'s `useFilters` , `usePagedPicker` and `applyFieldErrors` , with `DataTable` , `TableFrame` , `RowActions` , `ConfirmDialog` , `Combobox` , `Alert` , `Badge` , `DistributionPlot` and `TruncatedText` from `packages/ui` .

What changed recently

(`git log -15` over these paths)

- `f472a4d` — the board's shape as it stands: four Prisma reads became two raw panel reads plus one `count(*) FILTER` pass, with `status` interpolated literally because a bound enum cannot prove the partial index's predicate. Net: 10 statements → 8, and the planner actually uses `Attempt_live_by_test_idx`.
- `b96a89d`, `ad4139e`, `7c9edee` — `TestStat` stopped storing the mean, highest, lowest and the topper. All four are now counted live per request (`cohortCurveOf`, `rankOneSql`), and the stored sums are turned into averages on read.
- `0f94ae9` — the leaderboard collapsed to one per-paper board; `test-analytics.service.ts` now reads the topper's display name through `boardName`.
- `10f929d` — moved the live-ops and analytics screens next to the components only they use; no behaviour change.
- `06a5e80` / `eab64e1` — added the test-report workbook, then made the audit row state what was actually written (`results` / `absent` counts).

What is wrong with it

- `attempt-resolution.service.ts:124` — `state.take(attemptId)` discards its return. A void of a live sitting throws away everything saved to Redis since the last flush (up to ~25 s of answers), where `submit.service.ts:171-177` writes a stray state to the sheet before dropping the key. Input: void a sitting while the student is answering; `AttemptSheet.answers` then disagrees with what they did. Defensible for a voided sitting, but it is a silent loss and the opposite of what the sibling path does.
- `apps/admin/src/features/tests/test-analytics.tsx:37` — the badge reads `Updating: {evaluatedCount} of {liveEvaluatedCount} counted`, and `isSettling` is a `!==( test-analytics.ts:77 )`, so it fires in both directions. Input: void a ranked sitting; until the rebuild lands the screen reads "Updating: 20 of 19 counted".
- `rollup.service.ts:239-240` vs `:221` — `writeSectionTotals` filters `status='EVALUATED' AND isGraded`, while `writeTestTotals` uses `IN_COHORT`, which also requires `score IS NOT NULL`. An EVALUATED, graded sitting with a null score would be counted in `TestSectionStat` and not in `TestStat`. The scorer always writes a score, so this is latent rather than live.
- `apps/admin/src/features/live-ops/sitting-actions.tsx:68` — "Section timers start again from where the paper says" describes only the lost-key branch: with the key still present, `mergedOver` keeps `held.sections` (`attempt-state.service.ts:386`) and the section clocks do **not** restart. The copy is right for the case the action exists for and wrong for the case where the menu is still offered.
- `attempt-resolution.service.ts:78` — `extendedEndsAt` reads the **row's** `endsAt`, while `pushDeadline` overwrites Redis unconditionally. If Redis ever holds a later deadline than the row — a resume whose credit reached Redis (`attempts.service.ts:87`) but whose row update then threw (`:90`) — an extend would shorten the student's clock. Narrow, but the two writes are not symmetric.

Part IV — The clients

Three clients, one engine. The student web app and the Expo app are two compositions over the same exam engine in `packages/app-kit` — DOM-free on purpose, so the rules of a sitting cannot diverge between a phone and a browser. The admin app shares the tokens and the primitives but almost none of the composition: it is dense and data-first where the student language is airy and task-first.

What is worth reading closely here is the boundary — which code is allowed to live where, and what enforces it. Mobile may not import `packages/ui`, and that is a lint rule, not a convention.

The student portal

Where it lives

`apps/exams/src/{App.tsx,main.tsx,lib,providers,components,features}`; features are `auth`, `dashboard`, `tests`, `exam`, `performance`, `review`, `leaderboard`, `notifications`, `account`. The sitting's engine is not here — it is `packages/app-kit/src/exam/`.

- `apps/exams/src/App.tsx` — the whole route table; every authed screen is `React.lazy`.
- `apps/exams/src/lib/constants.ts` — `ROUTES`, `NAV_ITEMS`, `STORAGE_KEYS`, `PALETTE_LEGEND`.
- `apps/exams/src/features/exam/exam.tsx` — the exam hall: starts the sitting, builds the view, handles stand-down.
- `apps/exams/src/features/exam/exam-shell.tsx` — picks the skin, owns the fullscreen nag and the submit confirm.
- `apps/exams/src/features/exam/templates/shared/slots.tsx` — the slots both skins fill (`Header`, `SectionBar`, `QuestionPanel`, `Options`, `Palette`, `BottomBar`, `PaperWatermark`, private `Timer`).
- `apps/exams/src/features/performance/report.tsx` — `ReportShell`: one sitting, five tabs as routes.
- `apps/exams/src/components/ui/index.ts` — this app's own composition layer.
- `apps/exams/src/lib/queries.ts` — every student query, bound from app-kit's `createStudentQueries`.

What it is for

The SPA a student sits a mock test in: the walk up to a paper (`tests` → `about` → instructions), the sitting, and everything read afterwards. It owns none of the sitting's rules — `packages/app-kit/src/exam/` holds the engine and `packages/contracts` the clock and palette maths, so the Expo client behaves identically. It was `apps/test` until `7912c96`.

Route map (`App.tsx:110–203`; patterns `lib/constants.ts:7–43`)

PATH	SCREEN FILE	FRAME / PURPOSE
<code>/login</code>	<code>features/auth/login.tsx</code>	own; PIN, OTP, first-PIN setup
<code>/railway-preview</code>	<code>features/exam/railway-preview.tsx</code>	own; DEV-only (<code>App.tsx:112</code>), railway skin on a canned paper
<code>/tests/:testId/instructions</code>	<code>features/exam/test-instructions.tsx</code>	own, outside <code>AppShell</code>
<code>/tests/:testId/exam</code>	<code>features/exam/exam.tsx</code>	own, outside <code>AppShell</code>
<code>/</code>	<code>features/dashboard/dashboard.tsx</code>	<code>PageFrame</code> ; next sittable, recent results, trend
<code>/tests</code>	<code>features/tests/tests.tsx</code>	<code>PageFrame</code> ; a shelf per series — declared deviation from <code>ListView</code> (<code>tests.tsx:1–4</code>)
<code>/series/:seriesId</code>	<code>features/tests/series.tsx</code>	<code>PageFrame</code>
<code>/tests/:testId/about</code>	<code>features/tests/test-about.tsx</code>	<code>PageFrame</code> ; brief + when it opens
<code>/attempts/:attemptId/submitted</code>	<code>features/exam/submitted.tsx</code>	<code>PageFrame</code> ; handed in, marking queued
<code>/attempts/:attemptId/report</code>	<code>features/performance/report.tsx</code>	<code>PageFrame</code> ; index tab → <code>score-card.tsx</code>
<code>... /subjects</code> <code>/solutions</code> <code>/questions</code> <code>/compare</code>	<code>performance/subject-report.tsx</code> , <code>review/review.tsx</code> , <code>performance/question-report.tsx</code> , <code>performance/compare.tsx</code>	report tabs
<code>/attempts/:attemptId/{score-card,review,questions}</code>	<code>report.tsx</code> <code>ReportRedirect</code>	legacy links, <code><Navigate replace></code>
<code>/performance</code>	<code>features/performance/overview.tsx</code>	<code>PageFrame</code> ; figures across sittings
<code>/leaderboard</code>	<code>features/leaderboard/leaderboard.tsx</code> + <code>board.tsx</code>	<code>PanelFrame</code>
<code>/saved</code>	<code>features/review/saved.tsx</code>	<code>ListView</code> + <code>useListScreen</code>
<code>/notifications</code> , <code>/notifications/settings</code>	<code>features/notifications/*.tsx</code>	<code>PanelFrame</code>
<code>/profile</code> , <code>/account</code>	<code>features/account/{profile,account}.tsx</code>	<code>PageFrame</code>
<code>*</code>	<code>src/not-found.tsx</code>	own

- 1 Click `StartSitting`
`features/exam/start-sitting.tsx:22`
asks fullscreen on the gesture, links to the instructions.
- 2 GET `/me/tests/:testId/brief`
`features/exam/test-instructions.tsx:26` `briefQuery`
brief cached.
- 3 Prefetch `/me/tests/:testId/paper` and `import()` `./exam` + `./submitted`
`test-instructions.tsx:21-38`
paper and both chunks held before Begin.
- 4 Skin chosen for the pre-test screens too
`features/exam/instructions-shell.tsx:22`
`useInstructions` holds step, language, declaration.
- 5 Begin → `fullscreen.enter()` then `navigate('/tests:id/exam', { state: { languages } })`
`test-instructions.tsx:81-85`
- 6 `useStartedSitting(api, testId, { languages, tab: tabId(), resume })`
`apps/exams/src/features/exam/exam.tsx:35`
- 7 POST `/me/tests/:testId/attempt` with `holdsPaper`
`app-kit/src/exam/start-sitting.ts:65-82`
`controller @Actors(STUDENT) @SittingRateLimit()` `@Post @HttpCode(OK)` (`apps/api/src/attempt/attempt.controller.ts:63-66`); `staleTime: Infinity`, `gcTime: 0`, 3 tries only on a hang or fault. Server stamps `startedAt` / `endsAt` (`apps/api/src/attempt/attempt.service.ts:127-140`) and returns `serverNow` (`attempt.controller.ts:80`).
- 8 Paper resolved cheapest-first, `arrivedAt: Date.now()` stamped where the payload lands
`start-sitting.ts:86-98` + `exam/served-paper.ts:16-38`
held test paper reshuffled with `started.shuffleSeed`, else `started.paper`, else GET `/me/attempt/:id/paper`.
- 9 SITTING_ENDED on a `?resume=` start → `/attempt/:id/submitted`
`exam.tsx:43-44`
- 10 `useExamView(sitting, { api, focus, catalogQueryKey, tab, answerQueue })`
`exam.tsx:112-118`
queue backed by `sessionStorage`.
- 11 Local answers rebuilt from the stored queue
`app-kit/src/exam/use-attempt-state.ts:131-137`
- 12 Only a resumed sitting GETs `/me/attempt/:id/state`; `startedByThisCall` short-circuits it and sets `sectionsSettled`
`use-attempt-state.ts:166-200` (e20bb7e)
seed merges under local answers, revision via `seedRevision`, a retryable failure retries at `autosaveDelayMs()`.
- 13 The clock is `state.clock ?? paperClock` (`use-exam-view.ts:76-81`); `ExamTimer` runs `useClockCountdown` → `secondsLeft({endsAt, serverNow, arrivedAt}, Date.now())` (`packages/contracts/src/attempt.ts:265-269`). The client subtracts elapsed device time from a server-granted span; it never treats a wall clock as the deadline. A save's reply re-anchors it (`use-attempt-state.ts:256`)
that is how an extension lands. Sectional papers use `SectionTimer` + `useAnchoredCountdown`
- 14 Marking: `chooseOption` / `bubbleAnswer` / `markAndNext` / `clearResponse` → `record` (`use-exam-view.ts:213-216`, 256-278) → `state.answer` (`use-attempt-state.ts:386-398`)
banks seconds and `firstActionAt`, writes local state and `queue.put`, mirrored synchronously to `sessionStorage` (`exam/pending-answers.ts:40-57`). A failed save never takes an answer off the screen
- 15 Autosave: a rescheduling `setTimeout` at 25s ± 5s jitter (`autosave-policy.ts:8-27`, `use-attempt-state.ts:326-334`); an early flush at 20 unsent, only when nothing is in flight and the last save did not fail (`:393`).
- 16 `flush` waits out any in-flight save, then PATCH `/me/attempt/:id/state` with `{revision, answers≤200, sections, tab}` under a 15s abort; a backlog past `SAVE_BATCH_MAX` (200) goes up in consecutive whole batches
`use-attempt-state.ts:271-284` (6870d1d)
- 17 Ack: `applied !== false` → `queue.acknowledge(batch)`, deleting only the identical change objects, so one rewritten mid-flight survives
`pending-answers.ts:65-73`
- 18 Offline or refused: `failed()` sets `hasUnsaved` → a Not saved yet badge (`templates/shared/slots.tsx:54`). `SITTING_TAKEN_OVER` or `SITTING_SET_ASIDE` stands the tab down
it **stops saving and keeps the queue** (`use-attempt-state.ts:203-223`)
and the screen becomes an `EmptyState` with `stoodDownSays` and a `Continue` here that releases the leave guard and reloads with `?resume=<attemptId>` (`exam.tsx:121-153`), resending the queue as it stands. The per-question drop-on-resume was built (`ef74144`) and reverted (`29269a8`); the multi-device overwrite is an accepted limit.
- 19 Tab closing or backgrounding: `useLeaveGuard` (`packages/app-kit/browser/use-leave-guard.ts`) `preventDefault`s `beforeunload` while unsent and on `pagehide` calls `leave()` → one `keepalive` save (`use-attempt-state.ts:421-431`).
- 20 Navigation: the `Palette` slot (`slots.tsx:167-179`) → `features/exam/question-palette.tsx`
five swatches with counts, `canOpen` disabling seats; `SectionBar` tabs disabled unless in `view.reachable`. Changing section banks the open question and flushes (`use-exam-view.ts:190-199`)
- 21 Submit: `submit.ask` → `ConfirmDialog` in `exam-shell.tsx:38-47` (railway asks over its own paper) → `end()` behind an `ending` ref → `state.bankOpen()` then `state.finish(...)` (`use-exam-view.ts:136-155`).
- 22 `finish` waits ≤5s on an in-flight save, sets `stopped`, drains the backlog, then the last batch rides POST `/me/attempt/:id/submit` under a 10s abort with 1/2/4s retries
`use-attempt-state.ts:286-319`, `autosave-policy.ts:20-45`
- 23 Success: catalog marked stale without refetch, `focus.exit()`, `navigate('/attempt/:id/submitted', { replace: true, state: EndedSitting })`
`use-exam-view.ts:162-171`, `exam.tsx:94-96`. `SubmittedPage` draws per-section attempted/unattempted from that router state alone, plus an Alert that marking is queued

24 Failure after retries: `ending = false`, `state.resume()`, a danger `Alert` + Try again in `BottomBar` (`slots.tsx:184-191`).

Forward-only navigation — both halves verified

- Client: `forwardOnly = paper.navigation === FORWARD_ONLY` (`use-exam-view.ts:84`); `canOpen` → `mayOpenQuestion` (`packages/contracts/src/attempt.ts:378`); a reload lands on `furthestSeat` (`use-exam-view.ts:114-118`); `record` strips `marked` (`:215`); `nextQuestionId` gets `wrap: false`; legends and tallies filter through `isStateShown`.
- API: `withinReach` drops any change to a seat behind its section's furthest — `apps/api/src/attempt/attempt-state.ts:176-194` — called from `inReach`, which rebuilds the order from the per-process paper cache using the live key's `shuffleSeed` (`attempt-state.service.ts:216-229`). The rest of the batch still lands. Ceiling (`9123b02`): order is only visible *between* saves.

The two screens outside `AppShell` — `App.tsx:130-140` puts both inside `ProtectedRoute` but **above** `<Route element={}<AppShell />>`. The stated reasons: an exam hall has no navigation out of it (`lib/constants.ts:18`), and the walk up to a paper is not a page to browse (`:15`). So neither gets the nav rail, breadcrumbs, unread badge or `TourProvider`, and neither is subject to the default-PIN gate or the test-blocked banner (`components/app-shell.tsx:98-105`). The sitting additionally calls `useLockedZoom` (`features/exam/lock-zoom.ts`) and is `select-none h-dvh` (`exam-shell.tsx:31`).

The result screens

SCREEN	FILE	ENDPOINT
Score card	<code>features/performance/score-card.tsx</code> (<code>Hero</code> + <code>TileGrid</code>)	<code>GET /me/attempt/:id/scorecard</code>
Sections	<code>features/performance/subject-report.tsx</code> (<code>Section</code> + <code>DataTable</code>)	the same scorecard query
Solutions	<code>features/review/review.tsx</code> → <code>review-paper.tsx</code>	<code>GET /me/attempt/:id/solutions?sectionId=</code> (per section, <code>3e10825</code>) + bookmarks-in-attempt
Questions	<code>features/performance/question-report.tsx</code> (<code>tab fills</code> , <code>ListView</code> + <code>StatBand</code>)	<code>GET /me/attempt/:id/question-report</code>
Compare	<code>features/performance/compare.tsx</code>	<code>scorecard</code> + <code>GET /me/performance</code> + <code>GET /me/leaderboard?testId=</code> (only when <code>cohortSize > 0</code>)
Leaderboard	<code>features/leaderboard/leaderboard.tsx</code> + <code>board.tsx</code>	<code>GET /me/performance</code> for the test list, then <code>GET /me/leaderboard?testId=</code>
Overview	<code>features/performance/overview.tsx</code>	<code>GET /me/overview</code> + <code>GET /me/performance</code>

All five tabs route a 409 through `isMarkingPending` (`packages/app-kit/src/markings.ts`) so a still-queued scoring job reads as `No marks yet` with a `Retry`, not a failure. `ReportShell` owns the attempt `Combobox`, the breadcrumb tail, and a tooltip naming a retake as `unranked`.

The design language — `src/components/ui/` as it stands

Nine files, re-exported by `index.ts`: `PageBody` (the `gap-10` rhythm), `Hero` + `HeroFigure` (eyebrow/title/meta/figure/aside), `Section` (`SectionHeading` + content at `gap-4`), `SurfaceCard` (one card at `p-5`, never nested), `DividedList` + `DividedRow` (rows in **one** card parted by hairlines — the student answer to a stack of little cards), `Shelf` (sideways scroll inside the page's vertical one), `StatBand` (`MetricGroup` in one card), `TileGrid` + `StatTile`, and `skeletons.tsx` (`Band/Tiles/Block/BlockPair/Rows/Page/Report`). All are compositions over `packages/ui` primitives: no colour, no spacing value, no hex (`src/index.css:5-7` says so). Against admin's `PageHeader` + `ListView` + `DataTable` default, the default here is a feed or a shelf — `tests.tsx:1-4` and `notifications.tsx:1-5` each carry a written justification for deviating; only `saved.tsx` and the questions tab use `ListView`. `Hero`, `Section`, `SurfaceCard`, `DividedRow` and `Shelf` all take `meta`, never a `description`.

The data it touches

client storage only; the sitting's durable state belongs to the API module.

KEY	STORE	HOLDS	WHO WRITES
<code>iace.test.auth</code>	<code>localStorage</code>	<code>access</code> + <code>refresh token</code>	<code>createBrowserTokenStore</code> (<code>lib/api.ts:7</code>)
<code>iace.test.tab</code>	<code>sessionStorage</code>	this tab's <code>uuid</code> , sent as <code>tab</code>	<code>lib/tab-id.ts:6</code>
<code>iace.test.queued.<attemptId></code>	<code>sessionStorage</code>	JSON array of unsent <code>AnswerChange</code>	<code>pending-answers.ts:40-48</code>
<code>iace.test.tours</code>	<code>localStorage</code>	tours this device has seen	<code>TourProvider</code> (<code>app-shell.tsx:64</code>)

Section progress is **not** mirrored (`pending-answers.ts:58-60`): a reload loses an unsent section stamp, which the server's own stamp then supplies.

What must already be true

a student session; a test visible in `GET /me/catalog` and open; `hasDefaultPin` `false` (else every shell screen is replaced by `ChangePinCard`, `app-shell.tsx:111-119`); `isTestBlocked` `false` for a new attempt; `VITE_API_URL` (default `http://localhost:3000`, `lib/api.ts:10`); `crypto.randomUUID` (`/railway-preview` needs `import.meta.env.DEV`).

Why it is shaped this way

- The paper is fetched before `Begin` and reshuffled client-side from the start's seed — not one paper `GET` per sitting at the bell, which is the most expensive answer at a synchronised open.
- `arrivedAt` is stamped in the `queryFn`, not a render (`start-sitting.ts:88-94`) — not the device clock, so a skewed clock still counts down correctly.
- The queue is a plain list in `sessionStorage`, not a per-question version vector; the version attempt was reverted (`29269a8`) because the client cannot judge it, at the cost of a documented multi-device overwrite.
- Skins are tokens plus an `ExamTemplateConfig` (`slots.tsx:1-5`, `exam-shell.tsx:29`), not a second sitting screen per skin, which would drift in behaviour.

Is it optimal

Opening a paper is 2 requests warm (`holdsPaper` suppresses the server's copy) and 3 cold;

a fresh sitting makes **no** state GET (`e20bb7e`). Steady state is one `PATCH` per $\sim 25s \pm 5s$ plus one early flush per 20 answers, so a 100-question paper is $\sim 5-10$ saves, not 100. Submit is one `POST` carrying the last batch. Three report tabs share one `scoreCardQuery` — one request, not three. Solutions are per section (`3e10825`). Every authed chunk is lazy (`App.tsx:19-96`) and each suspense fallback is the shape it becomes. All read from source; nothing measured.

Shared machinery — `packages/app-kit/src/exam/*` (start, state, countdown, queue, focus guard); `app-kit/browser` (`ProtectedRoute` , `AppShell` , `useFullscreen` , `useLeaveGuard` , `PageCrumbs` , `usePageTour` , `useListScreen` , `useFilterSpec` , the figures); `createStudentQueries` and the named query keys, so both student clients cache identically; `createAppQueryClient({ notify: toast })` for one mutation-failure announcement; `packages/contracts` for the envelope, `ErrorCodes` , the clock and palette maths and the route constants; `packages/ui` for every primitive and token.

What changed recently

(`git log -15 -- apps/exams apps/test`)

- `0e4a1fe` — gave the app a component-test harness: `test/tsconfig.json` , `test/support/offline.ts` (refuses network before the api module captures `fetch`) and the first spec.
- `7912c96` / `34f8380` — `apps/test` → `apps/exams` , and `src/routes/*` + `src/components/*` folded into `src/features/<area>/` . No behaviour change; every older path is wrong.
- `e20bb7e` — a sitting the screen just started skips `GET /me/attempts/:id/state` and settles sections itself; a reload is unchanged.
- `35cae83` — the overview's standing figures are counted over the five newest sittings only.
- `3e10825` / `0f94ae9` — the answer key is held per paper and served one section at a time; the leaderboard is one paper's board.

What is wrong with it

- `lib/constants.ts:112` — the heading says "localStorage keys owned by this app", but `TAB` and `QUEUED_ANSWERS` go through `browserSessionStorage` (`lib/tab-id.ts:7` , `exam.tsx:117`). Two of the four are not localStorage.
- `features/exam/exam-shell.tsx:74-75` — two stacked one-line doc comments on `nagSays` ; the first ("Over the paper, because the answers it names are already gone from it") describes a submit summary, not the fullscreen nag. Leftover from a moved function.
- `src/not-found.tsx:16-17` — the button reads "Go to your tests" but links to `ROUTES.HOME` (`/`), the dashboard; the tests list is `/tests` .
- `packages/app-kit/src/exam/start-sitting.ts:102` — `forget` is returned but only `apps/mobile/app/exam/[testId].tsx:72` calls it; `apps/exams` relies on `gcTime: 0` plus the full `window.location.replace` in `continueHere` (`exam.tsx:150-153`). Dead for this app, not a bug.
- No dead screen or unused component found: every file under `src/components/ui/` and `src/features/exam/` is reachable. `railway-preview.tsx` and `templates/railway/preview-data.ts` are live but DEV-gated (`App.tsx:112`); the four PNGs in `templates/railway/assets/` are that skin's sprites.
- Pinned regression tests: `apps/exams/test/performance-report.dom.test.tsx` — 3 cases, all on the marking-pending vs read-failed split (a 409 makes `SubjectPanel` say "No marks yet"; `ComparePanel` says the same off the shared card; a 500 says "Your sections did not load" and Retry re-fetches, asserted by call count). The sitting has **no** test in this app — its cover is `packages/app-kit/test/{use-attempt-state,use-exam-view,use-countdown,start-sitting,pending-answers,autosave-policy,served-paper,use-leave-guard,use-fullscreen}` .

The admin portal

Where it lives

`src/App.tsx` (the whole route table; 32 `React.Lazy` chunks, login and

not-found static), `src/lib/constants.ts` (756 lines: `ROUTES` , `NAV_ITEMS` , `QUERY_KEYS` , 59 query-key builders, `STORAGE_KEYS` , label maps), `src/lib/api.ts` (16 lines), `src/providers/auth.ts` (33 lines), `src/components/app-shell.tsx` , `src/features/authoring/authoring-workspace.tsx` (735 lines — the card-per-question workspace shared by the bank question page and section authoring), `src/features/tests/test-builder.tsx` . Twelve `src/features/<area>/` folders hold their own screens plus the components nothing else imports; `src/components/` keeps the 13 pieces with 2+ consumers.

What it is for

Everything staff do that is not sitting a test: students and their imports, the

exam catalog, the question bank with its authoring/proof-reading pipeline, base configs, tests and series, live sitting operations, announcements, the audit log. It holds no authority of its own — every screen's gate is mirrored by `@RequiresFeature` or `@RequiresSuperAdmin` on the controller.

1. The route map

All rows sit inside `ProtectedRoute` → `AppShell` except `/login` and `*` . Paths are the `ROUTES` values at `src/lib/constants.ts:68-149` ; all 60 entries are referenced at least once.

ROUTE	SCREEN FILE (UNDER <code>APPS/ADMIN/SRC/</code>)	FOR
<code>/login</code>	<code>features/auth/login.tsx</code>	Email → OTP; no signup
<code>/</code>	<code>features/dashboard/dashboard.tsx</code>	Four bands, each drawn if its key is held
<code>/students</code>	<code>features/students/students.tsx</code>	The directory
<code>/students/import</code>	<code>features/imports/import-students.tsx</code>	Sheet preview, then commit
<code>/students/:id</code>	<code>features/students/student-detail.tsx</code>	One student, tabbed
<code>/students/:id/performance</code>	<code>features/students/student-performance-redirect.tsx</code>	9-line <code>Navigate</code> to the Performance tab

ROUTE	SCREEN FILE (UNDER <code>APPS/ADMIN/SRC/</code>)	FOR
<code>/branches</code>	<code>features/catalog/branches.tsx</code>	Branches
<code>/exams</code>	<code>features/catalog/exams.tsx</code>	Exams and stages
<code>/cohorts</code>	<code>features/students/cohorts.tsx</code>	Programs and events, one <code>?tab=</code>
<code>/cohorts/events/:id/import</code>	<code>features/imports/import-event-candidates.tsx</code>	Candidate sheet
<code>/cohorts/programs/:code/import</code>	<code>features/imports/import-program-students.tsx</code>	Enrol by program code
<code>/questions</code>	<code>features/questions/questions.tsx</code>	The bank list
<code>/questions/new</code>	<code>features/questions/bank-question.tsx</code>	Blank editable card
<code>/questions/import</code>	<code>features/imports/import-questions.tsx</code>	Question sheet
<code>/questions/taxonomy</code>	<code>features/questions/taxonomy.tsx</code>	Subjects and topics
<code>/questions/:id</code>	<code>features/questions/bank-question.tsx</code> (<code>readOnly</code>)	Read a question
<code>/questions/:id/edit</code>	<code>features/questions/bank-question.tsx</code>	Correct it
<code>/proofreading/assignments</code>	<code>features/authoring/assignment-queue.tsx</code> (<code>PROOFREADER</code>)	A reader's sections
<code>/proofreading/assignments/tests/:testId/sections/:sectionId</code>	<code>features/authoring/section-authoring.tsx</code>	Read that section
<code>/authoring</code>	<code>features/authoring/authoring-editor.tsx</code>	The typist's one-box editor
<code>/authoring/:id</code>	<code>features/authoring/authoring-editor.tsx</code>	Their own draft reopened
<code>/authoring/history</code>	<code>features/authoring/authoring-history.tsx</code>	What this author typed
<code>/authoring/assignments</code>	<code>features/authoring/assignment-queue.tsx</code> (<code>TYPIST</code>)	A typist's sections
<code>/authoring/assignments/tests/:testId/sections/:sectionId</code>	<code>features/authoring/section-authoring.tsx</code>	Type that section
<code>.../sections/:sectionId/import</code>	<code>features/imports/import-questions.tsx</code>	A sheet into one section
<code>/tests</code>	<code>features/tests/tests-and-series.tsx</code>	Tests and series, one toggle
<code>/tests/new</code> , <code>/tests/:id</code>	<code>features/tests/test-builder.tsx</code>	Setup → Paper → Offer
<code>/tests/:id/paper</code>	<code>features/tests/test-paper.tsx</code>	The paper alone
<code>/tests/:id/analytics</code>	<code>features/tests/test-analytics.tsx</code>	Cohort rollups, re-sync poll
<code>/tests/:testId/sections/:sectionId</code>	<code>features/authoring/section-authoring.tsx</code>	The owner's seat
<code>/tests/configs</code> , <code>/tests/configs/new</code> , <code>/tests/configs/:id</code>	<code>features/catalog/base-configs.tsx</code> , <code>base-config-form.tsx</code>	Stage blueprints
<code>/tests/series/new</code> , <code>/tests/series/:id</code>	<code>features/test-series/test-series-form.tsx</code>	One series, four views
<code>/tests/section-progress</code>	<code>features/authoring/section-progress.tsx</code>	Every section's state
<code>/live-ops</code>	<code>features/live-ops/live-ops.tsx</code>	Sittings board, 15s poll
<code>/admins</code>	<code>features/admins/admins.tsx</code>	Super-admin only
<code>/permissions</code>	<code>features/admins/permissions.tsx</code>	Super-admin only
<code>/announcements</code>	<code>features/announcements/announcements.tsx</code>	What was said, to whom
<code>/audit</code> , <code>/audit/imports</code>	<code>features/admins/audit.tsx</code> (<code>AuditActivityPage</code> , <code>AuditImportsPage</code>)	Row actions, import runs
<code>*</code>	<code>components/not-found.tsx</code>	Outside the gate AND the shell

2. The shell

`main.tsx:20` calls `mountApp` (`packages/app-kit/browser/mount-app.tsx:15-27`): `StrictMode` → `QueryClientProvider` → `ThemeProvider` → `TooltipProvider` → `BrowserRouter`, `Toaster` inside the theme and outside the router. `AuthProvider` wraps `<App />`.

`App.tsx:160` renders `ProtectedRoute` (`packages/app-kit/browser/protected-route.tsx:5`): a full-page `Spinner` while `isLoading`, else `<Navigate to="/login" state={{from: location.pathname}}>`. Tokens live in `localStorage` under `iace.admin.auth` (`constants.ts:751`) via `createTokenStore` (`packages/app-kit/src/token-store.ts:19`) — a per-SPA key, because both SPAs share one origin. Refresh is in the API core, not React: a 401 is peeked (`packages/contracts/src/client/core.ts:207`), `SESSION_REPLACED` ends the session at once, `UNAUTHENTICATED` goes through a single-flight `refreshTokens()` with a 10s timeout and jittered 1/2/4s backoff, then the request is retried once with the new token. A failed refresh calls `onUnauthorized` → `tokenStore.clear()` + `signOutSignal.emit` (`packages/app-kit/src/api-client.ts:37`), which `createAuth` answers with `queryClient.clear()` (`create-auth.tsx:86`) — the whole cache, not just the identity. `/auth/me` re-reads the identity at `staleTime` 5 min, so a permission change lands on reload.

Nav: `NAV_ITEMS` (`constants.ts:423`) is nine sections of leaves. `filterAdminNav` (`constants.ts:510`) strips `superAdminOnly` before the shell sees it; the shell then runs `filterNavByPermission(nav, can)` and `collapseLoneSections` (`app-shell.tsx:85` , `packages/app-kit/src/nav.ts:54,69`). Entries are **hidden**, **never disabled**, at READ level (`providers/auth.ts:30` defaults it), and a section emptied of children goes with them. `can` also requires `isActive` (`packages/contracts/src/admins.ts:140`); a deactivated admin additionally gets an empty nav and a `REFUSED` `EmptyState` in place of `<Outlet/>` (`components/app-shell.tsx:22,41`). Routes exist for everyone — `/admins` and `/permissions` wrap themselves in `SuperAdminOnly` (`features/admins/super-admin-only.tsx:6`).

3. Data fetching

One client from `createAppQueryClient({notify: toast})` (`main.tsx:18` , `packages/app-kit/src/query-client.ts:40`): `staleTime: 30_000` , `refetchOnWindowFocus: false` , `retry` once and only when `isWorthAskingAgain` (`httpStatus 0, 429, or ≥500`), delay doubling with a full base of jitter. `gcTime` is never set, so it is TanStack's 5-minute default. Overrides: exam and branch pickers 5 min (`features/students/use-exams.ts:25` , `lib/use-branches.ts:34`), an import draft `Infinity` (`features/authoring/import-workspace.tsx:110`), live-ops `refetchInterval 15s` (`features/live-ops/live-ops.tsx:70`), analytics a conditional 3s poll capped at 150s (`features/tests/test-analytics.tsx:90` , `constants.ts:557,560`).

Keys: `QUERY_KEYS` (`constants.ts:523`) is 30 roots, all prefixed `'admin'` except `ME` . Every key is built by a named helper below it (`constants.ts:585-744`). `QUERY_SCOPES` (`constants.ts:569`) — `named` , `picker` , `filter` , `held` , `all` — went module-private in 7f6eb16: a picker offers only live rows and a filter must reach retired ones, so the two cannot share an entry. A record's key nests under its list's root, which is the invalidation strategy: 53 `invalidateQueries` calls across 36 files mostly name a root prefix, and the detail entries fall out with it. `refetchType: 'none'` is used where the server's own answer was already written in by `setQueryData` (`test-builder.tsx:179,185` , `student-detail.tsx` , `test-series-access.tsx`).

Errors: no screen composes a message. `MutationCache.onError/onSuccess` toast once, honouring `meta.silent` , `meta.fields` and `meta.success` ; `QueryCache.onError` toasts a failed read after the retries. `bannerMessage(error, fields)` returns `null` when every message already sits on a form field (`packages/app-kit/src/form-errors.ts:37`), and `applyFieldErrors` feeds react-hook-form. `error.code` is read directly exactly once in the app — `features/authoring/section-authoring.tsx:132` , `ErrorCodes.NOT_FOUND` ; `test-builder-offering.tsx:307` instead keys off the `fieldErrors` key shape via `programOpeningField` .

4. Flow trace: finalize a test from the UI

- Admin on `/tests/:id` (Offer step) clicks **Done** — `features/tests/test-builder.tsx:364` — `BuilderFooter` — a `<Link to="/tests">` carrying an `onClick` .
- `finish` runs — `test-builder.tsx:221` — plain navigation when `unsavedOffer === 0` .
- Else `preventDefault` , re-judge the clock — `use-offer-draft.ts:46` — `recheck()` — `true` means an opening just passed; the fields show it and nothing opens.
- `setAsking(true)` — `test-builder.tsx:224` — the one confirm mounts.
- Each pending write listed in words — `test-builder-offering.tsx:285` — `changeLines()` .
- Confirm — `test-builder-offering.tsx:378` — `save.mutate()` ; `onMutate` clears any prior program refusal.
- Body from the held draft — `test-offer-draft.ts:82` — `offeringBodyOf()` — `{opensAt, programOpenings[], offered, expectedVersion}` .
- `api.admin.tests.saveOffering` — `packages/contracts/src/client/admin.ts:591` — `write('PUT', ...)` — Bearer token + `x-client-kind: WEB` .
- `PUT /admin/tests/:id/offering` — `apps/api/src/tests/tests.controller.ts:184` — `@Audit(TEST, UPDATE)` , `@RequiresFeature(TEST_MANAGEMENT, WRITE)` , `@Put('/:id/offering')` ; `class @Controller('admin/tests')` + `@Actors ADMIN` ; global `JwtAuthGuard` , `ActorGuard` , `FeaturePermissionGuard` , `SuperAdminGuard` , `ThrottlerGuard` (`app.module.ts:72-82`).
- `$transaction(TX_LIMITS.MEDIUM)` opens — `apps/api/src/tests/offering.service.ts:245` .
- `SELECT 1 FROM "Test" ... FOR UPDATE` — `apps/api/src/common/paper-edit.ts:18` — `beginPaperEdit()` — the app's first lock, Test before Question.
- Version check — `offering.service.ts:248` — `CONFLICT` if `test.version !== body.expectedVersion` .
- `writeOpening` — `offering.service.ts:274` — skips on same minute; else refuses if any `Attempt` exists, asserts the time is ahead, checks series order when `sequentialTests` , writes `Test.opensAt` .
- `writeProgramOpenings` — `offering.service.ts:294` — refuses duplicate codes; the kept set replaces `TestProgramUnlock` , dropping rows the new opening overtook.
- Branch `body.offered && status !== ACTIVE` — `offering.service.ts:255` — `finalizer.offerWithin()` .
- Outstanding proof-reading — `apps/api/src/tests/finalize.service.ts:100` — skipped for a super admin; else `VALIDATION_ERROR` naming the sections.
- Already finalized → only `status` returns to `ACTIVE` — `finalize.service.ts:51` .
- Else `assertPaperIsWhole` , then ONE write — `finalize.service.ts:61` — `finalizedAt = now, status = ACTIVE, version += 1` , so the two can never land apart.
- `version` incremented again by the caller — `offering.service.ts:261` ; `commit` .
- Re-read and announce — `offering.service.ts:442` — sets the audit entity id, emits `ACCESS_CATALOG_CHANGED {testSeriesId}` .
- `{status, opensAt, programUnlocks}` wrapped by `ResponseInterceptor` into `{success:true,data,meta:{requestId}}` .
- `onSuccess` — `test-builder-offering.tsx:352` — `offer.discard()` , `navigate('/tests')` , toast "Test offered to students."
- `onSettled` — `test-builder-offering.tsx:362` — `invalidateQueries({queryKey: QUERY_KEYS.TESTS})` , which also covers `testQueryKey(id)` . `TEST_PAPER` and `ASSIGNMENTS` are left alone.

What must already be true

a `Test` row; `owesAPaper(detail) === false` (`packages/contracts/src/tests.ts:394`);

no attempts if the opening is moving; no outstanding PROOFREADER assignment unless super admin; `expectedVersion` matching.

5. Flow trace: the authoring page

`features/questions/bank-question.tsx` on three routes, rendering `authoring-workspace.tsx` . Not the typist's `/authoring` editor — that is the separate one-box `authoring-editor.tsx` .

- A stem anywhere (bank list, question picker, a paper row, an analytics item) is a `QuestionLink` — `components/question-link.tsx:11` — `target="_blank"` `rel="noreferrer"` to `/questions/:id` , because the workspace draws no breadcrumb and a same-tab click would strand the reader.
- `BankQuestionPage({readOnly: true})` mounts — `bank-question.tsx:17` — keyed `'reading'` (line 98), so the cards are rebuilt on a mode change.
- `source` memoised — `bank-question.tsx:51` — one card `{key: id, lead, editable: false}` , no `create` .
- `CardBody` reads it — `authoring-workspace.tsx:494` — `useQuery(heldQuestionQueryKey(id))` , whose `queryFn` goes through `queryClient.fetchQuery(questionQueryKey(id))` , so the list the reader came from has usually already paid for it.
- `GET /admin/questions/:id` — `apps/api/src/questions/questions.controller.ts:131` — `@RequiresFeature(QUESTION_MANAGEMENT, READ)` — a `QuestionDetail` .
- Mapped to `Held` — `features/authoring/question-scaffold.ts` — `headerOf()` , `stateOf()` , `stamp: question.updatedAt` .
- `editable false` → `ReadBody` — `authoring-workspace.tsx:561` — one bar (lead, actions, a `SegmentedControl` of `EN/HI/TE`) over `AuthoringPreview` . The tabs are **languages, not sections**: that control and the editor's `toolbarEnd` (`question-panes.tsx:75`) are the same `LANGUAGE_ORDER` -driven widget.

8. Save is withheld — `authoring-workspace.tsx:390` — `editable || keys.length > 1`; a lone read-only card gets no button (7a95db8 removed a dead enabled "Next").
9. Edit question — `bank-question.tsx:112` — a `<Link>` to `/questions/:id/edit`; same component, key `'writing'`.
10. EditBody → QuestionPanels — `question-panes.tsx:23` — `ScaffoldEditor` left, `AuthoringPreview` + `AuthoringChecks` right, each pane scrolling alone.
11. Regions from `regionsFor` — `question-scaffold.ts:104` — stem `plain`, options `seat` (A...), then two `named` slots, Answer and Explanation.
12. The dotted rules — `packages/ui/src/components.css:260` — `border-top: 1px dotted` on *any* region followed by a `seat` (so a rule runs above option A, splitting it off the stem) and between two `named` regions (so Explanation does not run into Answer); `:266` keeps a **solid** rule plus `space-4` padding above the first `named` region, and that is what closes the options; `:223` hides a `plain` label, since a stem needs no word in front of it. 8fe3cd4's preview-side rule was reverted in 3f98226 — the preview is also what the read page draws.
13. A keystroke → `onRegions` → `stateFrom(current.state, language, regions)` — `authoring-workspace.tsx:640`.
14. `edit()` compares against the cached saved copy — `authoring-workspace.tsx:234` — `JSON.stringify` equality, so typing back to what is saved **deletes** the entry and "Unsaved" is exact, not sticky. A warning `Badge` shows in the header bar (line 315); `onDirtyChange` tells the page.
15. **Save** — `authoring-workspace.tsx:279` — `saveAndNext()` re-runs `validateQuestion` on what it is about to send, because the rendered issues lag the last keystroke.
16. `source.save` — `bank-question.tsx:70` — `api.admin.questions.update(key, {...toDraft(...), expectedUpdatedAt: held.stamp})` → `PATCH /admin/questions/:id` (`questions.controller.ts:155`, `@Audit(QUESTION, UPDATE)`, `WRITE`); a row that moved since the page read it is refused.
17. Then `invalidateQueries({queryKey: QUERY_KEYS.QUESTIONS})` and `navigate('/questions/:id')` — a save lands you back in read mode.
18. **Back to reading** — `bank-question.tsx:109` — when dirty, a destructive `ConfirmDialog` ("Discard the unsaved changes?") and only then `toReading()`; clean, it navigates at once (cc88ce0 — the shape used when the Offer step is left).
19. Alt+↑/↓ steps between cards app-wide — `authoring-workspace.tsx:221` — captured before the editor, whose plain arrows move within one question, and ignored inside a `[role="dialog"]`.

6. The composition language

`packages/ui` supplies every primitive — `TruncatedText` (192 uses), `Alert` (121), `ConfirmDialog` (63), `PageHeader` (59), `EmptyState` (56), `RowActions` (48), `TableFrame` (42), `ListView` (37), `FormDialog` (36), `Combobox` (31), plus `ScaffoldEditor`, `Stepper`, `FormPanel`, `ImportView`, `Metric` and the tokens/fonts/components/katex stylesheets imported at `main.tsx:3-7`. `packages/app-kit` supplies the session, the query client, `useListScreen` / `useFilters` / `useFilterSpec`, `useImportScreen`, `useLeaveGuard`, `bannerMessage` / `applyFieldErrors`, the nav algebra, tours and `useFullscreen` / `useWorkspace`; `packages/contracts` the typed client, the envelope, `ErrorCodes`, `FEATURE_KEYS`, `validateQuestion` and the IST helpers. `apps/admin/src/components` keeps only `access-picker`, `admin-multi-picker`, `app-shell`, `exam-picker`, `export-button`, `not-found`, `page-skeleton`, `picker-props`, `question-link`, `section-thread`, `stage-cell`, `taxonomy-picker`, `test-status-badges`.

List screens are uniform: `TableFrame` `header=<PageHeader breadcrumbs=<PageCrumbs nav={NAV_ITEMS}></> .../>` around one `ListView` given `list`, the **same** `filters` `spec` it was queried with, `columns`, `rowKey`, `empty` and `emptyFiltered`. 16 screens drive it through `useListScreen` → `useListQuery` (`packages/app-kit/src/use-list-query.ts:46`): one request per `{filters, page, pageSize}`, `keepPreviousData`, the page reset during render rather than in an effect, and the caller's `AbortSignal` riding the fetch. The spec's five kinds (`search`, `choice`, `multi`, `date`, `custom` / `customMulti`) are closed at `packages/ui/src/components/ui/list-view.tsx:57` — one primary search, the rest folded behind "Filters". Cells wrap in `TruncatedText`; row actions sit behind one `RowActions` menu; destructive items open a `ConfirmDialog` naming the consequence (`features/questions/questions.tsx:200-221` is the pattern). A failed page draws "Could not load this list." with `Retry` (`packages/ui/src/components/ui/table.tsx:161`), never an empty list.

The data it touches

no DB row directly. Durable state is `localStorage` (`constants.ts:750`):

`iace.admin.auth` (the token pair), `iace.admin.authoring.<adminId>` (the one-box editor's draft, restored only if something was typed — `authoring-editor.tsx:272`), `iace.admin.tours`. The finalize flow writes `Test.finalizedAt` (`timestampz`, set by the first offer and never cleared — the paper is frozen iff it is set, `prisma/schema.prisma:543`), `Test.status`, `Test.version` (twice per offer), `Test.opensAt`, and `TestProgramUnlock` rows (PK [`testId`, `programCode`], `schema.prisma:960`).

Why it is shaped this way

one 756-line `lib/constants.ts`, because cross-feature invalidation is why the keys were

centralised; splitting them reintroduces two names for one cache entry. No per-feature barrels — an SPA ships as one bundle and nothing is extracted from it; six cross-feature import edges remain on purpose. Read and edit are one component with a `key` swap, not two renderings, because three drawings of a question had drifted and two had lost the correct-option highlight. Nav is hidden, not disabled, since a disabled row advertises what the viewer cannot get — the cost being that hidden routes stay reachable by URL, so each page needs its own refusal.

Is it optimal

read, not measured. Lists are optimal: one request per filter set, previous rows

held, superseded searches aborted. The workspace keeps a live editor only within `LIVE_AROUND = 1` of the card in view (`authoring-workspace.tsx:106,346`). Extra work: the finalize flow's prefix invalidation refetches the tests list *and* the detail though the screen navigates away; live-ops is a 15s poll per open tab; analytics polls 3s while a re-sync lands.

What changed recently

`10f929d` is behaviourally a no-op: 54 route files and 27 loose components

became 12 feature folders, one rename to break a name collision, contents verified identical against the HEAD blobs apart from prettier re-wrapping six imports. 3f98226 / 8fe3cd4 moved the dotted rule above option A and between Answer and Explanation, and reverted the one briefly added to the preview. cc88ce0 added "Back to reading" with a discard confirm and keyed the workspace on the mode. 7a95db8 made `/questions/:id` the workspace in read mode, deleting ~740 lines and turning every stem into a new-tab link. 7f6eb16 un-exported `QUERY_SCOPES`. 0e4a1fe gave the app a test glob that can match `.tsx` at all (it was `test/**/*test.ts`, so a component test would have sat there never running) and optional-chained `import.meta.env` in `lib/api.ts`.

Tested screens — two: `LiveOpsPage` (`apps/admin/test/live-ops.dom.test.tsx`) and `TestSeriesFormPage` (`apps/admin/test/test-series-form.dom.test.tsx`). The other nine admin tests are pure logic (`assignment-filters`, `duration`, `nav-authoring`, `nav-cohorts`, `question-scaffold`, `student-detail-tabs`, `test-offer-draft`, `test-paper-view`, `test-schedule-draft`). Both DOM tests import `test/support/offline.ts` first — it replaces `globalThis.fetch` with a rejecting stub before the API client captures it — and both set `gcTime: 0`, `retry: false` with an explicit `client.clear()`. The four pinned regressions, each proved by reverting its fix: (1) the

series editor reverting a save on Edit → Cancel, the 63e5166 bug — `test-series-form.dom.test.tsx:81` ; its two fix lines are now mutually redundant backstops, so only reverting both fails it; (2) live-ops answering an outage with "Nobody is sitting this test right now" instead of a failure and a Retry — `live-ops.dom.test.tsx:45` ; (3) the report tabs going blank while marking is still queued — `apps/exams/test/performance-report.dom.test.tsx:46` (and `:55` , the same bug on the comparison card); (4) the report tabs going blank on a failed read — `apps/exams/test/performance-report.dom.test.tsx:66` .

What is wrong with it

- `packages/app-kit/browser/protected-route.tsx:21` sets `state={{from: location.pathname}}` and **no SPA reads it** (the only `location.state` read in either app is `test-builder.tsx:142` , for `step`). A deep link to `/tests/abc/paper` while signed out always lands on `/` afterwards: `features/auth/login.tsx:64` hard-codes `ROUTES.HOME` .
- `features/auth/login.tsx` never reads `signedOutReason` , though `createAuth` provides it and `apps/exams/src/features/auth/login.tsx:67` shows it in an `Alert` . An admin whose session was replaced elsewhere is bounced to sign-in with no explanation.
- `/tests/section-progress` is `superAdminOnly` in the nav (`constants.ts:468–474`) and `@RequiresSuperAdmin()` on the server (`apps/api/src/assignments/assignments.controller.ts:97`), but `features/authoring/section-progress.tsx` has no `SuperAdminOnly` wrapper. A non-super-admin holding TEST_MANAGEMENT who types the URL gets a toast and "Could not load this list." instead of the `REFUSED` `EmptyState` `/admins` and `/permissions` show.
- `GET /admin/authoring/tags` (`apps/api/src/questions/authoring.controller.ts:45` , backed by `authoringTagsSchema`) has **no** route constant in `ADMIN_AUTHORING_ROUTES` (`packages/contracts/src/authoring.ts:85`) and no client method; the Tags control is a plain `Input` (`features/authoring/authoring-header-bar.tsx:83–90`). The endpoint and its service query are dead from both SPAs.
- `packages/contracts/src/client/admin.ts:677` is an orphaned doc comment — "One decision over a page of drafts — one request, so nothing is half-approved" — followed by a blank line and then `uploadImage` . The bulk-approval method it described is gone.
- No screen reads an endpoint that no longer exists.** I scripted every `/admin/*` , `/auth/*` , `/me/*` route literal in `packages/contracts/src` against every `@Controller + @Get/@Post/@Patch/@Put/@Delete` pair in `apps/api/src/**/*controller.ts` ; the only unmatched string was `/students` inside a comment at `packages/contracts/src/students.ts:333` .

The mobile client

Where it lives

`apps/mobile/app/**` (expo-router routes), `src/{lib,providers,components}` , `webview/` (the bundled question page), `assets/` .

- `app/_layout.tsx` — splash hold, QueryClient, AuthProvider, TourProvider, and one `<Stack>` whose every screen sits under a `Stack.Protected` guard.
- `src/lib/api.ts` — the single API client + token store; resolves the base URL and **throws at module load** if it cannot.
- `src/lib/secure-storage.ts` — expo-secure-store behind app-kit's synchronous `KeyValueStorage` , via an in-memory mirror.
- `src/providers/auth.ts` — 20 lines: app-kit's `createAuth<StudentIdentity>` , pinned to `ActorTypes.STUDENT` .
- `app/exam/[testId].tsx` — the exam hall; runs app-kit's `useExamView` , hands each view to `ExamSkin` .
- `src/components/account/document-card.tsx` — picker + upload (the newest commit's subject).
- `src/lib/push-device.ts` + `notifications.ts` — FCM registration, deliberately best-effort.
- `webview/build.mjs` — esbuild step inlining the web's `richText` , tokens, KaTeX and Indic fonts into one CSP-locked HTML page; `package.json:7–10` run it before `dev` , `build` , `typecheck` .

What it is for

The Android-first student client for the same mock-test product `apps/exams` serves on the web: catalog, instructions, the sitting, the report, account and documents. It is a second *composition* over one engine, not a second implementation — sitting logic, autosave queue, start rule and submit copy all come from `packages/app-kit` . It exists because IACE students are phone-first, and because a native client can hold the answer queue in SQLite across a process kill. Not launched: no API is deployed, so only the dev-client profile builds.

Screen map

A root `Stack` (`app/_layout.tsx:63` , `headerShown: false`) holding a nested `Tabs` group (`app/(tabs)/_layout.tsx:36`) and nine pushed screens. Authorisation is structural: `Stack.Protected` `guard={Boolean(identity)}` wraps everything (`_layout.tsx:64–79`), `guard={!identity}` wraps `login` (`:80–82`).

ROUTE	FILE	WHAT IT IS
<code>/login</code>	<code>app/login.tsx</code>	Four-step card: sign in, mobile, code, set PIN
<code>/</code> (tab)	<code>app/(tabs)/index.tsx</code>	Home; <code>api.me.testDays</code>
<code>/tests</code> (tab)	<code>app/(tabs)/tests.tsx</code>	Catalog, filtered via <code>testsFilters</code>
<code>/performance</code> (tab)	<code>app/(tabs)/performance.tsx</code>	Chip-switched Overview / Leaderboard panels
<code>/saved</code> (tab)	<code>app/(tabs)/saved.tsx</code>	Saved + mistake questions
<code>/account</code> (tab)	<code>app/(tabs)/account.tsx</code>	Links, active devices, sign out
<code>/series/[id]</code>	<code>app/series/[id].tsx</code>	One series' tests
<code>/test/[id]</code>	<code>app/test/[id]/index.tsx</code>	Pre-clock test detail
<code>/test/[id]/instructions</code>	<code>app/test/[id]/instructions.tsx</code>	Rules + language choice; starts nothing
<code>/exam/[testId]?languages=</code>	<code>app/exam/[testId].tsx</code>	The sitting; <code>gestureEnabled: false</code>
<code>/attempts/[id]/submitted</code>	<code>.../submitted.tsx</code>	<code>headerBackVisible: false</code> , "Handed in"
<code>/attempts/[id]/report</code>	<code>.../report.tsx</code>	Tabbed report over one <code>scoreCardQuery</code>

ROUTE	FILE	WHAT IT IS
/profile , /change-pin , /notifications	app/*.tsx	Account's pushed screens

Tab order and labels: `MOBILE_NAV_ITEMS` (`src/lib/nav.ts:28-34`); paths: `ROUTES` / `DETAIL_ROUTES` / `ACCOUNT_ROUTES` , same file.

FLOW — PIN SIGN-IN, AND SURVIVING A RESTART

- Launch; splash held while the token mirror fills from SecureStore
`app/_layout.tsx:26-28` `hydrate()`; one registered key, `iace.mobile.auth` (`src/lib/api.ts:13`)
- `hydrated` flips, splash hides, providers mount
`_layout.tsx:31-34`
nothing read the token before this.
- `AuthProvider` seeds `hasToken` from the store
`packages/app-kit/src/create-auth.tsx:67`
this is how a restart restores a session.
- If it had one: `GET /auth/me`
`create-auth.tsx:71-80`
`enabled: hasToken` , `staleTime` 5 min, one retry at 1-2 s jitter. No token → the `!identity` guard shows `/login` .
- Mobile + PIN validated client-side
`app/login.tsx:114-116`
`studentLoginSchema` .
- Submit
`app/login.tsx:122` `api.auth.loginStudent(values)`
- Request built
`packages/contracts/src/client/core.ts:94`
headers carry `x-client: MOBILE` and a sanitised `x-device-name` from `Device.modelName` (`app-kit/src/api-client.ts:53-56` , `src/lib/api.ts:34`).
- `POST /auth/student/login`
`apps/api/src/auth/auth.controller.ts:77-85` `@Public()` , `@AuthRateLimit()` , `@Post('student/login')` , `@HttpCode(HttpStatus.OK)` , body via new `ZodBody(studentLoginSchema)`
- Logout check
`apps/api/src/auth/auth.service.ts:184` `pin.assertNotLocked(mobile)`
- Student found by `mobile` with `deletedAt: null` ; PIN verified, else `burnVerifyTime()` so an unknown number costs the same
`auth.service.ts:186-192`. The PIN is `**not unique**` across students; it is only checked against this one row
- Failure → `registerFailure` + `PIN_INVALID` ; inactive → `FORBIDDEN`
:193-196
- Tokens signed, session id minted
:315-327
access 15 m, refresh 30 d (`.env.example:84-85`).
- Session written to Redis
`apps/api/src/auth/session.service.ts:65`
`session:student:<studentId><sessionId>` , TTL = refresh TTL; id added to `sessions:student:<studentId>` (`:68-70`). First, `replaceSameKind` tombstones the previous `MOBILE` session: one session per client kind per student (`:50-52`).
- `{ success: true, data: { tokens, identity } }` validated
`packages/contracts/src/client/core.ts:149`
- `signIn(session)`
`create-auth.tsx:100-109`
`tokenStore.set` , `hasToken` true, identity seeded into `ME_QUERY_KEY` , then `onSignedIn` .
- Tokens persisted
`token-store.ts:31-36` → `secure-storage.ts:22-26`
mirror written synchronously, `setItemAsync` fire-and-forget with a `console.warn` on failure.
- `registerPushDevice()` fires
`src/providers/auth.ts:19`
- `identity` non-null → the protected branch mounts, `(tabs)` renders
`_layout.tsx:64-65`

Sign-out mirrors it: `dropPushDevice().then(() => signOut())` (`app/(tabs)/account.tsx:85`) — before the session ends, so the server drops the right token.

FLOW — UPLOAD A DOCUMENT (`7BEE05E`)

- Tap Upload / Replace this on `/profile`
`document-card.tsx:62` `choose()`; rendered twice there, once per kind (`app/profile.tsx:287,292`)
- Branch on kind
:45
photo → image library; `tenth-marksheet` → document picker, since it may be a PDF.
- Photo: media-library permission requested, refusal shows a native alert
:99-103

- 4 `launchImageLibraryAsync({ mediaTypes: ['images'], legacy: true })`
:106-109
legacy so the phone's gallery opens, not Google's picker.
- 5 File: `getDocumentAsync({ type: ACCEPTED_TYPES_FOR[kind], copyToCacheDirectory: true })`
:118-122
copied, or the URI is released before read.
- 6 Over 5 MB refused on device
:112/125 → :131-135
`DOCUMENT_MAX_BYTES` (`contracts/src/me.ts:22`).
- 7 The fix: `return new File(asset.uri)`
:114 and :127
expo-file-system's `File`, which implements `Blob`.
- 8 `upload.mutate(picked)`
:46 → :36 `api.me.uploadDocument(kind, file)`
- 9 `FormData` with one part named `file`
`contracts/src/client/me.ts:100-102`
the `as Blob` cast is gone because `UploadFile` is `Blob` (`contracts/src/me.ts:99`).
- 10 `send` detects `FormData` and omits `Content-Type` so the runtime sets its own boundary
`core.ts:91,98`
- 11 `POST /me/documents/:kind`
`apps/api/src/me/me.controller.ts:164-175` `@Audit(STUDENT_PROFILE, UPDATE)`, `@Post('documents/:kind')`, `@HttpCode(HttpStatus.OK)`, `@UseInterceptors(FileInterceptor('file', { limits: { fileSize: DOCUMENT_MAX_BYTES } }));` `class-level @Controller('me') + @Actors(ACTOR_TYPES.STUDENT)`. Subject is `user.id`, never a path id
- 12 `checkDocument`
`apps/api/src/me/documents.ts:54-91`
missing / over 5 MB / empty / wrong type each throw `VALIDATION_ERROR` with `fieldErrors.file`. The content type comes from **sniffing the bytes** (`sniffImage`, or `%PDF-`), never the claimed `mimetype`.
- 13 `assertExists(studentId)` before the upload
`apps/api/src/me/me.service.ts:77`
so no orphan object is ever written.
- 14 Key built
`documents.ts:30-38`
`students/<studentId>/<kind>-<epochMs>.<ext>`.
- 15 `storage.upload(key, file.buffer, contentType)`
`storage.service.ts:50-65`
one `PutObjectCommand`; the single upload path, MinIO only via `S3_ENDPOINT`.
- 16 `saveDocumentKey`
`students.service.ts:412-422`
`student.update` with a nested `profile.upsert`, writing `StudentProfile.photoUrl` or `.tenthMarksheetUrl`.
- 17 `auditContext.setEntityId(studentId)` explicitly, the route having no `:id`
`apps/api/src/me/me.service.ts:87`
- 18 Returns `this.profile(studentId)`; the client writes that into `PROFILE_QUERY_KEY` and invalidates `ME_QUERY_KEY`, so no second GET
`document-card.tsx:38-41`

What Expo's fetch refused. SDK 57 replaced `globalThis.fetch` with its `WinterCG` implementation, whose `FormData` takes a part only as a string, a `Blob`, or something exposing `bytes()`. RN's `{ uri, name, type }` descriptor — correct for RN's old XHR fetch — is none of the three, so `convertFormData` threw "Unsupported FormDataPart implementation" *before any request left the phone*. `core.ts:104-114` maps a throwing `fetch` to `INTERNAL / "Cannot reach the server. Check your connection."`, which is why the API log stayed empty. `expo-file-system`'s `File` implements `Blob` and reads the bytes natively. Narrowing `UploadFile` to `Blob` makes the crashing shape unwritable; the web caller (`apps/exams/src/features/account/document-card.tsx:25`) passes a browser `File`, which is a `Blob`.

The data it touches

MODEL.COLUMN	TYPE	HOLDS	WRITER
<code>StudentProfile.photoUrl</code>	<code>String?</code>	S3 key <code>students/<id>/photo-<ms>.jpg</code>	<code>saveDocumentKey</code>
<code>StudentProfile.tenthMarksheetUrl</code>	<code>String?</code>	same shape, <code>.pdf</code> allowed	<code>saveDocumentKey</code>
<code>PushDevice.token</code>	<code>String @unique</code>	FCM registration token	<code>PushService.registerDevice</code>
<code>PushDevice.platform</code>	<code>DevicePlatform</code>	<code>ANDROID</code> \	<code>IOS</code> ; mobile only sends <code>ANDROID</code> same
<code>PushDevice.deviceName</code>	<code>String?</code>	<code>Device.modelName</code>	same
<code>PushDevice.sessionId</code>	<code>String? @db.Uuid</code>	the sign-in that registered it; a push waits on that session being live	same
<code>PushDevice.lastSeenAt</code>	<code>Timestamptz(3)</code>	bumped on re-register	same

Redis (`apps/api/src/redis/redis.keys.ts:38-47`): `session:student:<id>:<sid>`, `sessions:student:<id>`, `session-replaced:student:<id>:<sid>` — all TTL `JWT_REFRESH_TTL` (30 d).

On device (`src/lib/constants.ts:2-10`): `iace.mobile.auth` in `SecureStore`; `iace.mobile.tab`, `.queued`, `.tours` in `expo-sqlite/kv-store` — SQLite because the answer queue is neither a secret nor small and must write synchronously (`src/lib/sitting-store.ts:6-13`).

What must already be true

`EXPO_PUBLIC_API_URL`, or `__DEV__` plus a Metro `hostUri`; else `src/lib/api.ts:24-28` throws before React mounts. `hydrate()` resolved before any token read. A Student with `pinHash` and `isActive`. For push: `FCM_PROJECT_ID` / `_CLIENT_EMAIL` / `_PRIVATE_KEY` on the API (empty in `.env.example:234-236`) and a `google-services.json` in the build. `webview/dist/question-page.html` generated (git-ignored).

Why it is shaped this way

- **Tokens in SecureStore, mirrored in memory.** app-kit's `KeyValueStorage` is synchronous so the API client can read a token from any call site; SecureStore is async. The alternative, an async store, forces every app-kit path to await. Cost: a write that does not land before a process kill is lost (`secure-storage.ts:10-11`).
- **Answers in SQLite, not SecureStore.** Alternative: one store for everything; rejected because the queue is large and must land synchronously.
- **No `googleServicesFile` in `app.json`.** Alternative: keep the key and gitignore the file — `prebuild` then *throws* in every checkout. Cost: push is dark, which `push-device.ts` tolerates by design.
- **Release profiles name no API URL.** Alternative: a guessed hostname, i.e. an APK that installs, opens and fails silently. Cost: a cloud release build must pass the URL as an EAS environment variable.

Which screen calls it

all mobile. `POST /auth/student/{login,otp/request,otp/verify,pin/set}` → `app/login.tsx`. `POST /me/documents/:kind` → `document-card.tsx` from `/profile`. `POST /DELETE /me/push-device` → `src/lib/push-device.ts`, no UI. `GET /me/notifications` + `PATCH .../:id/read` → `app/notifications.tsx`. `POST /me/pin` → `app/change-pin.tsx`. `GET /me/sessions` + `DELETE .../:id` → `src/components/account/active-devices.tsx`. `GET /me/push-subscription` is web-only.

Shared with the web, and not

Mobile imports **exactly two** workspace packages — `@iacle/app-kit` and `@iacle/contracts`. No import of `@iacle/ui` or `@iacle/app-kit/browser` exists anywhere; the only two mentions of `@iacle/ui` in the app are prose in comments (`global.css:1`, `src/lib/plural.ts:1`). Enforced mechanically: `eslint.config.js:8` pulls in `packages/config/eslint.no-web-ui.js`, which `no-restricted-imports` -errors on both and their subpaths. app-kit's root entry is `src/index.ts`; the only web-shaped thing inside is `src/nav.ts`'s `type { LucideIcon }` from `'lucide-react'`, erased at build — which is why `src/lib/nav.ts:19-26` declares its own `MobileNavItem` with a `lucide-react-native` icon instead of app-kit's `NavItem`.

From app-kit: the session factory (`createAuth`, `createTokenStore`, `signedOutMessage`), `createAppApiClient`, `createAppQueryClient`, `applyFieldErrors` / `LOGIN_FIELDS`, the **whole exam engine** (`useStartedSitting`, `useExamView`, `useAttemptState`, `tabIdFrom`, `stoodDownSays`, `submittingSays`, `TIMER_KIND`, `useClockCountdown`), `createStudentQueries` (`src/lib/queries.ts:6-16`), every shared query key (re-exported through `src/lib/constants.ts:13-33`), the filter specs, the derivations (`sittablesOf`, `resultsByTest`, `seriesProgress`, `questionReportInsights`, `verdictOf`, ...), the tour step machine, `useInfinitePages`.

Mobile's own: all 46 files under `src/components/` — a hand-built `ui/` tier styled with NativeWind against the **same** `packages/ui/src/tokens.css` via `tailwind.preset.native.js`; one exam skin; the WebView renderer. `metro.config.js:11-15` force-resolves `@tanstack/react-query` to the app's own copy so there is one `QueryClient` context; `:17-18` sets `inlineRem: 16`, since the tokens are authored at 16px/rem and NativeWind defaults to 14.

Push — can a push be received today? No. Two independent blockers.

Client: `registerPushDevice` returns before asking for anything in Expo Go, on a simulator, or on iOS (`push-device.ts:24`) — iOS hands back an APNs token FCM cannot address, so `DEVICE_PLATFORM.ANDROID` is the only value ever sent. On a real Android dev client it requests permission and calls `getDevicePushTokenAsync()` (`:59-71`), but `app.json` carries **no `googleServicesFile`** — removed in `6b16e63` because it named a git-ignored file absent from every checkout and `prebuild` throws rather than warns. Expo's GoogleServices plugin is a no-op when the key is absent, so the build succeeds with no FCM in the APK and the token call fails into the empty `catch` at `:36-38`. `expo-notifications` is never statically imported: `notifications.ts:16` loads it dynamically and returns `null` in Expo Go, where the module throws as it loads and used to take the router with it.

Server: `FcmSender.isConfigured` is `Boolean(projectId && clientEmail && privateKey)` (`fcm.sender.ts:51`); all three are `optional` (`env.schema.ts:196-198`) and empty in `.env.example`. Unconfigured, `PushService.reachOf` substitutes `[]` for the `pushDevice` query (`push.service.ts:115-120`) — no row read, no send attempted. The sender signs a service-account JWT itself rather than pulling in `firebase-admin`.

Registration is written and correct: `POST /me/push-device` (`me.controller.ts:136-144`) upserts on `token`, so the same phone re-registering is the same row moved to whoever is signed in, with `sessionId` from the JWT (`push.service.ts:68-76`). The in-app bell is the source of truth and is unaffected. `use-push-device.ts` routes a tapped notification to `/notifications` and invalidates `UNREAD_QUERY_KEY` on receipt — suppressed while `pathname.startsWith('/exam/')`, so one broadcast is not a read per sitting.

Build and release — `eas.json` has three profiles and **zero `env` blocks** (verified: no `env` key in the file).

- `development`: `developmentClient: true`, `distribution: internal`. **The only one that builds into a usable app:** it loads JS from Metro, and `devApiUrl` derives the API from Metro's `hostUri` on port 3000, rewriting `localhost / 127.0.0.1` to `10.0.2.2` on Android (`dev-api-url.ts:10-15`).
- `preview`: internal APK. `production`: app-bundle, `autoIncrement: true`, with `cli.appVersionSource: "remote"` so EAS owns `versionCode`.
- **What blocks both release profiles:** neither names `EXPO_PUBLIC_API_URL`, and Expo inlines `EXPO_PUBLIC_*` at *build* time. In a release build `__DEV__` is false, so `apiUrl` is undefined and `api.ts:24-28` throws on launch naming the variable. `817ed9f` did this deliberately — the previous hostnames were invented, and an APK that installs, opens and silently fails every request is worse than one that names the missing setting. `.env.example:9-12` records the same. **The brief's note is confirmed by the code, not just the comment.**
- Also blocking a store build: nothing in the repo signs the app (no `android/` or `ios/` — `prebuild` happens on EAS); the EAS project is linked (`projectId 243dc5bc-...`, `owner: indian-academy-of-competitive-exams`, so the upload keystore is org-owned, per `31b1872`); the icons are placeholders.
- **Icons:** `assets/icon.png` (14 KB) and `assets/adaptive-icon.png` (8 KB), copied from the student PWA at 512px where Expo asks 1024 — acceptable only because it is a wordmark on flat red (`#041fb5e`). `adaptiveIcon.backgroundColor: "#bf0d10"`; splash is `icon.png` at 160px on `#f6f7f9`.
- **Permissions:** `31b1872` set `microphonePermission: false` on the `expo-image-picker` plugin. The plugin adds `RECORD_AUDIO` unconditionally on Android (its picker can return video), `eas init` had echoed it into `android.permissions`, and `false` both skips the declaration and emits `tools:node="remove"` so a later library cannot merge it back.

Device-verified vs only written — Verified on hardware, per commit bodies: the document upload (`7bee05e`, "a photo and a PDF marksheet both upload and come back as On file"), sign-in against an unreachable API (`1a8cf21`, found when an Android emulator could not sign in and showed nothing), Android text rendering (`05437cd`, "the app had never rendered on a device until now"). Explicitly **not**: the five page tours and the tour renderer (`c88af90`, `5a2ffb4` both end "Nothing here is device-verified."), and every screen built before `16e6087` changed `inlineRem` to 16 — that commit says all of them "are worth a look on a device", i.e. they were reviewed 12.5% under size. **The exam sitting is written in full and is device-verified nowhere in the history.** The code is complete and non-trivial: `app/exam/[testId].tsx` starts the attempt on arrival, mounts `ExamSkin` before the paper lands so the WebView warns off the student's clock, intercepts Android Back into the submit confirmation (`:79-86`), flushes the queue on `AppState` `background` (`:175-182`), and offers "Continue here" when another device takes the sitting over (`:143-154`). Automated coverage is 8

unit files in `apps/mobile/test/` (51 tests, 51 passing per `.turbo/turbo-test.log`) over pure logic only — WebView message protocol, review protocol, image preload, deep-link language parsing, the secure-storage mirror, tour geometry, trend gaps, dev URL derivation. **No screen is rendered in any test.**

What changed recently

(`git log -15 -- apps/mobile`) — 7bee05e fixed every document upload. 31b1872 linked the org-owned EAS project and removed `RECORD_AUDIO`. 817ed9f removed the invented staging/production API hostnames so a misbuilt APK names the missing variable. 041fb5e took icon, adaptive foreground and splash from the student PWA's brand set. 6b16e63 created the three profiles and removed `googleServicesFile` — what unblocked Android builds at all, with dark push as the accepted price.

Is it optimal

`POST /me/documents/:kind:one S3 PutObject` plus three Postgres statements (existence check, nested profile upsert, profile re-read); the response is the refreshed profile, so no follow-up GET. The existence check is one extra round trip bought deliberately, against orphan objects. Login: one `findFirst` on the unique `mobile`, one argon2 verify, three Redis writes. `GET /auth/me` runs once per launch at a 5-minute `staleTime`, not per screen. The exam path matches the web's — client timer, Redis autosave, BullMQ scoring — and mobile adds nothing to Postgres on the hot path. Counts read off the source, not measured here.

What is wrong with it

- `src/lib/push-device.ts:24` — `Platform.OS !== 'android'` returns early, so an iOS build registers no device and `DEVICE_PLATFORM.IOS` is unreachable from this client, though `PushDevice.platform` (`prisma/schema.prisma:1243`) and `DEVICE_PLATFORM` (`contracts/src/access.ts:343`) both carry it. A dead enum half that reads as a shipped feature.
- `src/lib/use-push-device.ts:36-62` — the listener effect's deps are `[router, queryClient]`, not `signIn`, so listeners are installed while signed out. A tapped notification then calls `router.navigate('/notifications')`, a route the guard at `_layout.tsx:64` does not register when `identity` is null. Harmless today (no push can arrive); latent once Firebase is wired — a push tapped after the session expires navigates to a screen that is not in the tree.
- `app.json:5` pins `version: "0.1.0"` while `eas.json:3` sets `appVersionSource: "remote"`. EAS owns `versionCode`, but the user-visible `versionName` still comes from this hand-edited field — the one release value that can silently go stale.
- `src/lib/secure-storage.ts:25` — `setItemAsync` is fire-and-forget, so a sign-in followed immediately by a process kill leaves the mirror right and the keystore empty, and the next launch is signed out. Deliberate (`:10-11` warns), effectively untriggerable, but the one place where the session is not durable at the moment `signIn` returns.

What the three clients share

Where it lives

`packages/contracts/`, `packages/app-kit/`, `packages/ui/`, `packages/config/`.

- `contracts/src/envelope.ts` — `Errors` (19 members), `ERROR_CODE_STATUS`, `AppException`, `Paginated`, `PAGE_SIZE*`.
- `contracts/src/common.ts` — `ActorTypes`, `INSTITUTE_TIME_ZONE`, the Intl date helpers, `mobileSchema` / `pinSchema` / `newPinSchema`, the shared list-query parsers (`csvQuery`, `searchQuery`, `matchModeQuery`).
- `contracts/src/client/core.ts` — the transport: single-flight refresh, envelope parsing, `get` / `write` / `list` / `requestBlob`.
- `contracts/src/attempt.ts` + `exam-order.ts` — the exam rules both sides run (`secondsLeft`, `paletteCounts`, `mayOpenQuestion`, `openSections`, `servedQuestions`).
- `app-kit/src/exam/use-exam-view.ts` — the engine (407 lines); `use-attempt-state.ts` (516) the autosave under it; `pending-answers.ts` the React-free queue.
- `ui/src/index.ts` — the 155-name inventory (32 type-only) over 66 `.tsx` files in `components/ui` and 11 in `components/charts`; `ui/src/tokens.css` (715 lines) is the one place a value is defined.
- `config/eslint.no-dom.js`, `eslint.no-web-ui.js`, `eslint.api-boundaries.js` — the import boundaries, made mechanical.

What it is for

One API, three clients (NestJS, two Vite SPAs, one Expo app), one wire format. `contracts` is that format — schemas, route strings, error vocabulary, typed client — so a field renamed on the server is a compile error in both SPAs, not `undefined` on a screen. `app-kit` is the plumbing identical on web and phone (token store, API wrapper, form-error mapping, list paging, query policy) plus the exam engine, the one piece of product logic that must behave the same in a browser and in React Native. `ui` is the web design system; mobile cannot run it and reuses only the tokens. `config` holds the tsconfigs and ESLint configs that enforce those divisions.

Build output differs and the difference matters. `contracts` compiles (`package.json:18`) to **CommonJS** + `.d.ts` (`tsconfig.json:4-9`) because NestJS consumes it; both SPAs bypass `dist` and alias the package to TS source (`apps/exams/vite.config.ts:39-42`, `apps/admin/vite.config.ts:10`), with `optimizeDeps.exclude` for it and `@iace/ui`. `app-kit` and `ui` have **no build script** — exports point at `.ts` / `.tsx` / `.css` (`app-kit/package.json:9-12`, `ui/package.json:9-20`), so Vite and Metro compile them in-app. `config` ships raw JSON and `.js`. `turbo.json:11` has `build` depend on `^build`, which is why `contracts'` `dist` exists before the API compiles.

FLOW — A TAPPED OPTION BECOMES A DURABLE SAVE

- Candidate taps an option
the skin calls the handed view `view.chooseOption(id)`
handlers built once at `app-kit/src/exam/use-exam-view.ts:254`, so the view's identity never changes.
- Reads this render through a ref published in `useInsertionEffect`
`packages/app-kit/src/exam/use-exam-view.ts:249` `live.current.record(...)`
never stale, and no handler carries a dep list; a forward-only paper forces `{marked:false}` here (`:213`).
- Seconds on the open question banked, state derived
`use-attempt-state.ts:386` `answer() → changeFor() :457 → stateFor() :500`
an `AnswerChange` with accumulated `timeSpentSec` and earliest-wins `firstActionAt`.
- Queued and drawn at once
`:347` `record() → pending-answers.ts:54` `put()`
the Map holds it, `keep()` `:40` mirrors it to `iace.test.queued.<attemptId>` in `sessionStorage` (`apps/exams/src/lib/constants.ts:119`). A failed save never un-draws an answer.

- 5

Branch: 20 pending answers flush now, else the jittered timer
autosave-policy.ts:29 shouldFlushNow, :25 autosaveDelayMs (25s ± 5s)
early flush only when nothing is in flight and the last save did not fail (use-attempt-state.ts:393).
- 6

flush() awaits any save in the air, then sends whole batches
:271
a backlog over SAVE_BATCH_MAX goes up as consecutive saves; one oversized batch would be refused forever.
- 7

revision increments, abort armed at 15s
:241 save(), SAVE_TIMEOUT_MS autosave-policy.ts:17
- 8

api.me.saveAttemptState(attemptId, {...batch, tab})
contracts/src/client/me.ts:161 write('PATCH', ME_ATTEMPT_ROUTES.state(id), attemptSaveAckSchema, input, extra)
- 9

Transport adds Authorization, x-client, x-device-name
client/core.ts:83 send()
a thrown fetch becomes AppException(INTERNAL, httpStatus:0) :106, so callers need no second code path.
- 10

On 401: single-flight refresh
:207 tokenAfter(), :226 refreshInFlight ??=
SESSION_REPLACED throws without refreshing; only UNAUTHENTICATED is retried.
- 11

Server: @SittingRateLimit() @Patch('attempts/:id/state'), @Body(new ZodBody(saveAttemptStateSchema)), @CurrentUser(); class-level @Controller('me') + @Actors(ActorTypes.STUDENT)
apps/api/src/attempts/attempts.controller.ts:121-128
- 12

ZodBody.transform parses or throws VALIDATION_ERROR with fieldErrors
apps/api/src/common/zod-validation.pipe.ts:10
- 13

Compare-and-swap loop over Redis
apps/api/src/attempts/attempt-state.service.ts:246-257
key attempt:state:<attemptId>, TTL 172800s (attempt-state.ts:23); CONFLICT once the tries are spent.
- 14

Attempt id added to the attempt:dirty SET
:287
the flusher, not this request, writes Postgres.
- 15

ResponseInterceptor wraps {success, data, meta}
apps/api/src/common/response.interceptor.ts:13
- 16

Client validates both halves
client/core.ts:118 parse()
a refused shape becomes INTERNAL /"Unexpected response shape from API", never silently-typed data.
- 17

applied !== false acknowledges exactly the copies the batch carried
use-attempt-state.ts:259 → pending-answers.ts:65
an answer rewritten since the batch left stays queued.
- 18

The clock the save answered with replaces the paper's
:256 setClock
this is how an admin's extension reaches a live screen with no reload.

FLOW — A NEW ENDPOINT GETS A CONTRACT

- 1

Response schema + z.infer type in the domain file
contracts/src/saved.ts:13-28
- 2

Query/body schema, extending paginationQuerySchema for a list
saved.ts:30-37; ...Input is z.input (pre-coercion) for callers, ...Query is z.infer for the server
- 3

Path into the file's *_ROUTES as const
saved.ts:64-70; barrel the file in contracts/src/index.ts
- 4

A new failure mode gets one line in ErrorCodes, ERROR_CODE_STATUS and DEFAULT_MESSAGES
envelope.ts:12/39/66; all three are Record<ErrorCode, ...>, so omitting one is a type error as you type it
- 5

Server binds the same schema as a pipe and the same type as the return
apps/api/src/saved/saved.controller.ts:36, Promise<Paginated<SavedQuestion>>. All 32 controllers import @iace/contracts; 137 pipe bindings across them
- 6

Client adds one line to its group
list(SAVED_ROUTES.list, query, savedQuestionSchema)

Caught at author time: a service returning the wrong shape, a controller whose return drifted, a bad query argument, an unhandled ErrorCodes member. **Not** caught: an extra field the server sends — Zod strips it, and it surfaces as a missing column on screen.

The data it touches

no package writes a database row.

THING	WHERE	HOLDS	WRITTEN BY
attempt:state:<attemptId>	Redis, TTL 172800s	answers, section clocks, revision, pause facts	attempt-state.service.ts:254
attempt:dirty	Redis SET, no TTL	attempts Postgres has not seen	:287

THING	WHERE	HOLDS	WRITTEN BY
<code>iace.{test,mobile,admin}.auth</code>	<code>localStorage</code>	<code>{accessToken, refreshToken}</code>	<code>token-store.ts:31</code>
<code>iace.{test,mobile}.tab</code>	<code>sessionStorage</code> / device store	the tab or device a save is judged by	<code>exam/tab-id.ts:4</code>
<code>iace.{test,mobile}.queued.<attemptId></code>	<code>sessionStorage</code> / device store	undelivered <code>AnswerChange[]</code>	<code>pending-answers.ts:44</code>
<code>iace.*.tours</code>	<code>localStorage</code>	tours this device has seen	<code>src/tour.ts</code>

What must already be true

`contracts` built before the API typechecks or runs (`turbo.json:11`), and `prisma generate` for the enums `contracts` mirrors. Each app imports the four stylesheets once at its entry (`apps/*/src/main.tsx:3-7`) and extends the preset in its `tailwind.config.js` — a missing `content` glob silently emits no CSS, with no error (`tailwind.preset.js:13`). Each app passes its **own** auth storage key, because the two SPAs share one origin (`token-store.ts:18`). The exam engine needs four injected deps (`api` , `focus` , `tab` , `answerQueue`) and a mounted `QueryClientProvider` .

Why it is shaped this way

- The date helpers are Intl-only and `contracts` ' sole runtime dependency is `zod` (`package.json:25-27`). The alternative, `date-fns` / `@date-fns/tz` , stays server-side in `apps/api/src/common/time/institute-day.ts` ; here it would ship a date library to every student's browser.
- The engine is DOM-free so mobile reuses it verbatim. The cost is an adapter per capability (`FullscreenHandle` , `KeyValueStorage` , `SignOutSignal`) plus a second web tier (`app-kit/browser`). The alternative — a web implementation and a native copy — is two answers to "when does the clock expire".
- Exam ordering lives in `contracts` , not `app-kit` : `servedQuestions` runs in the browser for a prefetched paper (`app-kit/src/exam/served-paper.ts:32`) and on the server for a reload or report (`apps/api/src/attempt/attempt-paper.service.ts:159`). Two copies would be two answers to "same seed, same order".
- Two client factories, not one lazy namespace (`client.ts:9,16`), so the 1000-line admin group never enters a student bundle. The recording Proxy that did this before went in `608e15c` .

Exam engine: ownership — `useAttemptState` owns the mutable sitting: `answers` , `sections` , `sectionsSettled` , `clock` , `isSaving` , `hasUnsaved` , `takenOver` , `setAside` as state; `revision` / `inFlight` / `openQuestion` / `openedAt` / `stopped` / `heldElsewhere` as refs (`use-attempt-state.ts:135-158`). There is **no reducer** — transitions are named callbacks on `AttemptStateHandle` (`:61-97`): `answer` , `open` , `bankOpen` , `enterSection` , `closeSection` , `flush` , `finish` , `resume` , `leave` , `standDown` . `useExamView` owns navigation and ending (`sectionId` , `questionId` , `asking` , `ignoringFullscreen` , plus an `ending` ref so a clock and a tap cannot both submit) and returns one `ExamView` (`exam/exam-view.ts:90`) carrying every value a skin may draw and every move it may make — a skin holds no state, owns no clock, reaches for no mutation. `apps/exams/src/features/exam/exam.tsx:112` and `apps/mobile/app/exam/[testId].tsx:166` call the same hook, differing only in the `focus` adapter, the queue storage and the `tab` .

Which screen calls it

`@iace/contracts` : all four apps; `apps/api/package.json:20` lists it and nothing else from `packages/` . `@iace/app-kit` : `admin` (37 named imports), `exams` (98), `mobile` (79). `@iace/app-kit/browser` : `admin` and `exams` only (36 names). `@iace/ui` : `admin` , `exams` and `app-kit/browser` ; `apps/mobile/package.json` declares no dependency on it and imports none.

Counted over the 155 barrel names: **65** reach both SPAs (`Button` , `Input` , `Alert` , `Card` , `Combobox` , `ConfirmDialog` , `DataTable` , `DatePicker` , `Dialog*` , `EmptyState` , `Field` , `FormField` , `FormPanel` / `FormSection` , `ListView` , `LoadingState` , `MetricGroup` , `PageFrame` / `PanelFrame` / `TableFrame` , `PageHeader` , `PageErrorBoundary` , `PinField` , `RichContent` , `RowActions` , `Skeleton` , `StatRow` , `Tabs` , `Tooltip` , `TruncatedText` , `toast` , `cn` , `plural` ...), **52** reach admin but not `apps/exams` — the dense data-first language: the raw `Table*` family, `Stepper` , `SegmentedControl` , `FormDialog` , `ImportView` , `Accordion` , `RadioGroup` , `RatioBar` , `DateTimePicker` , `Kbd` , `FieldRow` , `ReadOnlyField` , `FormCombobox` , `BadgeList` , `Separator` , `INDIC_SCRIPTS` , `mathErrorIn` , `CAPPED_VIEWPORT` — and **14** reach `apps/exams` but not admin — among them `FillBubble` , `FILLS` , `Watermark` , `ComparisonCards` , `Label` , `PlotReference` , `Spinner` . **20** are reached only through `app-kit/browser` , among them `Breadcrumbs` , `Popover*` , `DonutPlot` , `DivergingBars` , `QuadrantPlot` , `TourSpotlight` , `ThemeProvider` , `Toaster` , `TooltipProvider` , `holdsASet` . **4** are exported and consumed by nothing outside the package (below).

One token set, two compositions: `tokens.css` has 17 numbered sections plus a dark block at `:397` , and the exam skins re-point the same variables under `[data-exam-template='default']` :553 and `[data-exam-template='ssc_railways']` :608 — swapping the attribute re-skins the screen with no component change. Mobile takes the tokens by **relative** `@import` (`apps/mobile/global.css:2`) and keeps a hand-written native twin preset (`apps/mobile/tailwind.preset.native.js`).

app-kit/browser **adapters** (48 exports; outside `src` because the DOM ban is scoped to `src/**`) — `browserStorage` / `browserSessionStorage` : Web Storage → `KeyValueStorage` . `browserSignOutSignal` : a `window` `CustomEvent` (`iace:signed-out`) → `SignOutSignal` , so the API client can end a session with no reference to the React tree. `createBrowserTokenStore` pairs them. `mountApp` : `createRoot` plus the fixed provider order. `AppShell` / `PageCrumbs` / `ProtectedRoute` : `react-router-dom` + `@iace/ui` . `useFullscreen` : the Fullscreen API → `FullscreenHandle` . `useLeaveGuard` : `beforeunload` + `pagehide` . `saveBlob` : an authenticated download → `<a download>` . `shrinkForUpload` : `canvas/WebP` . `useFilters` / `useLocalFilters` / `useFilterSpec` / `useListScreen` / `useScrollList` / `useImportScreen` : URL and list plumbing. The three figure modules: `Recharts` .

Import boundaries, mechanically enforced

BOUNDARY	WHERE
<code>contracts/src</code> and <code>app-kit/src</code> may not touch <code>window</code> , <code>document</code> , <code>localStorage</code> , <code>sessionStorage</code> (nor via <code>globalThis</code>)	<code>config/eslint.no-dom.js:18-30</code> , wired at <code>contracts/eslint.config.mjs:5</code> and <code>app-kit/eslint.config.js:5</code> ; scope <code>src/**</code> , which is why <code>app-kit/browser</code> exists
<code>apps/mobile</code> may not import <code>@iace/ui</code> or <code>@iace/app-kit/browser</code>	<code>eslint.no-web-ui.js:9-21</code> , wired at <code>apps/mobile/eslint.config.js:3</code> ; reinforced by that package.json listing neither
<code>apps/api/src</code> modules reach a sibling only through its public entry; <code>common/config/prisma/queue/redis/storage</code> exempt	<code>eslint-rules/api-module-boundaries.js:31</code> , wired at <code>eslint.api-boundaries.js:27-30</code>
No Postgres write on the live answer path	<code>eslint-rules/no-hot-path-db-write.js</code> , scoped to three files at <code>eslint.api-boundaries.js:19-35</code>
SPAs may not use raw <code>fetch</code> or <code>axios</code>	<code>eslint.react.config.js:44-51</code>
A controller may not build the envelope	<code>apps/api/eslint.config.mjs</code> , <code>no-restricted-syntax</code> on <code>*.controller.ts</code>

`eslint.react.config.js` also carries the design-system bans a type cannot state (`Loader2` , `animate-spin` , `<select>` , any date-typed input, `title` attributes, `ring-2` , raw hex, off-scale tracking and type sizes) and four repo-local rules: `confirm-destructive` , `no-narration` (error), `no-hardcoded-route` , `no-manual-focus` (warn,

and both off inside `packages/ui`).

Is it optimal

Measured in `608e15c`: no admin route string survives in the student bundle, and admin's entry chunk came out 551 bytes *smaller* than with the Proxy. `list()` recombines `data + meta` into one `Paginated` with no second request (`client/core.ts:244`); `queryString` drops empty arrays so an unset filter never becomes `in: []` server-side (`:21`). Refresh is single-flight with halved, jittered backoff (`:317-325`), so a hall of simultaneously-expired tokens makes one call per client and not in step. Autosave is jittered ± 5 s for the same reason (`autosave-policy.ts:25`). `useExamView` memoizes every paper walk (`:108-134`) because a sitting re-renders once a second. Bundle sizes: not measured this pass.

What changed recently

`7bee05e` narrowed `UploadFile` to `Blob` (`contracts/src/me.ts:99`) after Expo SDK 57's WinterCG fetch refused React Native's `{uri,name,type}` descriptor; the type is now the guard. `82505eb` gave the combobox search row `relative z-10` and an opaque background so the focus ring is no longer painted over — affects every `Combobox`, `MultiCombobox` and `DatePicker`. `e20bb7e` made `startedByThisCall` skip the state GET for a sitting the screen just started (7K empty requests at the bell) while still setting `sectionsSettled`. `608e15c` replaced the lazy-group Proxy with two factories, turning `api.admin.nope()` from a runtime `TypeError` into a compile error. `f1fa0b2` fixed the view's identity: ~14 handlers built once, reading the current render through a `useInsertionEffect` ref. `e83dab7` lifted the answer queue into `createPendingAnswers`. `dd680b3` un-exported 18 schemas composed only in their own file.

What is wrong with it

- `ui/src/tokens.css:5` says "v0.4 moves the brand red from #B83939 to #A8221B" and `:418` repeats it, but the live light `--primary` is `#bf0d10` (`:203`) — a value appearing in no ramp in the file (`--red-600` is `#a8221b`, `:33`). The header describes a move the semantic role did not take; `ui/README.md:44` quoting `#BF0D10` is the accurate one.
- `ui/README.md:12` names `StatTile` and `QuestionPalette` as `packages/ui` primitives. Both live in `apps/exams` (`src/features/tests/series.tsx:44`, `src/features/exam/question-palette.tsx:15`) and neither is in the barrel.
- `ui/src/tokens.css` numbering jumps `7 → 9` (`:255 → :264`); there is no section 8.
- `contracts/src/client/me.ts:98` — an orphaned doc comment ("Returns a FRESH session...", which belongs to `changePin` at `:106`) sits stacked above `uploadDocument`'s own comment.
- `contracts/src/client/core.ts:119,311,333` — the `204` branch (`noContentOf`) is unreachable against this API: no route uses `HttpStatus.NO_CONTENT` or `@HttpCode(204)` while 64 sites use `HttpStatus.OK`. Defensive, but nothing exercises it.
- `contracts/src/envelope.ts:254-261` — `STATUS_TO_CODE` has no 503 entry, so a bare gateway 503 with no envelope maps to `INTERNAL`, never `SERVICE_UNAVAILABLE`, though the code exists. A client branching on `SERVICE_UNAVAILABLE` will not see it from a proxy.
- `config/eslint.no-dom.js:10` bans only four globals. `fetch`, `FormData`, `File`, `Blob`, `Response`, `AbortController` and `URLSearchParams` are all used in `contracts/src` — correct, they are cross-runtime — but the rule therefore does not stop `app-kit/src` importing `@iace/ui`, `react-dom` or `react-router-dom`. Today none does (verified by grep), so the mobile-safety of `src` rests on discipline plus pnpm peer resolution, not on a rule.
- `apps/mobile/global.css:2` imports `packages/ui/src/tokens.css` by relative path, with a comment that the `@iace/ui` specifier hangs Metro's CSS transform — so mobile depends on `packages/ui` while declaring no dependency on it. Moving or renaming `tokens.css` breaks mobile with nothing in the dependency graph to warn.
- Barrel-exported but imported by nothing outside its own package. From `ui/src/index.ts`: `Pagination` (used only at `list-view.tsx:362`), `useTruncation` (`combobox-shell.tsx:10`), `FieldControl` (`form-field.tsx:13`), `PlotBand` (`line-plot.tsx:58`; mobile re-declares its own at `apps/mobile/src/components/ui/line-plot.tsx:24`). From `app-kit/src/index.ts`: `TEST_STATE_ITEMS`, `courseItems`, `seriesItems` (only `testsFilters`, `list-filters.ts:92-94`) and `continueWith`, `openNow`, `upNext` (only `waitingOn`, `catalog.ts:25-27`). From `browser/index.ts`: `useMediaQuery / DESKTOP_QUERY` (only `app-shell.tsx:21`). None is dead code; each is a wider public surface than it needs.
- `deliveryChannelSchema`, `authoringTagsSchema`, `healthResponseSchema`, `studentDataExportSchema`, `questionFilterSchema` are never parsed anywhere — only their `z.infer` types are read. A **deliberate** keep recorded in `dd680b3` (un-exporting them makes them unused-variable errors), worth stating so nobody "cleans" them. Consequence: those five responses are schema-validated on no client, because none has a method in a client group.
- `plural` exists twice — `ui/src/lib/utls.ts:13` and `apps/mobile/src/lib/plural.ts:2` — a knowing copy, because mobile cannot import `@iace/ui`. The Tailwind preset is duplicated the same way.

Part V — The platform underneath

This part answers the question "what are the common things, and why is the code cleaner for them". Every module in the earlier parts leans on the same small set of pieces: one response envelope, one guard chain, one queue table, one Redis key registry, one S3 path, one audit trail. The value of each is best measured by what every caller would otherwise have to write, so that is how they are described.

The machinery every module leans on

Where it lives

`apps/api/src/common/**`, `apps/api/src/{queue,redis,storage,prisma,health,audit}/**`, `main.ts`, `app.module.ts`. The client half is `packages/contracts/src/envelope.ts` and `packages/contracts/src/client/core.ts`.

- `app.module.ts` — the whole cross-cutting stack: one filter, three interceptors, five guards, two middlewares.
- `common/response.interceptor.ts` — wraps every handler return in the success envelope.
- `common/all-exceptions.filter.ts` — `@Catch()` with no argument: the single exit for everything thrown.
- `queue/queues.ts` — `QUEUE_NAMES`, `QUEUE_POLICY`, every job id and every cron.
- `redis/redis.keys.ts` — the entire keyspace in one object.
- `config/api-role.ts` — `API_ROLE` decides whether a container serves routes, runs workers, or both.
- `common/metrics/metrics.interceptor.ts` — latency histogram and the per-request log line.
- `audit/audit.service.ts` — the only writer of `RowActionLog`.

What it is for

Every feature module answers HTTP, throws for a refusal, pages a list, locks a record, counts a rate, enqueues work, writes to S3 and records who changed what. This is the one copy of each, so a feature module is a controller plus a service. The payoff is uniformity the client relies on: `client/core.ts:135` parses one failure shape for every endpoint, `core.ts:243` recombines one page shape for every list. The second payoff is that the live-test scaling rules live here — the Redis client, the queue policy, the request budget — rather than being re-decided per endpoint.

1. The response envelope

`{ success: true, data, meta }` / `{ success: false, error: { code, message, fieldErrors?, details? }, meta }` — `packages/contracts/src/envelope.ts:112–135`.

`ResponseInterceptor` (`response.interceptor.ts:13`) skips non-HTTP contexts (`:16`). `isPaginated` (`envelope.ts:178`) detects the exact `{ items, page, pageSize, total }` a list handler returns and `response.interceptor.ts:26–29` splits it: `items` → `data`, `counts` → `meta.page/pageSize/total`. A void handler still gets `data: null` (`:32`). Without it, ~400 handlers would each build a body.

`AllExceptionsFilter` (`:28`) rethrows for queue workers, which have no response (`:33`), then `translate` (`:91`) runs six cases in order: `AppException` (carries its own code and status); `ZodError` → `VALIDATION_ERROR` with `fieldErrors` keyed by the full dotted path (`:193`); `deadlock` → `CONFLICT`; `P2002` → `CONFLICT` with the colliding columns in `details`; `P2025 / P2023` → `NOT_FOUND` (a malformed uuid names nothing); Nest's `HttpException`; an Express middleware's `4xx` (`:168` — `5xx` deliberately excluded); else `INTERNAL` with a generic message (`:184`). A `5xx` logs the stack plus the body through `redact` and goes to Sentry (`:57–61`); a `4xx` is one debug line.

`ErrorCodes` — 20 members (`envelope.ts:12–32`), each with its status (`:39`) and default message (`:66`) beside it. `PIN_LOCKED` is 429, not 403: it is a throttle. `DRAW_SHORTFALL` and `EXPORT_T00_LARGE` are 422.

`meta.requestId` comes from `RequestIdMiddleware` (`request-id.ts:16`), which echoes an inbound `x-request-id` only against `/^[A-Za-z0-9._-]{8,128}$/` (`:7`) so a caller cannot forge a log line. `ensureRequestId` (`:28`) is idempotent, so the filter and the interceptor agree. `main.ts:52` adds the header to `exposedHeaders`.

Real call site: `AuditService.listRowActions` returns `paged(query, rows, total)` (`audit.service.ts:185`, `common/pagination.ts:7`); `AuditController.rowActions` (`audit.controller.ts:42`) returns it unchanged.

2. Guards and decorators

Registered at `app.module.ts:77–82`, run in that order:

1. `JwtAuthGuard` (`auth/guards/jwt-auth.guard.ts:20`) — honours `@Public()`, verifies the bearer token, checks the Redis session still exists (`:33`), then **re-reads an admin's grants from Postgres, not the token** (`:38`), so a revoke bites next request. Attaches `AuthenticatedUser` (`common/security/authenticated-user.ts:4`).
2. `ActorGuard` (`actor.guard.ts:11`) — `@Actors(...)`; a student token is a valid JWT and must not reach an admin route.
3. `FeaturePermissionGuard` (`feature-permission.guard.ts:22`) — checks `isActive` **before** the super-admin bypass (`:34`), then `can(user, key, level)` for any one key, then the extra `DATA_EXPORT` check for `@RequiresExport`.
4. `SuperAdminGuard` (`:11`).
5. `ThrottlerGuard` — last on purpose (`app.module.ts:81`), so `request.user` exists and a caller is counted as themselves.

Decorators in `common/security/decorators.ts`: `@Public()` `:21`, `@Actors()` `:24`, `@RequiresFeature` `:34`, `@RequiresAnyFeature` `:38`, `@RequiresExport` `:42` (one decorator, because a second `@RequiresFeature` would overwrite the first), `@RequiresSuperAdmin()` `:51`, `@CurrentUser()` `:54`. Plus `@Audit` (`audit/audit.decorator.ts:14`), the three rate-limit marks (`common/throttling/rate-limits.ts:22–28`), and `ZodBody` / `ZodQuery` / `ZodParam` (`common/zod-validation.pipe.ts:7,21,23`).

Interceptors: `RequestBudgetInterceptor` first so the budget bounds the others (`app.module.ts:74`), then `ResponseInterceptor`, then `AuditInterceptor` — innermost, so it taps the handler's own value rather than the envelope, pinned by `apps/api/test/audit-interceptor-order.e2e.test.ts`. `MetricsInterceptor` registers separately (`common/metrics/metrics.module.ts:20`); its position relative to those three is **not verified**.

3. The event bus

In-process only. `DomainEventBus.emit` (`common/events/domain-event-bus.ts:12`) wraps `EventEmitter2.emit` in try/catch and logs — fire and forget, so a failing listener cannot fail the producer's request. `wildcard: false` (`events.module.ts:9`), because the names are dotted and a wildcard would make `attempt.*` an accidental subscription.

Six events (`event-catalog.ts:3-16`), all wired:

EVENT	EMITTERS	HANDLER
<code>access.catalog_changed</code>	<code>access/test-series.service.ts:176,201,219,250,285</code> ; <code>tests/offering.service.ts:232,444</code> ; <code>tests/tests.service.ts:282,303</code>	<code>AccessCacheListener.onCatalogChanged</code> → <code>invalidateAll()</code>
<code>exam_stage.changed</code>	<code>configs/base-configs.service.ts:229</code> , <code>exam-stages.service.ts:128</code> , <code>exams.service.ts:120</code>	same listener (<code>access/access-cache.listener.ts:21</code>)
<code>student.pin_reset</code>	<code>auth/auth.service.ts:312</code>	<code>notifications/notification.listener.ts:39</code>
<code>student.signed_up</code>	<code>auth/auth.service.ts:119</code>	<code>notification.listener.ts:27</code>
<code>student.deactivated</code>	<code>students/students.service.ts:437</code> , <code>student-privacy.service.ts:121</code>	<code>auth/student-sessions.listener.ts:14</code> → <code>revokeAll</code>
<code>admin.deactivated</code>	<code>admins/admins.service.ts:230</code>	<code>auth/admin-sessions.listener.ts:14</code> → <code>revokeAll</code>

What the outbox's removal cost. `a5112aa` dropped `OutboxEvent`, its trigger and three SQL functions. The guarantee lost is at-least-once delivery across a crash: a listener's work is no longer in the producer's transaction and no relay re-reads a missed row. Its two non-re-derivable consumers were moved off first (per that commit body): notifications into their producer's own transaction (`20260929180000`), and a re-score onto `Attempt.scoredRevision` (`20260930140000`). For the four remaining listeners the code accepts the loss and says why at each site — a catalog bump is re-derivable because every held copy ages out (`access-cache.listener.ts:30`), a lost revocation is re-applied because refresh re-reads the flag (`student-sessions.listener.ts:19`). The durable replacement is the attempt sweeper, which re-asks for a score for as long as a sitting is ended-and-unscored (`attempts/scoring-queue.ts:1-5`).

4. The queue layer

Eight queues (`queue/queues.ts:4-13`), each registered with its own `jobOptionsFor(name)` (`queue.module.ts:18`). `QUEUE_POLICY` (`:20-37`):

QUEUE	CONC.	ATTEMPTS	BACKOFF	TRIGGER
<code>scoring</code>	8	5	exp 5s	<code>ScoringQueue.queue / .rescore</code>
<code>rollup</code>	2	5	exp 5s	<code>rollup-queue.ts:62</code> , debounced 5s / 60s
<code>attempt-flush</code>	1	3	exp 2s	scheduler every 60s
<code>attempt-sweep</code>	1	3	exp 2s	scheduler every 120s
<code>audit-archive</code>	1	3	exp 2s	cron <code>15 21 * * *</code> (02:45 IST)
<code>notification-prune</code>	1	3	exp 2s	cron <code>0 21 * * *</code> (02:30 IST), <code>tz: 'UTC'</code>
<code>notifications</code>	4	5	exp 5s	schedulers every 60s and 300s
<code>notification-delivery</code>	2	4	exp 15s	keyed per row, <code>limiter: { max: 10, duration: 1000 }</code>

Retention: a success 1h/1000 jobs (`:46`), a spent failure 7d/500 (`:40`). `keyedJob(jobId)` (`:60`) sets `removeOnFail: true`, because BullMQ silently drops an add whose retained failure holds the id.

Where workers run: the same process when `API_ROLE` is unset or `all` (`api-role.ts:26,33`) — dev and every test. In production every processor is behind `onRole([API_ROLES.WORKER], ...)` (`attempts.module.ts:81` , `notifications.module.ts:41` , `audit.module.ts:18`), as is scheduler registration (`attempts.module.ts:98`), so the container that runs the jobs schedules them. Queues get their **own** Redis connection with `maxRetriesPerRequest: null` (`queue.module.ts:14`), because blocking `BRPOPLPUSH` would stall the application client.

`ReportingWorkerHost` (`queue/reporting-worker-host.ts:9`) is subclassed by all eight: Nest's metadata scan walks the prototype chain, so one base class gives every worker its failed and `error` listeners. `QueueFailures.record` (`common/metrics/queue-failures.ts:17`) separates a retry (warn) from a spent job (error + stack) — the only place a dead letter is heard from.

5. Redis

One `RedisService` (`redis/redis.service.ts:68`), `@Global`, lazy-connect with capped retry backoff (`:79`). `onModuleInit` **refuses to boot production** against a Redis whose `maxmemory-policy` is not `noeviction` (`:91` , `eviction-policy.ts:11`) — an eviction drops a live sitting. Five Lua scripts give the compare-and-set primitives: `replaceJson` :144 , `removeIfUnchanged` :173 , `deleteIndexedSet` :190 , `releaseLock` :200 , `holdLock` with optional `steal` :205 .

Key inventory (`redis/redis.keys.ts`), TTLs as configured:

KEY	TTL
<code>otp:{actor}:{id}</code>	<code>OTP_TTL_SEC</code> , default 300s
<code>otp:cooldown:{actor}:{id}</code>	<code>OTP_RESEND_COOLDOWN_SEC</code> , default 45s
<code>otp:attempts:{actor}:{id}</code>	inherits the OTP's; deleted on a fresh code
<code>otp:daily:{mobile}</code> , <code>otp:daily:ip:{ip}</code> , <code>otp:daily:global</code>	86400s, set on the first INCR (<code>otp.service.ts:96</code>)
<code>pin:attempts:{mobile}</code>	the attempt window, set on the first failure
<code>pin:lock:{mobile}</code>	the ladder rung: 900 / 3600 / 86400s

KEY	TTL
pin:lockouts:{mobile}	run + PIN_LOCKOUT_DECAY_SEC (86400s)
pin:setup:{mobile}	PIN_SETUP_TTL_SEC , default 600s
session:{actor}:{sub}:{sid} , sessions:{actor}:{sub} , session-replaced:...	refresh lifetime, JWT_REFRESH_TTL default 30d
attempt:state:{id}	172800s = PAUSE_LIMIT_SEC , 48h (attempt-state.ts:23); the key /s the pause
attempt:dirty	none (a SET, drained by the flush worker)
sitting:claim:{studentId}	172800s
access:catalog:epoch	none (a counter)
ratelimit:{name}:{tracker}	the window, 60000ms
test:edit:{id} , base-config:edit:{id} , section:edit:{testId}:{sectionId}	EDIT_LOCK_TTL_SEC = 900s (redis.keys.ts:79)
audit:archive:{YYYY-MM-DD}	900s (audit-archive.processor.ts:28)

6. Storage

One `StorageService` (`storage/storage.service.ts:24`), @Global . **Confirmed never branched by environment** — no `isProduction` or `NODE_ENV` appears in the file. `MinIO` is reached by configuration alone: `S3_ENDPOINT` + `S3_FORCE_PATH_STYLE=true` locally (`.env.example:147,152`), `S3_ENDPOINT` empty in production so the SDK uses the real regional endpoint (`:37-39`). `signedWith` (`:14`) returns `undefined` when either half of the key pair is absent, handing the SDK its own credential chain — the instance role on EC2; `env.schema.ts:259` refuses a half-configured pair.

Surface: `upload` :50 ; `publicUrl(key)` :68 (unsigned `MEDIA_BASE_URL/key` , so the CDN caches one copy not one per student); `createDownloadUrl(key, expiresInSec = 900)` :72 (presigned GET); `read` :79 (for a file the API itself must re-read — an import committing the sheet it previewed, never for handing bytes to a browser); `remove` ; `objectSize` :91 (null on any HEAD failure); `ping` :103 (`HeadBucket` , used by `/health`).

7. Audit

The model is `RowActionLog` , not `AuditLog` (`prisma/schema.prisma:1015-1034`): `feature` (`AuditFeature` , 19 members — `packages/contracts/src/audit.ts:14`), `entityId` `uuid` , `entityLabel` `nullableText` , `action` (`AuditAction` , 9 members — `audit.ts:39`), `actorType` (the Prisma `ActorType` , which also admits `SCRIPT / SYSTEM`), `actorId` `nullable uuid` , `changed` `jsonb` , `importLogId` `nullable FK` `onDelete: Restrict` , `createdAt` `Indexes: [feature, entityId, createdAt]` , `[actorId, createdAt]` , `[createdAt]` , `[importLogId]` . `changed` is a `FieldDiff` — `Record<string, { from, to }>` (`audit.ts:63`), one entry per column that moved; `fieldDiff` drops a field null on both sides, so a `CREATE`'s `changed` is not a full snapshot (`audit.service.ts:131`).

Audited: 78 `@Audit(...)` sites across 18 controllers. `AuditInterceptor` (`audit.interceptor.ts:34`) reads the metadata, runs the handler, resolves the action (static, or `TOGGLE_ACTIONS.signIn` / `.tests` off the body — `audit.decorator.ts:18`), then the entity id from, in order, the `AuditContext` store, the created row's `id` , `params.id` , the payload's `id` (`:55`). The write is not awaited and its failure is swallowed (`:67`): the change already committed. `AuditContext` (`audit.context.ts:14`) is an `AsyncLocalStorage` established per request by `AuditContextMiddleware` (`audit.middleware.ts:10`) — middleware, not an interceptor, because `next()` runs the whole request inside the store. `setPatchDiff(null)` marks `unchanged` and the interceptor writes nothing (`audit.interceptor.ts:50`): a no-op save is not a row.

Scoping a read for a non-super admin: `rowActionWhere` (`audit.service.ts:81`) ignores `query.actorId` unless the viewer is a super admin (`:90`), then pins `where.actorId = viewer.id` **outside** the `AND` (`:100`) so an **ANY** match mode cannot widen past the viewer. The same pin scopes `listImports` (`:220`) and `importFile` (`:246`). `assertActive` (`:325`) refuses a signed-in-but-deactivated admin. Archive: 30 days hot, then one gzipped NDJSON object per institute day at `audit/row-actions/YYYY/MM/DD.ndjson.gz` (`audit-archive.ts:9-16`), paged 1000 rows at a time under a 900s Redis lock.

8. Logging and metrics

`f1d5ef4` made `common/logging.ts` the only place `stdout`'s shape is chosen. **The variable is `LOG_LEVEL`** , read off `process.env` because `appLogger()` runs before the config module exists (`:16-19`). Unset means `>=debug` outside production and `>=log` in it (`:10-13`); the value is `filterLogLevels` ' vocabulary (`'>=warn'` , `'>debug'` , `'log,error'` , one level). Production also switches `ConsoleLogger` to `json: true` (`:21`). Passed at `main.ts:25` .

What it turns on is the per-request line in `MetricsInterceptor.record` (`metrics.interceptor.ts:55-62`):

```
POST /attempts/:id/state → 200 42ms STUDENT:0192...ab [req=id]
```

method, **route pattern** (never the URL — `routeOf` :15 , so 5,000 ids are one series), status, rounded ms, actor as `ACTOR:id` or `anonymous` , request id in brackets. At `debug` , so it is every call in dev and staging and none in production. A success `≥1000ms` is a `warn` with `SLOW` appended (`:19,61`). Failures are skipped here (`:42`) — the filter already has the code, stack and body. The actor is inside the message string because this Nest emits each extra param as its own log line (`:57`).

`Metrics` (`metrics.service.ts`), prefixed `iace` on a private `Registry` plus `collectDefaultMetrics` (`:136`): `http_request_duration_seconds` (histogram, `method/route/status`, buckets `0.01...10`), `attempt_submits_total` , `otp_sends_total` , `queue_depth` , `queue_oldest_waiting_seconds` , `queue_job_failures_total` , `queue_connection_errors_total` , `redis_memory_bytes` , `redis_memory_ratio` , `redis_evicted_keys_total` , `live_attempts` , `db_connections` , `scoring_backlog` . The pull-only gauges are read on the scrape (`:179`): per-queue counts, Redis `INFO` memory / stats , `SCARD attempt:dirty` , `count(*) FROM pg_stat_activity` . `onModuleInit` resolves every queue by name off `QUEUE_NAMES` (`:135-148`), so a queue added later is measured without being listed again.

`GET /metrics` (`metrics.controller.ts:22`) is `@Public()` + `@SkipThrottle()` , answers `text/plain`; version=`0.0.4` via `@Res` , and requires `Authorization: Bearer ${METRICS_TOKEN}` whenever set (`:35`); `env.schema.ts:229` makes it mandatory in production.

`Health` (`health/health.controller.ts`): `GET /health` (liveness, `:36`) and `GET /health/ready` (`:51`). Both probe database, Redis, storage and the scoring queue in parallel (`:67`), each under a 2000ms race (`:94`) because a BullMQ client waits forever by design. `/health` always answers 200 with `status: 'ok' | 'degraded'` ; `/health/ready` throws `SERVICE_UNAVAILABLE` naming the down dependencies in `READINESS_DEPENDENCIES` . `HealthModule` imports the four infra modules itself (`health.module.ts:11`) so it travels with an extracted service.

9. Time

Storage UTC; every use is `Asia/Kolkata` (`packages/contracts/src/common.ts:12`). Three homes, split by what ships where:

- `packages/contracts/src/common.ts`** — `INSTITUTE_TIME_ZONE` , `civilDate` :23 (`Intl` , `en-CA` , the locale that formats `YYYY-MM-DD`), `instituteDayLabel` :36 , `instituteWallTime` :99 , `fromInstituteWallTime` :106 ; plus `todayISO()` at `students.ts:53` . Intl-based and dependency-free, because contracts ships into

both SPAs and mobile.

- `apps/api/src/common/time/institute-day.ts` — `startOfInstituteDay :13`, `endOfInstituteDay :18`, `instituteDayOf :23`, `shiftInstituteDay :28`, `toDateColumn :33`, `fromDateColumn :38`. Uses `date-fns` + `@date-fns/tz`, server-only: a date library in contracts lands in every student's browser.
- `apps/api/src/common/importing/date-cell.ts` — `toIsoDate` for a spreadsheet cell of unknown format (Excel serials from 61, month names, several orderings); it normalises, the schema judges validity after.

What breaks with the wrong one: UTC is 5.5h behind, so deriving a civil date from a stored instant with `getUTC*` or `toISOString().slice(0,10)` is wrong every night between 00:00 and 05:30 IST. A `YYYY-MM-DD` filter turned into a hand-built `T00:00:00.000Z` range loses the day's first 5.5h and gains the previous day's last 5.5h — `AuditService.parseDateOnlyBound` (`audit.service.ts:51`) is the correct pattern, picking the start or end helper by which end of the range it is. A `@db.Date` column is a civil date at UTC midnight by convention, so `toDateColumn` / `fromDateColumn` deliberately use UTC and must not be "fixed" to IST.

10. Throttling

`ThrottlerGuard` global and last (`app.module.ts:82`). Four throttlers, all 1-minute fixed windows (`throttling.module.ts:33–39`); each named one carries `skipIf: notMarkedWith(name)` (`rate-limits.ts:31`) so it runs only where its decorator is:

THROTTLER	PER MIN (DEFAULT)	ROUTES
default	<code>RATE_LIMIT_DEFAULT_PER_MIN = 300</code>	every route
otpRequest	<code>RATE_LIMIT_OTP_REQUEST_PER_MIN = 15</code>	<code>POST /auth/student/otp/request</code> (<code>auth.controller.ts:38</code>)
auth	<code>RATE_LIMIT_AUTH_PER_MIN = 120</code>	<code>POST /auth/student/otp/verify</code> , <code>/student/pin/set</code> , <code>/student/login</code> , <code>/admin/otp/request</code> , <code>/admin/otp/verify</code> , <code>/refresh</code> (<code>auth.controller.ts:50,61,78,91,101,114</code>)
sitting	<code>RATE_LIMIT_SITTING_PER_MIN = 60</code>	<code>POST /tests/:testId/attempt</code> , <code>POST /attempts/:id/submit</code> , <code>PATCH /attempts/:id/state</code> (<code>attempts.controller.ts:63,109,121</code>)

Defaults at `env.schema.ts:131–135`. A marked route is counted by both its named throttler and the default. `/health*` and `/metrics` are `@SkipThrottle()` at the class.

Tracker: `trackerFor` (`rate-limit.ts:12`) — `actor:id` when authenticated, `ip:{ip}` only as fallback, because a branch of 200 shares one address; which is also why `main.ts:45` sets `trust proxy` to `TRUST_PROXY_HOPS` (default 0). Counting is in Redis (`redis-throttler.storage.ts:22`, one `MULTI` of `INCR` + `PTTL`) so four containers are one limit, and it **fails open** on a Redis error (`:31`): a blip must not turn every request into a 429 mid-test.

What must already be true

`DATABASE_URL`, `REDIS_URL`, `JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET` (≥24 chars), `PIN_PEPPER`, `S3_BUCKET`, `MEDIA_BASE_URL` required by `env.schema.ts`; `CORS_ORIGINS` and `METRICS_TOKEN` additionally in production (`:229`). Redis must report `noeviction` or production refuses to boot. `API_ROLE` must be `worker` or unset on at least one container, or no job and no scheduler exists anywhere.

Why it is shaped this way

- **One envelope and one filter rather than per-endpoint shapes.** Costs a wrapper on every response and a duck-type check; buys a client parsing one failure shape and one page shape platform-wide. The alternative — HTTP status as the contract — was not taken, so `PIN_LOCKED` and `RATE_LIMITED` stay distinguishable while both answer 429.
- **In-process `EventEmitter2`, no broker and no longer an outbox.** Costs at-least-once delivery; buys zero infrastructure and no relay. The alternative kept until `a5112aa` was a transactional outbox with a trigger and three SQL functions.
- **Redis as the hot-path store, Postgres only via workers.** Costs a hard `noeviction` requirement and a 48h ceiling on resuming a sitting; the alternative is what the scaling rules exist to prevent.
- **One image with an `API_ROLE`, not separate services.** Costs a per-module `onRole` call; the alternative — a separate worker entry point — duplicates the whole bootstrap.

Which screen calls it

`GET /admin/audit/row-actions` and `../export` → `apps/admin/src/features/admins/audit.tsx` (`AuditActivityPage`), admin SPA route `/audit`. `GET /admin/audit/imports` and `../:id/file` → `AuditImportsPage`, route `/audit/imports`. Client methods at `packages/contracts/src/client/admin.ts:985–997`. `GET /health`, `/health/ready`, `/metrics` — **no UI** (probes and Prometheus only). Everything else here is cross-cutting with no screen; the client half is `client/core.ts`, wired per app by `packages/app-kit/src/api-client.ts`.

Is it optimal

- Envelope wrap: zero queries, one object spread. `optimal`.
- `GET /admin/audit/row-actions`: 2 queries in one `$transaction` (`findMany` + `count`, `audit.service.ts:174`) plus 2 for actor names — one `admin.findMany`, one `student.findMany` over the page's distinct ids (`:306`). No N+1. Index `[actorId, createdAt]` serves a non-super admin (the pin is always present), `[createdAt]` a super admin with no actor filter. Read from the schema, **not measured** with `EXPLAIN`.
- `GET /metrics`: 1 Postgres query, 2 Redis `INFO` + 1 `SCARD`, 2 BullMQ calls × 8 queues, all in `Promise.all` (`:179`) — ~19 round trips per scrape, which is why the route is token-gated rather than open.
- `GET /health`: 4 parallel probes each capped at 2000ms, so it answers in ≤2s regardless. `optimal`.
- Rate limiting: 1 Redis `MULTI` (2 commands) per throttler per request — 2 round trips on a marked route, 1 elsewhere. A Lua script would make it one; not taken.
- Audit write: 1 `INSERT` per audited request, off the response path (`audit.interceptor.ts:67` does not await); import rows go through one `createMany` (`:154`).

What it leans on

`packages/contracts` for `ErrorCodes`, `ActorTypes`, `FORM_LEVEL_FIELD`, `REQUEST_ID_HEADER`, `AppException`, `can()`, `FEATURE_KEYS`, the audit vocabulary and the time primitives: one definition the API and both SPAs enforce. `prom-client`, `helmet`, `compression`, `@nestjs/throttler`, `@nestjs/event-emitter` used as shipped; `85bd8d9` removed `@nestjs/config` (with `dotenv`, `dotenv-expand` and `lodash`) for Node's native `process.loadEnvFile` (`apps/api/src/load-env.ts`).

What changed recently

(last 5 touching these paths)

- `f1d5ef4` — `NestFactory.create` had no logger, so Nest printed every level in production. `logging.ts` now decides it from `LOG_LEVEL`, and `MetricsInterceptor` logs one line per call at debug (warn if $\geq 1000\text{ms}$).
- `85bd8d9` — env loading moved to `load-env.ts` and Node's own loader; three packages left the production tree.
- `a9ae56d` — the exam role now gives up: `statement_timeout=6s` appended to its Prisma URL (`prisma.service.ts:18`) and an 8s request budget answering `SERVICE_UNAVAILABLE` (`request-budget.ts:20`). Keyed on the role being *exactly* `exam`, never `servesRole`, because `all` also runs 120s BULK scoring transactions.
- `e42da86` — a 500 now logs the request body through `redact` (2KB, 6 levels, 20 items, word-level key matching) and attaches it to the Sentry event. 4xx stays one line.
- `4ead5d3` — `student.access_changed` deleted: eight producers emitting an event with no listener, plus `ACCESS_STUDENT_FIELDS` and the `DomainEventBus` dependency of three services holding it for that alone.

What is wrong with it

- `prisma/schema.prisma:1020` — `RowActionLog.entityLabel` is declared (DDL at `prisma/migrations/20260820000000_schema_realignment/migration.sql:541`) but **nothing writes or reads it**: a grep for `entityLabel` across `apps/`, `packages/`, `scripts/`, `prisma/` finds only those two lines. Its comment promises "the label as it was, so the log survives a rename or a delete" — a promise the code does not keep. Bites on any audit row whose entity is later renamed: the screen shows the raw uuid, because `toRowAction` (`audit.service.ts:261`) never selects a label.
- `health.controller.ts:41-46` — `GET /health` answers **200** with `status: 'degraded'` when a dependency is down, so a liveness probe keyed on the status code will never restart a container whose Redis is dead; only `/health/ready` returns 503. Safe only if the deployment reads the body or probes `/health/ready` — **not verified** from code.
- `queue/queues.ts:83-84` — `RELAY_BATCH`'s comment, "How many stranded events one relay pass hands on", is outbox-era wording. The constant is live but now bounds the notification sweep's claim batch (`notifications.processor.ts:83,117`); there is no relay and no stranded event.
- `app.module.ts:9` — `apps/api/src/events` is the **Event / EventCandidate domain feature** (who an EVENT series draws its roster from, `events/events.module.ts:7`), not the event bus, which is `apps/api/src/common/events`. The root module must alias one (`EventsModule` as `DomainEventsModule`) to compile. A naming collision, not a bug.
- `packages/contracts/src/envelope.ts:254-266` — `errorCodeForStatus` has no 422 entry, so a bare 422 with no envelope (from a proxy) reaches the client as `VALIDATION_ERROR`, though this API's own 422s are `DRAW_SHORTFALL` and `EXPORT_TOO_LARGE`. Harmless today: every 422 the API emits carries an envelope.

Telling a student something happened

Where it lives

`apps/api/src/notifications/**` (14 files), `apps/api/src/common/messaging/**` (9 files).

- `notifications.service.ts` — owns `Notification`; `tell()` is its only writer; bell list and mark-read.
- `notifications.processor.ts` — the sweep: claims unpushed rows, pushes them, books the paid chain.
- `push.service.ts` — owns `PushSubscription` + `PushDevice` and the free fan-out to both.
- `web-push.sender.ts` / `fcm.sender.ts` — the two transports; each exposes `isConfigured`.
- `notification-delivery.processor.ts` — the only place money is spent, plus `repairStalled`.
- `announcements.service.ts` — the admin broadcast: price, cap, fan-out, stats, export.
- `notification-policy.ts` — `SKIP_REASONS`, the paid-channel map, the grace window.
- `common/messaging/routed-message-sender.ts` — one `MessageSender` interface, a switch over providers.

What it is for

Every fact the platform learns about a student (a test opened, a result scored, a grant added, a PIN changed, an admin's broadcast) has to reach them. The `Notification` row is the source of truth; everything else pulls attention back to it. Free channels (web push, FCM) go immediately; a paid channel (SMS/WhatsApp) is bought only for an announcement whose admin opted in, and only after a 10-minute grace window the student can close by reading the bell. `NotificationDelivery` is the per-channel ledger that makes "what did this send cost, who did it reach" answerable.

FLOW — AN ANNOUNCEMENT GOES OUT

- 1 Admin submits the compose dialog
`apps/admin/src/features/announcements/compose-announcement-dialog.tsx:104` `api.admin.announcements.send(values)`
- 2 `POST /admin/announcements`
`apps/api/src/notifications/announcements.controller.ts:69` `send()`
`class @Controller('admin/announcements') + @Actors(ActorTypes.ADMIN) (:42-43); method @RequiresFeature(FEATURE_KEYS.NOTIFICATION_MANAGEMENT, PERMISSION_LEVELS.WRITE) @Post(), @CurrentUser, @Body(new ZodBody(createAnnouncementSchema))`
- 3 Validated
`packages/contracts/src/announcements.ts:31`
`title ≤120, body ≤1000, audience = the Students-screen filter, paidChannels defaults []`
- 4 Cap read
`apps/api/src/notifications/announcements.service.ts:70`
`NOTIFICATION_MAX_RECIPIENTS, 50000 (.env.example:214)`
- 5 Cohort = student filter AND `deletedAt: null`
`:255 cohortWhere()`

- 6 Ids fetched `take: cap + 1`
:74
| one query bounds and detects overflow
- 7 Branch
> cap → VALIDATION_ERROR (:80); === 0 → 'That reaches nobody' (:86)
| nothing written
- 8 Cost priced off the FIRST paid channel only
:136 priceOf()
| the rest are fallback, so unpriced
- 9 `TX_LIMITS.BULK` transaction opens (maxWait 10 s, timeout 120 s, `prisma/prisma.service.ts:11`)
:92
- 10 `Announcement` row written
:93
| audience, paidChannels, recipientCount, estimatedCostPaise, createdById
- 11 Recipients chunked 1000
:106, CHUNK at :37
- 12 One intent per student
:119 intentFor()
| `GENERIC`, `data: { message: body }`, `dedupeKey: announcement:<id>`
- 13 Bell rows written
`apps/api/src/notifications/notifications.service.ts:51`
| `createMany({ skipDuplicates: true })`; `pushedAt` `NULL`, `isRead` `false`
- 14 Commit
the whole cohort is told, or none of it
- 15 `detail(id)` re-read, stats counted off the ledger
:161, :202
| `notification.count` + one `groupBy`
- 16 Envelope interceptor wraps `{ success: true, data, meta }`; the student's app reads it later via `GET /me/notifications` (`apps/api/src/me/me.controller.ts:95`)

FLOW — A PUSH SEND

- 1 The sweep hands a page of claimed rows
`apps/api/src/notifications/notifications.processor.ts:92`
| `push.deliverAll()`; `studentId === null` dropped at :94
- 2 Whole body in a try/catch that only logs
`apps/api/src/notifications/push.service.ts:95`
| never throws back into the job
- 3 Three reads in one `Promise.all`
:103 reachOf()
| existing `WEB_PUSH` / `MOBILE_PUSH` ledger rows; subscriptions only if `sender.isConfigured`; devices only if `fcm.isConfigured`, else `[]`
- 4 Every target's session checked in one Redis `MGET`
:173 signedOut()
| `session:student:<studentId>:<sessionId>`, TTL = refresh lifetime (`auth/session.service.ts:65`); missing key = signed out
- 5 Refused (bad host or dead session) and orphaned targets deleted, matched on `(endpoint/token, studentId, sessionId)` as read
:131-148
| a target another sign-in rebound meanwhile survives
- 6 Survivors grouped by student with `Map.groupBy`
:152, :156
| `stdlib`; the local 9-line helper went in `382b365`, which moved the API tsconfig ES2023→ES2024
- 7 Page pushed in lanes of 8
:89, PUSH_LANES at :261
- 8 Two channels decided independently per notification
:184
| `open(channel)` false when a ledger row exists; no targets = absence, nothing booked
- 9 Payload is title + `/notifications` + id, never the body
:185, NOTIFICATION_INBOX_PATH at `contracts/src/access.ts:335`
| a lock screen must not render a score
- 10 Browser leg: `sendNotification` per subscription, TTL 6 h, timeout 5 s
`web-push.sender.ts:59-63`
| `404/410` → `GONE`
- 11 Phone leg: service-account JWT (RS256) traded at `oauth2.googleapis.com/token`, cached with a 1-min renew margin
`fcm.sender.ts:86-110`
| no `firebase-admin`; one POST to `fcm.googleapis.com/v1/projects/<id>/messages:send` (:113)

12 FCM outcome mapped

```
apps/api/src/notifications/fcm.sender.ts:130
| 404, or 400/403 whose error.details[0].errorCode is UNREGISTERED / INVALID_ARGUMENT / SENDER_ID_MISMATCH → GONE ; else FAILED
```

13 Each leg folded to one ledger row

```
apps/api/src/notifications/push.service.ts:289
| any SENT → SENT, sentAt, attempts 1 ; else FAILED, failedAt, lastError 'Nothing accepted the push (N tried)'
```

14 Dead targets pruned, ledger written once for the page

```
:238 settle()
| deleteMany by endpoint / by token, then createMany({ skipDuplicates: true })
```

15 No retry: a FAILED push row is terminal, and no paid channel is booked here

FLOW — DEVICE REGISTRATION

1 Web: the student flips the switch

```
apps/exams/src/features/notifications/notification-settings.tsx:128; the key comes from GET /me/push-subscription (:22 → me.controller.ts:112), returning { publicKey }, null when VAPID is unset
```

2 Web rebind on every signed-in load and every in-place session swap

```
apps/exams/src/lib/pwa.ts:118, :126
| once per page load, not per navigation ( 539ab6a )
```

3 POST /me/push-subscription

```
apps/api/src/me/me.controller.ts:118
| class @Actors(ACTOR_TYPES.STUDENT) ( :67 ); @Post , @HttpCode(HttpStatus.OK) , @CurrentUser , @ZodBody(pushSubscriptionSchema)
```

4 Endpoint host allow-listed in the schema itself

```
contracts/src/access.ts:307, :322
| https plus fcm.googleapis.com , push.services.mozilla.com , notify.windows.com , web.push.apple.com or a subdomain
```

5 Upsert keyed on endpoint

```
apps/api/src/notifications/push.service.ts:55
| writes p256dh , auth , userAgent , sessionId from the token; the same device resubscribing is the same row, moved to the live session
```

6 Mobile: Android only, real device only, not Expo Go

```
apps/mobile/src/lib/push-device.ts:24
| iOS returns an APNs token FCM cannot address
```

7 Permission asked, then getDevicePushTokenAsync(), accepted only when type === 'android'

```
:59-71
```

8 POST /me/push-device

```
apps/api/src/me/me.controller.ts:137
| pushDeviceSchema : token ≤4096, platform ANDROID/iOS, deviceName ≤120
```

9 Upsert keyed on token

```
apps/api/src/notifications/push.service.ts:71
| the token is the identity; it rotates, the phone does not
```

10 Teardown: DELETE /me/push-subscription by endpoint (me.controller.ts:127), DELETE /me/push-device by token (:146); both deleteMany scoped by studentId (push.service.ts:64 , :80)

11 Mobile drops its token before the session ends

```
apps/mobile/app/(tabs)/account.tsx:16
```

Two models, not one, per prisma/schema.prisma:1236-1238 : a browser subscription carries the keys that encrypt for it, a device token is one opaque string FCM owns and rotates. Different unique key (endpoint vs token), columns (p256dh / auth / userAgent vs platform / deviceName), transport, and death signal (410 vs UNREGISTERED). One table would mean nullable keys plus a discriminator.

FLOW — THE RELAY/SWEEP

1 Schedulers upserted at boot, worker role only

```
notifications.module.ts:62-73
| servesRole(API_ROLES.WORKER) gates it; fixed scheduler ids stop a redeploy stacking a second sweep
```

2 Repeat job relay-sweep every 60 s (queue/queues.ts:139); tests-opened-sweep every 5 min (:142); prune cron 0 21 * * * = 02:30 IST (:164)

3 Runs on the notifications queue, concurrency 4, attempts 5

```
notifications.processor.ts:47, queues.ts:31
```

4 Branch on job name

```
:64
| tests-opened-sweep → openings.sweep() and return
```

5 pushPending()

```
:78
| up to 25 pages a pass ( :35 ), stopping early on a short page
```

6 One page claimed and booked in one TX_LIMITS.SHORT transaction

```
:110
| raw UPDATE ... WHERE id IN (SELECT ... WHERE "pushedAt" IS NULL ORDER BY "createdAt" LIMIT 200 FOR UPDATE SKIP LOCKED) ; batch = RELAY_BATCH 200 ( queues.ts:84 ), on partial index Notification_unpushed_idx ( prisma/migrations/20260929180000_a_notification_is_written_where_it_happens/migration.sql:26 )
```

- 7 The same statement LEFT JOINS `Announcement` for `paidChannels` and books the FIRST channel only
:129
| one `PENDING` ledger row
- 8 Page pushed free
:92 (flow 2 above)
- 9 Booked paid rows re-read, then one `addBulk`
:142, :155
| job id keyed on the delivery row, `delay = escalationFor(...).deferSec * 1000` → 600 s, or 0 when `actBy` leaves under 600+1800 s (`notification-policy.ts:32-55`)
- 10 `repairStalled()` in the `finally`, after the pushes, SWEEP job only
:73
| moved after them in `539ab6a` because paid sends go one at a time
- 11 Repair selects `PENDING` paid rows older than 20 min, `LIMIT 25`
notification-delivery.processor.ts:159-165
| index `NotificationDelivery_pending_idx` (`20260926140000/migration.sql:21`); status is a raw literal so the planner can prove the partial predicate (:56)
- 12 Each stuck row forced to a decision sequentially at `ATTEMPT_CAP`
:168 (marked ponytail: at :167); a full batch logs a warning (:173)
- 13 Paid send decides first
:102-120
| already read → `SKIPPED/ALREADY_READ`; no template kind → `NO_TEMPLATE`; no mobile → `NO_CONTACT` + fall back
- 14 Row claimed `SENT` before the provider call
:123
| a retry that still found `PENDING` would pay twice
- 15 On error
MessageNotConfiguredError → skip + fall back (:141); under `ATTEMPT_CAP` 4, rethrow so BullMQ backs off 15 s on the same channel (:150, queues.ts:33); at the cap → `fallBack()` upserts the next channel and queues it (:213)
- 16 Nightly prune deletes settled rows older than `DELIVERY_RETENTION_DAYS 180` (`contracts/src/announcements.ts:64`) in pages of 1000, max 100 pages
notification-prune.processor.ts:46-57
| index `NotificationDelivery_settled_idx`

Call sites — `pushPending` has exactly **one** production caller: `process()` at `notifications.processor.ts:70`; the rest are `apps/api/test-db/notification-push.db.test.ts` (10, at lines 69, 78, 80, 90×2, 109, 124, 146, 156, 168, 170). `repairStalled` has exactly **one**: `notifications.processor.ts:73`, plus `test-db/notification-delivery.db.test.ts:243,257`. `tell` has **seven**: `access/student-grants.service.ts:145`, `attempts/scoring.processor.ts:176` and :194, `announcements.service.ts:107`, `notification.listener.ts:54`, `test-opening.service.ts:70`, `students/students.service.ts:383`.

The data it touches

COLUMN	TYPE	HOLDS	WRITTEN BY
<code>Notification.studentId / .adminId</code>	uuid?	exactly one recipient — <code>CHECK ((studentId IS NOT NULL)::int + (adminId IS NOT NULL)::int = 1)</code> (<code>20260907120000/migration.sql:72</code>)	<code>tell()</code> ; nothing writes <code>adminId</code>
<code>Notification.type</code>	enum	<code>TEST_ASSIGNED, RESULT_READY, ENROLLMENT_ADDED, GRANT_ADDED, GENERIC, RESULT_UPDATED, PIN_CHANGED, WELCOME</code> (<code>schema.prisma:164</code>)	<code>tell()</code>
<code>.title / .body</code>	text/text?	what the bell shows; only <code>title</code> leaves on a push	<code>tell()</code>
<code>.data</code>	jsonb?	<code>`Record<string, string></code>	<code>number></code> template vars; announcements write <code>{ message: <body> }</code> (<code>announcements.service.ts:129</code>), read back by <code>variablesOf</code> (<code>notification-delivery.processor.ts:226</code>) and handed to the provider. Other producers write their own keys. <code>tell()</code>
<code>.dedupeKey</code>	text?	<code>announcement:<id></code> , <code>test-open:<testId></code> , <code>welcome:<studentId></code> ; two partial uniques, one per recipient column (<code>20260907120000/migration.sql:75,79</code>)	<code>tell()</code>
<code>.actBy</code>	timestamptz?	when it stops being actionable; sole input to the grace window	<code>tell()</code> — no producer passes one
<code>.isRead</code>	bool	read receipt the paid send checks	<code>markRead()</code> (<code>notifications.service.ts:88</code>)
<code>.openedVia</code>	enum?	which channel brought them back	nothing
<code>.pushedAt</code>	timestamptz?	the sweep's claim stamp	<code>claimPage()</code> raw UPDATE

COLUMN	TYPE	HOLDS	WRITTEN BY
Announcement.audience	jsonb	the filter verbatim: <code>q, branchId[], programCode[], eventId[], course[], isActive, isTestBlocked, hasDefaultPin, preTestReady, profileCompleted, noAccess, match (studentListQuerySchema minus page/pageSize/sort — contracts/src/students.ts:212, announcements.ts:21)</code>	<code>send()</code>
.paidChannels	enum[]	WHATSAPP and/or SMS, in fallback order	<code>send()</code>
.recipientCount / .estimatedCostPaise	int	counted and priced at send time	<code>send()</code>
NotificationDelivery.(notificationId, channel)	unique	one row per channel that can fail; IN_APP has none	<code>claimPage(), settle(), fallback()</code>
.status	enum	PENDING/SENT/DELIVERED/FAILED/SKIPPED — DELIVERED never written , only counted (announcements.service.ts:219)	both processors
.skipReason	text?	NO_TEMPLATE, NO_CONTACT, ALREADY_READ (notification-policy.ts:11)	<code>skip()</code>
.providerMessageId	text?	—	nothing ; no reference in <code>apps/</code> or <code>packages/</code>
.attempts / .lastError	int/text?	retry count, last provider error	<code>recordFailure, ledgerRow</code>
PushSubscription.endpoint	text @unique	the browser's push endpoint	<code>subscribe()</code>
.p256dh / .auth	text	the browser's own encryption keys	<code>subscribe()</code>
PushSubscription.sessionId / PushDevice.sessionId	uuid?	the sign-in that registered it	<code>subscribe() / registerDevice()</code>
PushDevice.token	text @unique	FCM registration token, ≤4096	<code>registerDevice()</code>
.platform	ANDROID\	IOS	which store issued it; only ANDROID is ever sent (push-device.ts:24) <code>registerDevice()</code>

Redis: `session:<actor>:<subjectId>:<sessionId>` (`apps/api/src/redis/redis.keys.ts:38`), read-only here via `mgetJson`, TTL = refresh lifetime (set at `auth/session.service.ts:65`). This module writes no Redis key.

What must already be true

an `Admin` with `NOTIFICATION_MANAGEMENT` at READ (list/detail), WRITE (preview/send) or EXPORT (deliveries); `NOTIFICATION_MAX_RECIPIENTS` plus the two cost vars (`.env.example:210–214`); the sweep runs only where `servesRole(API_ROLES.WORKER)` and the controller mounts only under `API_ROLES.CORE` (`notifications.module.ts:37,62`); all three `VAPID_*` for web push, all three `FCM_*` for mobile; a paid send also needs `OTP_SENDER` ≠ console with `SMS_PROVIDER_URL`, and `WHATSAPP_INTERAKT_API_KEY` for WhatsApp, or `RoutedMessageSender` has no provider and throws (`routed-message-sender.ts:14`); migrations `Notification_unpushed_idx`, `NotificationDelivery_pending_idx`, `NotificationDelivery_settled_idx`.

Can a push be delivered today?

- **Web push: yes.** The path is complete and `VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY` are both non-empty in the working-tree `.env` (87 and 43 characters — I checked length only, never the value), so `WebPushSender.isConfigured` is true and `sendNotification` really goes out. `.env.example:225–226` ships them blank, so a fresh clone is dark until the pair is generated.
- **Mobile push (FCM): dark.** `FCM_PROJECT_ID`, `FCM_CLIENT_EMAIL`, `FCM_PRIVATE_KEY` are empty in both `.env.example:234–236` and the working-tree `.env`, so `FcmSender.isConfigured` is false (`fcm.sender.ts:51`), `reachOf` skips the `pushDevice` query (`push.service.ts:116`), and no `MOBILE_PUSH` row is ever booked. The transport is written and db-tested; the gap is credentials plus `google-services.json` (`.env.example:233`), not code.
- The prior audit's "push may be dark" is half right: true of FCM, not of web push.

What replaced the outbox — before `a5112aa`, a producer wrote a `notification.requested` row to `OutboxEvent` in its own transaction and a relay turned it into the `Notification` row a minute later. Now `tell(tx, ...)` writes the row directly in the producer's transaction (`notifications.service.ts:49`), so the bell no longer lags the fact. What survives of the relay is the **push** sweep: `Notification.pushedAt` plus `Notification_unpushed_idx` is the claim mechanism, and `claimPage()`'s `FOR UPDATE SKIP LOCKED` replaces the relay's row lease. `prisma/migrations/20260930170000_the_outbox_and_its_relay_are_gone/migration.sql` drops the table, its `BEFORE INSERT` trigger and the three SQL functions that served it.

Why it is shaped this way

- The bell row is written by its producer, not a relay. The outbox cost a minute's lag per notification and a second durable write per fact; the price now is `pushedAt` as a column on a hot table, and atomicity only as good as the producer's transaction.
- `dedupeKey` + `skipDuplicates` instead of an application "have I told them" read: a replay lands once with no extra query. Cost: two partial uniques because the recipient lives in two columns, and a keyless notification (PIN changed) can legitimately double up.
- The ledger is `UPDATED` on retry, never appended (`@unique([notificationId, channel])`), keeping it $O(\text{notifications} \times \text{channels})$. An attempt log would make per-announcement stats a window function instead of one `groupBy`, and the prune far larger.
- Paid channels are a fallback *chain*, not a fan-out (`firstChannelFor / nextChannelAfter`). Fanning out would be simpler code and double the invoice.

Which screen calls it

ENDPOINT	UI FILE	ROUTE
GET /admin/announcements	apps/admin/src/features/announcements/announcements.tsx:66	admin /announcements (apps/admin/src/lib/constants.ts:145 , nav "Sent" :496)
POST /admin/announcements/preview	compose-announcement-dialog.tsx:98	admin, same screen (dialog)
POST /admin/announcements	compose-announcement-dialog.tsx:104	admin, same screen
GET /admin/announcements/:id	announcements.tsx:177	admin, row detail
GET /admin/announcements/:id/deliveries/export	announcements.tsx:207	admin, detail download
GET /me/notifications	apps/exams/src/features/notifications/notifications.tsx:71 ; apps/mobile/app/notifications.tsx:35 ; badge apps/mobile/app/(tabs)/account.tsx:32	exams /notifications ; mobile /notifications
PATCH /me/notifications/:id/read	.../notifications.tsx:209 ; apps/mobile/app/notifications.tsx:41	both
GET /me/push-subscription	notification-settings.tsx:22	exams /notifications/settings (apps/exams/src/lib/constants.ts:38)
POST /me/push-subscription	notification-settings.tsx:128 ; rebind apps/exams/src/lib/pwa.ts:122	exams settings + every signed-in load
DELETE /me/push-subscription	apps/exams/src/lib/pwa.ts:137	exams settings
POST /me/push-device	apps/mobile/src/lib/push-device.ts:30 (via src/providers/auth.ts:4 , src/lib/use-push-device.ts)	mobile, no screen — on sign-in and on a restored session
DELETE /me/push-device	apps/mobile/app/(tabs)/account.tsx:16	mobile Account (sign-out)

No admin UI reads NotificationDelivery directly; the detail screen reads the stats object off GET /admin/announcements/:id .

Is it optimal

read from code and migration headers; I ran no EXPLAIN .

- preview 1 query (student.count), optimal. list 2 in one \$transaction , no per-row stats by design (announcements.service.ts:147), on Announcement_createdAt_idx . detail 3 — row, count, one groupBy over (status, skipReason) ; fae9fe4 collapsed a count per status.
- send : 1 findMany (ids, take: cap+1) + 1 create + [N/1000] createMany + 2 for detail . A 50 000-recipient send is 50 createMany statements in one 120 s transaction. No N+1. bd05f5d removed a second cohort count that priced "reachable" separately — it always equalled recipientCount .
- GET /me/notifications : 2, both on Notification_studentId_createdAt_isRead_idx (20260922100000), which answers the unread filter index-only.
- The sweep per 200-row page: 1 raw claim + 1 createMany in a transaction, then 3 parallel reads + 1 Redis MGET + ≤2 deleteMany + 1 createMany , then 1 findMany + 1 addBulk . **Constant query count per page whatever the page size** — the point of e2a2b49 . The real cost is one HTTP call per subscription and per token, 8 in flight.
- repairStalled : 1 indexed SELECT ... LIMIT 25 , then ≤25 sequential deliver() calls at ~2-4 queries each. 20260926140000 's header measures the index: 2131 buffers / 7.3 ms → 1 buffer / 0.3 ms. Prune: 1 SELECT ... LIMIT 1000 + 1 deleteMany per page, ≤100 pages a night.

What it leans on

the response envelope and AppException / ErrorCodes ; @Actors / @RequiresFeature / @RequiresExport / @CurrentUser ; ZodBody / ZodQuery over contracts schemas, which is where the push-endpoint allow-list lives so a bad host never reaches the service; common/pagination ; common/exporting ; common/prune-in-pages ; @Audit + AuditContext ; common/messaging 's MessageSender seam, so the processor names a channel and a kind and never a provider; BullMQ for the sweep, the delayed paid sends and the prune; ReportingWorkerHost + QueueFailures ; RedisService.mgetJson ; @nestjs/event-emitter for the two auth facts (notification.listener.ts); AppConfigService ; and api-role 's onRole / servesRole , which keeps the controller off the worker and the schedulers off the API boxes.

What changed recently

(git log --oneline -15 over both paths)

- 5ee4915 — repairStalled now warns on a full batch of 25, giving the deferral an observable trigger; the sequential loop is unchanged.
- 382b365 — behaviour-neutral: local groupBy → Map.groupBy ; API tsconfig ES2023→ES2024.
- bd05f5d — reachableCount and the second cohort count removed from preview and send; cost priced off recipientCount / the rows fetched. One fewer query each.
- 539ab6a — createAuth gained onSignedIn , so an in-place session swap (a PIN change revokes everything) re-registers the push target instead of leaving it on a dead session; the web rebind moved from every shell mount to once per page load, removing a PushSubscription upsert per navigation during a submit burst; repairStalled moved after the free pushes.
- 57533cc — push targets bound to a session and re-registered on every signed-in load (web) / sign-in (phone); a dead-target delete matches (target, studentId, sessionId) as read, so a concurrent rebind survives.

What is wrong with it

1. prisma/schema.prisma:1111-1112 — providerMessageId is documented "indexed for the webhook that will", but 20260922100000/migration.sql:20 dropped that index and nothing replaced it. The column is also never read or written anywhere in apps/ or packages/ .
2. apps/api/src/notifications/test-opening.service.ts:5 — the header still says announcedAt is stamped "in the same transaction as the outbox rows". No outbox rows exist since a5112aa ; it is stamped with the Notification rows (:84).
3. apps/api/src/queue/queues.ts:84 — RELAY_BATCH is documented as "how many stranded events one relay pass hands on", and NOTIFICATION_JOBS.SWEEP is still the literal 'relay-sweep' (:134). Vestigial outbox names; the constant is now the push claim size. Renaming the job string would orphan the live repeat scheduler, so

this one is not free.

4. `.env.example:220` — "served to the SPA by `GET /me/notification-preferences`". No such route; it is `GET /me/push-subscription` (`me.controller.ts:112`). Anyone wiring VAPID fresh looks for the wrong endpoint.
5. `prisma/schema.prisma:1061` — `Notification.openedVia` is written and read by nothing, yet its comment claims it exists "so the policy stops being an assumption". The policy never consults it.
6. `DeliveryStatus.DELIVERED` is counted at `announcements.service.ts:219` and exported as a column (`:250`) but written nowhere, so the announcement detail always shows `delivered: 0` beside a non-zero `sent`, and the export's "Delivered at" column is always blank.
7. `notifications.service.ts:27` + `notification-policy.ts:44-55` — `actBy` drives the "pay immediately rather than wait" branch, but no producer sets it (only the schema, the migration and `toRow` mention it). `escalationFor` therefore always takes the `Infinity` path and `deferSec` is always 600; the immediate-buy branch is reachable only from a test.
8. `notification-delivery.processor.ts:41` — `KIND_OF` covers `RESULT_READY`, `TEST_ASSIGNED` and `GENERIC`, so a `GRANT_ADDED` notification (`access/student-grants.service.ts:145`) could only ever be `SKIPPED/NO_TEMPLATE`. Unreachable today, because only an announcement supplies `paidChannels` and an announcement is always `GENERIC` — but the table reads as if the other kinds were live options.
9. `push.service.ts:304` — a push that found targets and had every one fail writes `FAILED` with no retry, and there is no `repairStalled` equivalent for the free channels. A transient web-push outage during one sweep page loses that page's pushes silently: `pushedAt` is already stamped, so the sweep never revisits the row. The bell row survives, which is the stated contract, but "best effort" here means exactly one attempt.

A student's own account

Where it lives

`apps/api/src/me/`, `apps/api/src/saved/`; the writes land in `apps/api/src/students/` (admin-owned module, reused). No student-facing route lives under `students/` — `apps/api/src/students/students.controller.ts:65-66` is `@Controller('admin/students')` + `@Actors(ADMIN)`.

- `apps/api/src/me/me.controller.ts` — 15 routes, class-level `@Actors(ActorTypes.STUDENT)`; no id in any path.
- `apps/api/src/me/me.service.ts` — profile read/update, document save; delegates to `StudentsService`.
- `apps/api/src/me/documents.ts` — kind→column map, S3 key shape, byte sniffing.
- `apps/api/src/saved/saved-questions.service.ts` — the only writer of `SavedQuestion`.
- `apps/api/src/saved/saved.controller.ts` — 5 routes at `me/saved`.
- `apps/api/src/access/access-resolver.service.ts` — the catalog ("my tests") and the one reach rule.
- `apps/api/src/students/students.service.ts` — owns `Student` / `StudentProfile`; `detail`, `update`, `saveDocumentKey`, `enrolmentOf`.
- `apps/api/src/students/student-privacy.service.ts` — the DPDP data export.
- `packages/contracts/src/me.ts`, `.../saved.ts` — schemas + `ME_ROUTES` / `SAVED_ROUTES`.

What it is for

Everything a signed-in student may read or change about themselves, with the subject taken from the token rather than from the request. It answers four questions: who am I and what do I stand on (`/me`), what tests can I reach right now (`/me/catalog`), what did I star for revision (`/me/saved`), and the account chores — PIN, devices, photo, marksheet. Both controllers are registered `onRole([API_ROLES.CORE], ...)` (`me.module.ts:24`, `saved.module.ts:11`), so an `API_ROLE=exam` container serves none of them.

FLOW — GET /ME

1	<code>GET /me</code> <code>apps/api/src/me/me.controller.ts:84-87</code> <code>profile()</code> <code>decorators: class @Controller('me') + @Actors(STUDENT); method only @Get()</code> .
2	Bearer token verified, Redis session checked <code>apps/api/src/auth/guards/jwt-auth.guard.ts:28-35</code> <code>canActivate()</code> <code> request.user = { id, actor, sessionId }; a student gets permissions: {} (:41).</code>
3	Actor gate <code>apps/api/src/auth/guards/actor.guard.ts:19-24</code> <code>canActivate()</code> <code> an ADMIN token here is FORBIDDEN.</code>
4	<code>MeService.profile(user.id)</code> <code>apps/api/src/me/me.service.ts:38-40</code> <code> row read next.</code>
5	<code>student.findFirst({ where: { id }, include: { profile, eventCandidacies.event } })</code> <code>students.service.ts:164-171</code> <code> one row or NOT_FOUND 'No such student'.</code>
6	<code>student.count({ id, ...HOLDS_OWN_ACCESS })</code> <code>students.service.ts:172, rule at students/own-access.ts:4-11</code> <code> hasOwnAccess boolean.</code>
7	<code>toSummary</code> derives <code>preTestReady</code> / <code>profileCompleted</code> from the profile columns <code>students.service.ts:514, readinessOf at packages/contracts/src/students.ts:95-101</code> <code> nothing stored; READINESS_FIELDS (:81-86) names the fields.</code>
8	<code>toProfileView</code> presigns <code>photoUrl</code> and <code>tenthMarksheetUrl</code> <code>students.service.ts:224,227,235-238</code> <code> two GET URLs valid DOCUMENT_URL_TTL_SEC = 300 s (:42); dob via fromDateColumn.</code>
9	<code>educationDetails</code> / <code>pastExamHistory</code> <code>safeParse</code> d, malformed → <code>null</code> <code>students.service.ts:229-231</code>

- 10 `withEnrolment` → `enrolmentOf`
me.service.ts:43-45, students.service.ts:184-200
| `Promise.all` of `program.findMany`, `exam.findMany`, `branch.findUnique`; an unmatched code keeps itself as its name.
- 11 `ResponseInterceptor` wraps
apps/api/src/app.module.ts:75
| `{ success: true, data: Me, meta: { requestId } }`.

FLOW — BOOKMARKING A QUESTION

- 1 Star pressed on the review surface
packages/app-kit/src/review.ts:42-48 `useBookmarks().toggle`
| posts when no `savedId` is held, else DELETES.
- 2 `POST /me/saved/bookmarks`
apps/api/src/saved/saved.controller.ts:48-55 `bookmark()`
| `@Post('bookmarks')`, `@HttpCode(HttpStatus.OK)`, `@Body(new ZodBody(bookmarkQuestionSchema))`; body is `{ attemptId, questionId }` (contracts/src/saved.ts:40-43).
- 3 JWT + actor guards as above
jwt-auth.guard.ts, actor.guard.ts
- 4 `servedPastTheGate` reads the sitting scoped by owner
saved-questions.service.ts:229-233
| `attempt.findFirst({ id, studentId })`; another student's id reads as `null`.
- 5 `paperQuestion.findUnique({ testId_questionId })`
:236-239
| proves the paper actually served it.
- 6 Missing sitting or missing paper row → `NOT_FOUND 'You did not sit this question'`
:240
- 7 `status !== EVALUATED` → `CONFLICT 'This sitting has not been marked yet'`
:241-243
- 8 `savedQuestion.create({ studentId, questionId, attemptId, paperQuestionId })`
:174-181
| row written.
- 9 A unique violation on `(studentId, questionId)` is swallowed
:182-184, `isUniqueViolation`
| the second star is the first row, so this is idempotent.
- 10 `findUniqueOrThrow({ studentId_questionId })` with `ROW_SELECT`
:186-189
| joins `question.subject.name`, `question.topic.name`, `question.currentVersion.content`.
- 11 `toSavedQuestion`
:268-280
| `stemPreview` via `stemPreviewOf`; `options` and `answerKey` deliberately absent (:31); `testId / testTitle / timeSpentSec` null on this path (no `sat` argument passed).
- 12 Client invalidates `savedQueryKey()`
packages/app-kit/src/review.ts:50
| the list and facets refetch.

Read back: `GET /me/saved` → `saved.controller.ts:33-39` → `list()` (`saved-questions.service.ts:53-79`): optional `attemptIdsOn` pre-query when `testId` is filtered (:82-88), then `$transaction([findMany, count])` ordered `createdAt desc, id desc` with `pageArgs` — served by `SavedQuestion_studentId_createdAt_idx` (`prisma/schema.prisma:1271` ; DDL `prisma/migrations/20260925090000_a_mistake_is_read_not_kept/migration.sql:26`), with the `id` tie-break sorted in memory. Then `satContext` (:91-140) adds `attempt.findMany` + `paperQuestion.findMany`. Covered by `apps/api/test-db/saved-questions.db.test.ts` — 12 cases including the FK-nulling one at :201.

FLOW — MY TESTS (GET /ME/CATALOG)

- 1 Screen mounts
apps/exams/src/features/tests/tests.tsx:41 `useQuery(catalogQuery)`
| options at `packages/app-kit/src/student-queries.ts:87`.
- 2 `GET /me/catalog`
me.controller.ts:90-93 `catalog()`
| `@Get('catalog')` only.
- 3 `MeService.catalog` → `AccessResolverService.catalog(studentId, new Date())`
me.service.ts:47-49, access-resolver.service.ts:149-154
- 4 `standingOf`
:273-285
| one `student.findUnique` selecting `'STANDING_FACTS'` (:110-118): reach facts plus `grants`, `eventCandidacies`, and every non-`VOIDED` attempt ordered by `attemptNo asc`, folded into `Map<testId, status>` so the latest sitting wins.
- 5 Absent/ `isActive: false` / `deletedAt` → `NOT_FOUND 'No such student'`
:151, `isPresent` at :288

- 6 `shared()`
:239-262
GET access:catalog:epoch from Redis, then one `testSeries.findMany({ isEnabled: true })` with `SHARED_SELECT` (:34-78): ACTIVE tests, `programUnlocks`, `baseConfig.sections`. Held per process, keyed on the epoch, max age 15 min (:31); a failed build is dropped (:258-260).
- 7 `reachedBy` → `sourcesOf` → `seriesSources`
:316, :302, :369-386
FREE reaches everyone; STANDARD needs `currentBranchId ∈ branchIds` and the stage's course in `enrolledCourses`; PROGRAM needs the code in `programs`; EVENT needs candidacy; a grant adds a source on top of any kind.
- 8 `toResolvedTest`
:473-488
`scopedDurationSec` / `scopedQuestionCount` / `scopedSections` applied; `opensAt` is the EARLIEST of this student's matching `programUnlocks`, else the test's own (`opensFor`, (:463-471); `attemptStatus` from the standing map.
- 9 `project`
:490-502
for `sequentialTests`, index of the first unsat test is the ceiling; everything past it is unreachable.
- 10 `projectTest`
:508-511
`canStart = reachable && testIsOpen(opensAt, now)`; `testIsOpen` allows a 5-minute grace (`contracts/src/access.ts:371-376`). `testBlocked` forces every `canStart` false (:499) but leaves the list intact.
- 11 `Response { testBlocked, series: [...] }`
`contracts/src/access.ts:450-454`
unpaginated; every reachable series with every ACTIVE test.
- 12 Bucketing is client-side
`contracts/src/access.ts:436-441`
`isSat` first (DONE), else `canStart` (OPEN) or LATER; `shutReason` (`app-kit/src/catalog.ts:140-143`) separates "Not open yet" from "Waiting its turn". Filtering and search are in-memory (`tests.tsx:52-59`, `app-kit/src/catalog.ts:82-88`).

The same resolution gates a start: `assertCanStart` (:157-168) projects ONE test and refuses with 'This test has not opened yet' vs 'This test is not open to you right now' (:514-517).

FLOW — PROFILE EDIT AND DOCUMENT UPLOAD

- 1 `PATCH /me`
`me.controller.ts:155-162` `update()`
`@Audit(AUDIT_FEATURE.STUDENT_PROFILE, AUDIT_ACTION.UPDATE)`, `@Patch()`, `@Body(new ZodBody(updateMeSchema))`.
- 2 `updateMeSchema` is `updateStudentSchema.pick({ fullName, profile })`
`contracts/src/me.ts:57-60`
an allowlist, so enrolment, branch, `studentType` and the verification booleans cannot arrive. Blank clears to null (`contracts/src/students.ts:272-278`).
- 3 `MeService.update` reads `before = students.detail()`
`apps/api/src/me/me.service.ts:53`
- 4 `students.update`
`students.service.ts:348-398`
`student.findFirst`, then `profile: { upsert: { create, update } }`; `dob` via `toDateColumn` (:560).
- 5 `$transaction(... , TX_LIMITS.SHORT)`
:377-392
the enrolment notification branch is dead on this route (`enrolledExams` cannot be patched here), so it is one UPDATE.
- 6 `auditContext.setChanged(fieldDiff(before, after, AUDITED_STUDENT_FIELDS))`
:395
- 7 `MeService` merges the profile diff over it
`me.service.ts:56-63`
`AUDITED_PROFILE_FIELDS` is `motherName, fatherName, dob, email, address, gender` (:19-26); the merge keeps a rename filed alongside.
- 8 `AuditInterceptor` writes the row after the handler returns
`audit/audit.interceptor.ts:48-69`
`actorType: STUDENT`, `entityId` from the store; an unchanged save records nothing (:50). Covered by `apps/api/test-db/audit-me.db.test.ts`.
- 9 `POST /me/documents/:kind`
`me.controller.ts:165-177` `@Audit(STUDENT_PROFILE, UPDATE)`, `@HttpCode(OK)`, `@UseInterceptors(FileInterceptor(DOCUMENT_FILE_FIELD, { limits: { fileSize: DOCUMENT_MAX_BYTES } })), @Param('kind', new ZodParam(documentKindSchema))`. The ceiling is also on the module (`me.module.ts:22`) so it bites while the body arrives
- 10 `checkDocument`
`me/documents.ts:54-91`
size 0, size > 5 MB, and type all refused as `VALIDATION_ERROR` with `fieldErrors.file`; the `bytes` decide via `sniffImage` + a `%PDF-` magic check (:40-51), never the browser's `mimetype`.
- 11 `students.assertExists` before the upload
`me.service.ts:78`, `students.service.ts:401-404`
so no orphan object lands in S3.
- 12 `documentKey`
`documents.ts:30-38`
`students/<studentId>/<kind>-<Date.now()>.<ext>`; a replacement is a NEW key, the old object is never deleted.

13	storage.upload	storage/storage.service.ts:50-65 one PutObjectCommand , the single upload path.
14	students.saveDocumentKey(id, columnFor(kind), key)	me.service.ts:84, students.service.ts:412-422 StudentProfile.photoUrl or .tenthMarksheetUrl upserted; isRecordNotFound → NOT_FOUND .
15	profile(studentId) re-read and returned	apps/api/src/me/me.service.ts:89 the client writes it straight into the cache (apps/exams/src/features/account/document-card.tsx:28).

What is NOT stored — Aadhaar and PAN images. DOCUMENT_KINDS has exactly two members, photo and tenth-marksheet (contracts/src/me.ts:12-15), and COLUMN_FOR (documents.ts:13-16) has no third column to write. StudentProfile carries aadhaarVerified / panVerified booleans only — the schema says so at prisma/schema.prisma:801 and 813-814 . Verified by grep: nothing anywhere in apps/ or prisma/ ever sets either to true ; the only write is anonymizedProfile() setting both false (students/anonymize.ts:41-42).

The data it touches

MODEL.COLUMN	TYPE	HOLDS	WRITTEN BY
Student.fullName	String?	the name	PATCH /me , admin, import
Student.pinHash , pinIsDefault	String? , Boolean	argon2 hash; still on an issued PIN	POST /me/pin (auth.service.ts:160-168)
Student.programs , enrolledExams , enrolledCourses , currentBranchId , isTestBlocked , isActive , deletedAt	arrays / uuid / bool	reach facts read by the catalog	admin + import only, never /me
StudentProfile.motherName , fatherName , email , address	String?	contact + parentage	PATCH /me
StudentProfile.dob	DateTime? @db.Date	civil date at UTC midnight	PATCH /me via toDateColumn
StudentProfile.gender	Gender?	enum	PATCH /me
StudentProfile.photoUrl , tenthMarksheetUrl	String?	S3 key, not a URL; presigned on read	POST /me/documents/:kind
StudentProfile.aadhaarVerified , panVerified	Boolean @default(false)	verification status; images never stored	nothing sets true
StudentProfile.educationDetails , pastExamHistory	Json?	arrays of educationEntrySchema (level , board? , year? , percentage? ...) / pastExamEntrySchema ; never queried	PATCH /me
SavedQuestion.id	uuid(7)	row id, the DELETE key	POST /me/saved/bookmarks
SavedQuestion.studentId	uuid	owner, onDelete: Cascade	same
SavedQuestion.questionId	uuid	the bank question, FK RESTRICT	same
SavedQuestion.attemptId	uuid?	provenance sitting, onDelete: SetNull	same
SavedQuestion.paperQuestionId	uuid?	the paper row it was served on, SetNull	same
SavedQuestion.createdAt	timestampz(3)	sort key	default

Redis: access:catalog:epoch (redis/redis.keys.ts:53) — a counter, no TTL, INCR 'd by any series/test write; session:<actor>:<subjectId>:<sessionId> and sessions:<actor>:<subjectId> (:38-43) — TTL = JWT_REFRESH_TTL , default 30d (config/env.schema.ts:107), read by GET /me/sessions . No /me route writes Redis directly.

What must already be true

A Student row with isActive: true and deletedAt: null ; a live Redis session (the JWT guard checks both). No feature key — those are admin-only. For the catalog to return anything, a TestSeries with isEnabled: true and Test.status = ACTIVE , plus a reach: a StudentGrant , a matching programs code, an EventCandidate , or currentBranchId ∈ series.branchIds with the stage's course in enrolledCourses . For a bookmark: an Attempt of their own at EVALUATED and a PaperQuestion for (testId, questionId) . For a document: S3_BUCKET / S3_ENDPOINT env and API_ROLE unset or core / all .

Why it is shaped this way

- Readiness (preTestReady , profileCompleted) is derived on every read rather than stored. The alternative — two columns on Student — existed and was dropped in e66ab91 : four writers had applied three different rules. Cost: every identity read now joins the profile for five columns.
- Document columns hold the S3 key, presigned for 300 s on read. The alternative, a stored public URL, would make the bucket public and give an identity document a permanent link. Cost: GET /me does two presigns, and a replaced document's old object is never deleted.
- SavedQuestion carries attemptId and paperQuestionId as nullable FKs with SetNull , not as plain strings. The alternative (plain columns) left dangling ids; RESTRICT would have blocked voiding a sitting. Cost: provenance silently becomes null , so a row's test and time-spent disappear.
- The catalog is one shared per-process snapshot plus one per-student read, epoch-busted. The alternative — a per-student cached catalog — needs per-student invalidation on every series write. Cost: a 15-minute worst-case staleness window (access-resolver.service.ts:31) for a field that changes without bumping the epoch.

Which screen calls it

(exact caller → SPA route)

ROUTE	CALLER	SCREEN
GET /me	apps/exams/src/features/account/profile.tsx:59 ; apps/exams/src/components/app-shell.tsx:30 ; apps/mobile/app/profile.tsx:54	/profile (exams) + the shell on every authed screen; mobile /profile
PATCH /me	profile.tsx:90 ; apps/mobile/app/profile.tsx:68	/profile
POST /me/documents/:kind	apps/exams/src/features/account/document-card.tsx:25 ; apps/mobile/src/components/account/document-card.tsx:36	/profile (both kinds, profile.tsx:244,249)
POST /me/pin	apps/exams/src/features/account/account.tsx:46 ; apps/mobile/app/change-pin.tsx:33	/account ; mobile /change-pin
GET /me/sessions	apps/exams/src/features/account/active-devices.tsx:22 ; apps/mobile/src/components/account/active-devices.tsx:35	/account ; mobile Account tab
DELETE /me/sessions/:id	active-devices.tsx:54 ; mobile active-devices.tsx:40	same
GET /me/catalog	packages/app-kit/src/student-queries.ts:87 → tests.tsx:41 , series.tsx:65 , test-about.tsx:90 , dashboard.tsx:66 ; mobile (tabs)/index.tsx:47 , (tabs)/tests.tsx:40 , series/[id].tsx:49 , test/[id]/index.tsx:52	/ , /tests , /series/:seriesId , /tests/:testId/about
GET /me/notifications	features/notifications/notifications.tsx:71 , app-shell.tsx:34 ; mobile notifications.tsx:35 , (tabs)/account.tsx:32	/notifications + the bell
PATCH /me/notifications/:id/read	notifications.tsx:209 ; mobile notifications.tsx:39	/notifications
GET /me/push-subscription	features/notifications/notification-settings.tsx:22	/notifications/settings (exams only)
POST /me/push-subscription	notification-settings.tsx:128 , lib/pwa.ts:122	/notifications/settings ; rebind on load
DELETE /me/push-subscription	lib/pwa.ts:137 ← app-shell.tsx:71	any authed screen
POST /me/push-device	apps/mobile/src/lib/push-device.ts:30 ← src/providers/auth.ts:19 , app/_layout.tsx:11	mobile only, on sign-in
DELETE /me/push-device	apps/mobile/src/lib/push-device.ts:53 ← (tabs)/account.tsx:85	mobile Account tab, on sign-out
GET /me/saved	features/review/saved.tsx:117 ; mobile src/components/saved/saved-list.tsx:47	/saved ; mobile Saved tab
GET /me/saved/facets	saved.tsx:58 ; mobile app/(tabs)/saved.tsx:21	same
POST /me/saved/bookmarks	packages/app-kit/src/review.ts:46	/attempts/:attemptId/report/solutions + questions tab
GET /me/saved/bookmarks/attempts/:id	packages/app-kit/src/review.ts:34	same
DELETE /me/saved/:id	saved.tsx:78 , review.ts:47 ; mobile saved-list.tsx:52	/saved and the report
GET /me/data-export	no UI	—

GET /me/data-export (me.controller.ts:78–82) has no client method in packages/contracts/src/client/me.ts and no caller anywhere: grep for data-export / dataExport across apps and packages returns only the controller itself. Its sibling POST /me/erasure was deleted in 62c882d for exactly that reason; the export was left behind.

Is it optimal

- GET /me — 5 Postgres round trips: student.findFirst , student.count (HOLDS_OWN_ACCESS), then Promise.all of program.findMany / exam.findMany / branch.findUnique , the last three skipped when the arrays are empty. Plus two local S3 presigns (HMAC, no network). Payload is one Me . No N+1. Read, not measured.
- PATCH /me — ~9 round trips, because detail() runs **twice**: once as before (me.service.ts:53) and once inside students.update 's tail (students.service.ts:397), so 4 queries and 4 presigns are spent on a diff of six scalar columns. The before read only needs AUDITED_PROFILE_FIELDS . Not optimal.
- GET /me/catalog — 1 Redis GET + 1 student.findUnique + (cold) 1 testSeries.findMany ; warm it is 2 calls regardless of catalog size. Index: the student read is by primary key. Payload is the whole reachable catalog with every section of every ACTIVE test — the one unbounded response in this module, and the reason bucketing, search and filtering are all client-side.
- GET /me/saved — 3 round trips (4 with a testId filter): the paged \$transaction , then attempt.findMany , then paperQuestion.findMany . Index SavedQuestion_studentId_createdAt_idx serves the equality + createdAt desc ; the id desc tie-break is an in-memory sort. The cost is satContext : it pulls every PaperQuestion row of every test the page's sittings belong to — 20 saved rows spanning twenty 100-question papers is 2 000 rows fetched to produce 20 timeSpentSec integers — and then rescans them per attempt (:113 , paperRows.filter inside attempts.map), which is O(tests × tests × questions). Correct, but the heaviest read in the module.
- GET /me/saved/facets — 2 round trips over the student's **whole** saved set, unpaged by design (:142). Bounded only by how much one student stars.
- POST /me/saved/bookmarks — 4 round trips (attempt, paperQuestion, create, re-read). The re-read exists so a duplicate star returns the row already held; optimal for the idempotency it buys.
- GET /me/sessions , DELETE /me/sessions/:id — Redis only, zero Postgres.

What it leans on

ResponseInterceptor + AllExceptionsFilter (app.module.ts:72–75) so no handler builds an envelope; JwtAuthGuard + ActorGuard for "the token is the subject"; ZodBody / ZodQuery / ZodParam (common/zod-validation.pipe) so fieldErrors feed react-hook-form directly (applyFieldErrors , profile.tsx:99); pageArgs / paged (common/pagination.ts) for the one paginated list; AuditContext + AuditInterceptor for the student's own edit; StorageService as the single S3 path; AccessResolverService so the catalog and the start guard cannot disagree; app-kit 's student-queries.ts and review.ts so web and mobile share every query key and the bookmark toggle; packages/ui ListView / TableFrame / MultiCombobox / TruncatedText on /saved , PageFrame / PageHeader / Alert elsewhere.

What changed recently

(`git log -15` over these paths)

- `7bee05e` — mobile document upload was failing before any request left the device: Expo SDK 57's WinterCG `fetch` refuses RN's `{ uri, name, type }` FormData part. Both pickers now hand over an `expo-file-system File`, and `UploadFile` narrowed to `Blob ( contracts/src/me.ts:99 )`.
- `62c882d` — `POST /me/erasure` removed; erasure is now `POST /admin/students/:id/erasure` behind `@RequiresSuperAdmin`. `AuthService.verifyStudentPin` went with it.
- `7730a31` — `saveDocument` stopped reading the student twice; `saveDocumentKey` now turns Prisma's record-not-found into `NOT_FOUND` itself.
- `e66ab91` — `Student.preTestReady / profileCompleted` columns dropped; both flags are now derived per read from `READINESS_FIELDS`. Identity reads gained a one-row profile lookup.
- `89b5c5f` — the predecessor: funnelled four writers through one `readinessOf` after three of them had applied different rules.

What is wrong with it

1. `apps/api/src/me/me.controller.ts:78-82` — `GET /me/data-export` is reachable and unused: no client method, no caller, nothing in either SPA or mobile. Any student token hits it today and gets their whole record including every sitting; it is the one `/me` route with no `@Audit` and the only one carrying `Cache-Control: no-store`, so it is also unlogged. Input that bites: a bare `curl -H "Authorization: Bearer <student token>" /me/data-export`.
2. `packages/contracts/src/students.ts:178-179` + `prisma/schema.prisma:813-814` — `aadhaarVerified / panVerified` are shown on `/profile` as a "Verified" badge (`apps/exams/src/features/account/profile.tsx:306-308`) but **nothing in the codebase ever sets either true**; the only write is `anonymize.ts:41-42` setting them false. Every student sees "Not verified" permanently, with no route to change it.
3. `apps/api/src/students/student-privacy.service.ts:114-118` — erasure nulls `photoUrl` and `tenthMarksheetUrl` but never calls `storage.remove`; `StorageService` is not even injected. The photo and the marksheet survive the erasure in S3 under `students/<studentId>/...`, and the id is still in the key. Input: erase a student who had uploaded a photo, then list the bucket prefix.
4. `packages/contracts/src/client/me.ts:87` — `/** Admin-only. A student token gets 403 from every one of these. */` sits directly on `meClient`, which is the opposite of the truth. Beside it, `:98` `/** Returns a FRESH session – the caller must store these tokens. */` is stranded above `uploadDocument` (it describes `changePin` at `:106`), and `:99` says "A photo or an identity document" when an identity document is precisely what this route will not store. Same class of defect at `me.controller.ts:58` (the class comment is captured by `interface UploadedFileLike`) and `:164` (`/** The student's photo. */` over a route that also takes the marksheet).
5. `apps/api/src/me/me.service.ts:75` — `if (!file) throw new AppException(VALIDATION_ERROR, ...)` is unreachable: `checkDocument` on the line above already throws on `!file ( documents.ts:58-62 )`. It is there only to narrow the type, since `checkDocument` returns `string` rather than asserting. Harmless, but the duplicate copy means a future message change has two places to edit.
6. `apps/api/src/students/students.service.ts:377-392` — `MeService.update` routes through `StudentsService.update`, which opens a transaction solely so the enrolment notification commits with the row. `updateMeSchema` cannot carry `enrolledExams`, so `added` is always empty on this path: every student profile save pays for a transaction that can only ever contain one UPDATE.

Part VI — The data model

`prisma/schema.prisma` is the target of record, and the hand-written SQL under `prisma/migrations` is the record for everything Prisma cannot say. Both are read here directly.

The part worth your attention is the last one: the triggers, functions, views, partial uniques and CHECKs. They are where this system keeps its promises. A rule that lives in a service method holds only for the paths that call it; a rule that lives in a trigger holds for every path, including the one somebody writes next year and the one somebody types into psql at midnight.

Every table, and what the database refuses

Where it lives

`prisma/schema.prisma` (1275 lines; 41 models, 36 enums) and

`prisma/migrations/**` (105 applied revisions, baseline `20260820000000_schema_realignment`, head `20260930170000_the_outbox_and_its_relay_are_gone`).

- `prisma/schema.prisma` — ten section banners are the ownership map (`:290` taxonomy, `:332`
- `20260820000000_schema_realignment/migration.sql` — every table, and the first locked-config
- `20260917100000_a_sitting_keeps_its_answers_on_one_sheet/migration.sql` — the positional answer
- `20260918120000_every_id_is_a_uuid/migration.sql` — cuid → native `uuid`, 105 columns over 40
- `20260918110000__section_scores_are_stored_by_position/migration.sql` — objects → packed arrays.
- `2026091190000_rank_is_counted_in_postgres/migration.sql:20` — `sitting_percentile()`, made
- `20260918140000_an_edited_version_resyncs_every_paper_that_pins_it/migration.sql` — the

Facts marked **(catalog)** were read from the running dev Postgres (`iace-postgres`, `_prisma_migrations` at head) with read-only `pg_catalog` queries — the final state of all 105 migrations; the migration file is cited for wording.

What it is for

It holds an institute's exam catalog, its question bank with authoring

workflow, the papers built from both, and the sittings taken against them. Two guarantees are pushed into Postgres rather than a service: a blueprint stops moving once locked, and a paper stops moving once anybody has sat it — because an answer sheet stores *positions* into the paper and into each row's option list, so a shifted paper would silently re-interpret stored answers. Everything on the live-test path is shaped for page count: one row per sitting instead of one per answer, packed arrays instead of keyed objects, 16-byte uuids instead of 26-byte cuids.

Flow trace: an admin rewrites a question version that unopened papers pin (the only flow that lives entirely in SQL)

- `UPDATE "QuestionVersion" SET options = ...` — `question_version_sat_guard` fires `BEFORE UPDATE ... WHEN (options/answerKey/content distinct)` **(catalog)** — nothing written yet.
- It locks every pinning test `ORDER BY "id" FOR UPDATE`, then refuses if any has an `Attempt` — `20260918160000...:20`, `20260917100000...:146` — a sat paper's content, key and options are frozen; otherwise the `UPDATE` proceeds.
- `question_version_option_ids_fanout` fires `AFTER UPDATE OF options` — `20260918140000...:33` — `UPDATE "PaperQuestion" SET "optionIds" = "optionIds" WHERE "questionVersionId" = NEW.id`.
- Naming the column fires `paper_question_option_ids BEFORE UPDATE OF ..., "optionIds"` — `20260917100000...:58` — rebuilds `optionIds` from the version JSON in stored order.
- `paper_question_sat_guard` sees a changed row (alphabetical `BEFORE` order puts `o` before `s`) and takes its full `FOR UPDATE` path — `20260917100000...:123` — no `Attempt`, so it passes. Durable result: every pinning `PaperQuestion.optionIds` matches the new option order, so no sheet decodes to the wrong option.

1. Model inventory, by owning module

Row labels are the cardinality the keys imply, not measured counts. **PK strategy:** 30 of 41 models are `String @id @default(uuid(7)) @db.Uuid`; 11 borrow a parent's key or are composite (*borrowed*). `uuid v7` over `v4` because its leading timestamp keeps inserts at the index's right edge — measured 90% leaf density and 5.11 MB WAL per 20,000 inserts against 68% and 18.93 MB (`20260918120000...:14-17`).

MODEL	PURPOSE	ROWS	PK
<code>Exam</code>	An exam under an <code>ExamCourse</code> ; <code>code</code> is what <code>Student.enrolledExams</code> holds	tiny	<code>uuid7</code>
<code>ExamStage</code>	The stage/paper level carrying configs, series and tests	tiny	<code>uuid7</code>
<code>BaseConfig</code>	A stage's reusable blueprint; locks at first finalize	tiny	<code>uuid7</code>
<code>BaseConfigModule</code>	Session layer above sections (runtime deferred)	tiny	<code>uuid7</code>
<code>BaseConfigSection</code>	A section's counts, marks, timing, merit/qualifying	tiny	<code>uuid7</code>
<code>Subject / Topic</code>	Bank taxonomy; a topic is a pool, not a scope	tiny	<code>uuid7</code>
<code>Question</code>	Stable identity + tags, <code>stemHash</code> , <code>status</code>	per - bank - item	<code>uuid7</code>
<code>QuestionVersion</code>	Immutable content/options/answerKey per edit	~1-3/question	<code>uuid7</code>
<code>Test</code>	A paper's identity: scope, source, version, revision, <code>opensAt</code>	per - test	<code>uuid7</code>
<code>PaperQuestion</code>	One question on one paper, with marks and <code>optionIds</code>	per - paper - row	<code>uuid7</code>

MODEL	PURPOSE	ROWS	PK
QuestionAssignment	Who types / proof-reads one section of one test	2/section/test	uuid7
QuestionReview	A proof-reader's per-question verdict	per-paper-row	uuid7
SectionComment	The thread on one section of one test, with revisions	small	uuid7
Attempt	Live state and scored result in one row	per-attempt	uuid7
AttemptSheet	One sitting's whole answer set + verdicts	per-attempt	borrowed
Student	Identity, enrolment arrays, PIN hash, soft delete	per-student	uuid7
StudentProfile	PII + verification booleans (no Aadhaar/PAN images)	per-student	uuid7
Admin	Admin identity + AdminRole	tiny	uuid7
AdminFeaturePermission	One feature key an admin holds, at one level	tiny	composite
Branch / Program / Event	Access vocabularies	tiny	uuid7
TestSeries	The unit of offering; kind decides reach	tiny	uuid7
StudentGrant	Per-student override of series access	per-student	composite
EventCandidate	Who an EVENT series reaches	per-student	composite
TestProgramUnlock	Per-program stagger on one test's opening	tiny	composite
ImportLog	One import run, its counts and target	per-import	uuid7
ImportRowEdit	A previewed row corrected or left out pre-commit	per-import-row	composite
RowActionLog	Central CRUD audit; actor ids without FKs	per-write	uuid7
Notification	One in-app message to exactly one recipient	per-attempt+	uuid7
Announcement	One send, with audience filter and priced cost	tiny	uuid7
NotificationDelivery	One row per channel that can fail	per-notif×channel	uuid7
StudentStat	Per-student rollup sums	per-student	borrowed
StudentSubjectStat	Per-student per-subject per-scope bucket	per-student×subject	composite
TestStat	Ranked cohort count + time sum	per-test	borrowed
TestSectionStat	Per-test per-section rollup	per-test×section	composite
TestQuestionStat	Item analysis and distractor counts	per-paper-row×test	composite
PushSubscription	A browser push endpoint + its keys	per-student×device	uuid7
PushDevice	One phone's FCM token (DevicePlatform)	per-student×device	uuid7
SavedQuestion	Bookmarks/mistakes keyed by bank questionId	per-student×question	uuid7

2. Enums (36)

Gender MALE/FEMALE/OTHER · DifficultyLevel LOW/MEDIUM/HIGH · SupportedLanguage EN/HI/TE (lowercase inside content JSON) · QuestionType SINGLE_MCQ, TEXT_FIELD (typed, no options) · AnswerMode EXACT (case/space-folded) / NUMERIC (parsed, tolerance either way) — lives inside answerKey JSON, never a column (:45) · QuestionStatus ACTIVE/ARCHIVED · TestUi CBT, OMR, GENERIC, TYPING (both of the last two render the CBT screen; TYPING reserved, :58-65) · NavigationPolicy FREE/FORWARD_ONLY · LanguageMode SINGLE/DUAL · ExamTemplate DEFAULT / SSC.RAILWAYS (skin only) · TimerTemplate COMPOSITE_FREE (may return), SECTIONAL_LOCKED and SESSION_MODULE_LOCKED (may not), PER_ITEM_TIMED (forward-only) · ExamCourse SSC/RRB/BANKING/ AP_TS_POLICE · ExamMode CBT/OMR/PSYCHOMETRIC/SKILL/PHYSICAL/INTERVIEW/DESCRIPTIVE · MeritType MERIT/QUALIFYING · StageDisposition CONDUCTED/PARTIAL/CATALOG_ONLY · TestStatus DRAFT/ACTIVE/ INACTIVE · TestScope FULL/MODULE/SECTIONAL · PaperSource FRAMED (typed) / PICKED (drawn) · PaperQuestionStatus ACTIVE/DROPPED/BONUS · AttemptStatus IN_PROGRESS, SUBMITTED, EVALUATED, EXPIRED (**reserved, never written** — a stranded sitting ends SUBMITTED so it is still scored, :149), VOIDED (admin-archived, out of every rollup) · AnswerState NOT_VISITED/NOT_ANSWERED/ ANSWERED/MARKED_REVIEW/ANSWERED_MARKED · NotificationType TEST_ASSIGNED, RESULT_READY, ENROLLMENT_ADDED, GRANT_ADDED, GENERIC, RESULT_UPDATED, PIN_CHANGED, WELCOME — all eight written · DeliveryChannel IN_APP/WEB_PUSH/MOBILE_PUSH (FCM)/EMAIL/SMS/WHATSAPP · DeliveryStatus PENDING/ SENT/DELIVERED/FAILED/SKIPPED (a decision not to spend) · StudentType ONLINE/OFFLINE/NON_IACE · BranchType PHYSICAL/VIRTUAL · ImportSource INDIVIDUAL/SHEET/SCRIPT/SELF_SIGNUP · TestSeriesKind STANDARD/FREE/PROGRAM/EVENT (reach rule, :215) · PermissionLevel READ/WRITE (WRITE implies READ) · AuditFeature 17 values naming a changeable record kind · AuditAction CREATE/UPDATE/DELETE/ACTIVATE/DEACTIVATE/BLOCK/UNBLOCK/IMPORT/EXPORT · ActorType ADMIN/STUDENT/ SCRIPT/SYSTEM · AssignmentRole TYPIST/PROOFREADER · SendBackReason SPELLING/DATA_CORRECTION/ ANSWER_OPTION · AdminRole TYPIST/PROOFREADER/ADMIN/SUPER_ADMIN (seeds grants, decides isSuperAdmin) · DevicePlatform ANDROID/IOS.

3. Every jsonb column, field by field

AttemptSheet.answers (NOT NULL , CHECK jsonb_typeof = 'array') — slot *N* is paper row *N* in PaperQuestion.order , whatever order the screen showed. null for a question never touched, else a 5- or 6- element array (apps/api/src/attempts/answer-sheet.ts:24-41): [0] state index into NOT_VISITED, NOT_ANSWERED, ANSWERED, MARKED_REVIEW, ANSWERED_MARKED ; [1] option — an **integer position** into PaperQuestion.optionIds , or the raw id string when the row does not hold it (which then scores wrong), or null ; [2] timeSpentSec ; [3] firstActionSec after Attempt.startedAt , nullable; [4] answeredSec , nullable; [5] typedAnswer , **present only when given**. Written whole, only by AttemptSheetService.write (apps/api/src/attempts/attempt-sheet.service.ts:33); seeded blankSheet(size) in the start txn.

AttemptSheet.verdicts — parallel array, `[isCorrect: boolean|null, marksAwarded: number]` per slot; scorer only (`answer-sheet.ts:44` , `scoring.processor.ts:158`).

Attempt.sectionScores — array of `[baseConfigSectionId, score, correctCount, wrongCount, unattemptedCount, timeSpentSec]` (`score-paper.ts:43`). `packedSections` writes, `sectionScoresIn` is the only reader; a row still in the old object shape reads as *no* sections, never half-decoded.

Attempt.sectionState — **dead**: declared at `schema.prisma:703` , written and read by nothing.

QuestionVersion.content — `{ <lang>: { stem: RichContent, solution?: RichContent } }`, lang lowercase (`en / hi / te`); RichContent is `[{ type: 'TEXT', text: string }]` where text is editor HTML (`packages/contracts/src/questions.ts:103–114, 187–194` ; confirmed against a stored row). **options** — `[{ id: uuid, position: int, isCorrect: bool, text: { <lang>: RichContent } }]` (`questions.ts:435–441`). **answerKey** — `{ mode: EXACT|NUMERIC, answers: { <lang>: string }, tolerance?: number|null }` (`:442–448`), TEXT_FIELD only.

Test.scopeRef — `{ moduleId?, sectionId? }` (`contracts/src/tests.ts:75–78`), read only via `scopeRefOf` (`apps/api/src/common/prisma-json.ts:21`).

Test.questionPoolFilter — `{ sections: { [sectionId]: { topicIds?: string[], tags?: string[], mix?: { LOW, MEDIUM, HIGH } } } }` (`tests.ts:108–120`).

BaseConfig.featureFlags — **dead**: no reader or writer outside the schema, the baseline SQL and the locked-config trigger's column list.

SectionComment.revisions (`Json[]`) — `{ body, at }` per earlier wording, oldest first, current never included (`contracts/src/assignments.ts:201–205`).

StudentProfile.educationDetails — `[{ level, board?, institution?, year?, percentage? }]`; **pastExamHistory** — `[{ exam, year?, result? }]` (`contracts/src/students.ts:137–159` , `≤12` rows). **RowActionLog.changed** — `Record<field, { from, to }>` (`contracts/src/audit.ts:63`). **ImportLog.errors** — `{ fileErrors?: string[], message?: string }` (`apps/api/src/imports/imports.service.ts:465–470`). **ImportRowEdit.draft** — a whole `QuestionDraft` (`contracts/src/questions.ts:363–377`); null when the row was only left out. **Notification.data** — template variables, `Record<string, string|number>` (`notification-delivery.processor.ts:226`); the one writer puts `{ message }` (`announcements.service.ts:129`). **Announcement.audience** — the student list filter minus `page` , `pageSize` , `sort` (`contracts/src/announcements.ts:21–25`). **TestQuestionStat.optionCounts** — `Record<optionId, count>` (`attempts/test-analytics.ts:49`).

4. Everything Prisma cannot express

Triggers (12, catalog). Five hold the locked blueprint. `base_config_guard` (`BEFORE DELETE OR UPDATE ON "BaseConfig"`) refuses deleting a locked config and any change to 18 named shape columns — *"base config % is locked; clone it to change its shape"*. `base_config_section_guard` / `base_config_module_guard` (`BEFORE INSERT OR DELETE OR UPDATE`) run `base_config_child_guard`, which calls `base_config_assert_unlocked` on both configs **lowest id first**, so two transactions re-parenting in opposite directions queue instead of deadlocking. `base_config_shape_guard` (deferred constraint trigger, `AFTER UPDATE OF "timerTemplate"`) and `base_config_section_shape_guard` (deferred, `AFTER INSERT OR UPDATE`) both call `base_config_assert_section_shape`: a `moduleId` is forbidden unless the template is `SESSION_MODULE_LOCKED` and required when it is; `durationSec` is required for `SECTIONAL_LOCKED`, `perQuestionSec` for `PER_ITEM_TIMED`. Three `*_truncate_guard` statement triggers raise unconditionally, because `TRUNCATE` fires no row trigger. `apps/api/src/configs/base-configs.service.ts` is the code path; without these, raw SQL, a worker or a psql session could rewrite a locked paper shape.

Three guards make positional sheets safe. `paper_question_sat_guard` has a fast path for a status-only update (`to_jsonb(NEW) - 'status' IS NOT DISTINCT FROM ...` , so a drop or bonus never blocks a start), then takes `FOR UPDATE` on the `Test` row — deliberately, because an `Attempt` insert's FK check takes `KEY SHARE` on that row, so a paper edit and the first sitting serialize instead of racing — and raises *"A paper somebody has sat cannot change: only a question's status may move"*. `question_version_sat_guard` does the same for a pinned version's options, key or content. `paper_question_option_ids` (`BEFORE INSERT OR UPDATE OF "questionVersionId", "optionIds"`) copies the version's option ids in stored order onto the paper row; `question_version_option_ids_fanout` (`AFTER UPDATE OF options`) re-assigns `optionIds` to itself on every pinning row to re-fire it. The fan-out leans on alphabetical `BEFORE` order (`o` before `s`), and `20260918140000...:16–23` warns that renaming either reverses it and drops `sat_guard`'s own check silently. Without the fan-out, an in-place version edit leaves stale `optionIds` on unopened papers and every stored position decodes to the wrong option.

SQL functions. `sitting_percentile(outscored, tied, cohort)` RETURNS numeric IMMUTABLE PARALLEL SAFE — `100` for a cohort `≤1`, else `ROUND(LEAST((outscored + tied)/cohort, 1) * 100, 2)` with every input clamped; read at `apps/api/src/attempts/ranking-sql.ts:63, 108, 160`. `PARALLEL SAFE` came from `20260912010000`, so a parallel plan can use it. `"tagsText"(text[])` — `array_to_string($1, ',')`, IMMUTABLE PARALLEL SAFE STRICT, exists only to back `Question_tags_trgm_idx` (`20260925120000...:33`). The three outbox helpers (`notification_request_written`, `notification_from_request`, `uuid_or_null`) and their `OutboxEvent_notification_request` trigger were **dropped** with the table (`20260930170000...:19–23`).

View. `AttemptSheetAnswer` decodes one sheet slot per row: state name, `selectedOptionId` (`position` → `paper.optionIds[pos+1]`, or the raw string), `typedAnswer`, `timeSpentSec`, `firstActionAt` / `answeredAt` as `startedAt + make_interval`, and `isCorrect` / `marksAwarded` from `verdicts`. Explicitly *"for support and ad-hoc SQL"* (`20260917100000...:170`) — **no application code reads it**; the only repo references are `docs/02-domain-rules.md:269` and the doc-check test. Recreated by `20260918120000...:418` after the uuid conversion.

Partial uniques. `Attempt_graded_per_test_key` (`testId, studentId`) WHERE `isGraded` — one ranked sitting per student per test, which every rollup and board counts over. `BaseConfig_default_per_stage_key` (`examStageId`) WHERE `isDefault`. `BaseConfigSection_flat_order_key` (`baseConfigId, order`) WHERE `moduleId IS NULL` — the `@@unique([baseConfigId, moduleId, order])` cannot order module-less sections, because NULLs are distinct. `Student_mobile_live_key` (`mobile`) WHERE `deletedAt IS NULL` and `Student_externalRef_live_key` — a soft-deleted row releases its number. `QuestionAssignment_active_role_key` (`testId, baseConfigSectionId, role`) WHERE `replacedAt IS NULL` — one active typist and one proof-reader per section. `Notification_student_dedupe_key` / `_admin_dedupe_key` on (`recipient, dedupeKey`) — what makes a replayed `tell()` land once. Two expression uniques: `TestSeries_name_key` (`lower(name)`) and `Test_testSeriesId_title_key` (`testSeriesId, lower(title)`) — case-insensitive, untitled drafts unconstrained because NULLs are distinct (`20260912170000...:12–14`).

CHECKS. `AttemptSheet_answers_is_array` — a non-array `answers` would make every positional read silently empty. `Notification_one_recipient_check` — (`studentId IS NOT NULL`)::int + (`adminId IS NOT NULL`)::int = `1`. Four on `TestSeries`: `branches_are_standard_only` (`kind = 'STANDARD' OR cardinality(branchIds) = 0`), `event_kind_names_its_event` and `program_kind_names_its_program` (biconditional, (`kind = X`) = (`col IS NOT NULL`)), `only_free_spans_no_stage` (`kind = 'FREE' OR examStageId IS NOT NULL`). These are what `access-resolver.service.ts` can assume instead of re-checking.

GIN (11). Array containment: `Question_tags_idx`, `Student_enrolledCourses_idx`, `Student_enrolledExams_idx`, `Student_programs_idx`, `TestSeries_branchIds_idx`. Trigram, for leading-wildcard `ILIKE`: `Question_questionCode_trgm_idx`, `Question_tags_trgm_idx` on `"tagsText"(tags)`, `QuestionVersion_content_trgm_idx` on `content::text`, `Student_fullName_trgm_idx`, `Student_mobile_trgm_idx`, `Test_title_trgm_idx`.

Composite FKs (eight, `ON UPDATE NoAction`): `Test` → `BaseConfig(id, examStageId)`; `PaperQuestion` → `Test(id, baseConfigId)`, → `BaseConfigSection(baseConfigId, id)`, → `QuestionVersion(questionId, id)`; `Question` → `QuestionVersion(questionId, id)`; `BaseConfigSection` → `BaseConfigModule(baseConfigId, id)`; `QuestionAssignment` → `Test(id, baseConfigId)` and → `BaseConfigSection(baseConfigId, id)`. Each stops a child pointing at a parent belonging to a different config, stage or question — a single-column FK cannot.

Storage parameters (catalog). `fillfactor` `60` on `TestStat`, `TestSectionStat`, `TestQuestionStat` with `autovacuum_vacuum_threshold = 50`, `scale_factor = 0`, `cost_delay = 0` (`20260908090000`, for HOT updates on unindexed counters). `fillfactor = 70` on `AttemptSheet` — `50` at creation, raised by `20260918100000...:54`: a sheet is rewritten ~25 times then never again, so reserving half a page is waste (`14.85` → `13.02` MB per 5,000 sittings). `Attempt` gets fixed vacuum thresholds of 5,000 rows rather than 20% of the table, because scoring a 5K exam leaves ~15,000 dead rows and until vacuum runs a standing read fetches 5,044 pages instead of 80.

5. Index inventory

Confirmed: `Attempt_ranking_idx` exists — (testId, score DESC, timeTakenSec, id) INCLUDE (studentId, submittedAt) WHERE (isGraded AND status = 'EVALUATED' AND score IS NOT NULL) . Created 20260911190000...:31 ; dropped and recreated with the INCLUDE by 20260929090000...:31-32 because every board read those two columns and went back to the heap per ranked row. It serves rank, percentile, standings and the topper (ranking-sql.ts ; 20260930120000...:6 calls the topper one Index Only Scan probe). **Confirmed: `Attempt_testId_score_idx` was dropped** — created 20260820000000...:793 , dropped 20260918100000...:52 as redundant beside the ranking index (6.5 MB per 105,000 sittings; one exam's WAL fell 26.45 → 24.51 MB).

Other `Attempt` indexes → query: `Attempt_live_by_test_idx` (testId, endsAt) WHERE IN_PROGRESS → live-ops running/overdue counts (129 pages → 3-7); `Attempt_expiring_idx` (endsAt) WHERE IN_PROGRESS → the two-minute expiry sweeper, previously a 7,815-page sequential scan; `Attempt_unscored_idx` (submittedAt) WHERE SUBMITTED AND score IS NULL → the unscored backlog; `Attempt_rescore_idx` (testId, scoredRevision, id) WHERE EVALUATED → the re-score sweep, paged along its own index (20260930160000); `Attempt_student_drift_idx` (studentId, updatedAt) WHERE status IN (EVALUATED, VOIDED) AND updatedAt > evaluatedAt → the per-student drift sweep (rollup.service.ts:186-187), narrowed to moved sittings by 20260928203000 ; `Attempt_testId_status_idx` → cohort reads, kept deliberately because the ranking index covers only graded scored rows; `Attempt_testId_updatedAt_idx` → the cohort sweep's watermark (rollup.service.ts:155); `Attempt_studentId_createdAt_idx` → a student's sitting list.

Added for a named query: `Question_status_idx` (the dashboard's whole-bank status count, 468 buffers → 45), `Question_createdAt_id_idx` , `Student_createdAt_id_idx` , `Student_fullName_id_idx` (list sorts that previously sorted the whole table), `SavedQuestion_questionId_idx` (the RESTRICT FK to Question), `EventCandidate_eventId_createdAt_idx` , `NotificationDelivery_pending_idx` and `_settled_idx` (the sender and the nightly prune), `Notification_unpushed_idx` (createdAt) WHERE pushedAt IS NULL (the push sweep), `Notification_studentId_createdAt_isRead_idx` (the bell, and an index-only unread count).

No reader found (five, all small): `Student_anonymizedAt_idx` — anonymizedAt is written (students/anonymize.ts:23) and read per row (student-privacy.service.ts:109), never filtered on; `AdminFeaturePermission_featureKey_idx` — every query leads with adminId or selects the whole table (admins.service.ts:240,261,300); `BaseConfig_isDefault_idx` — the only isDefault predicate is { examStageId, isDefault: true } (base-configs.service.ts:394), already served by `BaseConfig_default_per_stage_key` ; `Student_mobile_idx` and `Student_externalRef_idx` duplicate the leading column of their `_live_key` partial uniques, adding only soft-deleted rows. 20260926150000...:18-21 says the rest of that question needs `pg_stat_user_indexes.idx_scan` from real traffic (scripts/index-usage.mjs) — the dev database's counters are not evidence. **No hot query without an index was found**: every sweep, board and list read I traced names an index above.

Four were dropped as readerless by 20260922100000 (`Notification_actBy_idx` , `NotificationDelivery_providerMessageId_idx` , `NotificationDelivery_status_queuedAt_idx` , `StudentSubjectStat_subjectId_idx`) — and 20260929090000...:12-15 admits the third *did* have a reader, the nightly prune, which came back as the partial `_settled_idx` . Three prefix-redundant indexes went in 20260926150000 : `ExamStage_examId_idx` , `QuestionAssignment_testId_idx` , `Test_testSeriesId_idx` .

6. Notable data moves

- 20260917100000 — `AttemptQuestion` (a row per answer) → `AttemptSheet` (a row per sitting).
- 20260918110000 — `sectionScores` objects → packed arrays. Same numbers, same order; what
- 20260918120000 — every id to native `uuid` . A `DO` block scans every table and **refuses to
- 20260912170000 — series and test names made unique case-insensitively, **renaming**
- 20260903100000 then 20260904130000 — every test moved onto the series that holds it,
- 20260930140000 — re-scores moved from `OutboxEvent` rows to `Attempt.scoredRevision` . The
- 20260930170000 — DROP TABLE "OutboxEvent" plus its three functions; the file warns it must
- Smaller: 20260829100000 (exam codes canonicalised and `Student.enrolledExams` rewritten),

7. Relationship map (every FK, catalog)

ON UPDATE is CASCADE except on the eight composite FKs, which are NoAction .

→ **Admin** · AdminFeaturePermission.adminId (Cascade) · Announcement.createdById (Restrict) · Notification.adminId (Cascade) · Question.createdById (Restrict) · QuestionAssignment.assignedId (Restrict) · SectionComment.authorId (Restrict)

→ **Student** · Attempt.studentId (Restrict) · Notification.studentId , EventCandidate.studentId , PushDevice.studentId , PushSubscription.studentId , SavedQuestion.studentId , StudentGrant.studentId , StudentProfile.studentId , StudentStat.studentId , StudentSubjectStat.studentId (all Cascade) · Student.currentBranchId → Branch.id (Restrict)

→ **Test** · Attempt.testId (Restrict) · Notification.testId (**SetNull**) · QuestionReview.testId , SectionComment.testId , TestProgramUnlock.testId , TestStat.testId , TestSectionStat.testId , TestQuestionStat.testId (Cascade) · PaperQuestion.(testId, baseConfigId) and QuestionAssignment.(testId, baseConfigId) → Test(id, baseConfigId) (Cascade) · Test.(baseConfigId, examStageId) → BaseConfig(id, examStageId) (Restrict) · Test.examStageId → ExamStage.id (Restrict) · Test.testSeriesId → TestSeries.id (Restrict)

→ **Attempt** · AttemptSheet.attemptId (Cascade) · SavedQuestion.attemptId (**SetNull**)

→ **Question / QuestionVersion** · QuestionVersion.questionId (Restrict) · Question.(id, currentVersionId) → QuestionVersion(questionId, id) (Restrict) · PaperQuestion.questionId (Restrict) and .(questionId, questionVersionId) → QuestionVersion(questionId, id) (Restrict) · QuestionReview.questionId (Cascade) · SavedQuestion.questionId (Restrict) · TestQuestionStat.questionId (Restrict) · Question.subjectId → Subject.id (Restrict) · Question.topicId → Topic.id (Restrict) · Question.assignmentId → QuestionAssignment.id (**SetNull**)

→ **BaseConfig / sections** · BaseConfigModule.baseConfigId , BaseConfigSection.baseConfigId (Cascade) · BaseConfig.examStageId → ExamStage.id (Restrict) · BaseConfig.clonedFromId → BaseConfig.id (Restrict, self) · BaseConfigSection.(baseConfigId, moduleId) → BaseConfigModule(baseConfigId, id) (Restrict) · BaseConfigSection.subjectId → Subject.id (Restrict) · PaperQuestion.(baseConfigId, baseConfigSectionId) and QuestionAssignment.(baseConfigId, baseConfigSectionId) → BaseConfigSection(baseConfigId, id) (Restrict) · TestSectionStat.baseConfigSectionId → BaseConfigSection.id (Restrict)

Rest · ExamStage.examId → Exam.id (Restrict) · Topic.subjectId → Subject.id (Restrict) · StudentSubjectStat.subjectId → Subject.id (Restrict) · TestSeries.examStageId → ExamStage.id (Restrict) , .programCode → Program.code (Restrict) , .eventId → Event.id (Restrict) · StudentGrant.testSeriesId → TestSeries.id (Cascade) · Notification.testSeriesId → TestSeries.id (**SetNull**) · Notification.announcementId → Announcement.id (Cascade) · NotificationDelivery.notificationId → Notification.id (Cascade) · EventCandidate.eventId → Event.id (Cascade) · TestProgramUnlock.programCode → Program.code (Restrict) · TestQuestionStat.paperQuestionId → PaperQuestion.id (Cascade) · SavedQuestion.paperQuestionId → PaperQuestion.id (**SetNull**) · ImportRowEdit.importLogId → ImportLog.id (Cascade) · RowActionLog.importLogId → ImportLog.id (Restrict)

What must already be true

the only extension is `pg_trgm`, created by the migration itself

(`20260925120000...:31` , `CREATE EXTENSION IF NOT EXISTS`) and not declared in `schema.prisma` , so the migrating role must be able to create one; the server holds exactly `plpgsql` and `pg_trgm` . A fresh clone needs `pnpm db:migrate:deploy` from scratch, and `20260918120000` will **abort** against any database holding rows. The first super admin is inserted by pure SQL. The composite uniques (`Test_id_baseConfigId_key` , `BaseConfig_id_examStageId_key` , `BaseConfigSection_baseConfigId_id_key` , `BaseConfigModule_baseConfigId_id_key` , `QuestionVersion_questionId_id_key`) exist only to back the composite FKs and must not be pruned as "redundant".

Why it is shaped this way

- **One positional sheet row per sitting.** The alternative, `AttemptQuestion` , was 500,000 rows at
- **Packed `sectionScores` , keeping the section id.** Dropping the id too saves ~60 bytes a
- **Rank and percentile counted live off `Attempt_ranking_idx` .** A stored `rank` column existed
- **`uuid(7)` , not cuid or `uuid(4)` .** cuid cost 10 bytes a row and sorted under `en_US.utf8`

What changed recently

(`git log -15 -- prisma/schema.prisma prisma/migrations`) — `a5112aa` dropped

`OutboxEvent` and its three trigger functions; `cdfab62` replaced outbox-driven re-scoring with `Attempt.scoredRevision` plus `Attempt_rescore_idx` ; `7c9edee` , `ad4139e` , `b96a89d` removed stored rollup columns (per-column sums, averages, the stored topper, a test's mean/highest/lowest) for live counts; `e66ab91` dropped two stored student-readiness flags, now derived from profile columns; `63f3f72` moved notification writes into the producer's own transaction (its trigger and helpers are what `a5112aa` then removed).

What is wrong with it

- `prisma/schema.prisma:703` — `Attempt.sectionState Json?` is written and read by nothing
- `prisma/schema.prisma:362` — `BaseConfig.featureFlags Json?` likewise has no reader or writer,
- `prisma/schema.prisma:3-6` — the header says the hand-written parts are "partial-unique indexes,
- `AttemptSheetAnswer` is maintained (recreated through the uuid migration, its name pinned by
- `prisma/schema.prisma:149` — `AttemptStatus.EXPIRED` is declared and never written; the comment
- `prisma/schema.prisma:45-51` — `enum AnswerMode` exists in the database but no column uses it; the

Part VII — What it runs on

This part has two halves, and they disagree.

The first is what `docs/04-infrastructure.md` currently plans: EC2 with Docker Compose behind Caddy, one shared Valkey box, one shared RDS Postgres, and a monthly bill small enough to be unremarkable. That plan was re-costed and re-argued repeatedly over the last fortnight and it is current.

The second is what the repository actually contains — and the deployment story does not survive contact with it. The findings are listed plainly, with the worst one first: **the API image does not build**.

The infrastructure, the cost, and where the plan has drifted

Subject: `docs/04-infrastructure.md`, 760 lines, 16 sections. Last touched by `f1d5ef4`.

Half 1 — what the plan now says

Preamble (L1–47). Prices are USD/month, ap-south-1, 730 h, from the public AWS Price List bulk files, re-pulled 28 Sep 2026 against list version 20260925 and **unchanged** from the 22 Sep pull. The EC2 bulk file is 313 MB and must be streamed; the public-IPv4 charge is **not** in it — it is `AP53-PublicIPv4:InUseAddress` under AmazonVPC. Sized for `CLAUDE.md`'s load (~3K normal, 6K handled, 8K with additions, 10K inside limits), every derived figure worked against **8,000 candidates/event, 30 events/month**. Two decisions are declared closed: **data stores split off the API box (26 Sep)** and **compute settled on EC2 + Docker Compose behind Caddy, Fargate/ALB not returning (29 Sep)**.

\$1 Cost. Today, staging only: Box A' `t4g.medium` \$16.35 + 30 GB gp3 & EIP \$6.39; Box B' `t4g.micro` \$4.09 + 10 GB gp3 & IPv4 \$4.56; RDS `db.t4g.micro` 20 GB gp3 7-day PITR \$17.95; S3 + three CloudFront distributions ~\$1; Route 53/ECR/CloudWatch/two free Budgets \$2.40 — **\$52.74**. With production: Box A' `t4g.large` + 30 GB + EIP \$39.09; Box A' `t4g.small` \$14.57; Box B' `x2 t4g.medium` 20 GB \$21.82; RDS `db.t4g.small` two databases \$33.28; frontend/media/DNS SG \$3.40 — **\$112.16. +18% GST** (AWS India) → ~\$62 and ~\$132. Staging starts `medium` because `pnpm install` + two Vite builds want 2–4 GB; drops to `small` once images come from ECR. The old ALB+Fargate shape was ~\$142 for production alone. Egress is inside the free 100 GB: measured 25 Sep, a 100-question bilingual sitting is 41 KB paper + 56 KB solution report gzipped, and the score card **re-measured 29 Sep at 1 KB gzipped / 2.7 KB raw** regardless of paper length — under 100 KB total, 0.8 GB per event, ~24 GB/month; without `compression()` ~125 GB and billable.

\$2 Shape. One VPC, two AZs only because an RDS subnet group demands two subnets in two zones; everything runs in one zone, so inter-AZ traffic is zero. Box A: public subnet, Elastic IP, Caddy terminating TLS and routing by path to exam or core; SG 80/443 from the world, 22 from one address. Box B: SG accepts 6379 and 6380 **from the API boxes' security group by group membership**, nothing else; public address only to pull images. RDS: private subnet, 5432 from the API SG. **No NAT**. Valkey has its own box for **recovery**, not memory: the API box becomes disposable, a dead box is a three-minute AMI launch plus an EIP remap, and live sittings survive.

\$3 Compute. `API_ROLE (apps/api/src/config/api-role.ts)` selects the role; one image, unset = all. **exam** 0.8 vCPU, heap 1536, scales with the box, serves `AttemptsController` + `MeLeaderboardController`. **core** 0.6 vCPU, heap 768, `auth/me/saved/results/ admin/* /imports`. **worker** 0.4 vCPU, heap 768, **exactly one, never more**, no routes but health and metrics, all eight processors and schedulers. The singleton is a constraint: `concurrency: 1` in `QUEUE_POLICY` is the only thing stopping the 60 s flush and 120 s sweep overlapping, and it works in one process only; a fourth scoring-only role is the escape hatch, not a second worker. Exam gets the largest share because of CFS throttling. **Exam alone runs a budget: 6 s `statement_timeout`, 8 s request budget → 503**, both under the client's 10 s submit / 15 s save give-up; keyed on the role being exactly `exam`. Peak ~350–500 req/s at 8,000 candidates. Scoring measured 25 Sep with `scripts/bench-scoring.mjs`: 5,000 sittings of 100 questions reach `EVALUATED` in **4.75 s** at concurrency 8, p95 9.4 ms, **1.99 ms Postgres CPU + 2.46 ms Node CPU each**, 3.4 transactions, 61 tuples read / 28.5 written; sweep 2→488/s, 4→798, 8→1,052, 16→1,254, 24→1,301 with p95 4.7→26.6 ms. On a 10-core M4 with Postgres on loopback; Graviton2 assumed 3–4× slower, so read it as **25–32 s of database time and 2–3.5 min of worker time at 0.4 vCPU**. Two-box latency 0.1–0.3 ms vs loopback's 50 μs over **nine round trips per attempt** = +15–40%. Scaling is vertical: `t4g.2xlarge` serves ~4,000 req/s, eight times the 10K ceiling, so **the box runs out of availability, not capacity**. Caddy uses a `dynamic a` upstream so `--scale exam=4` needs no config change. Only EVENT-series tests are pre-warmed, and pre-warming here means a scheduled resize before `opensAt`.

\$4 Ingress. Caddy in a container, per-hostname Let's Encrypt over ACME HTTP; no ALB, ACM or target groups. `deploy/Caddyfile` is the source of record. **The split is not prefix-clean:** `/me/attempts/:id/scorecard`, `/solutions`, `/question-report` are core under the same prefix as the exam role's `/paper`, `/state`, `/submit`, so the matcher lists leaf suffixes. Given up: one Caddy is not self-healing, a reboot is an outage. One escape hatch — a **warm standby behind a Route 53 health check at \$0.50/month**. Let's Encrypt refuses `*.amazonaws.com` (Public Suffix List), and the student app is a PWA, so every environment needs a real name; `*.cloudfront.net` is the exception. The API is deliberately **not behind CloudFront**: 15–30M autosave requests/month would add \$10–30 and buy nothing. Anything in front must pass `x-client / x-device-name`, and `TRUST_PROXY_HOPS` must count the real hops.

\$5 Frontend. Two SPAs, one **private S3 bucket, two CloudFront distributions**, pay-as-you-go (always-free 1 TB / 10M requests / 2M function invocations); flat-rate plans refused because the Free plan's 1M requests per distribution is passed in a busy month. `/assets/*` one year immutable; `index.html`, `sw.js`, `manifest` `no-cache`; **old assets never deleted**; page routes reach `index.html` through a **CloudFront Function**, not the 404-to-index rule; compression and HTTP/3 are CloudFront's.

\$6 Database. RDS PostgreSQL 17, **one instance, a database per environment** — not a container, because backups, PITR and storage autoscaling. Now `db.t4g.micro` (2 vCPU, 1 GB) 20 GB gp3 \$15.33 + \$2.62; production `db.t4g.small` (2 GB) 20→100 GB \$30.66 + \$2.62. 7-day PITR on both, staging included, so the restore gets rehearsed. **Connections decide the size:** ~112 `max_connections` on 1 GB, ~225 on 2 GB; budget **exam 8, core 25, worker 25 = 58** + 10 staging = 68. 17 not 16 because Extended Support is \$0.114/vCPU-hour (16 leaves 28 Feb 2029, 17 on 28 Feb 2030). Scoring is 9 round trips, 3.4 transactions, 28.5 inserts + 1.7 updates, ~16 s of database CPU for an 8,000-candidate event. Start/flush/submit and the per-sitting disk/WAL figures are **void** (\$16).

\$7 Cache and queues. Valkey on Box B, **one container per environment**: production `:6379` `maxmemory 1.2 GB`, staging `:6380` 256 MB, both `appendonly yes` `fsync 1 s`, `maxmemory-policy noeviction`, **a password each** (`requirepass`) as the third lock after the SG and `PRIVATE_IP` binding. The API carries it in `REDIS_URL` as `redis://:PASSWORD@host:port`; the healthcheck takes `REDISCLI_AUTH` because `valkey-cli` does not read the URL. **A process each, not two database indexes**, because `maxmemory` is per process and `noeviction` makes a full instance **fail writes**. `noeviction` is BullMQ's requirement and the API refuses to boot in production against anything else. ElastiCache Serverless is unusable (no parameter groups); Cluster mode is refused because the flusher's `MGET` and settle script span `attempt:state:*` and `attempt:dirty` with no hash tags; `cache.t4g.small` is \$23.94 and comes back empty. Box B's size is decided by the **AOF rewrite fork**, never CPU (~960 ops/s against 100,000+): `t4g.micro` 1 GB / 10 GB now; `t4g.small` 2 GB / 10 GB before the first load test (~100 MB dataset, fork can double it); `t4g.medium` 4 GB / 20 GB at 8,000 live sittings. **Disk is sized by the AOF, not the OS** — about three times the dataset plus ~3 GB; gp3 gives 3,000 IOPS and 125 MB/s at any size.

\$8 Networking. No NAT: both boxes are public-subnet with their own address, so SMS, WhatsApp, push, Sentry, log shipping and ECR need no gateway — the old shape's `t4g.nano` NAT at \$6.42 is gone. **Security groups, not subnets**, are the boundary: a public subnet is only a route table, and three things must line up (address, SG rule, NACL). Box A takes 80/443 and nothing else inbound; Box B 6379 and 6380 and RDS 5432 **from Box A's SG by group**. **Docker cannot open a hole in a security group** — published ports bypass the host's iptables INPUT chain via PREROUTING DNAT (why `ufw` fails on containers), but an SG is enforced at the interface, outside the instance. **No inbound SSH:** `AmazonSSMManagedInstanceCore` + Session Manager, free, with a transcript. **IMDSv2 required at launch:** `HttpTokens=required,HttpPutResponseHopLimit=2` — hop limit 2 because the caller is inside a container. A private subnet is refused: it needs the load balancer \$4 removed plus a NAT, \$16–25 more; Box B's egress alternatives all beat its \$3.65 address (NAT instance \$6.42 + address, NAT gateway ~\$32, ECR+logs endpoints ~\$22). Egress \$0.1093/GB after 100 GB free.

\$9 Logs, metrics, alarms. `LOG_LEVEL` decides stdout and `apps/api/src/common/logging.ts` is the only place the shape is chosen: unset means `>=debug` outside production, `>=log` in it, and production also switches `ConsoleLogger` to `json: true`. `MetricsInterceptor` logs a line per request at `debug` — so every call in dev/staging, none in production, where 3K students autosaving is ~150 lines/s — and a success slower than **1,000 ms** at `warn` instead. `AllExceptionsFilter` owns failures: 4xx one line, 5xx with stack **and request body**. Bodies pass `apps/api/src/common/redact.ts` first (2 KB, six levels, twenty array items) and `instrument.ts` sets `sendDefaultPii: false`. Containers use `json-file` capped in `deploy/compose.yml` at **10 MB × 3**; an environment shipping to CloudWatch overrides to `awslogs`, 14-day retention, and nothing in the repo assumes it. Caddy's own JSON access log rolls at 100 MiB — no bucket, no lifecycle. **~ten alarms at \$0.10:** 5xx rate and latency from `lace_http_request_duration_seconds`; container-down from EC2 status checks + Docker restart count; **CPUSurplusCreditsCharged** (the largest expected-vs-worst-case invoice gap); RDS CPU/storage/connections; Valkey memory and evictions (\$0.30 custom metric); **CloudFront bytes out**. **No ALB 5xx, no unhealthy-target count, no ECS running count, and there will not be**. Grafana Cloud's free tier scrapes `/metrics` (AMP + AMG would be \$14–19). Off deliberately: Container Insights, VPC flow logs.

\$10 Media. One private bucket behind CloudFront over an OAC; a question image is a **stable unsigned URL** = `MEDIA_BASE_URL` + key. Reason is **caching, not privacy**: a per-request signature means 8,000 students fetch the same diagram 8,000 times from S3 at \$0.1093/GB — \$37–64/month — and never hit browser cache. Nothing is given away: the key is a uuid, the bucket does not list, and what protects a paper is the server refusing it before `startedAt`. The escalation is CloudFront **signed cookies**, never a URL signature. Uploads carry `public`, `max-age=31536000`, `immutable`; `minio-init` opens `questions/images` for anonymous download so the one code path holds. Admin imports and student documents keep presigned URLs. Diagrams re-encoded to WebP in the admin's browser (`75868e0`): 2,045 KB → 229 KB at 1600 px, files under 200 KB and GIFs untouched. Lifecycle: imports expire at 90 days, audit archives to Glacier at 90, incomplete uploads abort at 7.

\$11 Environment, secrets, DNS, registry. The environment file is the unit, not the variable: one file per environment at `deploy/.env`, root-owned, `chmod 600`, read twice by Compose (`--env-file` for `${...}` substitution, `env_file` per service for the process environment). **Nothing fetches configuration at boot** — `env.schema.ts` validates `process.env` and nothing else. Files: `.env.example` (tracked, every schema variable, `dev_only_` secrets), `deploy/.env.example` (tracked, the deployed subset with the pool arithmetic), `deploy/.env` (never tracked, `.gitignore` line 32, exists on the box and in SSM), `deploy/valkey/.env` (never tracked, `PRIVATE_IP` + a password per process). **Per-role values are deliberately not in the file** — `API_ROLE`, `UV_THREADPOOL_SIZE`, `NODE_OPTIONS`, `cpus`, `mem_limit` live per service in `compose.yml`. The SPAs have **no runtime environment**: `VITE_API_URL` is compile-time, so an environment is a build and a staging artifact cannot be promoted. **SSM Parameter Store holds the master copy and nothing reads it at runtime**: one `SecureString` per environment at `/iace/<env>/env` plus `/iace/valkey/env`, each a whole file; Standard tier caps at **4 KB**, so strip comments or pay \$0.05 for Advanced; Secrets Manager at \$0.40 each is refused, and the KMS key is AWS-managed. **The instance role is the one credential never in the file** — `S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY` stay unset in production (`847db3e`), and the profile carries exactly five things: the media bucket, `ssm:GetParameter` on `/iace/<env>/*` with `kms:Decrypt`, CloudWatch Logs write, ECR read, `AmazonSSMManagedInstanceCore`. **Production refuses to boot without**: `connection_limit` on `DATABASE_URL`, non-empty `CORS_ORIGINS`, `METRICS_TOKEN`, `TRUST_PROXY_HOPS` > 0, a Valkey that cannot evict, and any of `JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET` / `PIN_PEPPER` still carrying `dev_only_`; it **warns** when `NODE_OPTIONS` ' heap does not match the container's memory limit, naming the role. Route 53 \$0.50 (free ALIAS to CloudFront, an A record per environment to its EIP; ACM only in us-east-1, free; the EIP is the failover mechanism). ECR \$0.10/GB-month, last ten images, under \$0.20. **GitHub Actions pushes through an OIDC role**.

\$12 Non-production. Its own API box, sharing production's Valkey box and RDS instance — one `t4g.medium` running Caddy and the same three containers, because staging exists to rehearse production. Its own database with `CONNECTION LIMIT 10`, `statement_timeout 30 s`, `idle_in_transaction_session_timeout 60 s` on the role. Its own Valkey process on 6380 at 256 MB. `t4g.medium` while building on the box, `t4g.small` from ECR (three containers need ~850 MB at rest). `NODE_ENV=development`, `OTP_SENDER=console` → the API returns `devCode` on the request response, so a tester signs in with no provider; `CORS_ORIGINS` is still enforced. **Heaps 384 MB per container** rather than production's 1536/768/768. One DNS record per environment. **Seeded and generated data only, never a restore of production**. This box is also the load-testing rig, worth trusting because the database is across a network hop; resize to the production candidate for an afternoon (~\$0.33 for four hours at 4× size), against one burstable and one non-burstable family.

\$13 Growth and commitments. Box A' `t4g.medium` → `t4g.small` when images build in CI and come from ECR. Box A → `t4g.large` when production exists, → `t4g.xlarge` for an event, **scheduled, never reactively**. Box B `t4g.micro` 10 GB → `t4g.small` before the first full-scale load test → `t4g.medium` at 8,000 live sittings. RDS `db.t4g.micro` → `db.t4g.small` on a **second exam container — connections, not load. Commit to nothing at launch**; after 4–8 weeks commit to 70–80% of baseline. On demand vs 1-year no-upfront: `t4g.large` \$32.70/~\$23.50; `t4g.small` \$8.18/~\$5.91; `t4g.medium` \$16.35/~\$11.75; `db.t4g.small` \$30.66/\$24.09. A Compute Savings Plan covers any region and family so resizing stays covered; an RDS reservation is family-tied. **t4g unlimited mode is the default**: past the 20%-per-vCPU baseline it is ~\$0.04 per surplus vCPU-hour (~\$47/month for a pegged box) — the right trade for an exam, but it needs the `CPUSurplusCreditsCharged` alarm.

\$14 Release. (1) Build both targets from `apps/api/Dockerfile`: `runtime` 600 MB / 108 MB pulled, `migrate` 731 MB / 163 MB; the Prisma CLI is only in the migration image. (2) Build the SPAs — `VITE_API_URL` is compile-time. (3) Run the **migrate** image to completion; the API does not start until it has. (4) Recreate the API containers, each `tini`-led so SIGTERM closes Nest (`b2029d4`); **there is no draining** — `up -d` recreates with a gap. (5) Upload hashed assets **first** (`max-age=31536000`, `immutable`), then `index.html` / `sw.js` / `manifest` `no-cache`; invalidate those three paths only (1,000/month free). (6) Old `/assets` never deleted. (7) Push a changed `deploy/.env` back to SSM in the same breath. **The API goes before the SPAs** — a new bundle meeting the old API can send a field it strips, and a submit's `last` batch was exactly that. No `skipWaiting`: a new service worker does not activate until every old tab closes. A release bumping `STEM_HASH_VERSION` refreshes the bank in the worker at boot, logging `Rehashed N question stems` — hold imports until that line. Never during an event window.

\$15 Deferred. (a) The service worker never prunes its cache (`iace-shell-v1` fixed name) — stamp the build id into `sw.js`. (b) **Nobody has measured a real paper's image weight**, the one line that can move the invoice by an order of magnitude: CloudFront's free 1 TB over thirty 8,000-candidate events is **4.4 MB of images per sitting**; 23 MB would be ~\$450/month. Knobs are `MAX_WIDTH` and `QUALITY` in `shrink-image.ts`. (c) **Batching scoring**: nine round trips whose statements total ~0.7 ms against 1.99 ms of database CPU, so most of it is parse/plan/transaction overhead; batching collapses it to ~six statements, and `foldMistakes` already takes an array. (d) Whether `SavedQuestion` needs a row per wrong answer — 28 inserts/attempt, 140,000/event, derivable from `AttemptSheet.verdicts`. (e) Redis as a hash per sitting instead of one JSON document; revisit above ~100 Mbps sustained. (f) Archiving old answer sheets and pruning read notifications past ~200 GB. (g) **PgBouncer, when connections rather than load bind** — not needed on one box; it arrives with a second API box or past ~80% of `max_connections`. Measured 28 Sep with PgBouncer 1.25.2 in transaction mode: all of `pnpm test:db` passes unchanged and scoring runs ~1,390/s vs ~1,360 direct, given `max_prepared_statements = 200`, **no pgbouncer=true on the url** (the flag drops the statement cache and scoring fell to ~730/s at twice the database CPU), `search_path` in `track_extra_parameters`, and a **direct url for the migrate container** (the advisory lock stayed held on an idle server connection). RDS Proxy is not the alternative — Prisma pins every connection.

\$16 Not re-measured. Exam HTTP throughput 545 req/s at 0.5 vCPU / 2,076 at 1 vCPU, p99 400 ms vs 4–30 ms (undated, pre-answer-sheet). Start/flush/submit database CPU 0.13 / 0.35 / 0.41 ms. Per-sitting disk and WAL 3.75 KB and 14 KB. Redis per sitting 10.2 KB JSON / 12.1 KB stored. SPA first load 817 KB raw / 238 KB brotli at `3fca444`. **Graviton 3–4× slower — an assertion, never benchmarked, and every "on AWS" figure rests on it**.

Half 2 — where the document and the repo disagree

Blocking

1. **The release runbook's first step cannot run: the image does not build.** Doc §14 step 1 (docs/04-infrastructure.md:674) and deploy/README.md:3,82 . Reality: apps/api/Dockerfile:21 is COPY apps/test/package.json apps/test/ , and apps/test does not exist — 7912c96 (2026-09-30) renamed it to apps/exams and updated deploy/publish-spas.sh:18 but not the Dockerfile. apps/exams/package.json is never copied either, so removing line 21 alone would still break pnpm install —frozen-lockfile (pnpm-lock.yaml:255 has an apps/exams importer). **Verified by running it:** docker build -f apps/api/Dockerfile --target deps . fails at Dockerfile:21 with "/apps/test/package.json": not found . **The repo is wrong.** It also voids deploy/README.md:124–130 ("Verified 26 September 2026 ... all three images build") and makes §14's image sizes (600/108 MB, 731/163 MB) unverifiable today.
2. **§6's per-role connection budget cannot be expressed by the shape §11 describes.** Doc §6:311–314 budgets "exam 8, core 25, worker 25". But connection_limit lives in DATABASE_URL , which §11:534 declares is **one shared file** (deploy/compose.yml:38,62,82 give all three services the same env_file: .env , and only migrate sets DATABASE_URL explicitly at deploy/compose.yml:27). apps/api/src/prisma/prisma.service.ts:18–21 adds only statement_timeout , and only for the exam role — nothing adjusts the pool per role. So all three containers get whatever the single URL says. **The doc is wrong**, or the shape needs a per-service override it does not have.
3. **deploy/.env.example's staging pool contradicts §6 and §12.** deploy/.env.example:51 is the staging URL (iace_staging , cf. API_HOST=api.staging.iace.co.in at line 13) with connection_limit=25 — and three containers reading it request 75. §6:313 and §12:607 both budget staging at **10**, enforced as CONNECTION_LIMIT 10 on the role. With the role cap in place, connections are refused. **The example file is wrong**, and the comment above it (deploy/.env.example:47–50) restates the *production* budget beside a staging URL.

Stale claims

4. **§15's first deferred item is already done, and quotes a name that no longer exists.** Doc §15:709–712 says "The service worker never prunes its cache (iace-shell-v1 is a fixed name) ... Stamp the build id into sw.js and it cleans itself up." 3c57ef5 (2026-09-29) did exactly that: apps/exams/vite.config.ts:9,29 rewrites the placeholder to iace-shell-<buildId> at build time and throws if sw.js stops naming it; apps/exams/public/sw.js:9 is iace-shell-dev and the built artifact apps/exams/dist/sw.js:9 is iace-shell-01ac1578309d . The activate handler (apps/exams/public/sw.js:26–33) therefore does fire. **The doc is wrong.**
5. **§2 says the worker takes no HTTP; §3's own table says it serves health and metrics.** Doc §2:119 ("the worker takes no HTTP at all") vs §3:156 ("no routes but health and metrics"). apps/api/src/main.ts:66 listens unconditionally; HealthModule (apps/api/src/health/health.module.ts:12) and MetricsModule (apps/api/src/common/metrics/metrics.module.ts:16) register their controllers with **no role gate**; deploy/compose.yml:85,93 gives the worker API_PORT: 3000 and a wget ... /health healthcheck. **§2 is wrong**; §3 is right.
6. **§9 says failures never depend on the log level — a 4xx does.** Doc §9:462 ("Failures never depend on the level. AllExceptionsFilter owns them — a 4xx as one line"). apps/api/src/common/all-exceptions.filter.ts:65 logs a 4xx with this.logger.debug(...) , which production's >=log (apps/api/src/common/logging.ts:11,19) drops entirely. Only the 5xx branch (:57 logger.error) survives. **The doc is wrong.** Input that bites: any 400/403/409 in production — a rejected submit leaves no log line at all.
7. **§9 repeats itself.** Doc §9:460 and §9:469–470 state "Nothing logs a successful response body: at 6K students ... tens of GB an event" twice, almost verbatim. Editorial, but it will print twice.

Present-tense claims with no repo artifact

8. **§11:599 "GitHub Actions pushes through an OIDC role" — there is no such workflow.** .github/workflows/ contains exactly one file, ci.yml , with three jobs (verify:31 , database:92 , docs-check:167), permissions: contents: read (:17–18), no id-token , no aws-actions/configure-aws-credentials , no ECR login, no image build or push, and no deploy job. §13:645's trigger ("Images build in CI and come from ECR") is therefore unreachable today. **The doc states an intention as a fact.**
9. **§5:288 "Page routes reach index.html through a CloudFront Function" — nothing in the repo contains one** (deploy/publish-spas.sh only syncs and invalidates). Console-side config; *not verified* .
10. **§9:476 "Caddy's access log ... rolled by Caddy itself at 100 MiB" —** deploy/Caddyfile:34–37 sets output file /var/log/caddy/access.log and format json with **no roll_size** . 100 MiB is Caddy's documented default, not something the repo sets. Accurate by default, not by configuration.
11. **§12:607's CONNECTION_LIMIT 10 , statement_timeout 30s , idle_in_transaction_session_timeout 60s on the staging role —** no SQL, script or migration in the repo sets these. Manual DBA steps; *not verified* .

Verified correct (so the handbook can assert them)

- API_ROLE selection: apps/api/src/config/api-role.ts:33 reads process.env.API_ROLE at module
- The Caddyfile matcher is **exactly** the exam role's route set. deploy/Caddyfile:11 lists
- Worker singleton rationale: ATTEMPT_FLUSH_EVERY_MS = 1 min / ATTEMPT_SWEEP_EVERY_MS = 2 min
- Exam budget: REQUEST_BUDGET_MS = 8_000 , STATEMENT_TIMEOUT_MS = 6_000 , keyed on the role being
- §11's boot refusals: all five in apps/api/src/config/env.schema.ts:252–287
- .env.example carries all 70 schema variables — **zero missing**. The 14 extras are all read
- §9's logging claims, all of them: appLogger() defaults and json: production
- §10: one upload path, publicUrl = MEDIA_BASE_URL + '/' + key
- §14: tini ENTRYPOINT on both targets (apps/api/Dockerfile:56,67); migrate gated by
- §6's Postgres 17: postgres:17-alpine in both docker-compose.yml:15 and ci.yml:100 .

Internal arithmetic worth a second look

- §7:385 says 8,000 sittings is a ~**100 MB** dataset, and §7:388–391's own rule is "three times the
- §11:545 says "**Three** files, and only one of them is real" above a table with **four** rows.
- `apps/api/Dockerfile:1` still reads "for ARM — **Fargate** runs it on Graviton", which §3:41
- `apps/api/src/config/api-role.ts:29–30` warns and falls back to `ALL` on an unknown `API_ROLE`, but

Part VIII — What is wrong with it

Twenty-two readers went through this repository and came back with **141 findings**. Each one is written up in its own chapter, under *What is wrong with it*, with the path and line that proves it. This part is the triage: what to look at first, and what can wait.

Nothing here is a style opinion. A finding earned its place by naming a concrete input that produces a wrong result, a dead path, or a sentence in the repository that is not true of the code beside it.

Note The nine findings in the first two tables were re-checked by hand against the working tree rather than taken on trust. Where a reader's claim did not survive that check it was dropped, and two of them were.

Blocking — fixed in this pass

WHAT	WHERE	WHAT IT DID
The API image could not build. COPY apps/test/package.json apps/test/ named a directory that no longer existed, and apps/exams/package.json was never copied in its place.	apps/api/Dockerfile:21	docker build failed at that line. 7912c96 renamed the student app and updated deploy/publish-spas.sh but not this, so every release step downstream of the image was unreachable.

This was the single most urgent finding in the sweep and it is **fixed** — the line now names apps/exams , and all three targets (deps , runtime , migrate) were built to prove it, with dist/main.js confirmed present in the runtime image. The same commit drops a stale reference to Fargate in the file's header; the boxes are Graviton t4g instances under Docker Compose.

Nothing else in this part is fixed. Everything below is open.

Correctness — a user gets a wrong answer

WHAT	WHERE	THE INPUT THAT BITES
A retake re-locks the later papers of a sequential series. The standing map keeps the /latest sitting, so starting a retake of test 1 turns its status back to IN_PROGRESS and test 2 stops being startable.	apps/api/src/access/access-resolver.service.ts:283 and :492	Finish test 1, start a retake of it, look at test 2.
A proof-reader can clear a tick on a different section's question. check and sendBack both call requireOnPaper ; uncheck does not, and its updateMany keys on (testId, questionId) alone. fixed has the same hole.	apps/api/src/questions/section-work.service.ts:339 and :384	DELETE /admin/sections/<testId>/<sectionA>/questions/<a question on section B>/check . The released reader of section B is never pulled back, because the un-tick path does not reopen reading.
Any admin holding TEST_MANAGEMENT write can delete a work-free assignment on any test in the institute . The route takes no testId and the service reads the row by id alone.	apps/api/src/assignments/assignments.controller.ts:127 , apps/api/src/assignments/assignments.service.ts:312	DELETE /admin/assignments/<any assignment id> .
A test-blocked student is counted in an admin's reach figure and is sent the "this test is open now" notification, for a test their own catalog marks unstartable. audienceOf filters deletedAt and isActive but never isTestBlocked .	apps/api/src/access/access-resolver.service.ts:407	Block a student, then open a test their series reaches.
Five wrong PIN guesses inside the cooldown leave the student told to "request a new code" while the next request is refused RATE_LIMITED . The attempt-cap branch deletes the OTP and attempt keys but not the cooldown; the success path does clear it.	apps/api/src/auth/otp.service.ts:185	Five bad guesses within 45 seconds.
Setting NOTIFICATION_COST_SMS_PAISE to 0 makes the platform-wide OTP spend ceiling Infinity . The variable is nonnegative() , and the kill switch divides by it.	apps/api/src/config/env.schema.ts:186 , apps/api/src/auth/otp.service.ts:129	A zero in the environment file silently removes the cap.

Privacy and exposure

WHAT	WHERE	WHAT IS EXPOSED
Erasure does not erase the documents. The anonymiser nulls photoUrl and tenthMarksheetUrl on the row, but StorageService is never imported into that service — verified, zero references. The photo and the marksheet stay in S3 under students/<studentId>/... , with the student id still in the key.	apps/api/src/students/student-privacy.service.ts , apps/api/src/students/anonymize.ts:36	Erase a student who uploaded a photo, then list the bucket prefix. For a DPDP erasure that is the whole point of the operation.
GET /me/data-export is live, returns a student's entire record including every sitting, is the only /me route with no @Audit , and has no caller anywhere — verified: the only two matches in apps/ and packages/ are its own definition. Its sibling POST /me/erasure was deleted for exactly this reason; the export was left behind.	apps/api/src/me/me.controller.ts:78	One curl with any student token. Unlogged.

WHAT	WHERE	WHAT IS EXPOSED
<code>requestAdminOtp</code> sits only on the shared per-IP auth bucket and is metered by none of the daily caps, so it answers "is this address an IACE admin" for free.	<code>apps/api/src/auth/auth.service.ts:210</code>	Enumeration against the admin OTP endpoint.

Prose that is not true of the code

This class matters more than its severity suggests: each one is a sentence that would send a reader down the wrong path, and two of them describe **safety gates that were deleted**.

THE CLAIM	WHERE	THE TRUTH
"Gated. Empty and null until <code>solutionsAreOpen</code> " — twice.	<code>packages/contracts/src/stats.ts:686</code> , <code>:717</code>	No such symbol, column or check exists. It was deleted in <code>7b71af8</code> , "a test opens and never shuts". The real gate is two lines: your own sitting, status <code>EVALUATED</code> . An early submitter can read the answer key while the rest of the hall is still writing, and nothing prevents it.
<code>docs/03-conventions.md</code> §5: "tests moves a <code>Question</code> in-use counter on the offer, once per question served."	<code>docs/03-conventions.md:168</code>	There is no such column. <code>Question</code> has no counter of any kind; in-use is derived live as <code>paperQuestions.length + questionStats.length > 0</code> . This line is mine — I rewrote its timing in the last pass and kept a premise that was already false. It is corrected in this session's commit.
<code>RowActionLog.entityLabel</code> "keeps the label as it was, so the log survives a rename or a delete".	<code>prisma/schema.prisma:1020</code>	Nothing writes or reads it. After a rename the audit screen shows the raw uuid.
<code>Notification.openedVia</code> exists "so the policy stops being an assumption".	<code>prisma/schema.prisma:1061</code>	The policy never consults it; nothing writes it.
<code>providerMessageId</code> is "indexed for the webhook that will...".	<code>prisma/schema.prisma:1111</code>	The index was dropped and never replaced, and the column is never read or written.
A migration header calls <code>SectionComment</code> "append-only by construction".	migration <code>20260919140000</code>	A later migration made it editable, and an admin whose assignment row is <code>replacedAt</code> can still reword an old message.
Comments naming a transactional outbox, a relay, and "stranded events".	<code>apps/api/src/queue/queues.ts:83</code> , <code>apps/api/src/notifications/test-opening.service.ts:5</code>	The outbox, its trigger and three SQL functions were deleted in <code>a5112aa</code> . The job id <code>relay-sweep</code> cannot be renamed without orphaning the live scheduler, so that one is not free — the comments are.

Dead weight

Columns, routes and exports that nothing reaches. Each is cheap to remove and each currently reads to a newcomer as a feature.

- `Attempt.sectionState` — a jsonb column written and read by nothing.
- `DeliveryStatus.DELIVERED` — counted and exported, never written, so every announcement shows `delivered: 0` beside a non-zero `sent`.
- `Notification.actBy` — the sole input to the "pay now rather than wait" branch, and no producer ever sets it, so that branch is reachable only from a test.
- `scoreHistogramSchema` — describes a column dropped by `20260924140000`.
- `aadhaarVerified` / `panVerified` — rendered as a "Verified" badge on the student's profile; nothing in the codebase ever sets either to true, so every student sees "Not verified" permanently with no route to change it.
- `DEVICE_PLATFORM.IOS` — a dead enum half: the mobile client returns early on anything but Android.

Everything else

The remaining findings sit in their own chapters. They are real, but they are the kind that cost a query, a comment or a little clarity rather than a correct answer.

CHAPTER	FINDINGS	THE THEME
The question bank	7	Search cost, and the version-rewrite rule's edges
Building a test	10	The largest count in the document — mostly the freeze's edges
Importing in bulk	7	A run's counts disagree between the screen and the audit row; a CSV downloads named <code>.xlsx</code>
The platform	5	<code>/health</code> answers 200 while degraded, so a liveness probe never restarts a container with a dead Redis
Shared packages	11	Mostly exports nothing imports any more
The two SPAs	12	Stale copy, dead props, one button whose label and destination disagree
Rank and rollups	4	A percentile computed for 5,999 rows and discarded
The mobile client	4	Push is dark by configuration, not by code

Note None of this was found by `pnpm docs:check`. That gate validates that a backticked name exists and that a `docs/03 §N` citation resolves — it cannot read a sentence. Every finding above is prose naming real things and saying something untrue about them, or code nothing calls. There is no mechanical check for either, and this document is the substitute.